



课程重点内容回顾

授课教师：张圣林

南开大学



成绩评定



- 平时成绩：10%
 - 出勤、课堂表现
- 实验成绩：30%
 - 实验结果
- 期末考试：60%
 - 闭卷考试



考试信息



- 考试时间：2024年12月30日（周一）上午10:00-11:40
- 考试地点：泰达4区3阶和4区4阶
- 考试题型
 - 选择（10道题，20分）
 - 填空（30空，30分）
 - 简答&计算&综合题&编程题（4道题，50分）



第一章 引言

授课教师：张圣林

南开大学



本章目标



1. 了解计算机网络在经济生活中的应用范围及其重要性
2. 了解网络的基本概念，掌握典型交换方式及其优缺点
3. 掌握分层结构和网络协议（核心内容）
4. 掌握网络参考模型（核心内容）
5. 掌握计算机网络主要度量的含义
6. 了解网络安全与威胁
7. 了解制定网络协议的标准化组织
8. 了解国内外互联网发展史，以史为鉴



本章内容



1.1 初识互联网

1.2 网络实例

1.3 协议与分层结构

1.4 参考模型

1.5 计算机网络度量单位

1.6 网络安全与威胁

1.7 标准化组织

1.8 互联网发展史与启示

1. 协议设计目的
2. 协议分层结构
3. 服务原语
4. 服务与协议的关系



协议设计目的

➤ 网络协议

- 为进行网络中的数据交换而建立的规则、标准或约定，即网络协议(network protocol)
- 通信双方需要共同遵守，互相理解

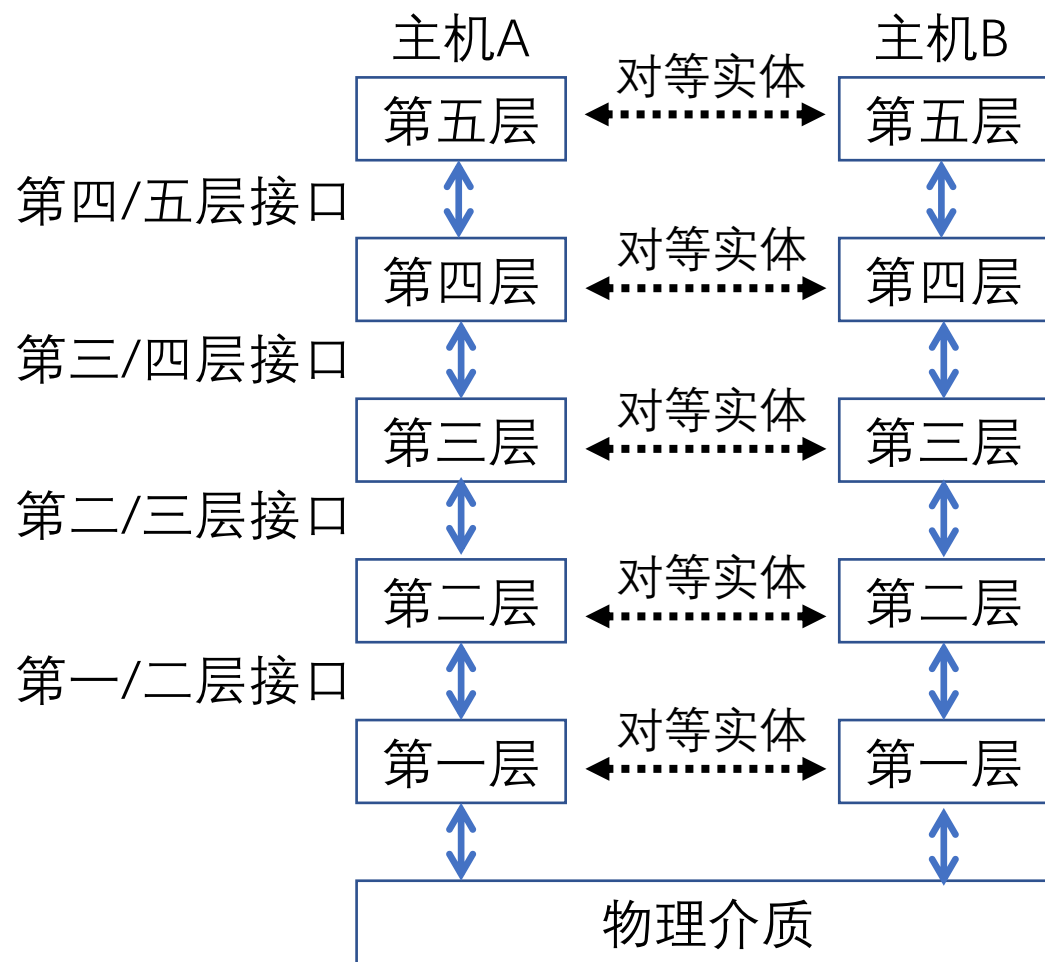
➤ 三要素

- 语法：规定传输数据的格式（如何讲）
- 语义：规定所要完成的功能（讲什么）
- 时序：规定各种操作的顺序（双方讲话的顺序）

通信双方要说的事情多种多样，导致网络协议异常复杂

协议分层结构

- 层次栈
 - 为降低网络设计的复杂性，网络使用层次结构的协议栈，每一层都使用其下一层所提供的服务，并为上层提供自己的服务
- 对等实体
 - 不同机器上构成相应层次的实体成为对等实体
- 接口
 - 在每一对相邻层次之间的是接口；接口定义了下层向上层提供哪些服务原语
- 网络体系结构
 - 层和协议的集合为网络体系结构，一个特定的系统所使用的一组协议，即每层的协议，称为协议栈

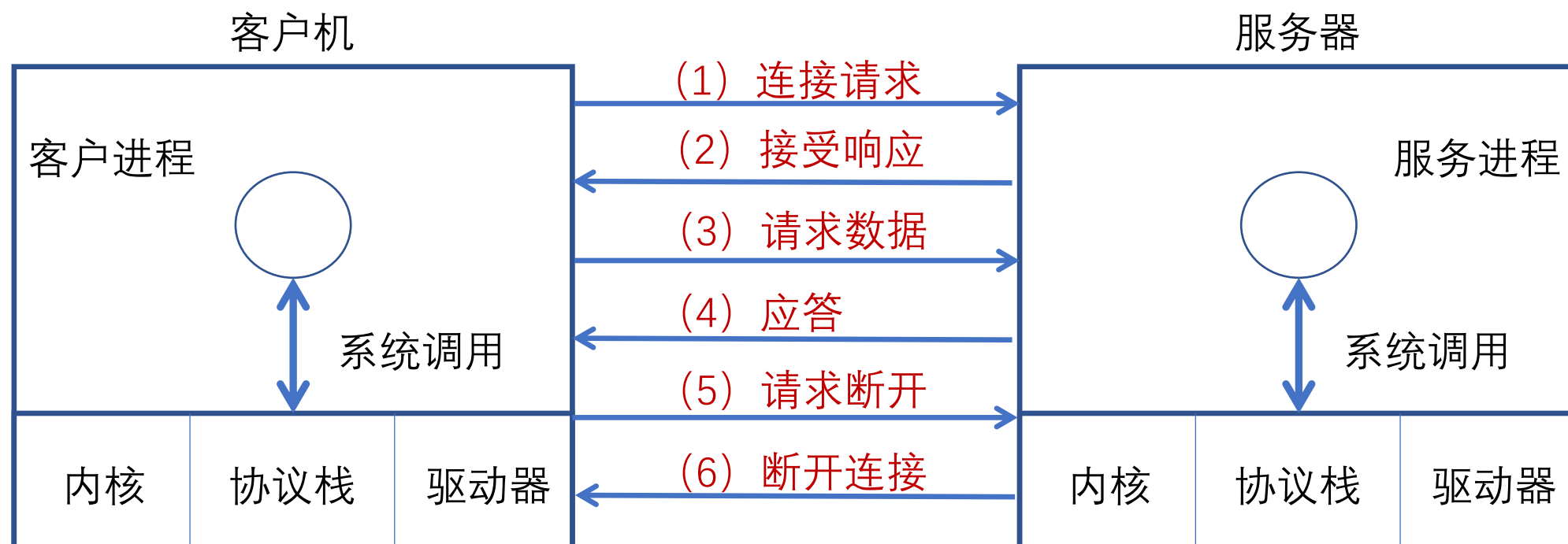


服务原语

➤ 服务

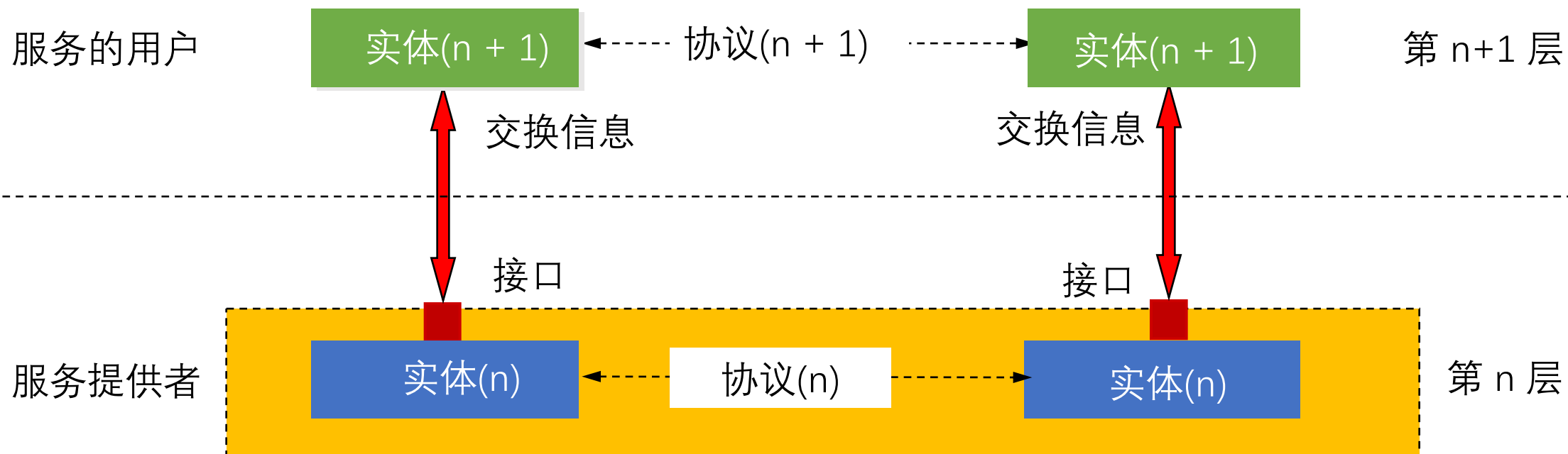
- 典型服务：面向连接传输服务，无连接传输服务
- 原语告诉服务执行某些操作或报告对等实体所采取的操作

➤ 六个核心服务原语（以面向连接服务为例）



服务与协议的关系

- 协议是“水平”的，服务是“垂直”的
- 实体使用协议来实现其定义的服务
- 上层实体通过接口使用下层实体的服务





目 录



1.1 初识互联网

1.2 网络实例

1.3 协议与分层结构

1.4 参考模型

1.5 计算机网络度量单位

1.6 网络安全与威胁

1.7 标准化组织

1.8 互联网发展史与启示

1. OSI参考模型

2. TCP/IP参考模型

3. OSI模型TCP/IP模型对比

4. 课程内容的分层组织

5. 模型与网络实例



OSI 参考模型



➤ OSI 7 层模型

- OSI: Open Systems Interconnection
- Day, Zimmermann, 1983

➤ 物理层 (Physical Layer)

- 定义如何在信道上传输0、1: Bits on the wire
- 机械接口 (Mechanical) : 网线接口大小形状、线缆排列等
- 电子信号 (Electronic) : 电压、电流等
- 时序接口 (Timing) : 采样频率、波特率、比特率等
- 介质 (Medium) : 各种线缆、无线频谱等
- ...

应用层 Application Layer
表示层 Presentation Layer
会话层 Session Layer
传输层 Transport Layer
网络层 Network Layer
数据链路层 Data Link Layer
物理层 Physical Layer



OSI 参考模型



- 数据链路层 (Data Link Layer)
 - 实现相邻 (Neighboring) 网络实体间的数据传输
 - 成帧 (Framing) : 从物理层的比特流中提取出完整的帧
 - 错误检测与纠正: 为提供可靠数据通信提供可能
 - 物理地址 (MAC address) : 48位, 理论上唯一网络标识, 烧录在网卡, 不便更改
 - 流量控制, 避免“淹没” (overwhelming) : 当快速的发送端遇上慢速的接收端, 接收端缓存溢出
 - 共享信道上的访问控制 (MAC) : 同一个信道, 同时传输信号。如同: 同一间教室内, 多人同时发言, 需要纪律来控制
 - ...





OSI 参考模型



➤ 网络层 (Network Layer)

- 将数据包跨越网络从源设备发送到目的设备 (host to host)
- 路由 (Routing)：在网络中选取从源端到目的端转发路径，常常会根据网络可达性动态选取最佳路径，也可以使用静态路由
- 路由协议：路由器之间交互路由信息所遵循的协议规范，使得单个路由器能够获取网络的可达性等信息
- 服务质量 (QoS) 控制：处理网络拥塞、负载均衡、准入控制、保障延迟
- 异构网络互联：在异构编址和异构网络中路由寻址和转发

思考：为何在唯一的MAC地址之外，还需要唯一的IP地址？





OSI 参考模型



➤ 传输层 (Transport Layer)

- 将数据从源端口发送到目的端口（进程到进程）
- 网络层定位到一台主机（host），传输层的作用域具体到主机上的某一个进程
- 网络层的控制主要面向运营商，传输层为终端用户提供端到端的数据传输控制
- 两类模式：可靠的传输模式，或不可靠传输模式
- 可靠传输：可靠的端到端数据传输，适合于对通信质量有要求的应用场景，如文件传输等
- 不可靠传输：更快捷、更轻量的端到端数据传输，适合于对通信质量要求不高，对通信响应速度要求高的应用场景，如语音对话、视频会议等





OSI 参考模型



- 会话层 (Session Layer)
 - 利用传输层提供的服务，在应用程序之间建立和维持会话，并能使会话获得同步
- 表示层 (Presentation Layer)
 - 关注所传递信息的语法和语义，管理数据的表示方法，传输的数据结构
- 应用层 (Application Layer)
 - 通过应用层协议，提供应用程序便捷的网络服务调用



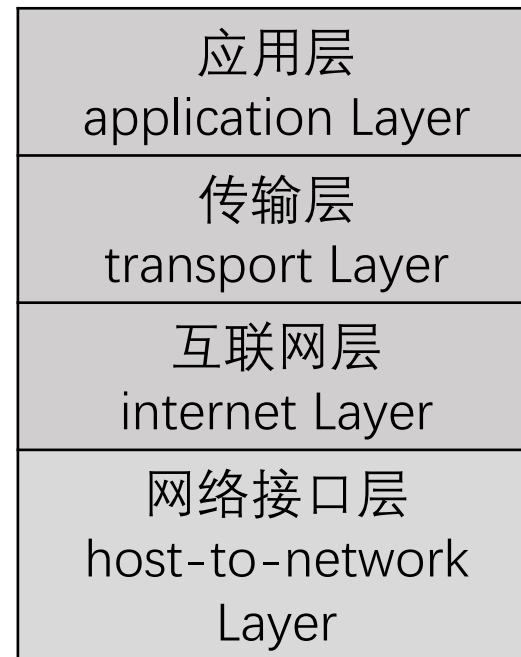


TCP/IP参考模型



- TCP/IP参考模型：ARPANET所采用
 - 以其中最主要的两个协议TCP/IP命名
 - Vint Cerf和Bob Kahn于1974年提出
- 网络接口层（Host-to-network Layer）
 - 描述了为满足无连接的互联网络层需求，链路必须具备的功能
- 互联网层（Internet Layer）
 - 允许主机将数据包注入网络，让这些数据包独立的传输至目的地，并定义了数据包格式和协议（IPv4协议和IPv6协议）
- 传输层（Transport Layer）
 - 允许源主机与目标主机上的对等实体，进行端到端的数据传输：TCP，UDP
- 应用层（Application Layer）
 - 传输层之上的所有高层协议：DNS、HTTP、FTP、SMTP...

ARPNET
最终采用TCP和IP
为主要协议



- 先有TCP/IP协议栈，然后有TCP/IP参考模型
- 参考模型只是用来描述协议栈的

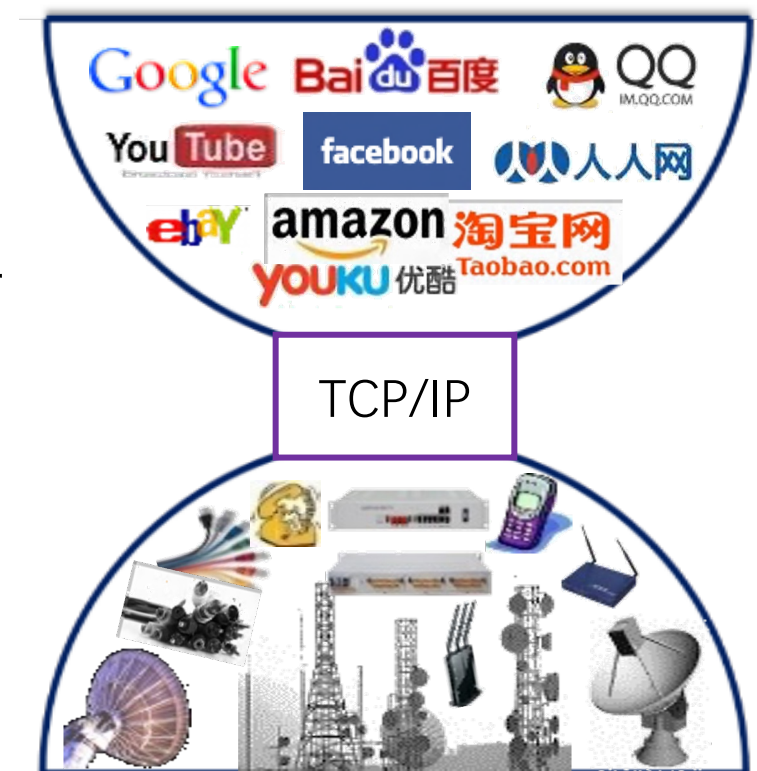
TCP/IP参考模型

➤ TCP/IP参考模型

- 摒弃电话系统中“笨终端&聪明网络”的设计思路
- **端对端原则**：采用**聪明终端&简单网络**，由**端系统**负责丢失恢复等，简单的网络大大提升了可扩展性
- 实现了建立在简单的、不可靠部件上的可靠系统

➤ IP分组交换的特点

- 可在各种底层物理网络上运行(**IP over everything**)
- 可支持各类上层应用(**Everything over IP**)
- 每个IP分组携带各自的目的地址，网络核心功能简单（通过路由表转发分组），适应爆炸性增长



TCP/IP的沙漏模型



OSI 模型与TCP/IP模型比较

➤ 7层模型与4层模型

- TCP/IP模型的网络接口层定义主机与传输线路之间的接口，描述了链路为无连接的互联网层必须提供的基本功能
- TCP/IP模型的互联网层、传输层与OSI模型的网络层、传输层大致对应
- TCP/IP模型的应用层包含了OSI模型的表示层与会话层

➤ 基本设计思想：通用性与实用性

- OSI：先有模型后设计协议，不局限于特定协议，**明确了服务、协议、接口等概念**，更具通用性
- TCP/IP模型：仅仅是对已有协议的描述

➤ 无连接与面向连接

- OSI模型网络层能够支持无连接和面向连接通信
- TCP/IP模型的网络层仅支持无连接通信（IP）

应用层
表示层
会话层
传输层
网络层
数据链路层
物理层

OSI 7层模型

应用层
传输层
互联网层
网络接口层

TCP/IP 4层模型



OSI模型与TCP/IP模型比较



OSI的失败：糟糕的时机、技术、实现、政策

OSI模型的不足

- 从未真正被实现
 - TCP/IP已成为事实标准，OSI缺少厂家支持
- 技术实现糟糕
 - OSI分层欠缺技术考虑：会话层、表示层很少内容；数据链路层、网络层内容繁杂。模型和协议过于复杂
 - 分层间功能重复：差错控制、流量控制等在不同层反复出现
- 非技术因素
 - TCP/IP实现为UNIX一部分，免费
 - OSI被认为是政府和机构的强加标准

TCP/IP模型的不足

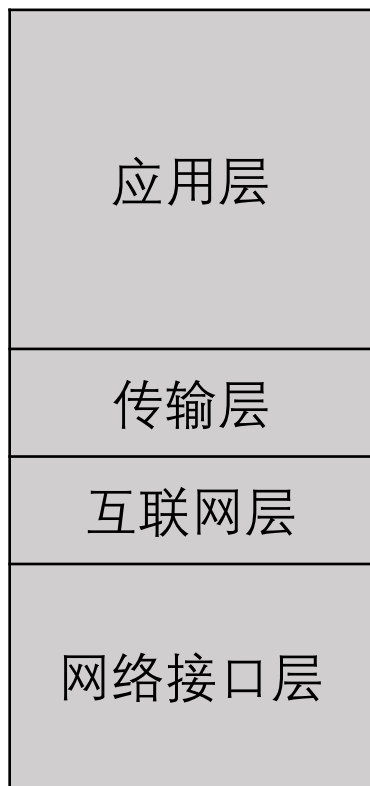
- 核心概念未能体现
 - 未明确区分服务、接口和协议等核心概念
- 不具备通用性
 - 不适于描述TCP/IP之外的其它协议栈
- 混用接口与分层的设计
 - 链路层和物理层一起被定义为网络接口层，而非真正意思上的分层
- 模型欠缺完整性
 - 未包含物理层与数据链路层
 - 物理层与数据链路层是至关重要的部分



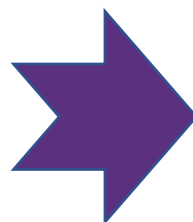
本教程内容的分层组织



OSI 7层模型



TCP/IP 4层模型



本教程的分层组织

- 突出核心概念
- 区分接口与分层
- 体现完整性
- 体现通用性
- 简化分层，易于教学



本章内容



1.1 初识互联网

1.2 网络实例

1.3 协议与分层结构

1.4 参考模型

1.5 计算机网络度量单位

1.6 网络安全与威胁

1.7 标准化组织

1.8 互联网发展史与启示

- 比特率
- 带宽
- 包转发率
- 时延
- 往返时间 RTT
- 时延带宽积
- 吞吐量
- 丢包率
- 利用率
- 时延抖动



第三章 数据链路层基础

授课教师：张圣林

南开大学



本章目标



- 了解数据链路层在网络体系结构中的位置及基本功能和服务
- 掌握差错检测和纠正的基本原理和典型的编码方法（核心内容）
- 掌握无错信道和有错信道上停等协议的设计和实现方法（核心内容）
- 理解停等协议的性能问题及滑动窗口协议的基本思想
- 掌握回退N和选择重传两种典型滑动窗口协议的工作机制（核心内容）
- 了解点到点链路层协议PPP与PPPoE



本章内容



3.1 数据链路层的设计问题

3.2 差错检测和纠正

3.3 基本的数据链路层协议

3.4 滑动窗口协议

3.5 数据链路协议实例

1. 数据链路层在协议栈中的位置
2. 数据链路层的功能
3. 数据链路层提供的服务
4. 成帧
5. 差错控制
6. 流量控制



数据链路层在协议栈中的位置

- 向下：利用物理层提供的位流服务
- 向上：向网络层提供明确的 (well-defined) 服务接口



参考协议栈



数据链路层的功能

➤ 成帧 (Framing)

- 将比特流划分成“帧”的主要目的是为了检测和纠正物理层在比特传输中可能出现的错误，数据链路层功能需借助“帧”的各个域来实现

➤ 差错控制 (Error Control)

- 处理传输中出现的差错，如位错误、丢失等

➤ 流量控制 (Flow Control)

- 确保发送方的发送速率，不大于接收方的处理速率
 - 避免接收缓冲区溢出



本章内容



3.1 数据链路层的设计问题

3.2 差错检测和纠正

1. 差错检测与纠正概述

2. 典型的纠错码

3. 典型的检错码

3.3 基本的数据链路层协议

3.4 滑动窗口协议

3.5 数据链路协议实例



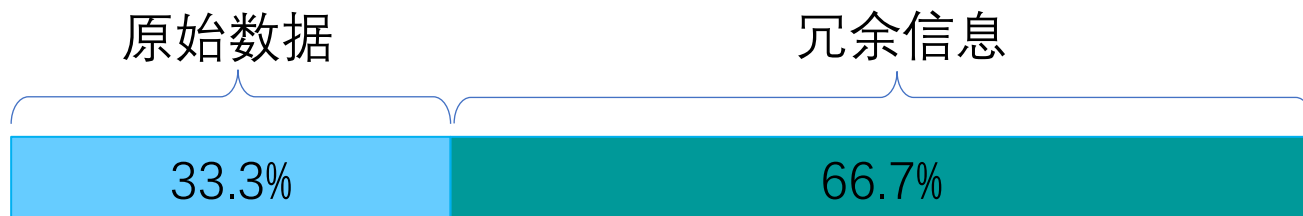
差错检测和纠正概述

➤ 如何解决信道传输差错问题

- 通常采用增加冗余信息（或称校验信息）的策略
- 简单示例：每个比特传三份，如果每比特的三份中有一位出错，可以纠正

0	1	0	0	1	0	1	0	1	0	1	1	0	1
0	1	0	0	1	0	1	0	1	0	1	1	0	1
0	1	0	0	1	0	1	0	1	0	1	1	0	1

携带2/3的冗余信息！
冗余信息量大！





差错检测和纠正概述



- **目标：** 保证一定差错检测和纠错能力的前提下， 如何减少冗余信息量？
- **考虑的问题**
 - 传输需求
 - 冗余信息的计算方法、携带的冗余信息量
 - 计算的复杂度等
- **两种主要策略**
 - 检错码 (error-detecting code)
 - 纠错码 (error-correcting code)



差错检测和纠正概述



- 检错码 (error-detecting code)
 - 在被发送的数据块中，包含一些冗余信息，但这些信息只能使接收方推断是否发生错误，但不能推断哪位发生错误，接收方可以请求发送方重传数据
 - 主要用在高可靠、误码率较低的信道上，例如光纤链路
 - 偶尔发生的差错，可以通过重传解决差错问题



差错检测和纠正概述



➤ 纠错码 (error-correcting code)

- 发送方在每个数据块中加入足够的冗余信息，使得接收方能够判断接收到的数据是否有错，并能纠正错误（**定位出错的位置**）
- 主要用于错误发生比较频繁的信道上，如无线链路
- 也经常用于物理层，以及更高层（例如，实时流媒体应用和内容分发）
- 使用纠错码的技术通常称为**前向纠错**（FEC, Forward Error Correction）



差错检测和纠正概述

- 码字 (code word): 一个包含 m 个数据位和 r 个校验位的 n 位单元
 - 描述为 (n, m) 码, $n=m+r$
- 码率 (code rate): 码字中不含冗余部分所占的比例, 可以用 m/n 表示
- 海明距离 (Hamming distance): 两个码字之间不同对应比特的数目
 - 例: 0000000000 与 0000011111 的海明距离为 5
 - 如果两个码字的海明距离为 d , 则需要 d 个单比特错就可以把一个码字转换成另一个码字
 - 为了检查出 d 个错 (比特错), 可以使用海明距离为 **$d+1$** 的编码
 - 为了纠正 d 个错, 可以使用海明距离为 **$2d+1$** 的编码



差错检测和纠正概述

➤ 例如：

- 一个只有4个有效码字的编码方案：0000000000, 0000011111, 1111100000, 1111111111
- 海明距离为5，可以检测4位错，纠正2位错
- 如果已知只有1位或2位错误，接收方接收0000000111
 - 则可知原码字为：0000011111
- 如果发生至多3位错误，例如0000000000变成0000000111，接收方无法纠正错误，但可以检测出错误



典型检错码



➤ 常用的检错码包括：

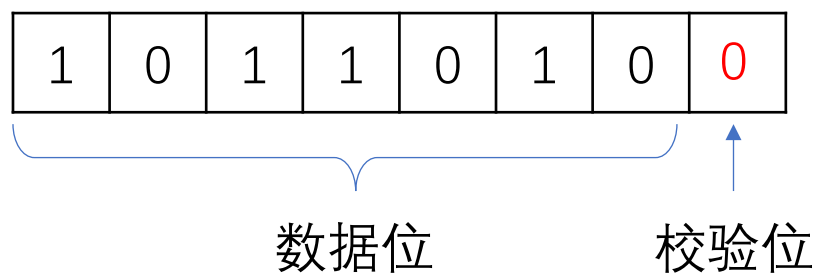
- **奇偶检验** (Parity Check): 1位奇偶校验是最简单、最基础的检错码
- **校验和** (Checksum): 主要用于TCP/IP体系中的网络层和传输层
- **循环冗余校验** (Cyclic Redundancy Check, CRC): 数据链路层广泛使用的校验方法



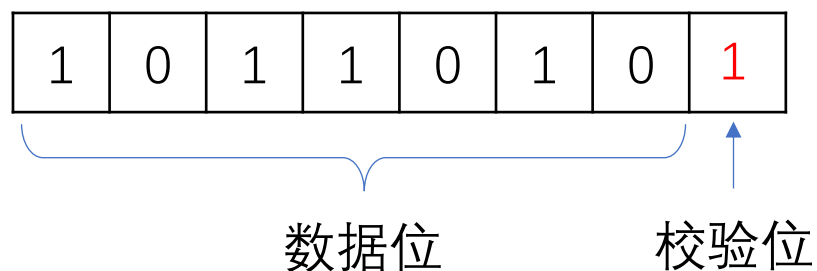
典型检错码—奇偶校验

➤ 1位奇偶校验：增加1位校验位，可以检查奇数位错误

- 偶校验：保证1的个数为偶数个，例如：



- 奇校验：保证1的个数为奇数个，例如：





典型检错码—校验和

➤ TCP/IP体系中主要采用的校验方法

发送方：进行 16 位二进制补码求和运算，计算结果取反，随数据一同发送

接收方：进行 16 位二进制补码求和运算（包含校验和），结果非全1，则检测到错误

数据
1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

数据
1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

① 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1

① 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1

校验和
1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0
0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1

0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1

1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

未检测到错误

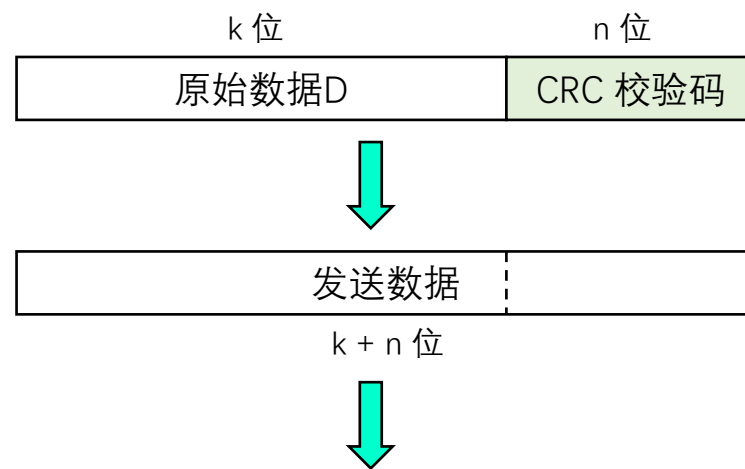


典型检错码—循环冗余校验CRC



➤ CRC校验码计算方法

- 设原始数据D为k位二进制位模式
- 如果要产生n位CRC校验码，事先选定一个n+1位二进制位模式G (称为生成多项式，收发双方提前商定)，G的最高位为1
- 将原始数据D乘以 2^n （相当于在D后面添加n个0），产生k+n位二进制位模式，用G对该位模式做模2除，得到余数R（n位，不足n位前面用0补齐）即为CRC校验码



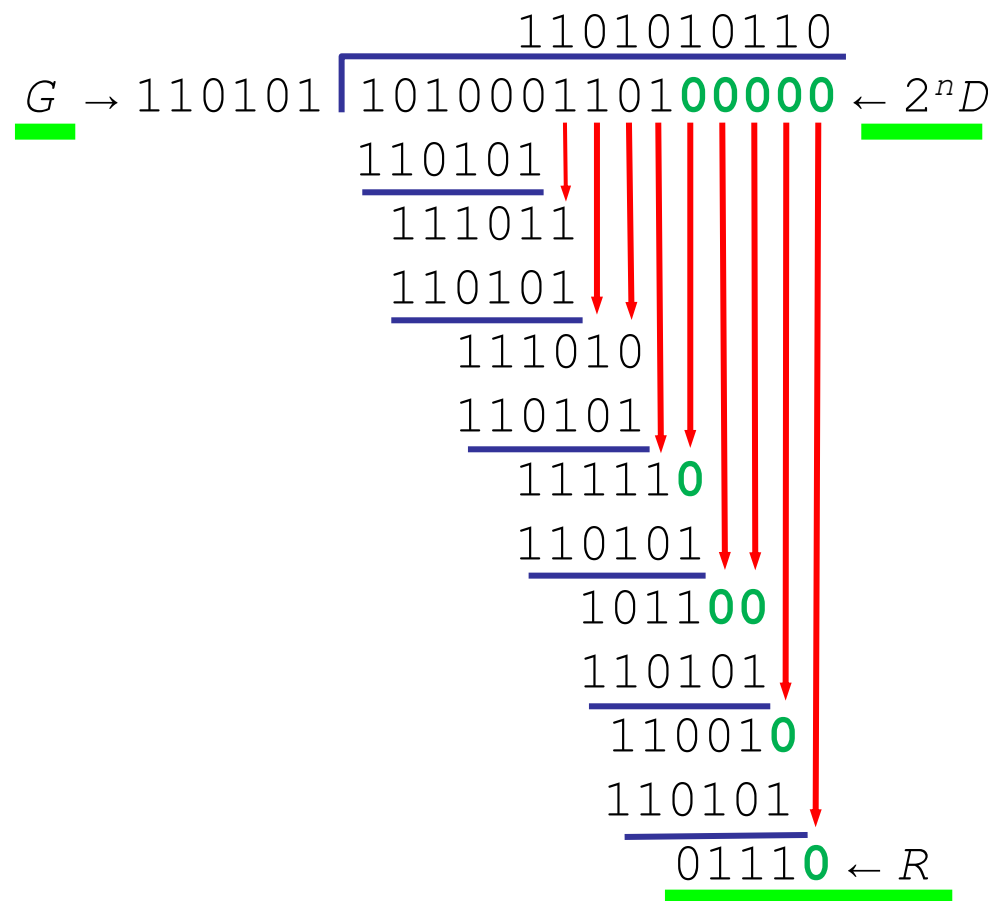
CRC校验能力：能检测出少于n+1位的突发错误



检错码—循环冗余校验CRC

➤ CRC校验码计算示例

- $D = 1010001101$
- $n = 5$
- $G = 110101$ 或 $G = x^5 + x^4 + x^2 + 1$
- $R = 01110$
- 实际传输数据: 101000110101110





检错码—循环冗余校验CRC

➤ 四个国际标准生成多项式

- $\text{CRC-12} = x^{12} + x^{11} + x^3 + x^2 + x + 1$
- $\text{CRC-16} = x^{16} + x^{15} + x^2 + 1$
- $\text{CRC-CCITT} = x^{16} + x^{12} + x^5 + 1$
- $\text{CRC-32} = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

例如，以太网、无线局域网使用CRC-32生成多项式



典型纠错码



➤ 设计纠错码

- 要求：m个信息位，r个校验位，纠正**单比特错**
- 对于任何一个有效信息，有n个与其距离为1的无效码字：
 - 可以考虑将该信息对应的合法码字的n位逐个取反，得到n个距离为1的非法码字和1个合法码字，需要r个校验位来标识
- 因此有： $n + 1 \leq 2^r$,即 $m + r + 1 \leq 2^r$
- 在给定m的情况下，利用该式可以得出纠正单比特错误校验位数的下界



典型纠错码—海明码

- **目标：**以奇偶校验为基础，如何找到出错位置，提供1位纠错能力
- 理解海明码编码过程，以 (15, 11)海明码为例
 - 例如：11比特的数据01011001101
 - 11比特数据按顺序放入数据位
 - **校验位：**2的幂次方位（记为p1, p2, p4, p8）
 - 每个校验位对数据位的子集做校验，缩小定位错误的范围
 - **问题：**每个校验位如何计算？

			0
	1	0	1
1	1	0	0
1	1	0	1

 校验位
 数据位

右下角数字为码字中位序号

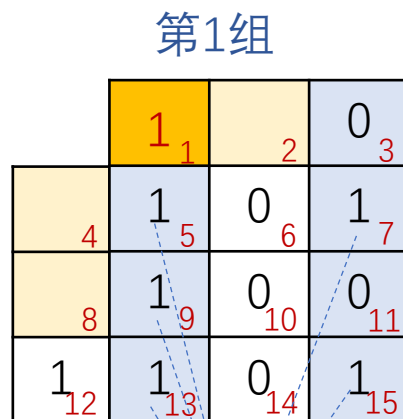


典型纠错码—海明码

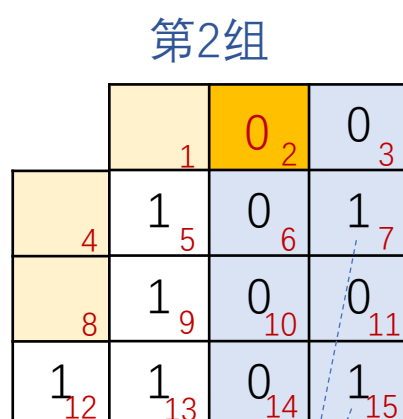
➤ 子集的选择与校验位计算

- 海明码缺省为偶校验（也可以使用奇校验）

 每组的数据位
 每组的校验位



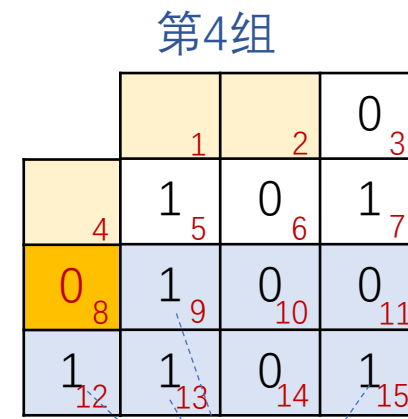
5个1
p1=1



2个1
p2=0



5个1
p4=1



4个1
p8=0



典型纠错码—海明码

➤ 码字的发送与接收

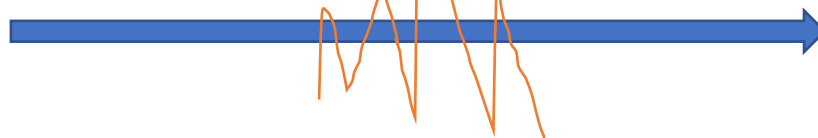
- 如果发送过程中第7位出现差错，如何定位错误？

	1 ₁	0 ₂	0 ₃
1 ₄	1 ₅	0 ₆	1 ₇
0 ₈	1 ₉	0 ₁₀	0 ₁₁
1 ₁₂	1 ₁₃	0 ₁₄	1 ₁₅

校验位 数据位

发送方

101100111010001



	1 ₁	0 ₂	0 ₃
1 ₄	1 ₅	0 ₆	0 ₇
0 ₈	1 ₉	0 ₁₀	0 ₁₁
1 ₁₂	1 ₁₃	0 ₁₄	1 ₁₅

校验位 数据位

接收方

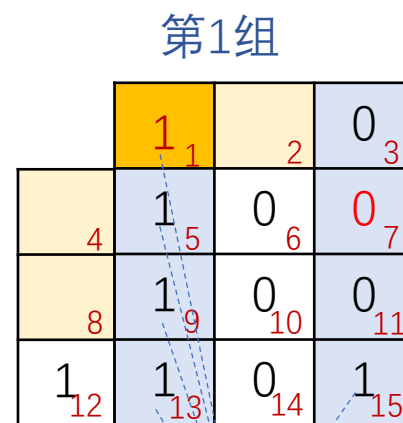


典型纠错码—海明码

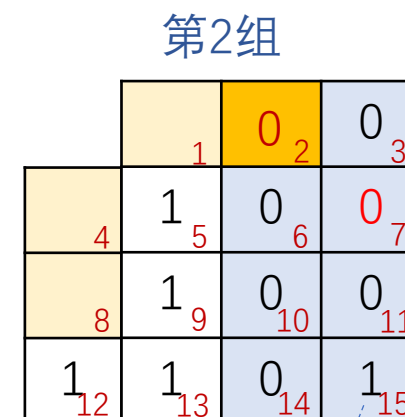
➤ 定位错误与纠正

- 组1和组2的校验结果可以定位错误所在的列
- 例如：组1和组2校验结果都指明存在错误，
可定位错误位于第4列
- 其他导致组1和组2出错的情况判断

出错列	2	3	4
组1	×	√	×
组2	√	×	×



5个1
有错误



1个1
有错误



典型纠错码—海明码

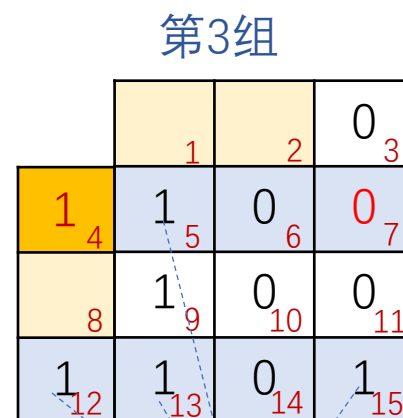
➤ 定位错误与纠正（续）

- 组3和组4的校验结果可以定位错误所在的行
- 例如：组3校验结果指明存在错误，组4校验结果指明无错误，**可定位错误位于第2行**

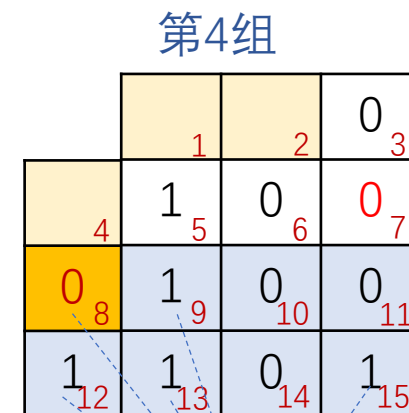
➡ 结论：错误位于第2行、第4列（位置7）

- 其他导致组3和组4出错情况判断

出错行	2	3	4
组3	×	✓	×
组4	✓	×	×



5个1
有错误



4个1
无错误



典型纠错码—海明码



➤ 定位错误与纠正（续）

- 总体的定位错误列表

出错位	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
组1	×	√	×	√	×	√	×	√	×	√	×	√	×	√	×
组2	√	×	×	√	√	×	×	√	√	×	×	√	√	×	×
组3	√	√	√	×	×	×	×	√	√	√	√	×	×	×	×
组4	√	√	√	√	√	√	√	×	×	×	×	×	×	×	×



典型纠错码—海明码

➤ 海明码纠正的实现过程

- 每个码字到来前，接收方计数器清零
- 接收方检查每个校验位 k ($k = 1, 2, 4 \dots$)的奇偶值是否正确（每组运算）
- 若 p_k 奇偶值不对，计数器加 k
- 所有校验位检查完后，若计数器值为0，则码字有效；若计数器值为 j ，则第 j 位出错。例：若校验位 p_1 、 p_2 、 p_8 出错，则第11位变反



本章内容



3.1 数据链路层的设计问题

3.2 差错检测和纠正

3.3 基本的数据链路层协议

3.4 滑动窗口协议

3.5 数据链路协议实例

1. 定义与假设

2. 乌托邦式单工协议

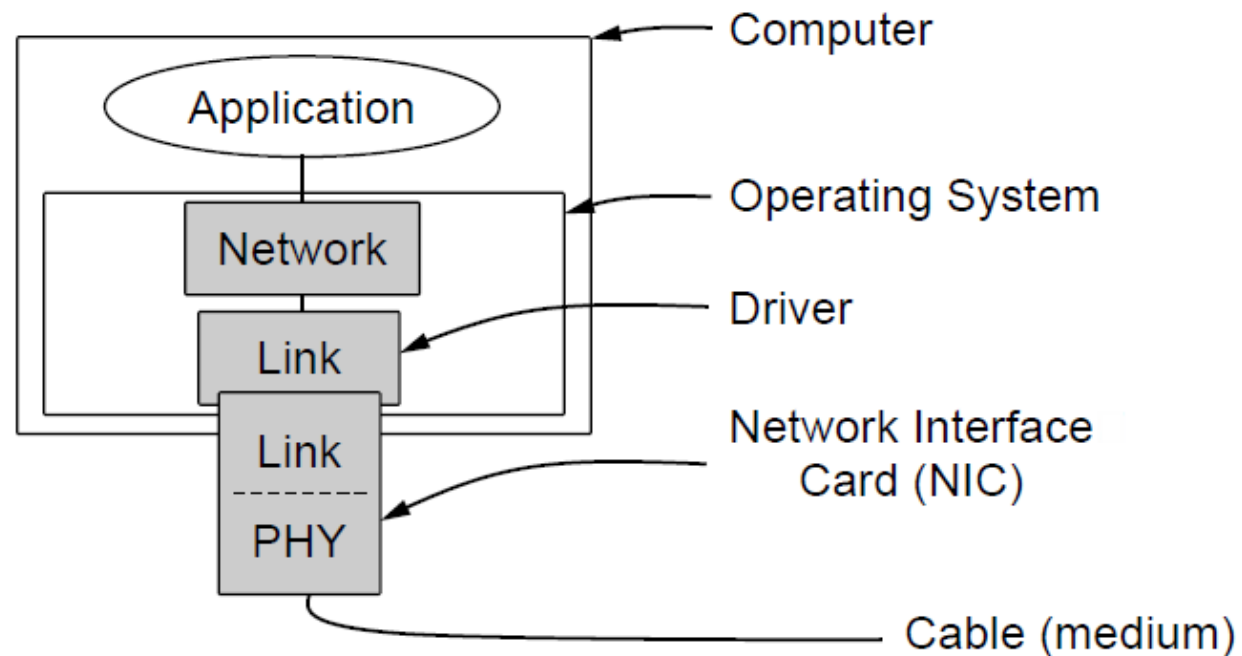
3. 无错信道单工停止-等待协议

4. 有错信道单工停止-等待协议



物理层、数据链路层和网络层的实现

- 物理层进程和某些数据链路层进程运行在专用硬件上（网络接口卡）
- 数据链路层进程的其他部分和网络层进程作为操作系统的一部分运行在CPU上，数据链路层进程的软件通常以设备驱动的形式存在





乌托邦式单工协议（1）

➤ 假设

- 单工（Simplex）协议：数据单向传输
- 完美信道：帧不会丢失或受损
- 始终就绪：发送方/接收方的网络层始终处于就绪状态
- 瞬间完成：发送方/接收方能够生成/处理无穷多的数据

➤ 乌托邦：完美但不现实的协议

- 不处理任何流量控制或纠错工作
- 接近于无确认的无连接服务，必须依赖更高层次解决上述问题



乌托邦式单工协议（2）



➤ 发送方

- 在循环中不停发送
- 从网络层获得数据
- 封装成帧
- 交给物理层
- 完成一次发送

```
frame s;  
packet buffer;  
  
while (true) {  
    from_network_layer(&buffer);  
    s.info = buffer;  
    to_physical_layer(&s);  
}
```

➤ 接收方

- 在循环中持续接收
- 等待帧到达 (frame_arrival)
- 从物理层获得帧
- 解封装，将帧中的数据传递给网络层
- 完成一次接收

```
frame r;  
event_type event;  
  
while (true) {  
    wait_for_event(&event);  
    from_physical_layer(&r);  
    to_network_layer(&r.info);  
}
```




无错信道上的停等式协议 (1)

➤ 不再假设

- 接收方能够处理以无限高速进来的数据
- 发送方以高于接收方能处理到达帧的速度发送帧，不会导致接收方被“淹没”(overwhelming)

➤ 仍然假设

- 通信信道不会出错 (Error-Free)
- 数据传输保持单向, 需要双向传输链路 (半双工物理信道)



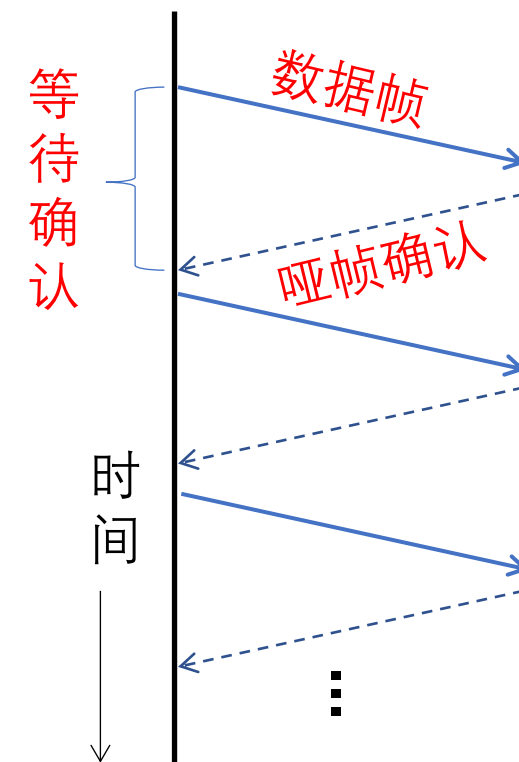
无错信道上的停等式协议（2）

➤ 停-等式协议（stop-and-wait）

- 发送方发送一帧后暂停，等待**确认**（Acknowledgement）到达后发送下一帧
- 接收方完成接收后，回复**确认**接收.
- 确认帧的内容是不重要的：哑帧（dummy frame）

发送方

接收方





无错信道上的停等式协议（3）



➤ 发送方

- 完成一帧发送后
- 等待确认到达
- 确认到达后，发送下一帧

```
while (true) {  
    from_network_layer(&buffer);  
    s.info = buffer;  
    to_physical_layer(&s);  
    wait_for_event(&event);  
}
```

➤ 接收方

- 完成一帧接收后
- 交给物理层一个哑帧
- 作为成功接收上一帧的确认

```
while (true) {  
    wait_for_event(&event);  
    from_physical_layer(&r);  
    to_network_layer(&r.info);  
    to_physical_layer(&s);  
}
```



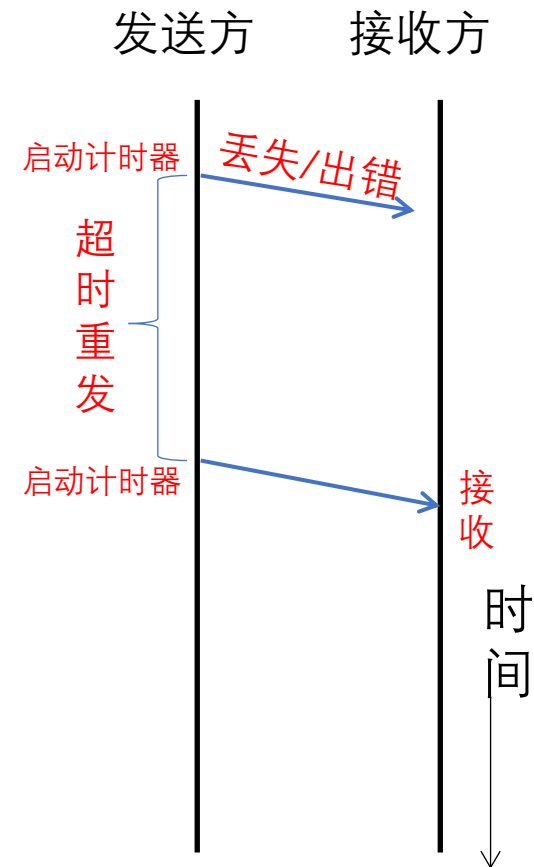
有错信道上的单工停等式协议 (1)

➤ 假设

- 通信信道可能会出错，导致：
 - 帧在传输过程中可能会被损坏，接收方能够检测出来
 - 帧在传输过程中可能会丢失，永远不可能到达接收方

➤ 一个简单的解决方案

- 发送方增加一个**计时器(timer)**，如果经过一段时间没有收到确认，发送方将超时，于是再次发送该帧





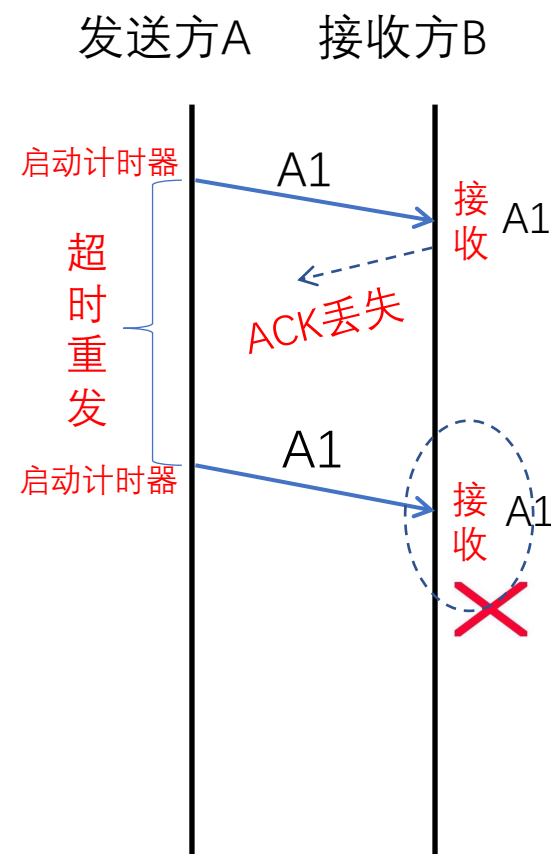
有错信道上的单工停等式协议 (2)

➤ 考虑一个特别场景

- A发送帧A1
- B收到了A1
- B生成确认ACK
- ACK在传输中丢失
- A超时，重发A1
- B收到A1的另一个副本（并把它交给网络层）

➤ 其他的场景

- 另一个导致副本产生的场景是过长的延时 (long delay)





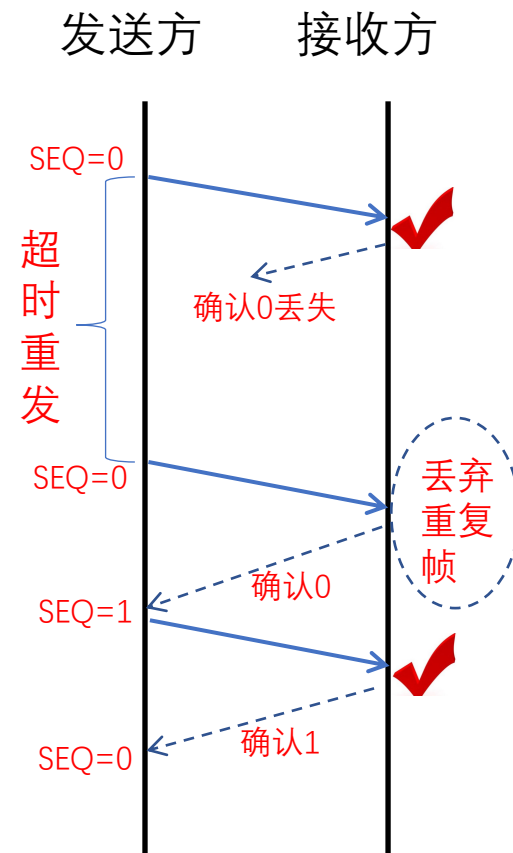
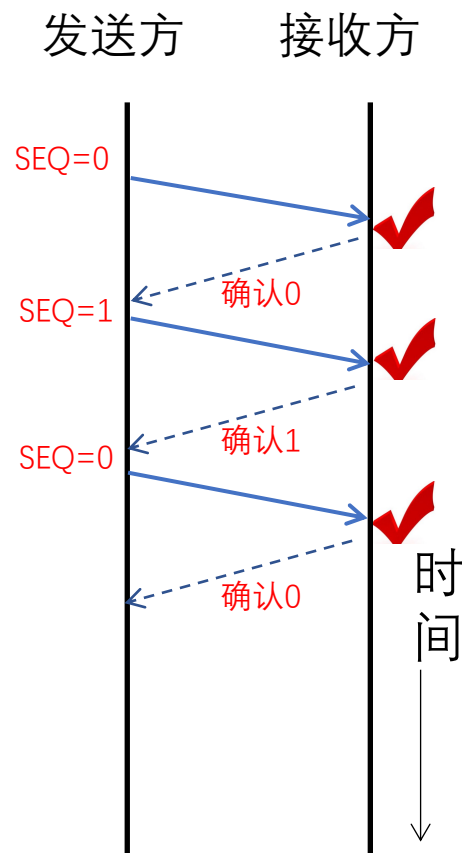
有错信道上的单工停等式协议 (3)

➤ 序号 (SEQ: sequence number)

- 接收方需要确认到达的帧是否第一次发来的新帧
- 让发送方在发送的帧的头部放一个序号，接收方可以检查它所收到的帧序号，由此判断这是个新帧还是应该被丢弃的重复帧。

➤ 序号所需要的最小位数 (bits)

- 序号需要区分当前帧 (序号m) 和它的直接后续帧 (序号m+1)
- 1 bit序号(0或1)就足以满足要求。





有错信道上的单工停等式协议 (4)



➤ 发送方

- 初始化帧序号0, 发送帧
- 等待: 正确的确认/错误的确认/超时
- 正确确认: 发送下一帧
- 超时/错误确认: 重发

```
next_frame_to_send = 0;
from_network_layer(&buffer);
while (true) {
    s.info = buffer;
    to_physical_layer(&s);
    start_timer(s.seq);
    wait_for_event(&event);
    if (event == frame_arrival){
        from_physical_layer(&s);
        if (s.ack == next_frame_to_send){
            from_network_layer(&buffer);
            inc(next_frame_to_send);
        }
    }
}
```

➤ 接收方

- 初始化期待0号帧
- 等待帧达到
- 正确帧: 交给网络层, 并发送该帧确认
- 错误帧: 发送上一个成功接收帧的确认

```
frame_expected = 0;
while (true) {
    wait_for_event(&event);
    if (event == frame_arrival){
        from_physical_layer(&r);
        if (r.seq == frame_expected){
            to_network_layer(&r.info);
            inc(frame_expected);
        }
        s.ack = 1 - frame_expected;
        to_physical_layer(&s);
    }
}
```



本章内容



3.1 数据链路层的设计问题

3.2 差错检测和纠正

3.3 基本的数据链路层协议

3.4 滑动窗口协议

3.5 数据链路协议实例

1. 停等协议的性能问题
2. 滑动窗口协议
3. 回退N协议
4. 选择重传协议



滑动窗口协议—协议基本思想

➤ 协议基本思想

- 窗口机制

- 发送方和接收方都具有一定容量的缓冲区（即窗口），发送端在收到确认之前可以发送多个帧

- 问题

- 发送端一次可以发送多少个帧？
- 某个帧出错或丢失怎么处理？



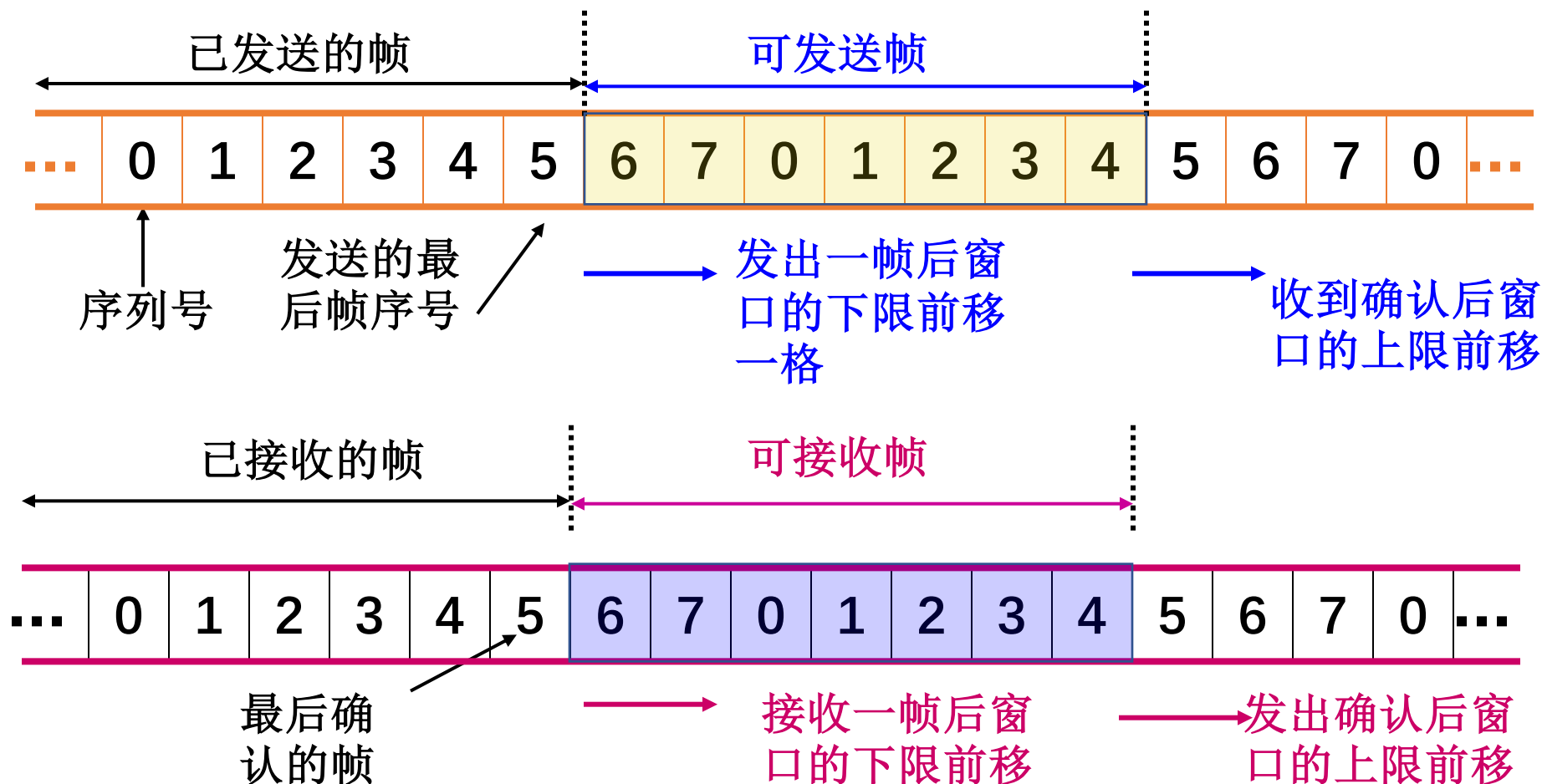
滑动窗口协议—协议基本思想

- 目的
 - 对可以连续发出的最多帧数（已发出但未确认的帧）作限制
- 序号使用
 - 循环重复使用有限的帧序号
- 流量控制：接收窗口驱动发送窗口的转动
 - 发送窗口：其大小记作 W_T ，表示在收到对方确认的信息之前，可以连续发出的最多数数据帧数
 - 接收窗口：其大小记作 W_R ，为可以连续接收的最多数数据帧数
- 累计确认：不必对收到的分组逐个发送确认，而是对按序到达的最后一个分组发送确认



滑动窗口协议—协议基本思想

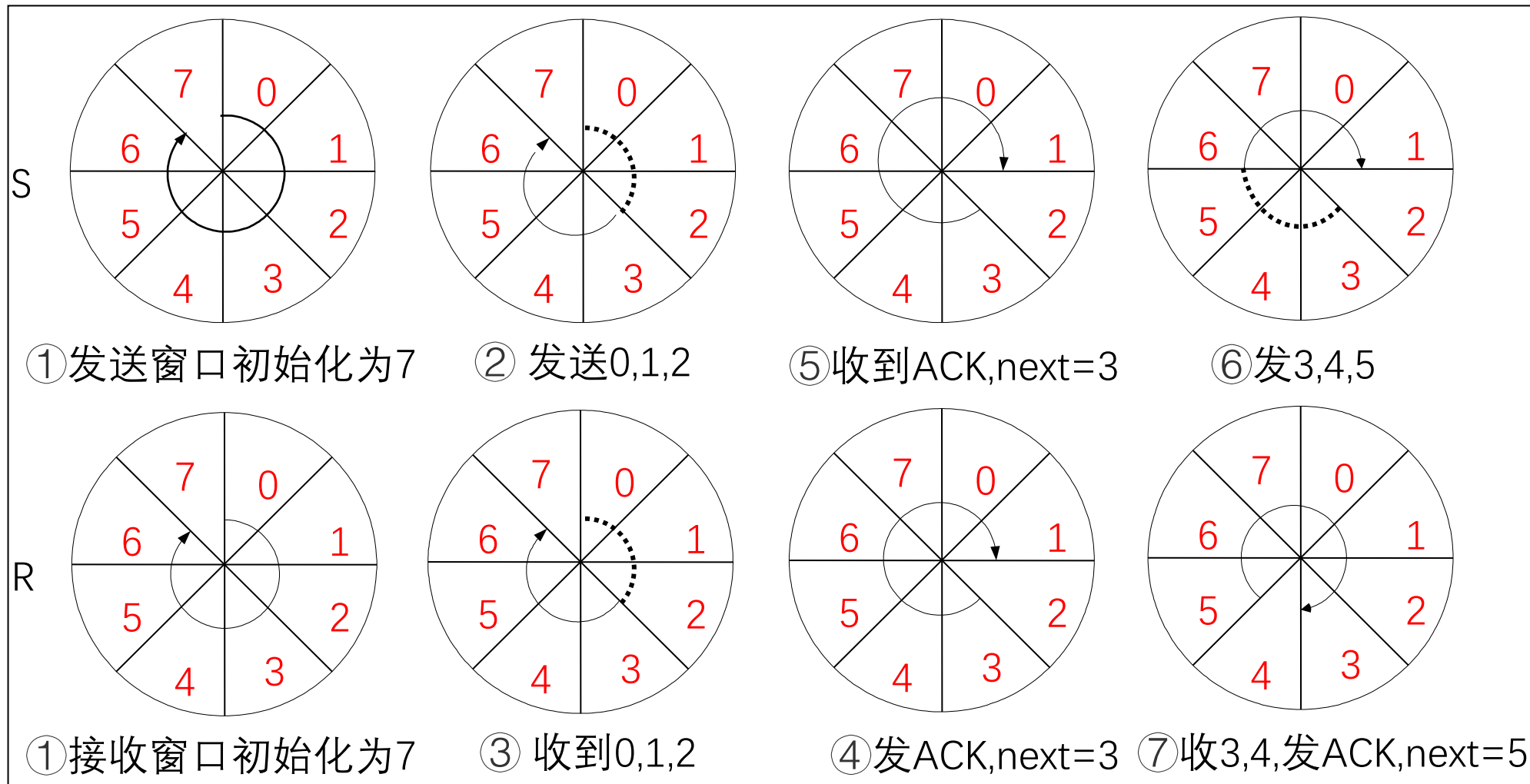
➤ 发送窗口与接收窗口(窗口大小7)





滑动窗口协议—协议基本思想

➤ 发送窗口与接收窗口(窗口大小7)





回退N协议—协议设计思想

➤ 出错全部重发

- 当接收端收到一个出错帧或乱序帧时，丢弃所有的后继帧，并且不为这些帧发送确认
- 发送端超时后，重传所有未被确认的帧

➤ 适用场景

- 该策略对应接收窗口为1的情况，即只能按顺序接收帧

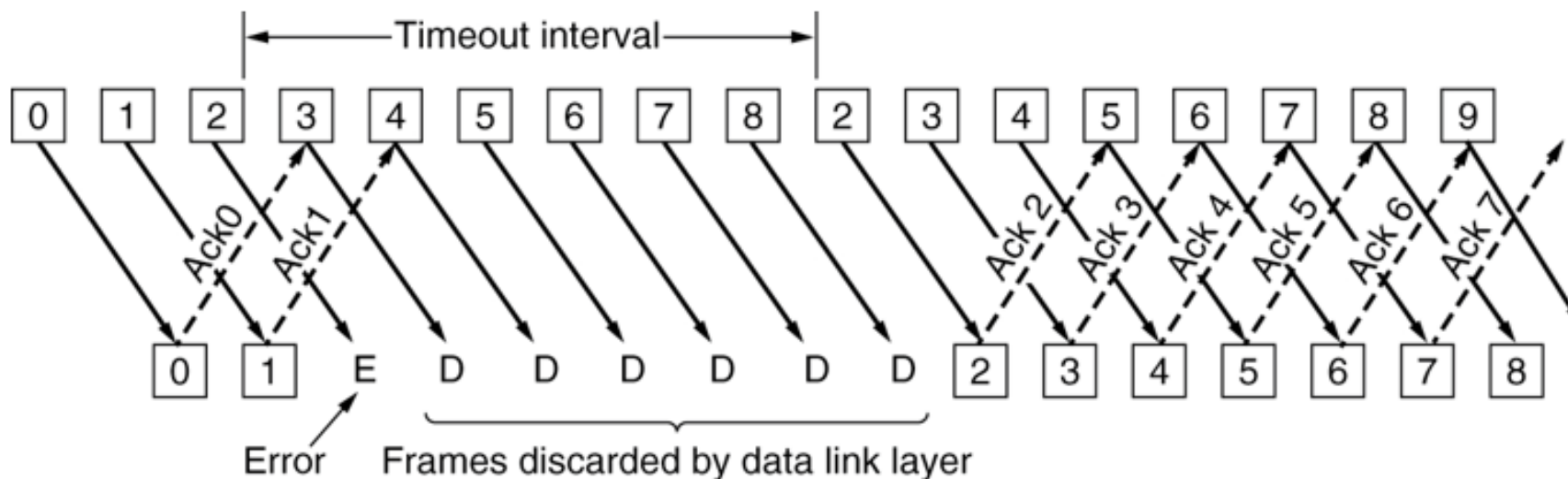
➤ 优缺点

- 优点：连续发送提高了信道利用率
- 缺点：按序接收，出错后即便有正确帧到达也丢弃重传

回退N协议—协议原理分析

➤ 基本原理

- 当发送方发送了N个帧后，若发现该N帧的前一个帧在计时器超时后仍未返回其确认信息，则该帧被判为出错或丢失，此时发送方就重新发送出错帧及其后的N帧。





选择重传协议—协议设计思想

➤ 设计思想

- 若发送方发出连续的若干帧后，收到对其中某一帧的否认帧，或某一帧的定时器超时，则只重传该出错帧或定时器超时的数据帧

➤ 适用场景

- 该策略对应接收窗口大于1的情况，即暂存接收窗口中序号在出错帧之后的数据帧

➤ 优缺点

- 优点：避免重传已正确传送的帧
- 缺点：在接收端需要占用一定容量的缓存



选择重传协议—协议原理分析

➤ 基本原理

- 在发送过程中，如果一个数据帧计时器超时，就认为该帧丢失或者被破坏；接收端只把出错的的帧丢弃，其后面的数据帧保存在缓存中，并向发送端回复NAK；发送端接收到NAK时，只重传出错的帧
- 如果落在窗口内的帧从未接受过，那么存储起来，等比它序列号小的所有帧都正确接收后，按次序交付给网络层
- 接收端收到的数据包的顺序可能和发送的数据包顺序不一样，因此在数据包里必须含有顺序号来帮助接收端进行排序。



第四章 介质访问子层

授课教师：张圣林

南开大学



本章目标



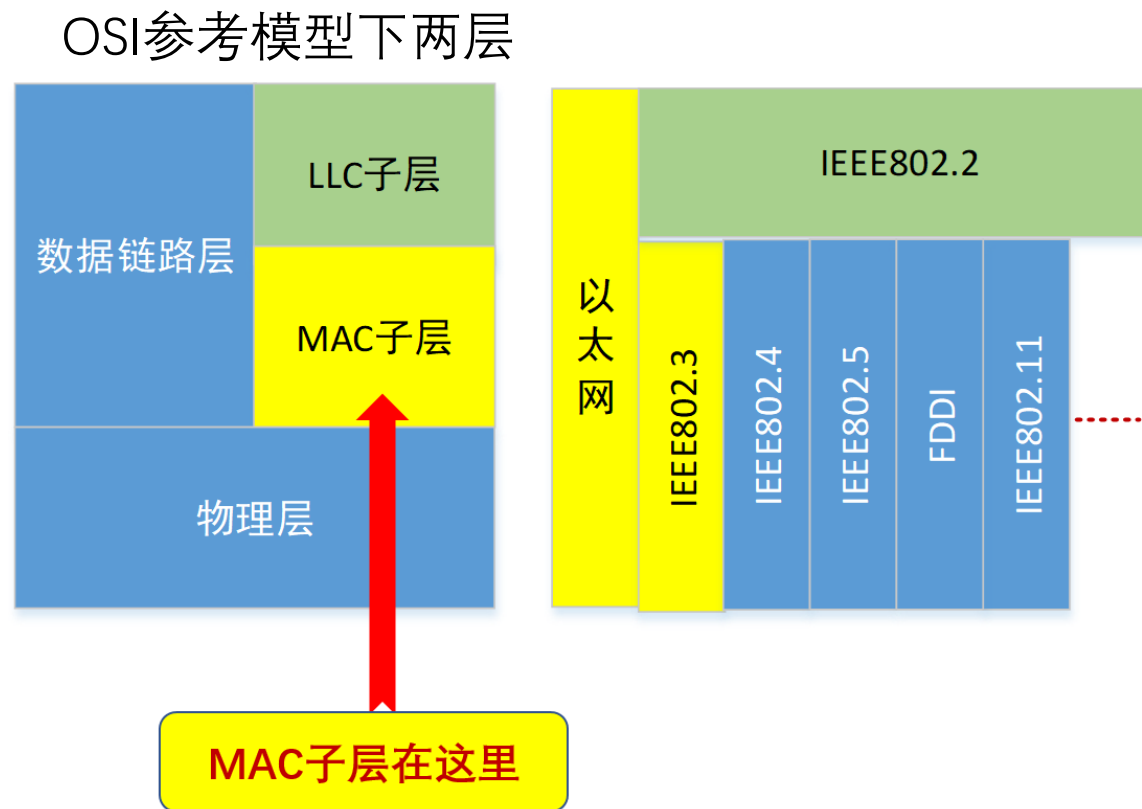
1. 了解MAC子层的位置和功能
2. 掌握两种ALOHA协议的工作原理
3. 掌握CSMA工作原理及核心所在
8. 掌握经典以太网的拓扑
9. 掌握经典以太网的帧结构
10. 掌握交换式以太网其特征
11. 掌握数据链路层交换的原理
12. 掌握MAC地址表的维护
13. 了解无线局域网



MAC子层在哪里？



- 数据链路层分为两个子层：
 - MAC (media access control)子层： 介质访问
 - LLC (logical link control)子层： 承上启下（弱层）
- 以太网和IEEE802.3
 - 覆盖的层数不同（1.5层 vs 2层）
 - 帧的结构有细微不同
 - 以太网：事实上的标准，而802.3系列让接入有了无限的延展性
- 局域网： 以太网、无线局域网……





本章内容



- 4.1 信道分配问题 (P4)
- 4.2 多路访问协议 (P11)
- 4.3 以太网 (P44)
- 4.4 数据链路层交换 (P73)
- 4.5 无线局域网 (P133)

1. 多路访问协议
2. 受控访问协议
3. 有限竞争协议



三大类多路访问协议及小分类

➤ 4.2.1 随机访问协议

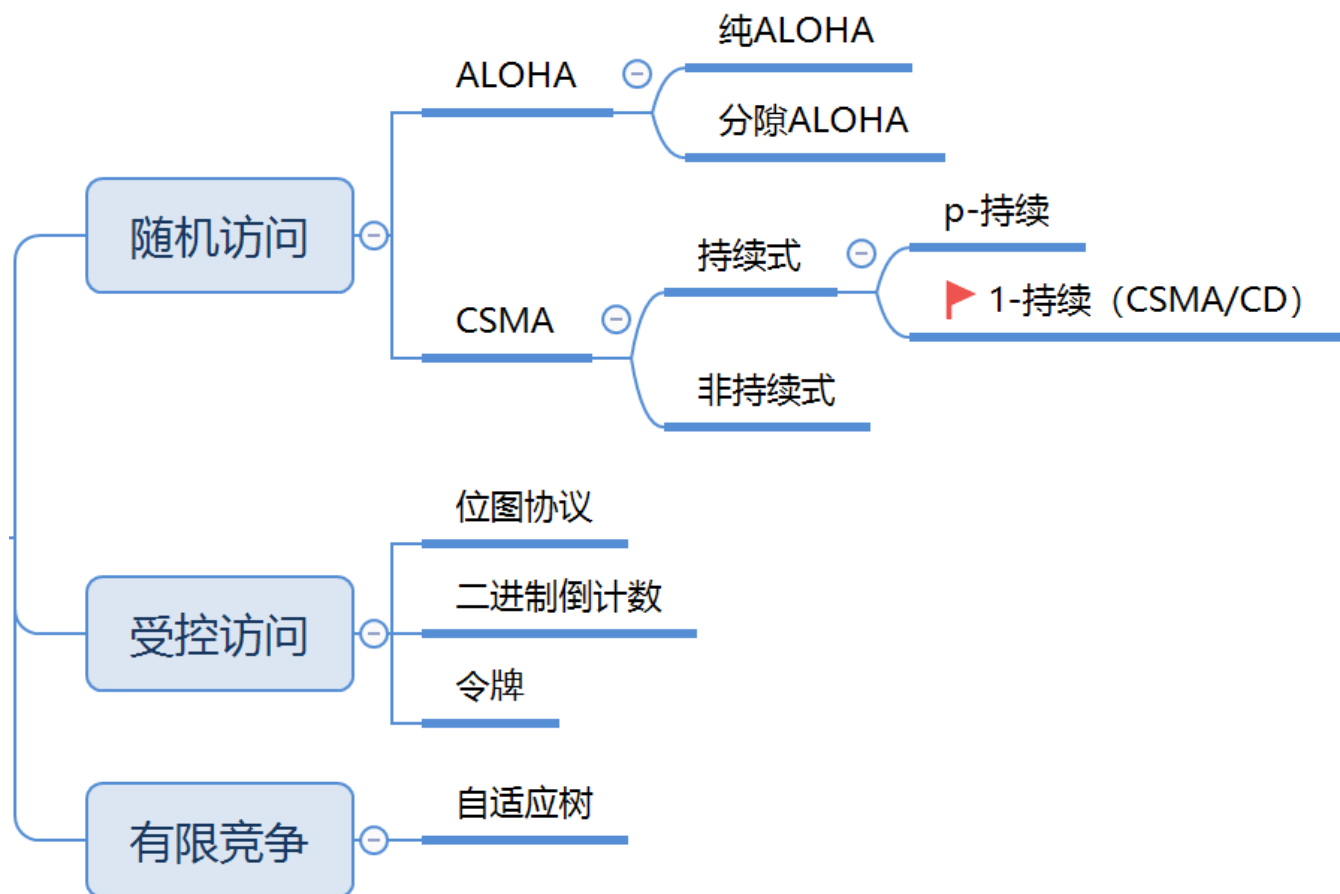
- 特点：冲突不可避免

➤ 4.2.2 受控访问协议

- 特点：克服了冲突

➤ 4.2.3 有限竞争协议

- 利用上述二者的优势

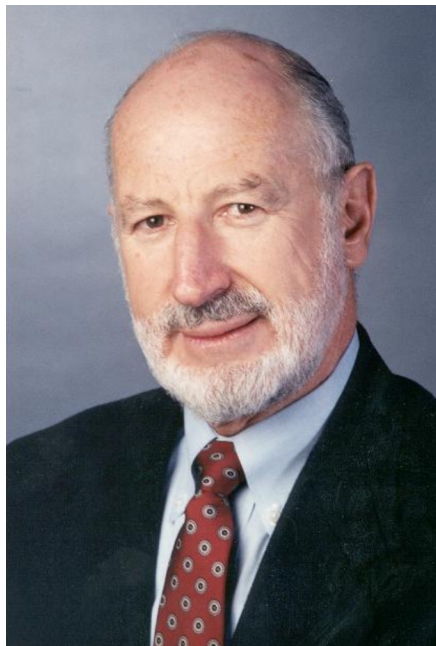




ALOHA协议的由来



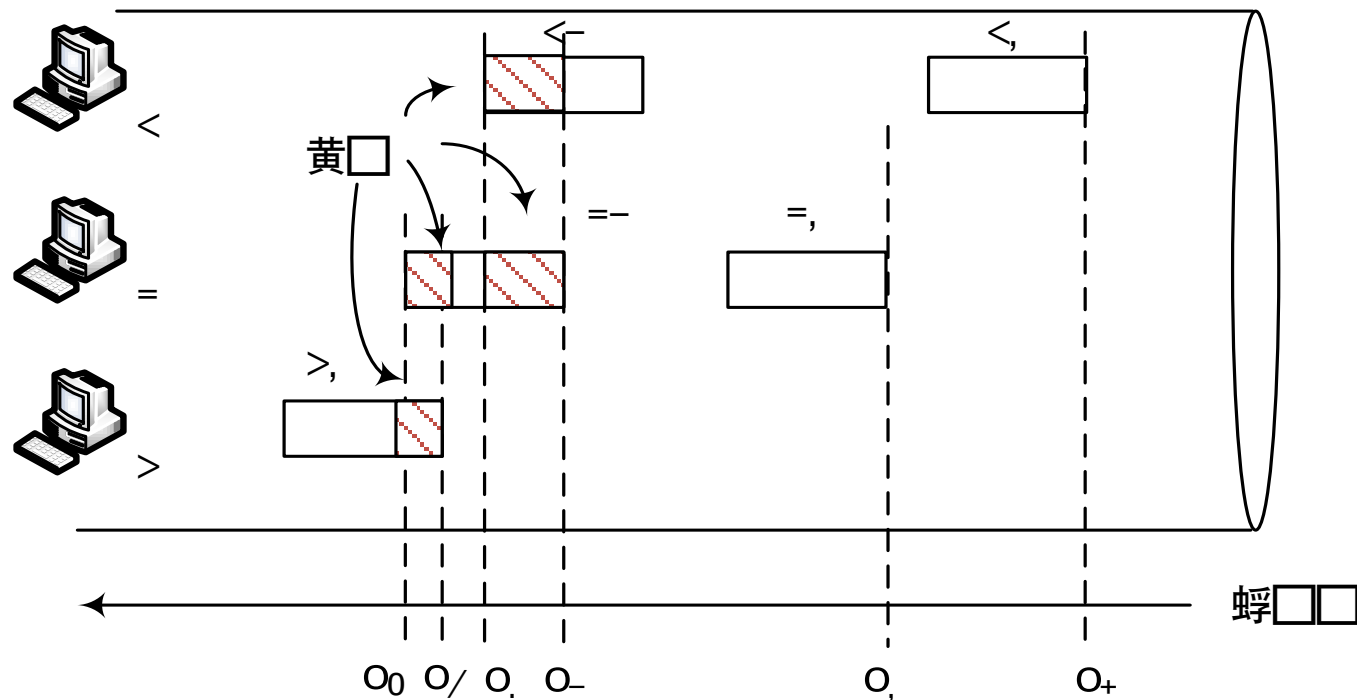
- 夏威夷大学Norman Abramson及他的同事设计
- ALOHANet: 连接檀香山和其它岛屿
- 两个版本
 - 纯ALOHA协议
 - 分隙ALOHA协议





纯ALOHA协议工作原理：任性！

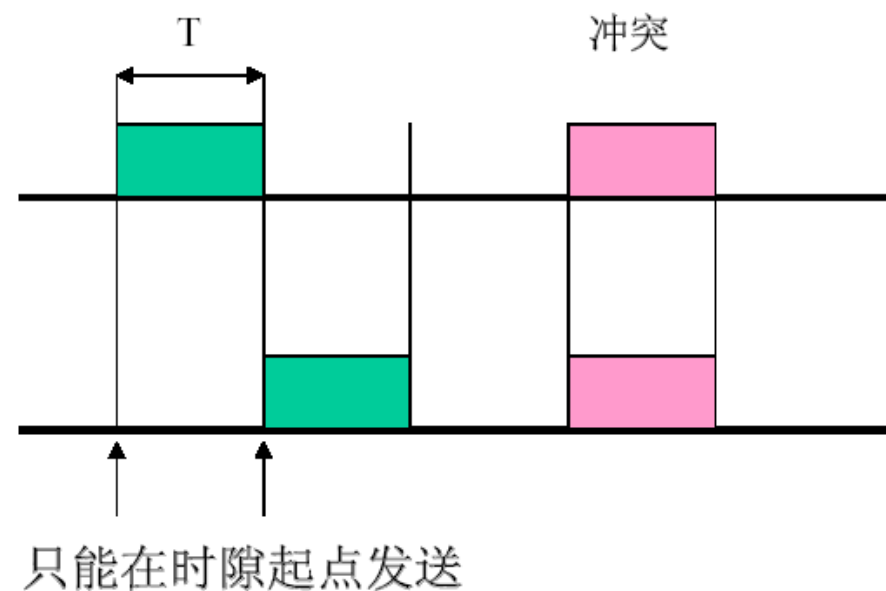
- 原理：想发就发！
- 特点
 - 冲突：两个或以上的帧
 - 随时可能冲突
 - 冲突的帧完全破坏
 - 破坏了的帧要重传





分隙ALOHA (Slotted ALOHA) 工作原理

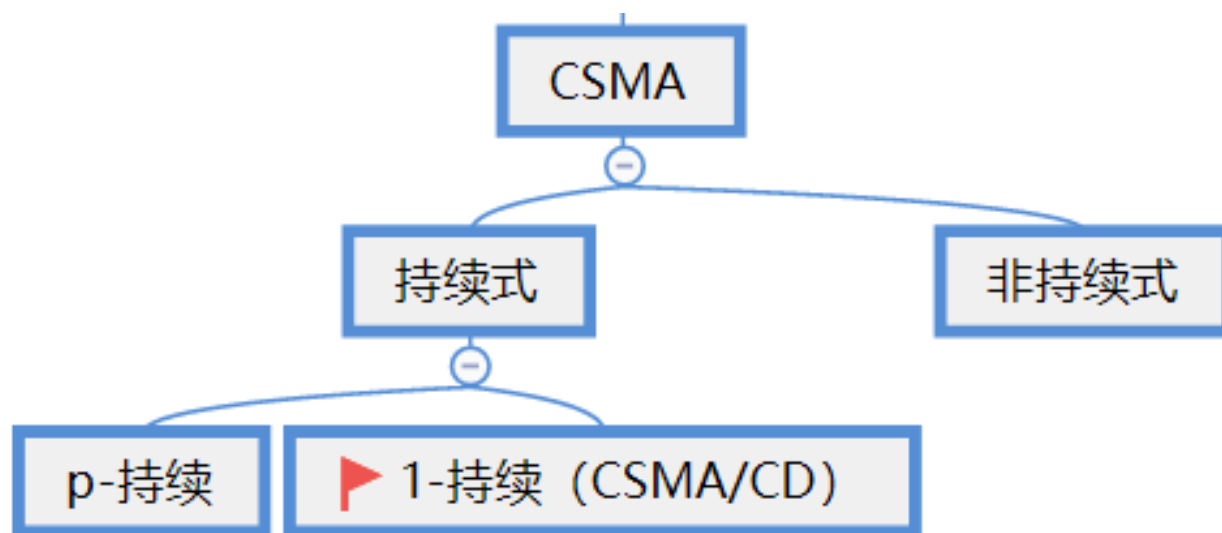
- 分隙ALOHA是把时间分成时隙 (时槽)
- 时隙的长度对应一帧的传输时间。
- 帧的发送必须在时隙的起点。
- 冲突只发生在时隙的起点



冲突危险期: D

载波侦听多路访问协议

- CSMA: Carrier Sense Multiple Access
- 特点: “先听后发”
 - 改进ALOHA的侦听/发送策略分类



不再任性!
变得礼貌了!



非持续式CSMA

➤ 特点

- ①经侦听，如果介质空闲，开始发送
- ②如果介质忙，则等待一个随机分布的时间，然后重复步骤①

➤ 好处

- 等待一个随机时间可以减少再次碰撞冲突的可能性

➤ 缺点

- 等待时间内介质上如果没有数据传送，这段时间是浪费的



1-持续式CSMA

➤ 1-Persistent CSMA

➤ 原理：“先听后发、边发边听”

➤ 过程

- ①经侦听，如介质空闲，则发送。
- ②如介质忙，持续侦听，一旦空闲立即发送。



p-持续式CSMA

➤ 特点

- ①经侦听，如介质空闲，那么以 p 的概率发送，以 $(1-p)$ 的概率延迟一个时间单元发送
- ②如介质忙，持续侦听，一旦空闲重复①
- ③如果发送已推迟一个时间单元，再重复步骤①

➤ 注意

- 1-持续式是 p -持续式的特例



问题：先听再发，避免了冲突吗？

- CSMA：如侦听到介质上无数据发送才发送，发送后还会发生冲突吗？
- 肯定会！
- 两种情形
 - (1) 同时传送； (2) 传播延迟时间



不再任性的CSMA还会发生冲突吗？



CSMA/CD



➤ CSMA/CD: Carrier Sense Multiple Access with Collision Detection

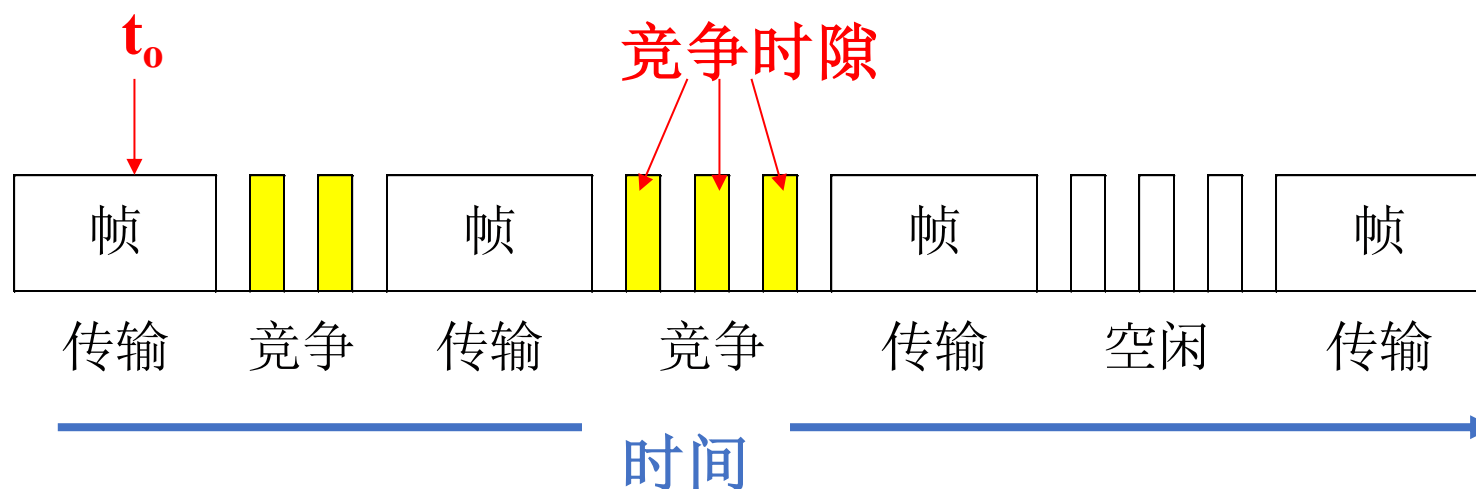
➤ 特点

- ①经侦听，如介质空闲，则发送
- ②如介质忙，持续侦听，一旦空闲立即发送
- ③如果发生冲突，等待一个随机分布的时间再重复步骤①



CSMA/CD概念模型

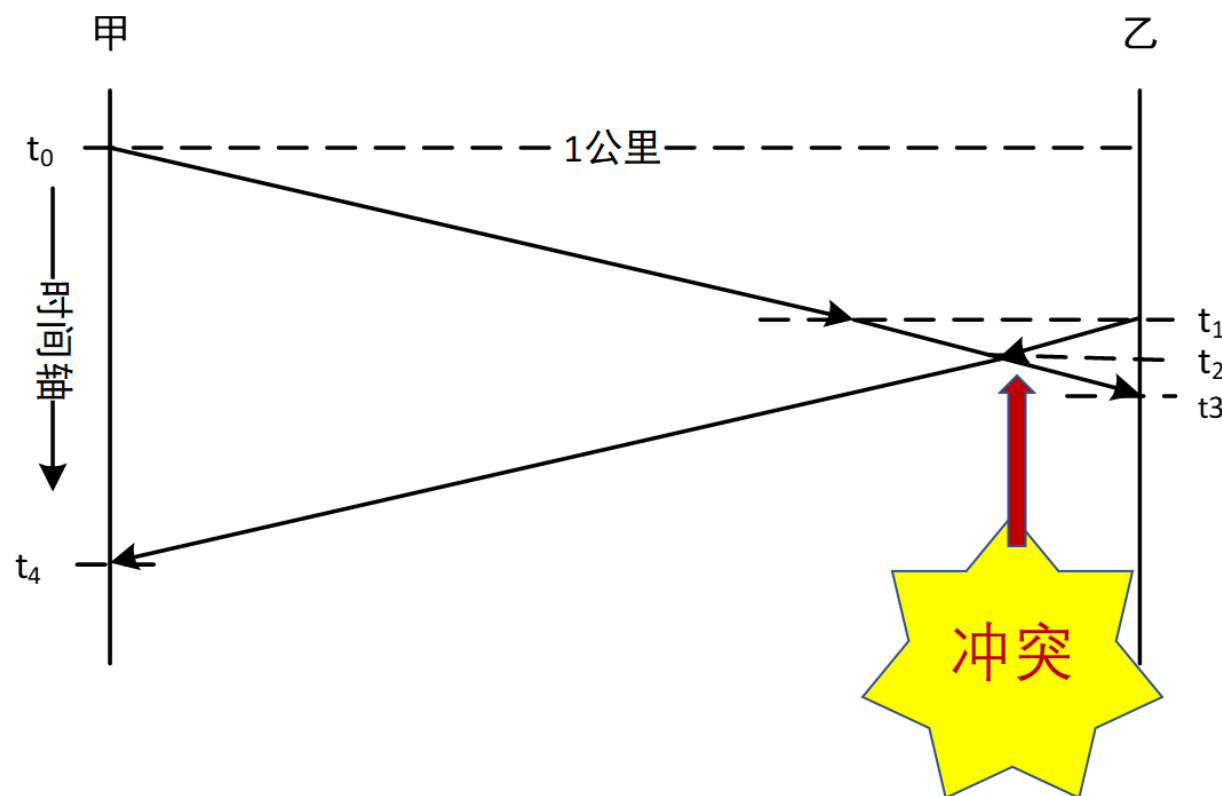
- **传输**周期：一个站点使用信道，其他站点禁止使用
- **竞争**周期：所有站点都有权尝试使用信道，争用时间槽
- **空闲**周期：所有站点都不使用信道





传播延迟对载波侦听的影响

- 信号传输速度: $0.65C$
- t_0 时刻: 甲侦听后发送, 到达乙约需5微妙
- t_1 时刻: 乙侦听后发送
- t_2 时刻: 冲突
- t_3 时刻: 乙检测到冲突
- t_4 时刻: 甲检测到冲突





冲突窗口

- 即发送站发出帧后能检测到冲突（碰撞）的最长时间。
- 是一个时间区间
 - 可能侦听到发出的帧遭到冲突（碰撞）
- 数值上：等于最远两站传播时间的两倍，即 $2D$ （ D 是单边延迟）
 - $2D$ 相当于1个来回传播延迟RTT：Round Trip Time



各种CSMA的性能比较

- 以太网采用了CSMA/CD
 - 冲突：比ALOHA少，比P-持续式高
 - P-持续式付出了高延迟的代价

**先听后发，边发边听；
一旦冲突，立刻停发；
等待时机，然后再听。**

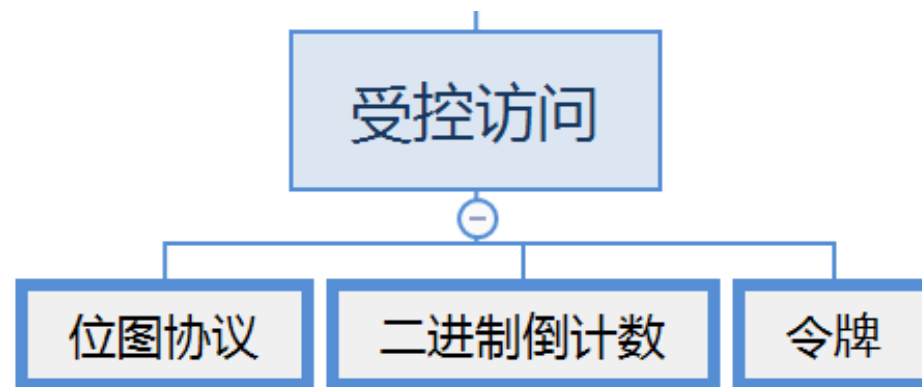


受控访问协议



➤ 4.2.2 受控访问（无冲突）协议

- 4.2.2.1 位图协议（预留协议）
- 4.2.2.2 令牌
- 4.2.2.3 二进制倒数计数协议





位图协议（预留协议）图示

➤ 竞争期：在自己的时槽内发送竞争比特

- 举手示意
- 资源预留

➤ 传输期：按序发送

- 明确的使用权，避免了冲突

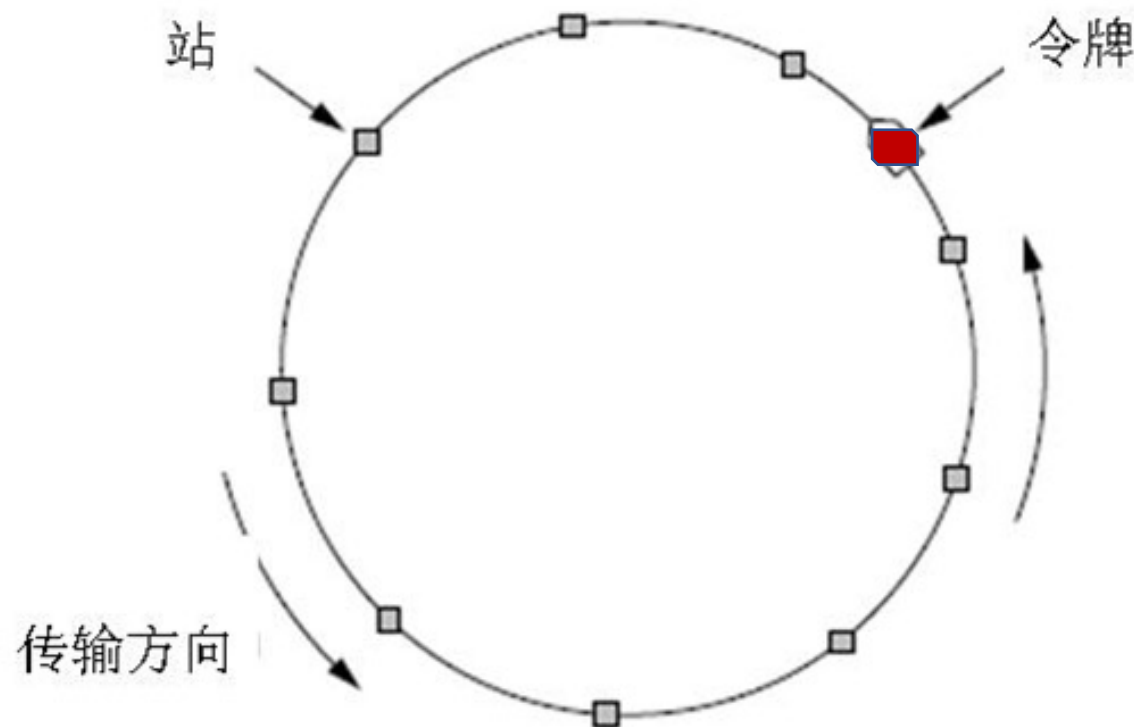




令牌传递



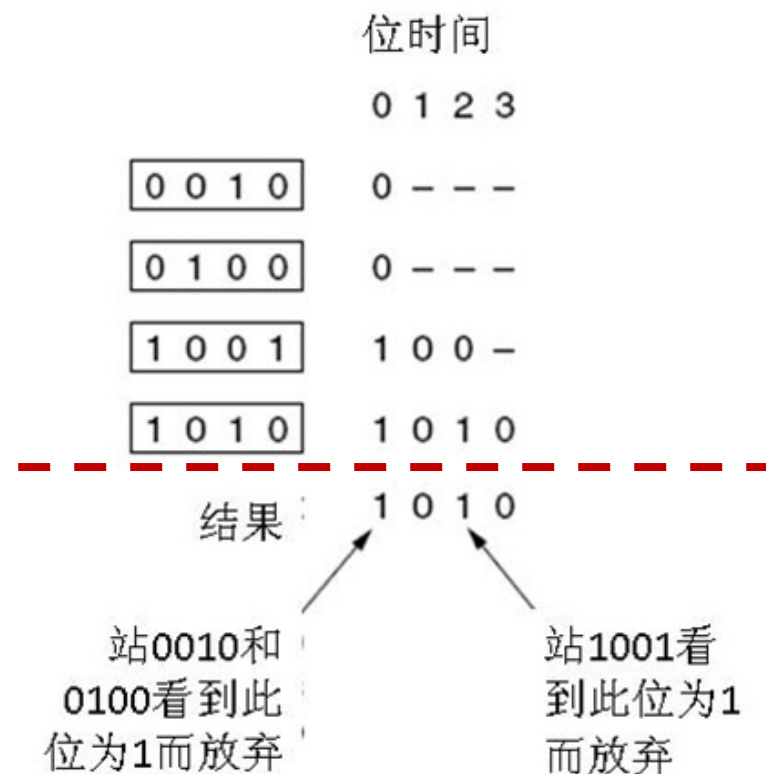
- **令牌**：发送权限
- 令牌的**运行**：发送工作站去抓取，获得发送权
 - 除了环，令牌也可以运行在其它拓扑上，如**令牌总线**
- 发送的帧需要**发送站**将其从共享信道上去除；防止无限循环
- **缺点**：令牌的维护代价





二进制倒数计数协议

- 站点： **编序号**，序号长度相同
- 竞争期：有数据发送的站点从高序号到低序号排队，高者得到发送权
- 特点： **高序号站点优先**
 - 站点轮流使用序号

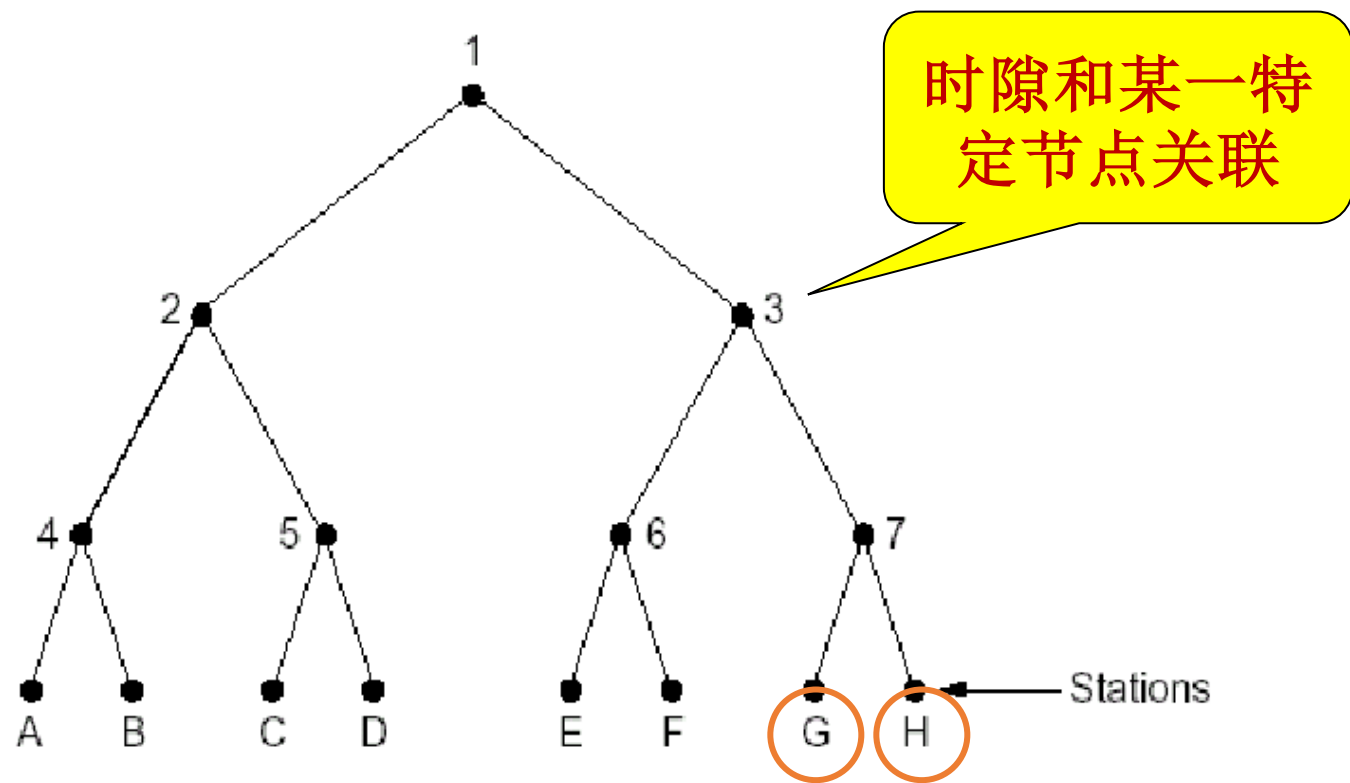




自适应树搜索协议 (Adaptive Tree Walk Protocol)



- 在一次成功传输后的第一个竞争时隙，**所有站点**同时竞争。
- 如果**只有一个站点**申请，则获得信道。
- **否则**在下一竞争时隙，有一半站点参与竞争（递归），下一时隙由另一半站点参与竞争
- 即所有站点构成一棵**完全二叉树**。





目 录

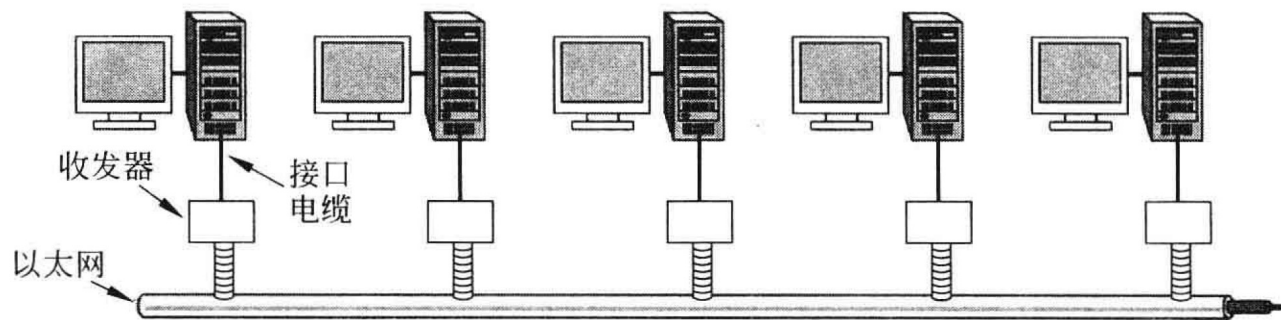


- 4.1 信道分配问题 (P4)
- 4.2 多路访问协议 (P11)
- 4.3 以太网 (P44)
- 4.4 数据链路层交换 (P73)
- 4.5 无线局域网 (P133)

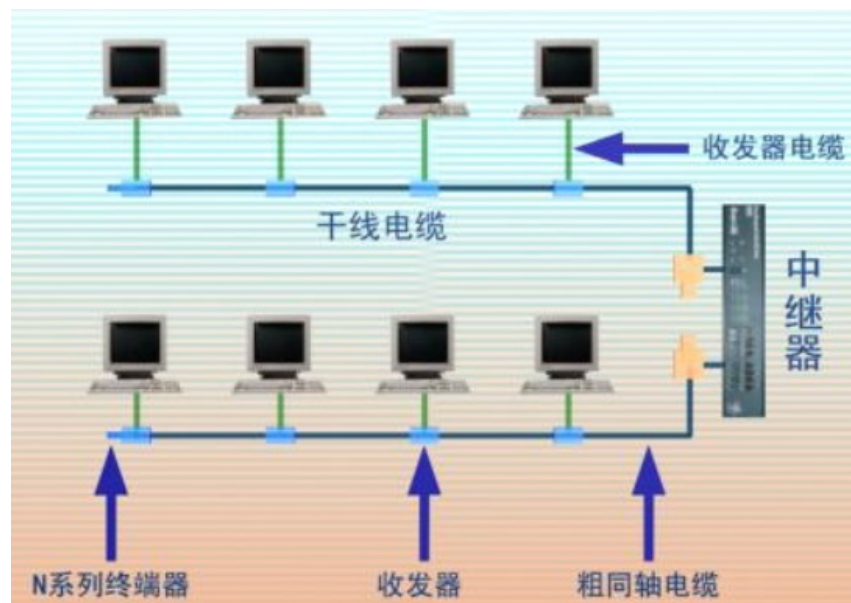


经典以太网 — 经典以太网的物理层

- 最高速率10Mbps
- 使用曼彻斯特编码
- 使用同轴电缆和中继器连接



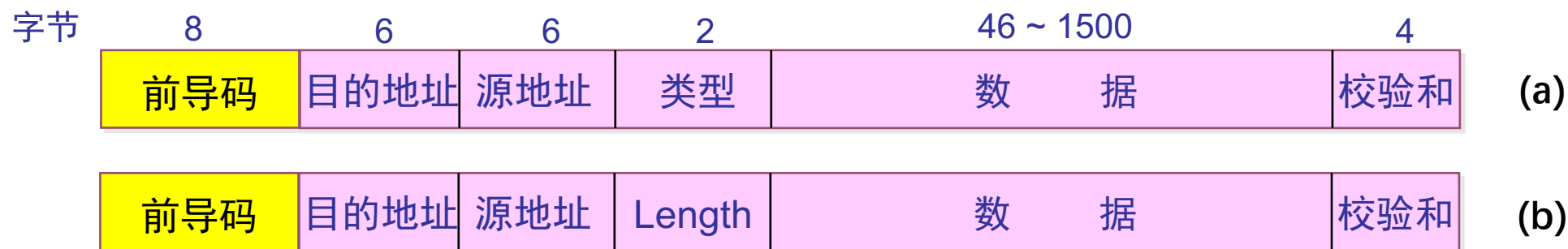
粗以太网 (thick Ethernet)





经典以太网 – MAC子层协议

- 主机运行CSMA/CD协议
- 常用的以太网MAC帧格式有两种标准：
 - DIX Ethernet V2 标准（最常用的）
 - IEEE 的 802.3 标准



MAC帧格式
(a) DIX Ethernet V2 (b) IEEE 802.3



经典以太网 - MAC子层协议



- 硬件地址又称为物理地址，或 MAC 地址
- MAC帧中的源地址和目的地址长度均为6字节

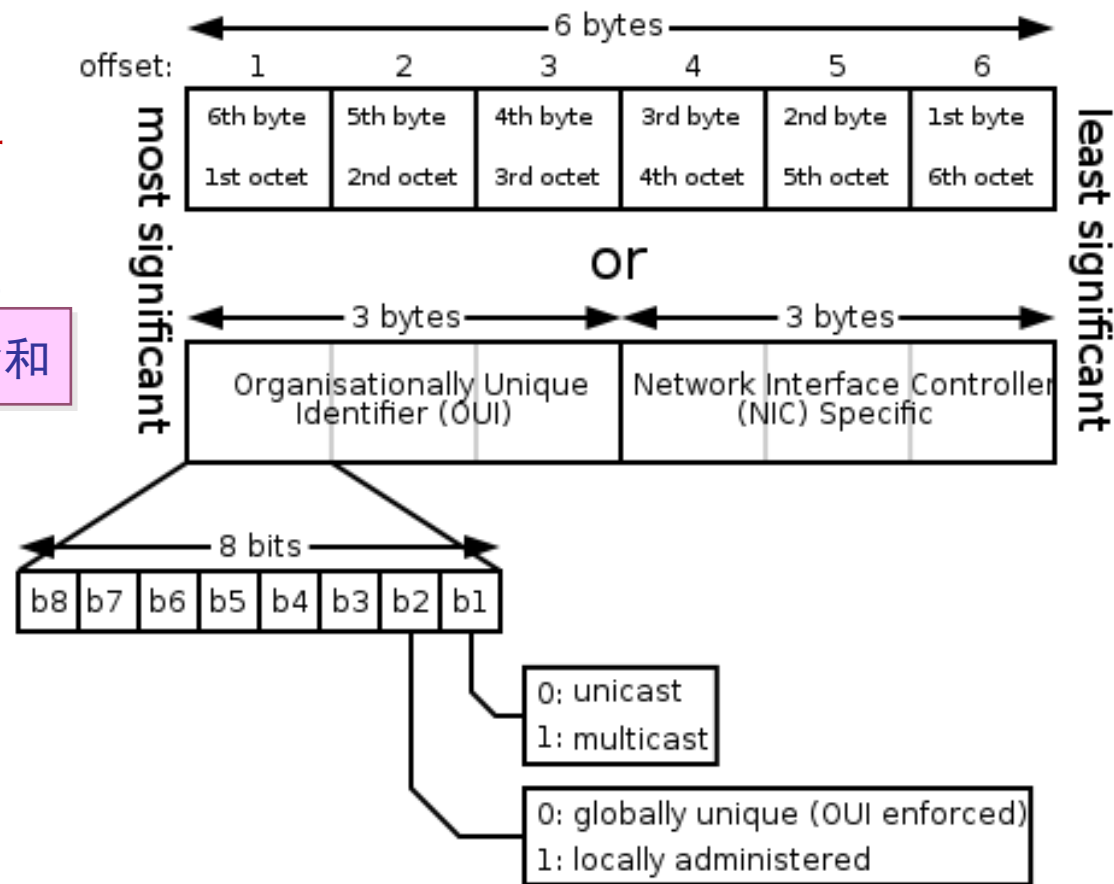


MAC地址举例

单播 (unicast) : 5C-26-0A-7E-4E-4C

广播 (broadcast) : FF-FF-FF-FF-FF-FF

组播 (multicast) : 01-00-5E-00-00-00



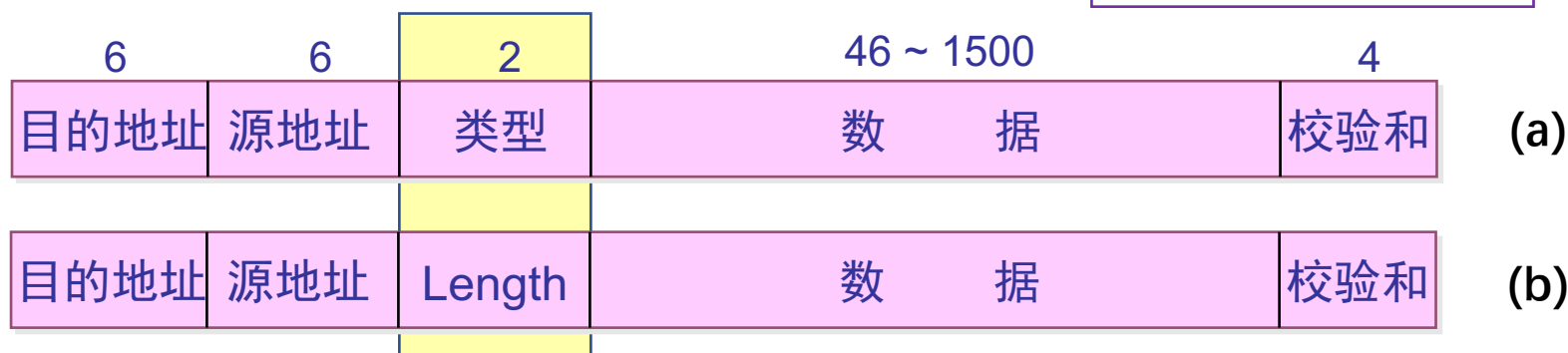


经典以太网 - MAC子层协议

- 源地址后面的两个字节，Ethernet V2将其视为上一层的协议类型，IEEE802.3将其视为数据长度。

- 可在[这里](#)查询协议类型（Ether Types）的值

IPv4: 0x0800
ARP: 0x0806
PPPoE: 0x8864
...



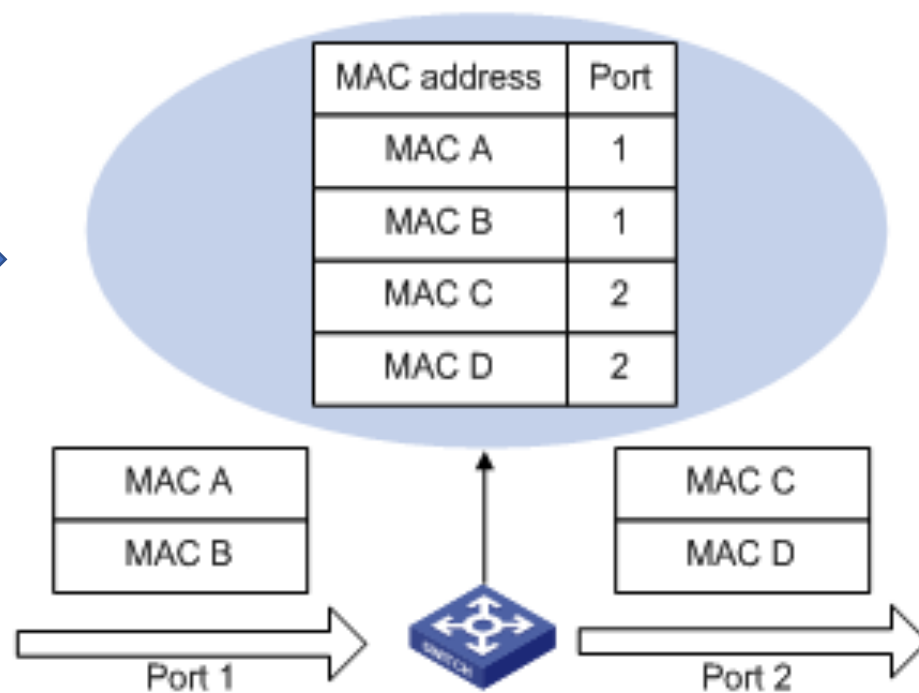
MAC帧格式

(a) DIX Ethernet V2 (b) IEEE 802.3



交换式以太网

- 交换式以太网的核心是**交换机 (Switch)**
 - 工作在数据链路层，检查MAC 帧的**目的地址**对收到的帧进行转发
 - 交换机通过高速背板把帧传送到目标端口





快速以太网



- fast Ethernet(IEEE 802.3u, 1995)
 - 带宽 10Mbps → 100Mbps
 - 保留原来的工作方式（帧格式、接口、过程
 - 自动协商（autonegotiation）
 - 线缆类型

与之前10Mbps版本的以太网相比，10倍的速度非常快。千兆以太网诞生后，100Mbps就不再是最快的以太网了，但人们仍习惯于称100Mbps以太网为“快速以太网”。

名称	线缆	最大长度	编码方式	优点
100Base-T4	双绞线	100米	8B6T	可用3类UTP
100Base-TX	双绞线	100米	4b/5b	全双工速率100Mbps(5类UTP)
100Base-FX	光纤	2000米	4b/5b	全双工速率100Mbps，距离长



千兆以太网



➤ gigabit Ethernet(IEEE 802.3ab, 1998)

- 100Mbps → 1000Mbps(1Gbps)
- 保留原来的工作方式（帧格式、接口、过程规则）
- 全双工和半双工两种方式工作。
 - 在半双工方式下使用 CSMA/CD（为了向后兼容），增加载波扩充和帧突发
 - 全双工方式不需要使用CSMA/CD（缺省方式）
- 流量控制和巨型帧（Jumbo frame）

• 线缆类型

名称	线缆	最大长度	编码方式	优点
1000Base-SX	光纤	550米	8b/10b	多模光纤（50、62.5微米）
1000Base-LX	光纤	5000米	8b/10b	单模光纤（10微米） 或多模光纤（50、62.5微米）
1000Base-CX	2对STP	25米	8b/10b	屏蔽双绞线
1000Base-T	2对UTP	100米	4D-PAM5	标准5类UTP

万兆以太网

➤ 10-Gigabit Ethernet(IEEE 802.3ae, 2002)

- 1Gbps → 10Gbps
- 常记为10GE, 10GbE 或 10 GigE
- 只支持全双工, 不再使用CSMA/CD
- 保持兼容性
- 重点是超高速的物理层



10GBASE-SR SFP +收发器

名称	线缆	最大长度	编码方式	优点
10GBase-SR	光纤	最多300米	64b/66b	多模光纤 (0.85微米)
10GBase-LR	光纤	10千米	64b/66b	单模光纤 (1.3微米)
10GBase-ER	光纤	40千米	64b/66b	单模光纤 (1.5微米)
10GBase-CX4	4对双轴	15米	8b/10b	双轴铜缆
10GBase-T	4对UTP	100米	64b/65b	6a类UTP



本章内容



- 4.1 信道分配问题 (P4)
- 4.2 多路访问协议 (P11)
- 4.3 以太网 (P44)
- 4.4 数据链路层交换 (P73)
- 4.5 无线局域网 (P133)

1. 数据链路层交换原理
2. 链路层交换机
3. 虚拟局域网

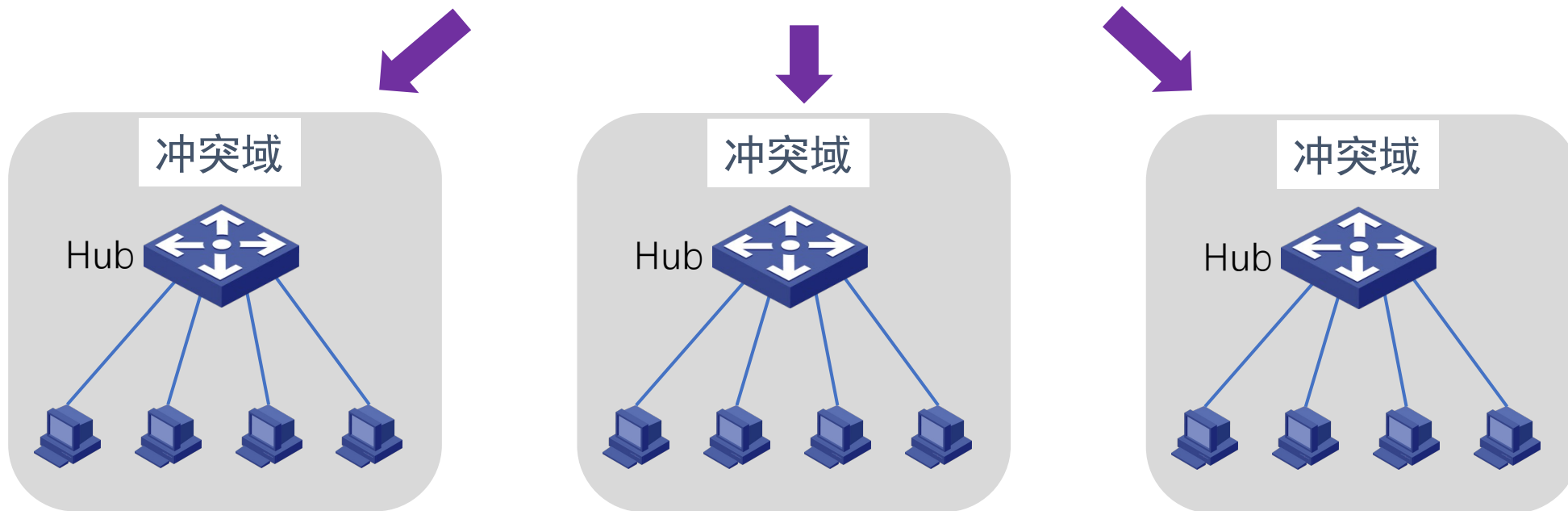


数据链路层交换原理



➤ 物理层设备扩充网络

三个独立的冲突域



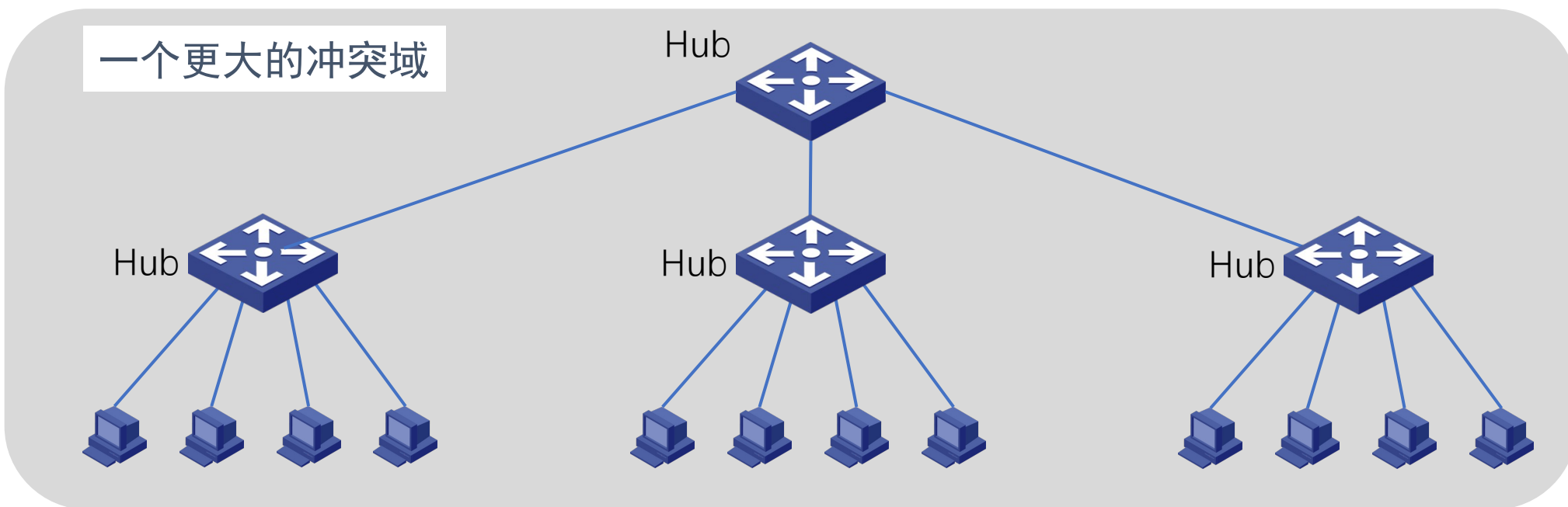


数据链路层交换原理



➤ 物理层设备扩充网络

- 扩大了冲突域，性能降低，安全隐患

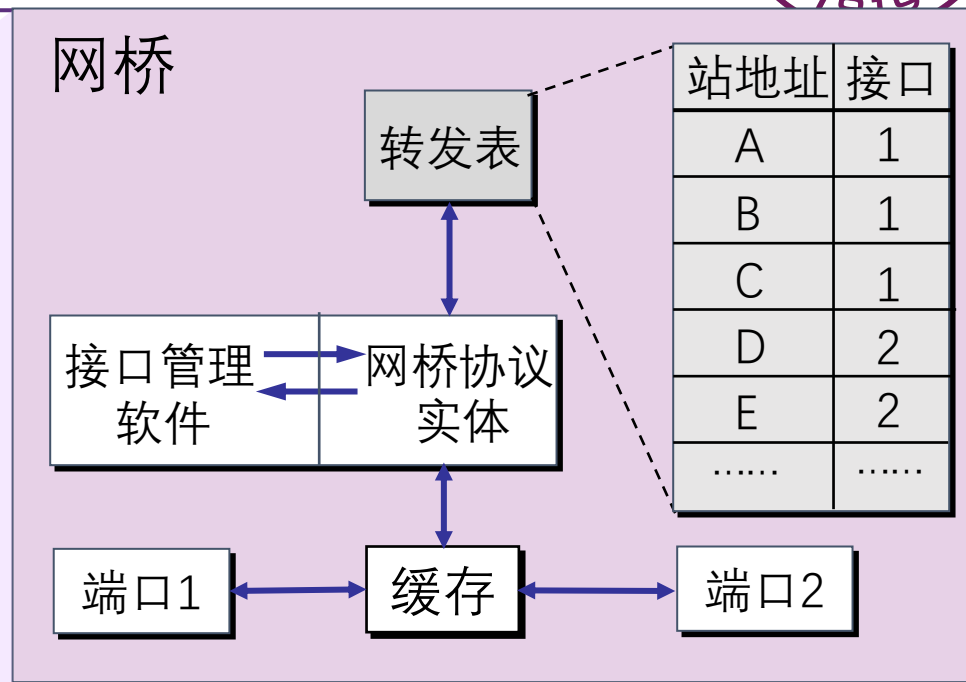
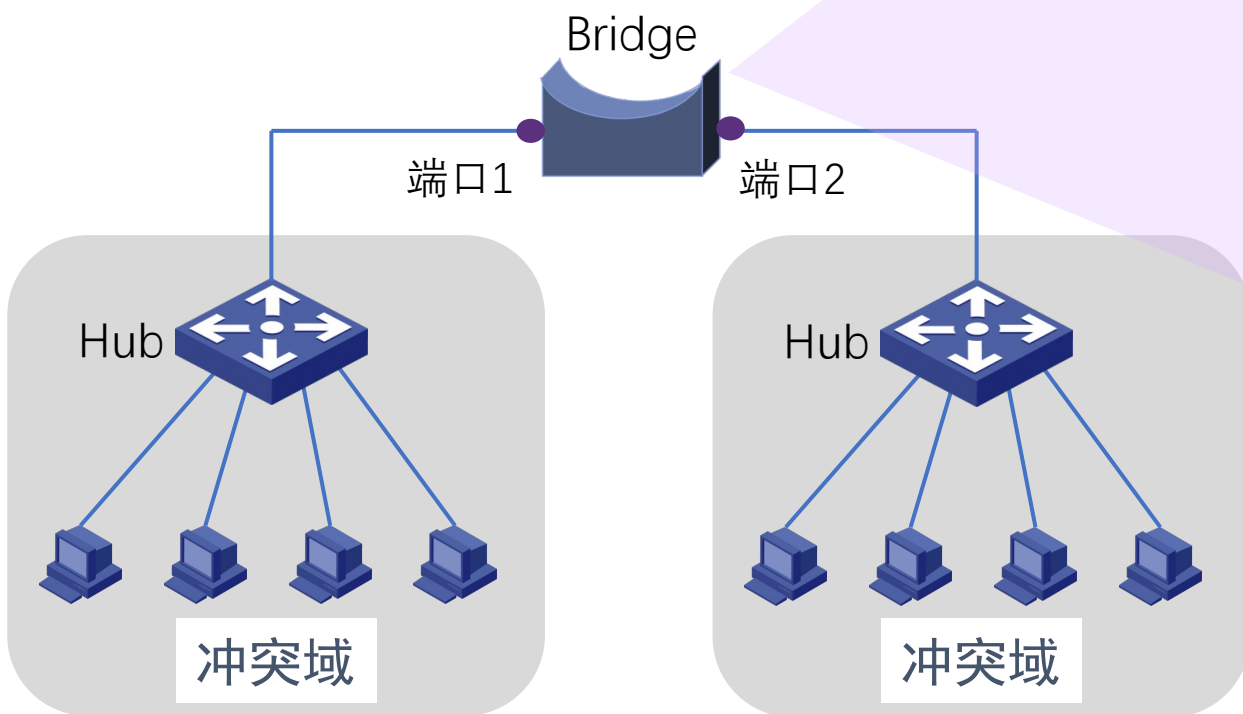




数据链路层交换原理

➤ 数据链路层设备扩充网络

- 网桥或交换机
- 分隔了冲突域





数据链路层交换原理



- 理想的网桥是透明的。
 - 即插即用，无需任何配置
 - 网络中的站点无需感知网桥的存在与否

怎么做到透明的？



数据链路层交换原理

➤ MAC地址表的构建-逆向学习源地址

A→B发出数据帧

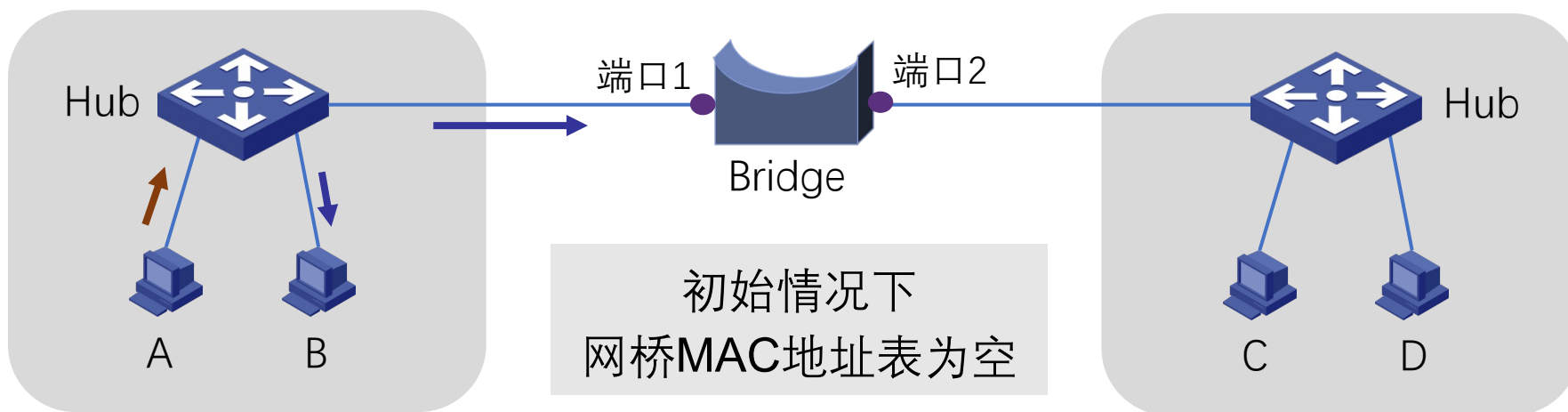


MAC地址表	
MAC地址	端口
MAC_A	1

记录帧到达时间

设定老化时间（默认300s）

当老化时间到期时，该表项会被清除。





数据链路层交换原理

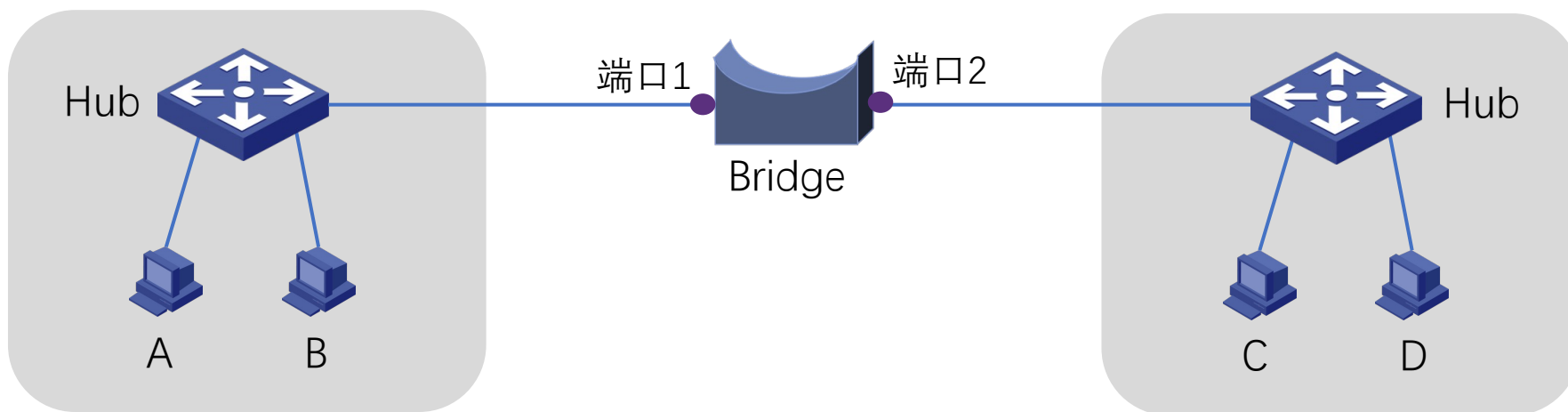


➤ 发送帧的站MAC地址被学习

网桥怎样学习到B\C\D的
MAC地址?

MAC地址表	
MAC地址	端口
MAC_A	1
MAC_B	1
MAC_C	2
MAC_D	2

主机向外发送数据时
其MAC地址就会被学习



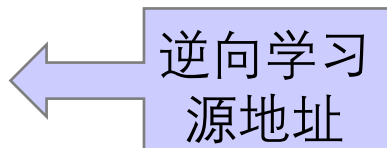


数据链路层交换原理



➤ 网桥构建MAC地址表的过程

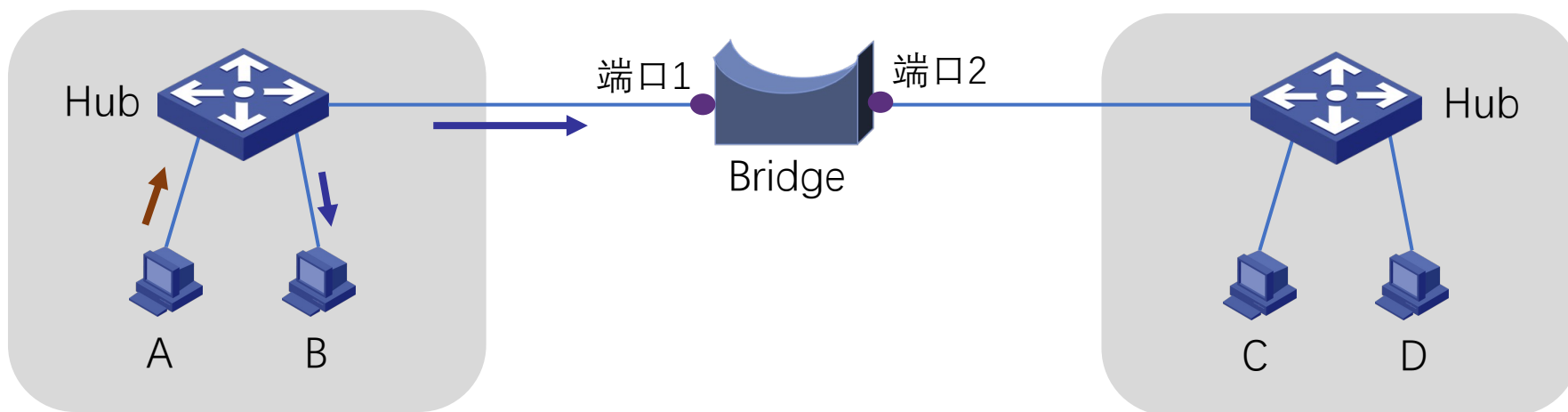
再次：A发出数据帧



MAC地址表	
MAC地址	端口
MAC_A	1
MAC_B	1
MAC_C	2
MAC_D	2

网桥发现MAC_A已在表中！

更新该表项的帧达到时间，
重置老化时间





数据链路层交换原理



➤ MAC地址表的构建

- 增加表项：帧的源地址对应的项不在表中
- 删除表项：老化时间到期
- 更新表项：帧的源地址在表中，更新时间戳

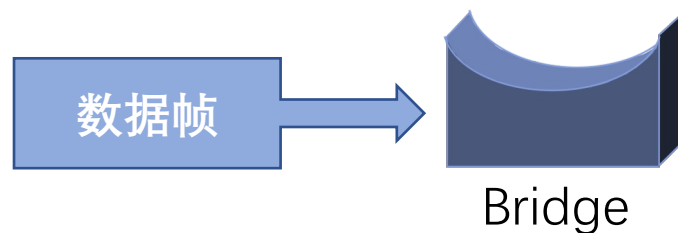


数据链路层交换原理



- 网桥通过逆向学习帧的源地址
 - 获知主机所处的位置
 - 构建MAC地址表

网桥如何利用MAC地址表进行数据帧的转发？



网桥对于入境帧的处理过程 (forwarding、filtering、flooding)



数据链路层交换原理

➤ Forwarding (转发)

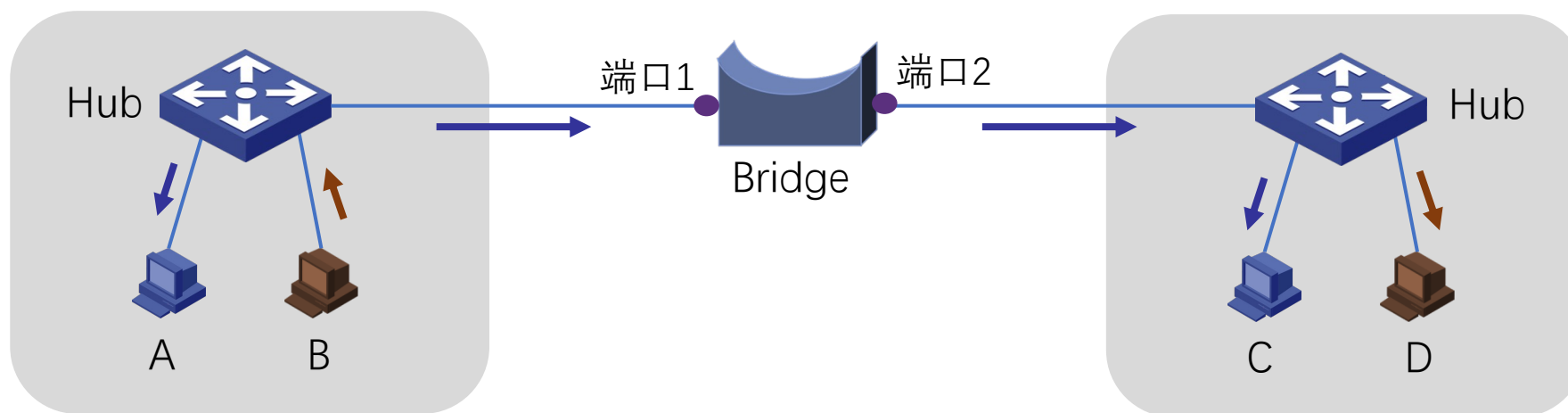
B → D 发出数据帧

逆向学习源地址
并根据目的地址
查询MAC地址表

MAC地址表完善时

MAC地址表	
MAC地址	端口
MAC_A	1
MAC_B	1
MAC_C	2
MAC_D	2

找到匹配项！
从对应端口2转发出去





数据链路层交换原理

➤ Filtering (过滤)

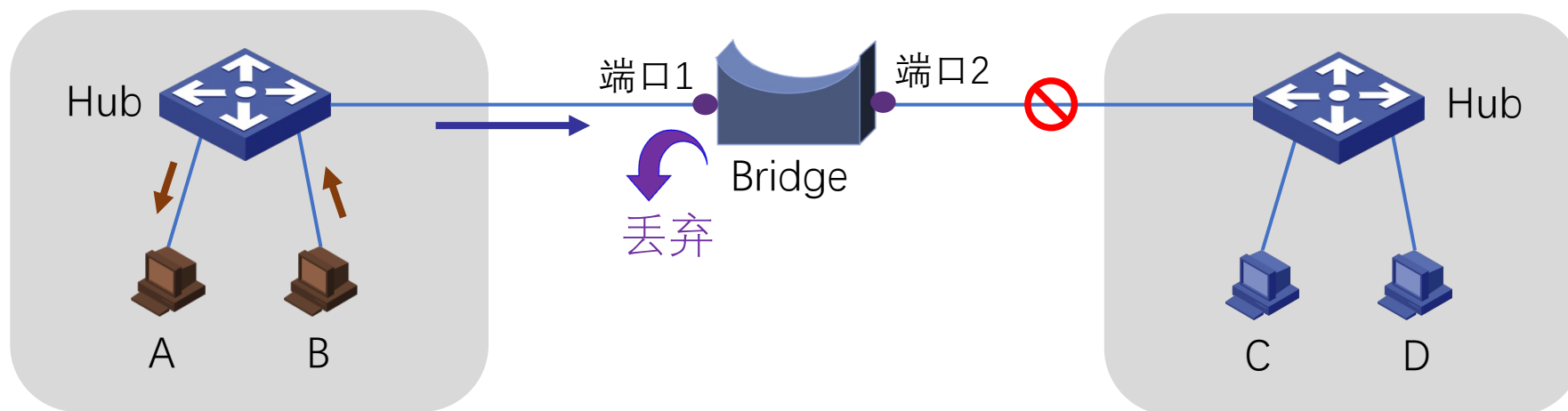
B → A 发出数据帧

逆向学习源地址
并根据目的地址
查询MAC地址表

MAC地址表完善时

MAC地址表	
MAC地址	端口
MAC_A	1
MAC_B	1
MAC_C	2
MAC_D	2

找到匹配项！
入境口=出境口，丢弃！





数据链路层交换原理



➤ Flooding（泛洪）

A → B 发出数据帧

MAC地址表不完善时

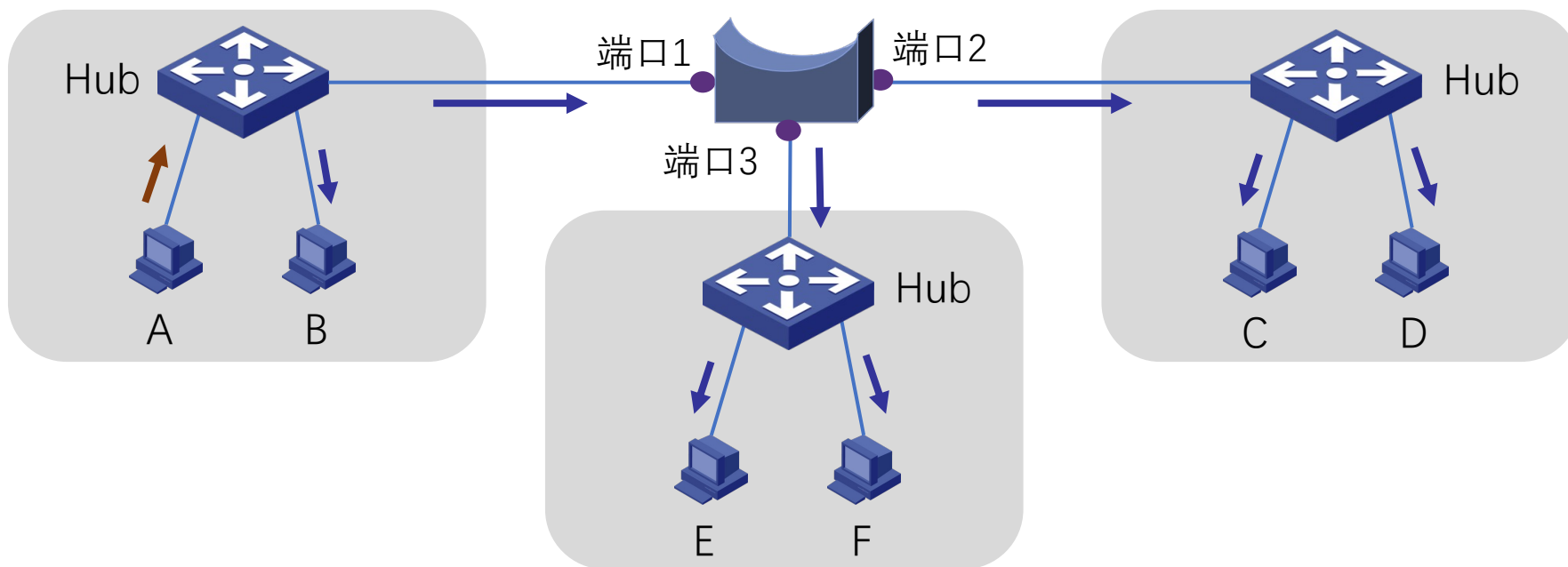
MAC地址表	
MAC地址	端口
MAC_A	1

找不到匹配表项！

从所有端口（除了入境口）发送出去
一个网段的数据被发送到无关网段

存在安全隐患

浪费网络资源





数据链路层交换原理



➤ Flooding（泛洪）

- 两种目的地址的帧，需要泛洪：
 - 广播帧：目的地址为FF-FF-FF-FF-FF-FF的数据帧
 - 未知单播帧：目的地址不在MAC地址转发表中的单播数据帧

泛洪的两种情形！



数据链路层交换原理



➤ 透明网桥工作原理（小结）

• 逆向学习

根据帧的**源地址**在MAC地址表查找匹配表项，

✓如果没有，则**增加**一个新表项（源地址、入境端口、帧到达时间），

✓如果有，则**更新**原表项的帧到达时间，重置老化时间。

• 对入境帧的转发过程（**三选一**），查帧的目的地址是否在MAC地址表中

✓如果有，且入境端口 $\neq$ 出境端口，则从对应的出境端口**转发帧**；

✓如果有，且入境端口 = 出境端口，则丢弃帧（即**过滤帧**）；

✓如果没有，则向除入境端口以外的其它所有端口**泛洪帧**。



第五章 网络层

授课教师：张圣林

南开大学



本章目标



1. 学习网络层服务：无连接和面向连接服务，数据包和虚电路网络
2. 学习Internet网络层协议：IPv4/IPv6, ICMP, DHCP, NAT, ARP
3. 掌握链路状态、距离矢量等路由算法，了解层次路由结构
4. 掌握Internet路由协议：OSPF、RIP、BGP
5. 了解路由器工作原理：控制层和数据层，报文转发机制，交换结构
6. 了解网络拥塞及拥塞控制思想
7. 了解网络服务质量及设计思想
8. 了解其它重要的网络层相关机制：三层交换、VPN
9. 了解IPv6相关机制



本章内容



5.1 网络层服务

5.2 Internet网际协议

5.3 路由算法

5.4 Internet路由协议

5.5 路由器工作原理

5.6 拥塞控制算法

5.7 服务质量

5.8 三层交换与VPN

5.9 IPv6技术



本章内容



5.1 网络层服务

5.2 Internet网际协议

5.3 路由算法

5.4 Internet路由协议

5.5 路由器工作原理

5.6 拥塞控制算法

5.7 服务质量

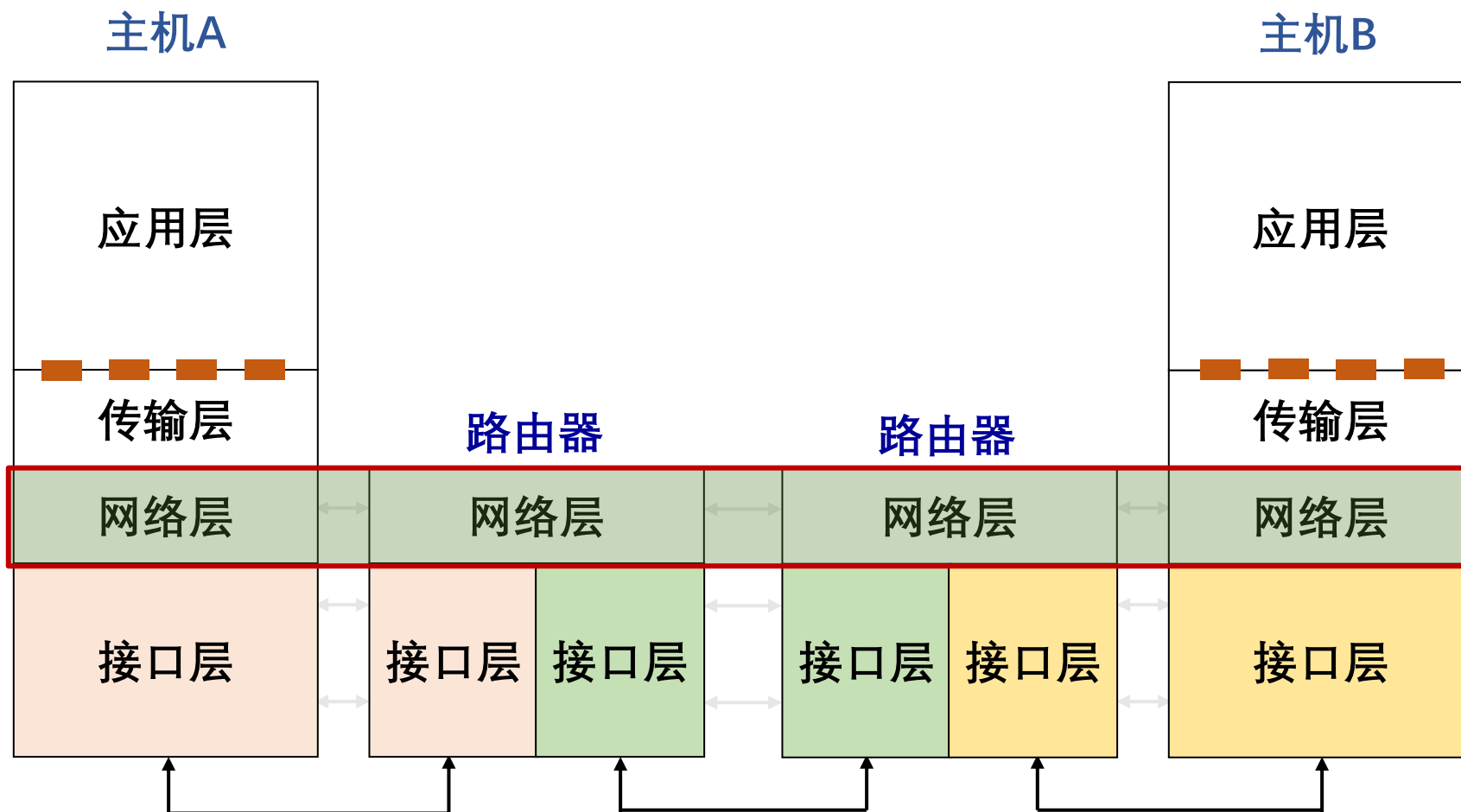
5.8 三层交换与VPN

5.9 IPv6技术

1. 网络层服务概述
2. 无连接服务的实现
3. 面向连接服务的实现
4. 虚电路与数据包网络的比较



网络层服务概述

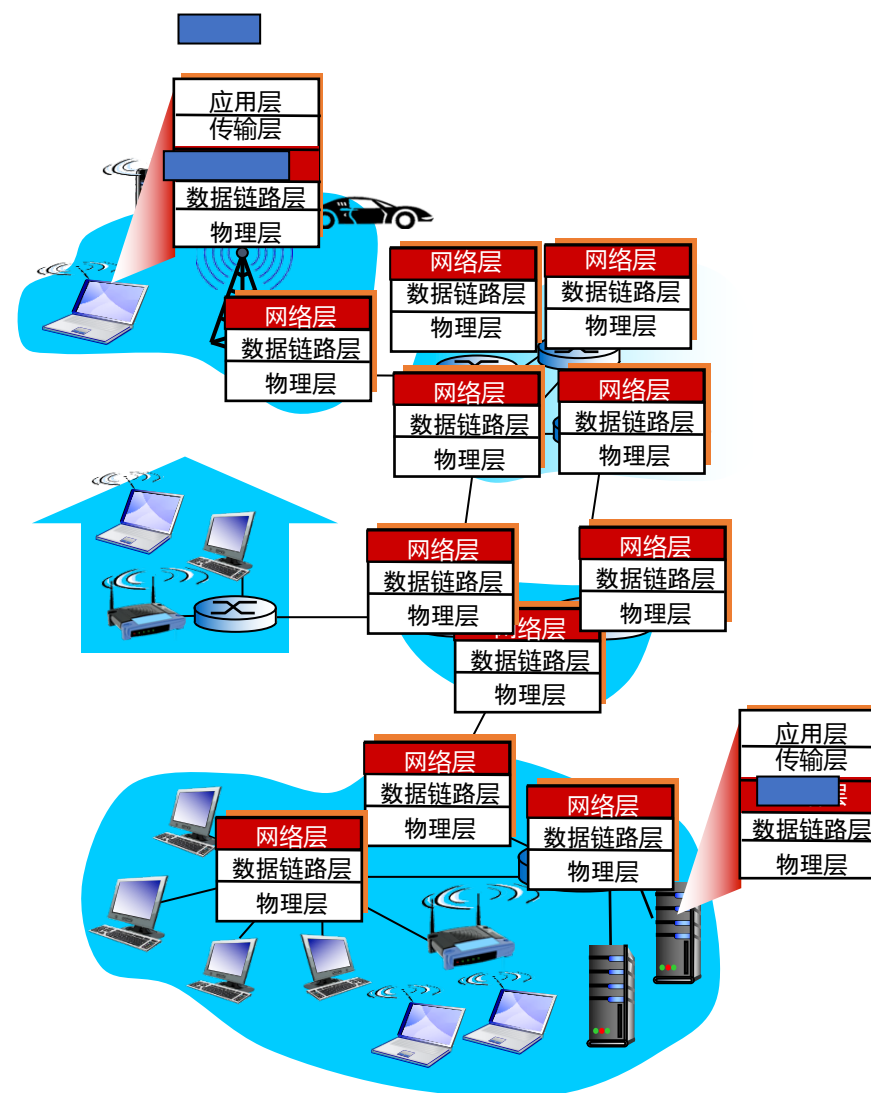


接口层通常包括数据链路层和物理层



网络层服务的实现

- 网络层实现端系统间多跳传输可达
- 网络层功能存在每台主机和路由器中
 - 发送端：将传输层数据单元封装在数据包中
 - 接收端：解析接收的数据包中，取出传输层数据单元，交付给传输层
 - 路由器：检查数据包首部，转发数据包



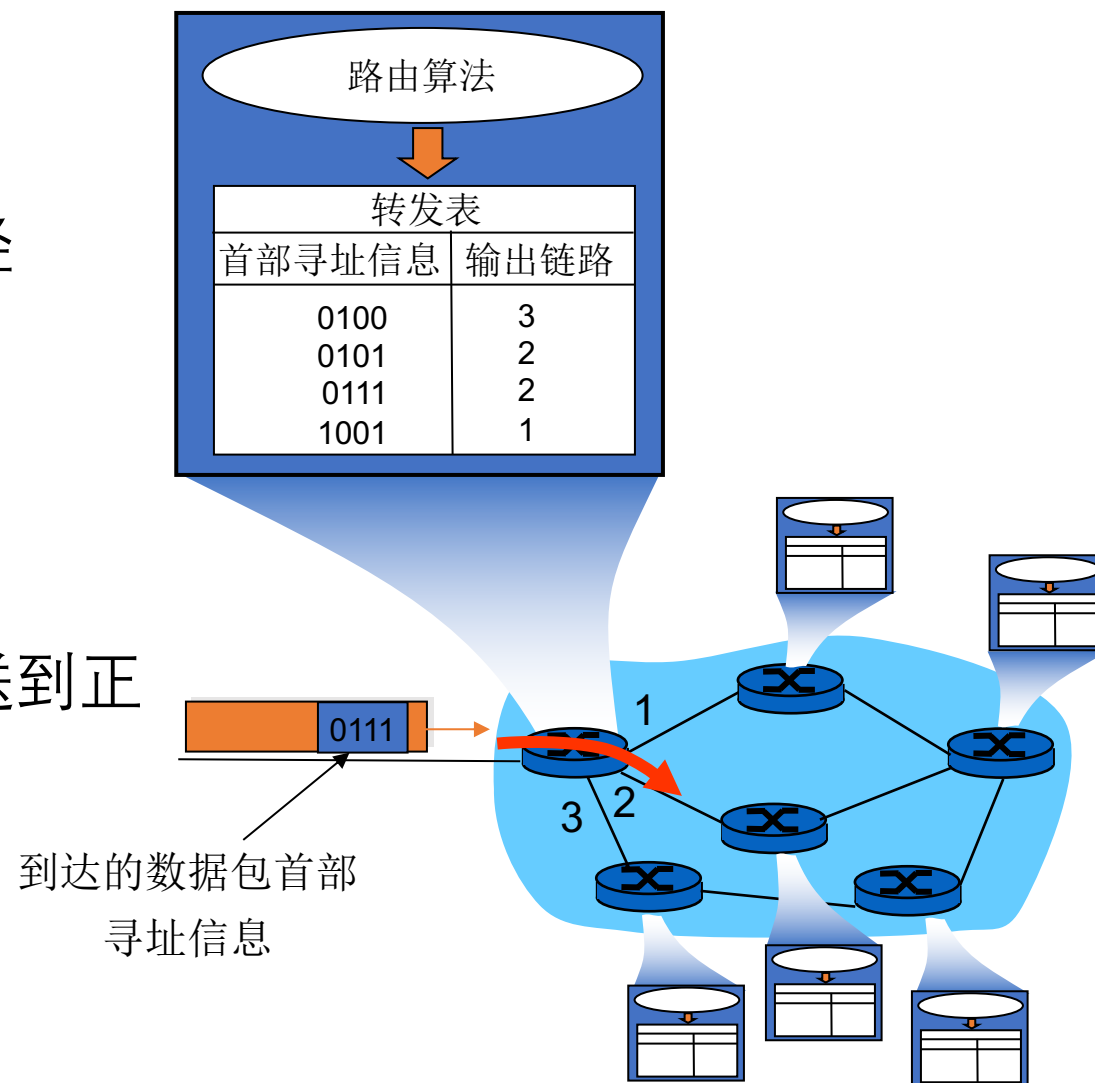
网络层关键功能

➤ 路由（控制面）

- 选择数据包从源端到目的端的路径
- 核心：路由算法与协议

➤ 转发（数据面）

- 将数据包从路由器的输入接口传送到正确的输出接口





本章内容



5.1 网络层服务

5.2 Internet网际协议

5.3 路由算法

5.4 Internet路由协议

5.5 路由器工作原理

5.6 拥塞控制算法

5.7 服务质量

5.8 三层交换与VPN

5.9 IPv6技术

1. IPv4协议

2. IP地址

3. ARP

4. NAT

5. DHCP

6. Internet控制报文协议

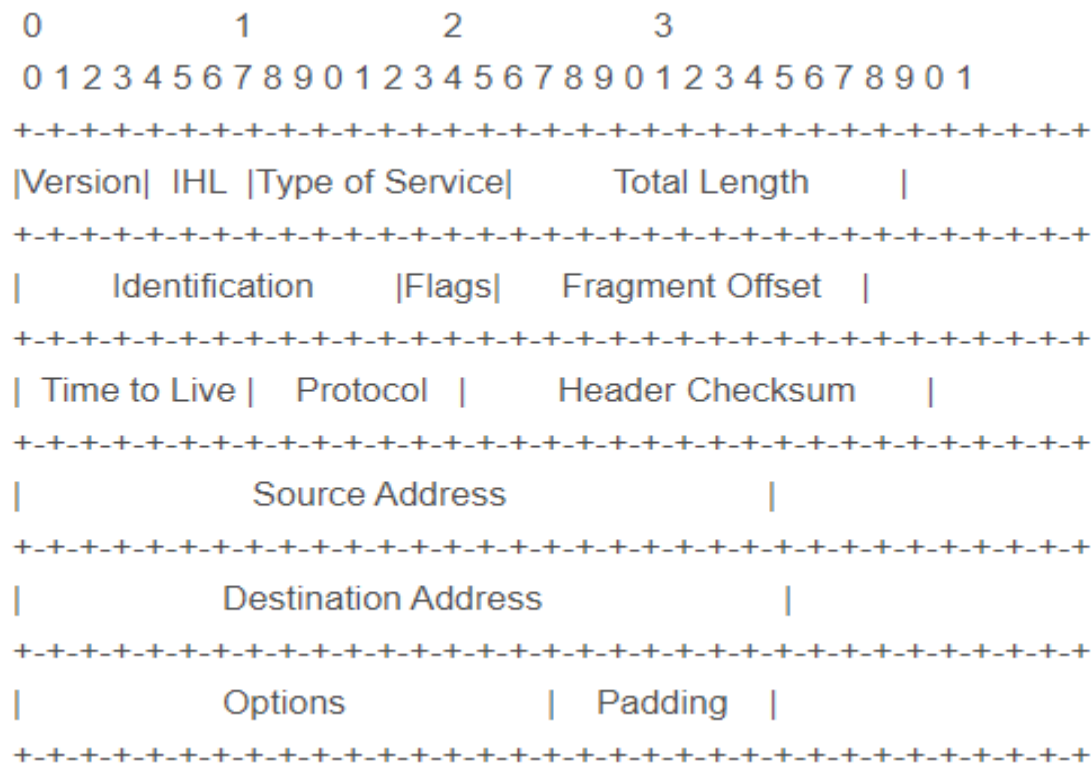


IPv4协议



- IPv4协议，网际协议版本4，一种无连接的协议，是互联网的核心，也是使用最广泛的网际协议版本，其后继版本为IPv6

- Internet协议执行两个基本功能
 - 寻址(addressing)
 - 分片(fragmentation)



RFC 791



IPv4数据包格式

- **版本**: 4bit, 表示采用的IP协议版本
- **首部长度**: 4bit, 表示整个IP数据包首部的长度
- **区分服务**: 8bit, 该字段一般情况下不使用
- **总长度**: 16bit, 表示整个IP报文的长度,能表示的最大字节为 $2^{16}-1=65535$ 字节
- **标识**: 16bit, IP软件通过计数器自动产生, 每产生1个数据包计数器加1; 在IP分片以后, 用来标识同一片分片
- **标志**: 3bit, 目前只有两位有意义; MF, 置1表示后面还有分片, 置0表示这是数据包片的最后1个; DF, 不能分片标志, 置0时表示允许分片
- **片偏移**: 13bit, 表示IP分片后, 相应的IP片在总的IP片的相对位置

IP 数据包由首部和数据两部数据包成



DF: Do not fragment
MF: More fragment



IPv4数据包格式

- **生存时间TTL(Time To Live)**：8bit,表示数据包在网络中的生命周期，用通过路由器的数量来计量，即跳数（每经过一个路由器会减1）
- **协议**：8bit，标识上层协议（TCP/UDP/ICMP…）
- **首部校验和**：16bit，对数据包首部进行校验，不包括数据部分
- **源地址**：32bit，标识IP片的发送源IP地址
- **目的地址**：32bit，标识IP片的目的地IP地址
- **选项**：可扩充部分，具有可变长度，定义了安全性、严格源路由、松散源路由、记录路由、时间戳等选项
- **填充**：用全0的填充字段补齐为4字节的整数倍

IP 数据包由首部和数据两部分数据包成





数据包分片

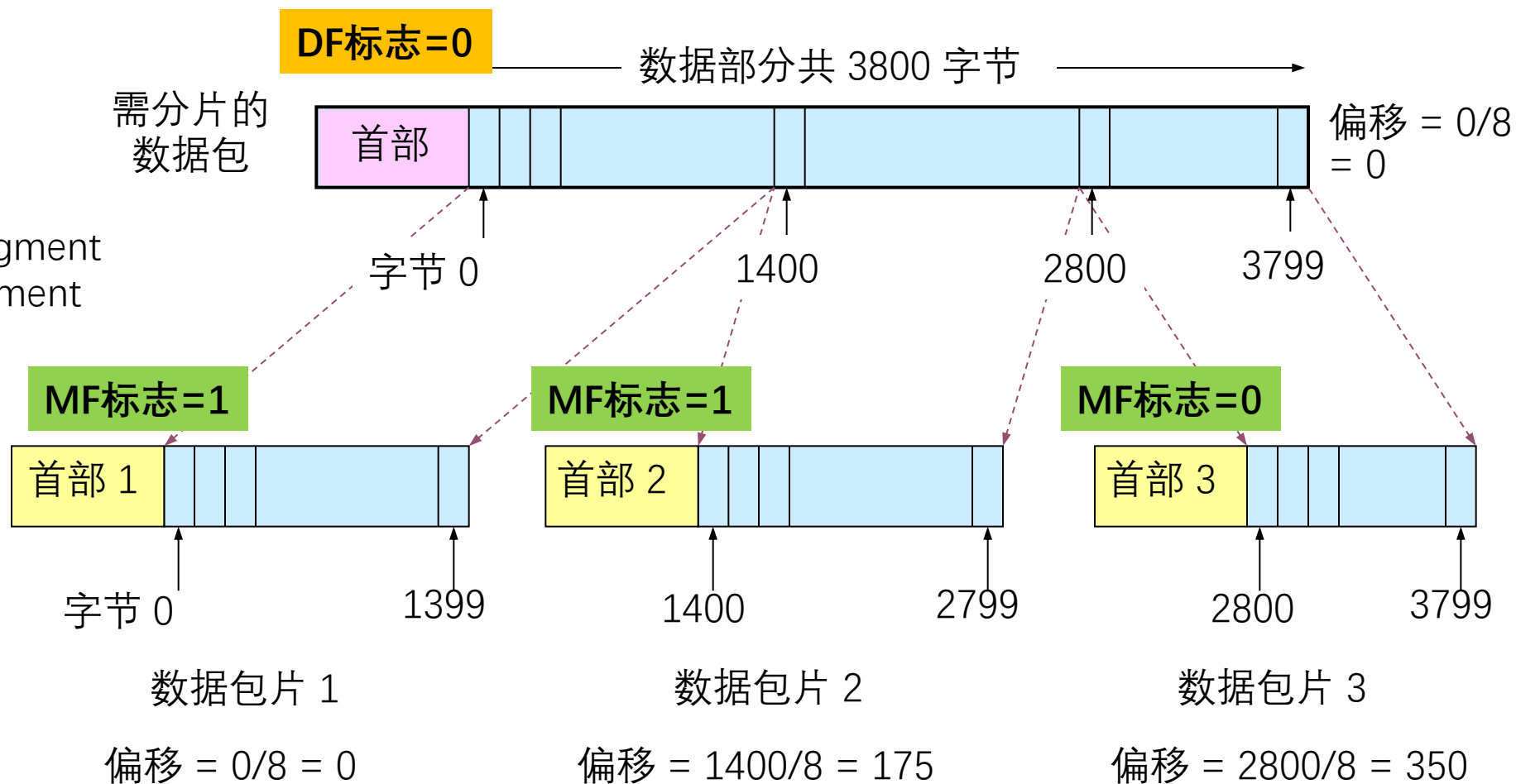


- MTU (Maximum Transmission Unit) , 最大传输单元
 - 链路MTU
 - 路径MTU (Path MTU)
- 分片策略
 - 允许途中分片：根据下一跳链路的MTU实施分片
 - 不允许途中分片：发出的数据包长度小于路径MTU（路径MTU发现机制）
- 重组策略
 - 途中重组，实施难度太大
 - 目的端重组（互联网采用的策略）
 - 重组所需信息：原始数据包编号、分片偏移量、是否收集所有分片



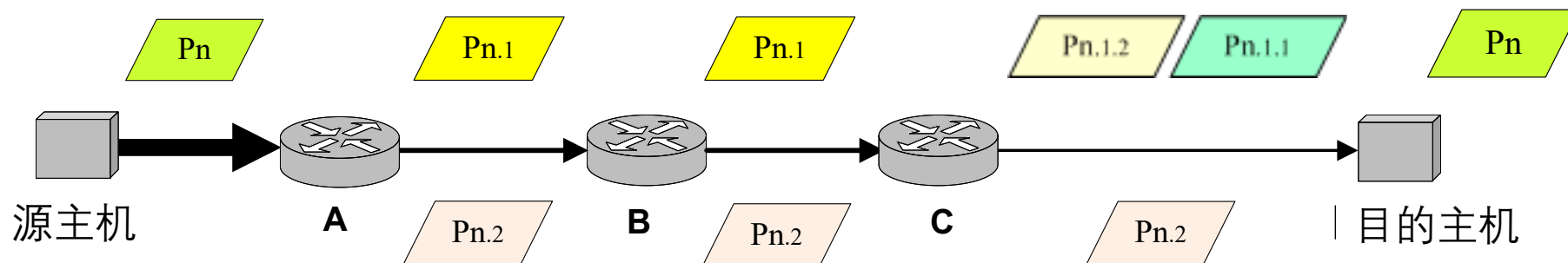
数据包分片

DF: Do not fragment
MF: More fragment





数据包分片



- IPv4数据包在传输途中可以多次分片
 - 源端系统，中间路由器（可通过标志位设定是否允许路由器分片）
- IPv4分片只在目的IP对应的目的端系统进行重组
- IPv4分片、重组字段在基本IP头部
 - 标识、标志、片偏移



IP协议功能及报头字段总结

网络层基本功能

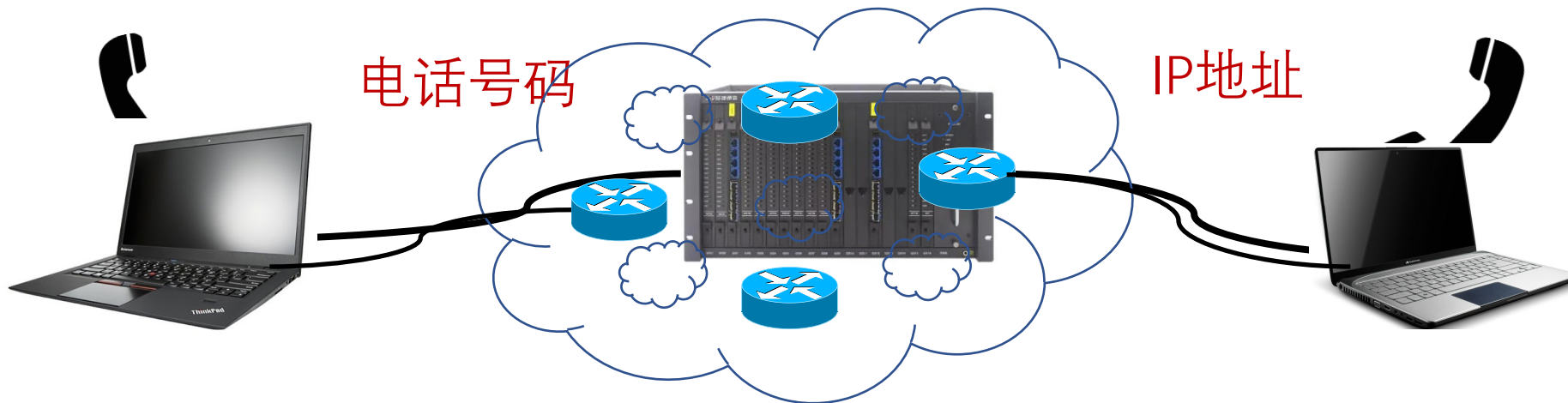
- 支持多跳寻路将IP数据包送达目的端：目的IP地址
- 表明发送端身份：源IP地址
- 根据IP头部协议类型，提交给不同上层协议处理：协议

其它相关问题

- 数据包长度大于传输链路的MTU的问题，通过分片机制解决：标识、标志、片偏移
- 防止循环转发浪费网络资源（路由错误、设备故障…），通过跳数限制解决：生存时间TTL
- IP报头错误导致无效传输，通过头部校验解决：首部校验和



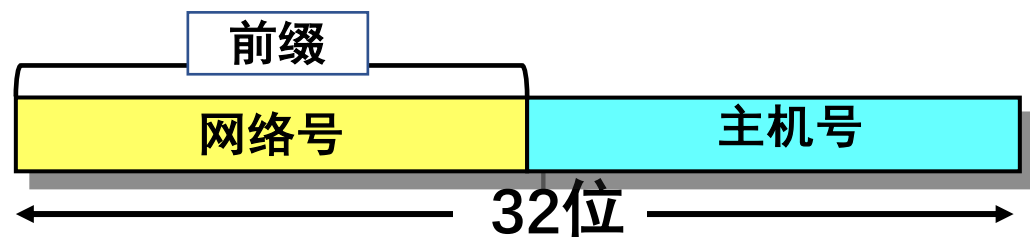
- **IP地址**，网络上的每一台主机（或路由器）的每一个接口都会分配一个全球唯一的32位的标识符





IP地址

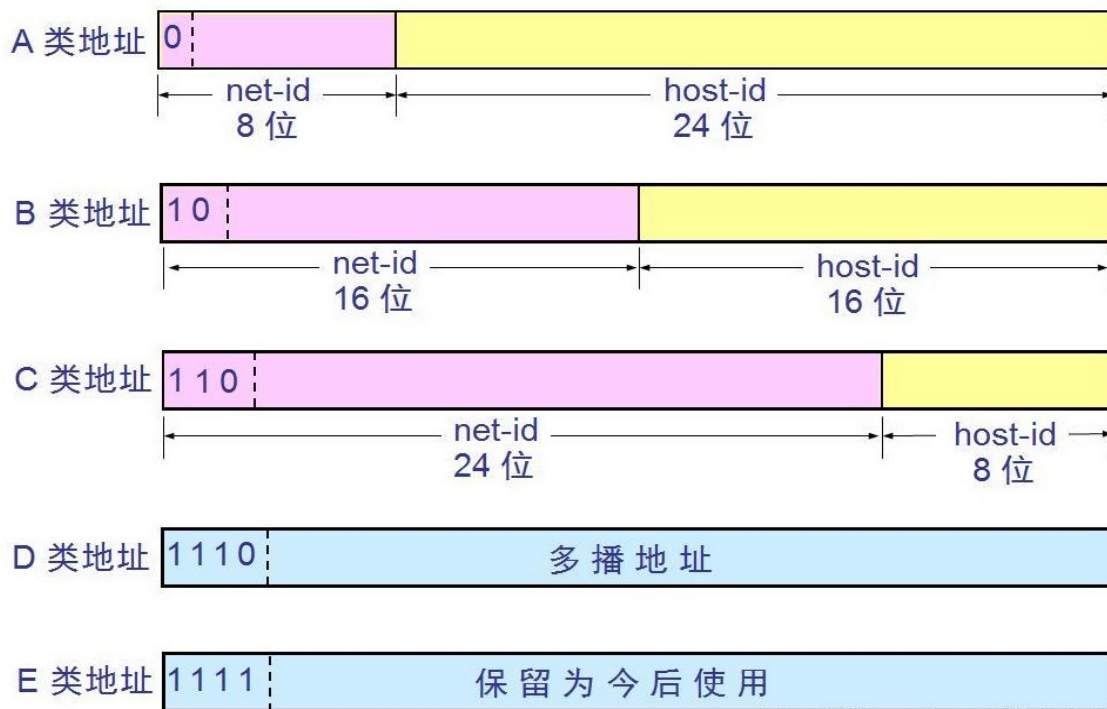
- **IP地址**，网络上的每一台主机（或路由器）的每一个接口都会分配一个全球唯一的32位的标识符
- 将IP地址划分为固定的类，每一类都由两个字段组成
- 网络号相同的这块连续IP地址空间称为地址的**前缀**，或**网络前缀**





分类的IP地址

- IP地址共分为A、B、C、D、E五类，A类、B类、C类为单播地址
- IP地址的书写采用点分十进制记法，其中每一段取值范围为0到255



1.0.0.0~127.255.255.255

128.0.0.0~191.255.255.255

192.0.0.0~223.255.255.255

A类

B类

C类



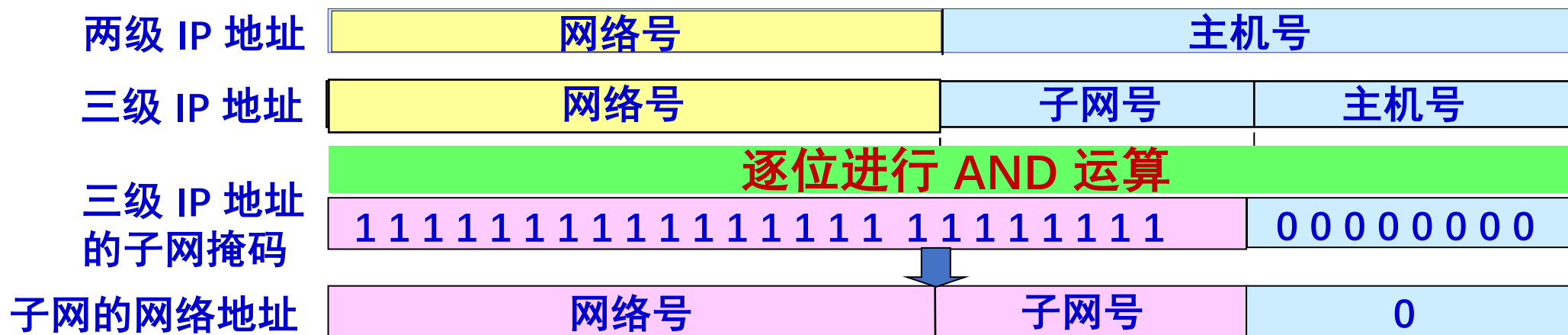
IP特殊地址



地址	用途
全0网络地址	只在系统启动时有效，用于启动时临时通信，又叫主机地址
127.X.X.X	指本地节点(一般为127.0.0.1)，用于测试网卡及TCP/IP软件，这样浪费了1700万个地址
全0主机地址	用于指定网络本身，称之为网络地址或者网络号
全1主机地址	用于广播，也称定向广播，需要指定目标网络
0.0.0.0	指任意地址
255.255.255.255	用于本地广播，也称有限/受限广播，无须知道本地网络地址

子网划分

- 子网划分 (subnetting), 在**网络内部**将一个网络块进行划分以供多个内部网络使用, 对外仍是一个网络
- 子网(subnet), 一个网络进行子网划分后得到的一系列结果网络称为子网
- 子网掩码(subnet mask), 是32 bit 的二进制数, 置1表示网络位, 置0表示主机位
- 子网划分减少了 IP 地址的浪费, 网络的组织更加灵活, 便于维护和管理





子网划分



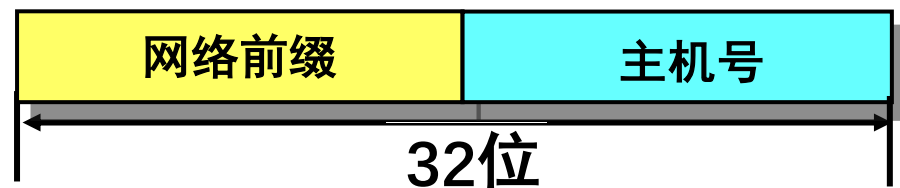
IP地址	172	16	2	160	
子网掩码	255	255	255	192	
	10101100	00010000	00000010	10100000	逐位进行 AND 运算
	11111111	11111111	11111111	11000000	
前缀←	10101100	00010000	00000010	10000000	→主机位
	10101100	00010000	00000010	10111111	
主机位全0, 子网地址	172.16.2.128	主机位全1, 广播地址	172.16.2.191	可分配IP地址范围	子网拥有主机数量
			172.16.2.128+1~	2 ⁿ -2=62 (n=6)	
			172.16.2.191-1		



无类域间路由

➤ CIDR (Classless Inter-Domain Routing)

- 将32位的IP地址划分为前后两个部分，并采用斜线记法，即在IP地址后加上“/”，然后再写上网络前缀所占位数



- 一个 CIDR 地址块可以表示很多地址，这种地址的聚合常称为路由聚合 (route aggregation)，也称为构成超网 (supernet)
- 聚合技术在Internet中大量使用，它允许前缀重叠，数据包按具体路由的方向发送，即具有最少IP地址的最长匹配前缀



最长前缀匹配

最长前缀匹配 (Longest prefix match)

- CIDR可变长子网掩码以及路由聚合，需要最长前缀匹配来实现最精确匹配
- IP地址与IP前缀匹配时，总是选取子网掩码最长的匹配项
- 主要用于路由器转发表项的匹配，也应用于ACL (access control list) 规则匹配等

IP前缀 (2种描述方式)		出接口号
200.23.16.0/21	11001000 00010111 00010	0
200.23.24.0/24	11001000 00010111 00011000	1
200.23.24.0/21	11001000 00010111 00011	2
Otherwise 0.0.0.0/0	--	3

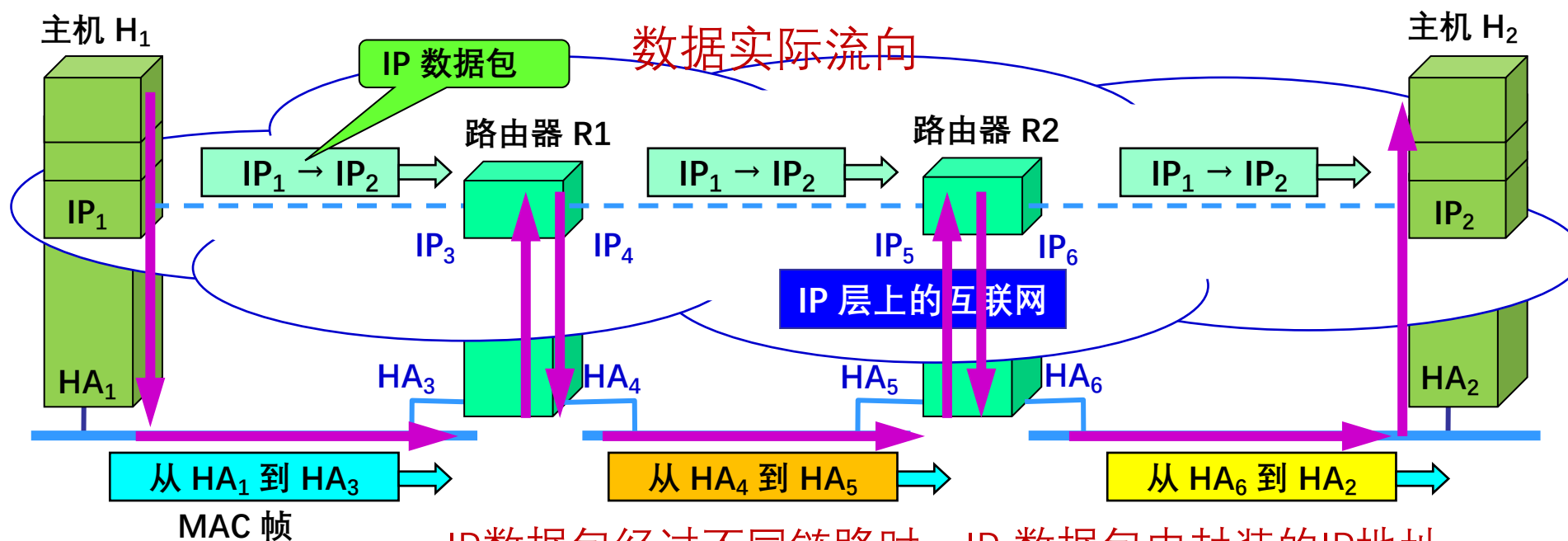
IP地址: 200.23.22.161 (11001000 00010111 00010110 10100001) , 接口0

IP地址: 200.23.24.170 (11001000 00010111 00011000 10101010) , 接口1



IP 与 MAC地址

IP 地址放在 IP 数据包的首部，而硬件地址则放在 MAC 帧的首部



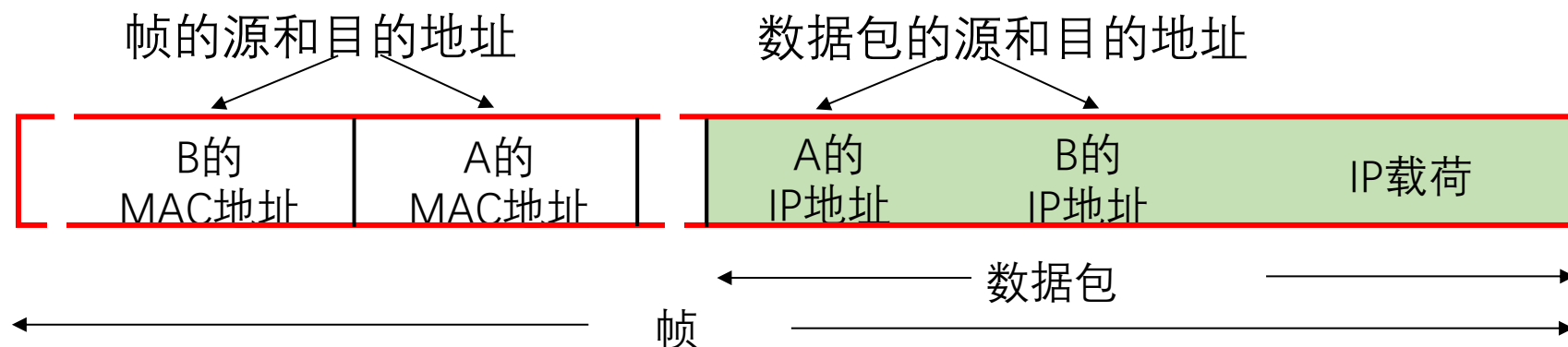
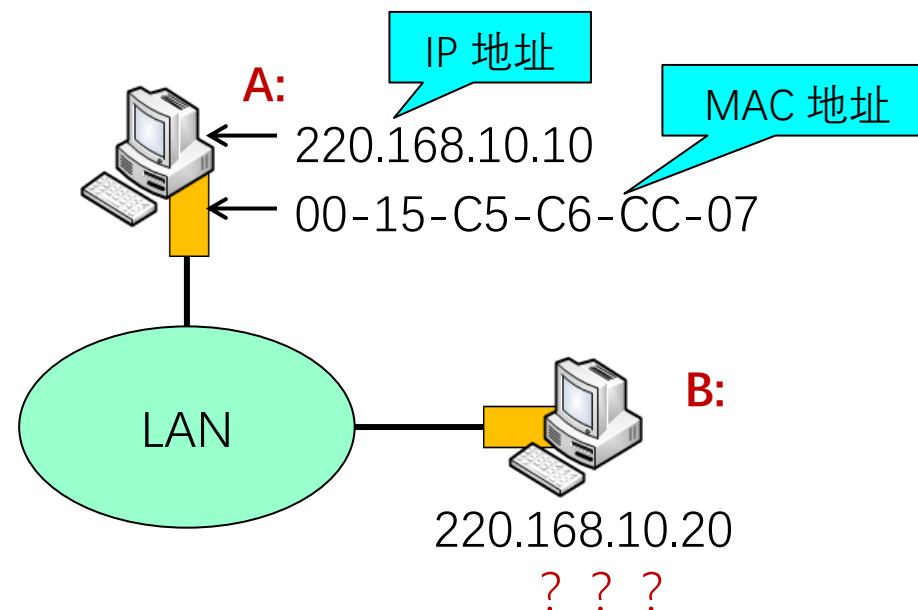
IP数据包经过不同链路时，IP 数据包中封装的IP地址不发生改变，而Mac帧中的硬件地址是发生改变的



ARP地址解析协议

- IP数据包转发：从主机A到主机B
- 检查目的IP地址的网络号部分
 - 确定主机B与主机A属相同IP网络
 - 将IP数据包封装到链路层帧中，直接发送给主机B

问题：给定B的IP地址，如何获取B的MAC地址？





ARP地址解析协议

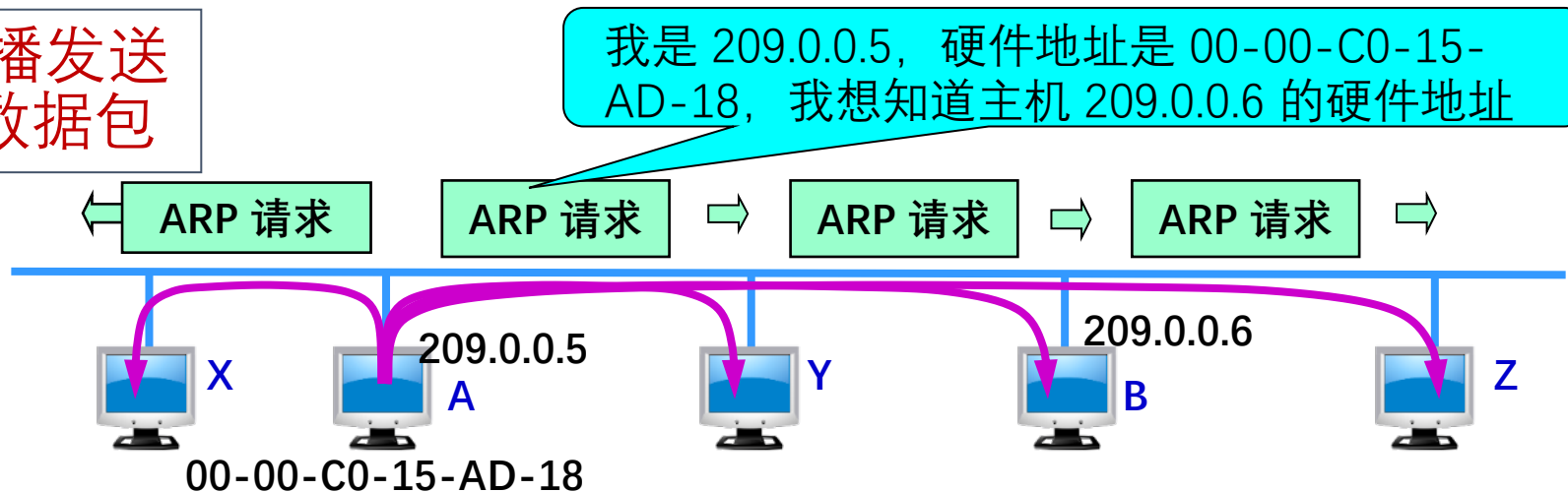


- A已知B的IP地址，需要获得B的MAC地址（物理地址）
- 如果A的ARP表中缓存有B的IP地址与MAC地址的映射关系，则直接从ARP表获取
- 如果A的ARP表中未缓存有B的IP地址与MAC地址的映射关系，则A广播包含B的IP地址的ARP query数据包
 - 在局域网上的所有节点都可以接收到ARP query
- B接收到ARP query数据包后，将自己的MAC地址发送给A
- A在ARP表中缓存B的IP地址和MAC地址的映射关系
 - 超时删除

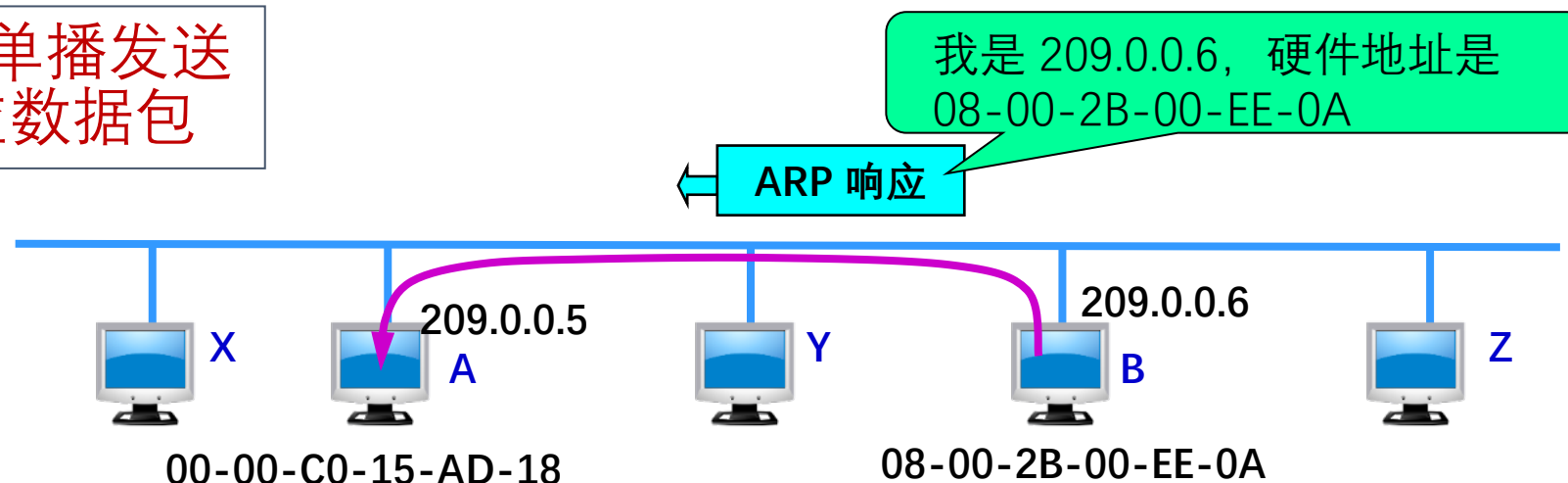


ARP协议工作过程

主机 A 广播发送
ARP 请求数据包



主机 B 向 A 单播发送
ARP 响应数据包

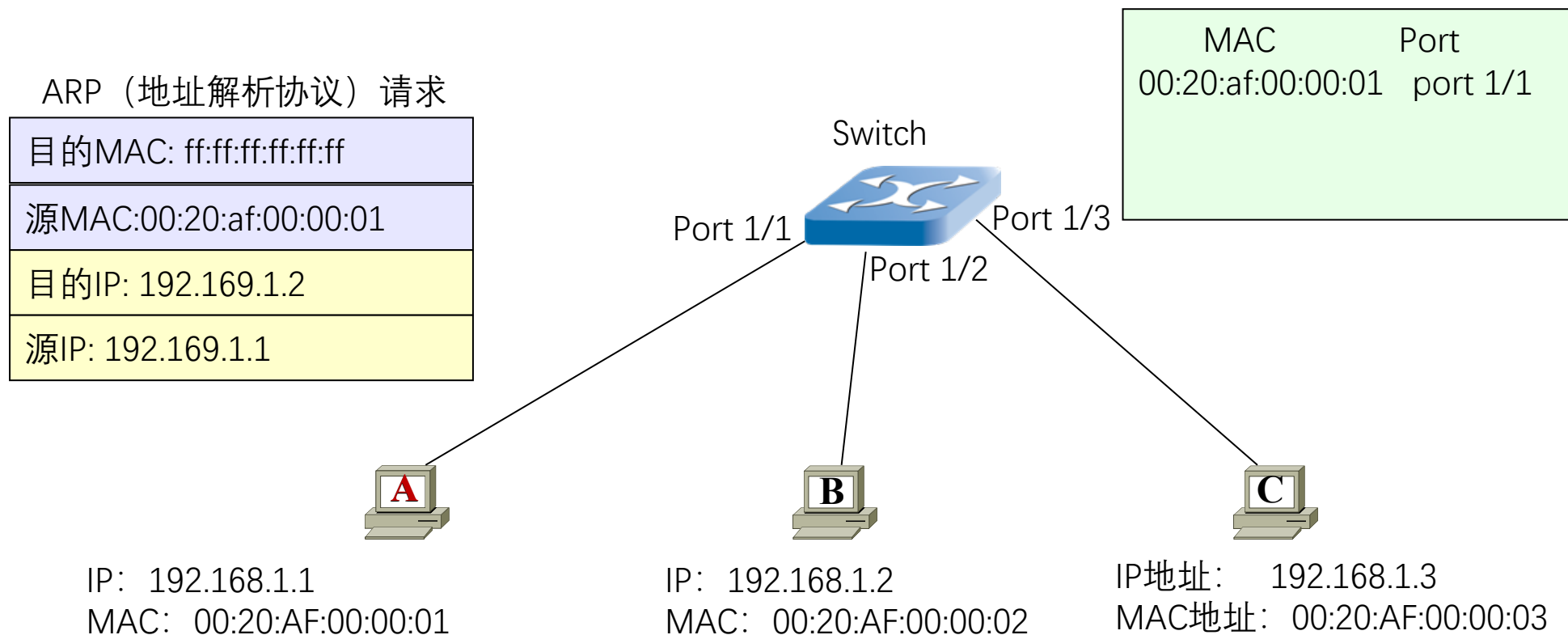




IP包转发



➤ 直接交付：与目的主机在同一个IP子网内



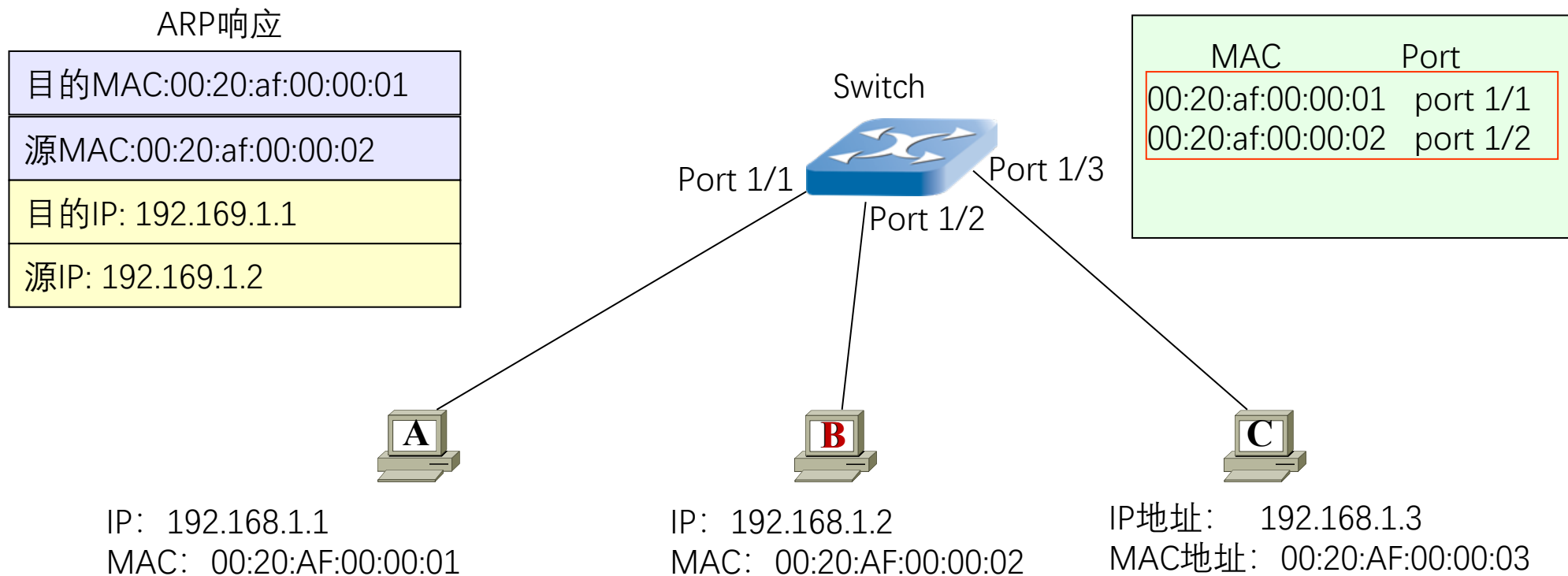
ARP: Address Resolution Protocol



IP包转发



➤ **直接交付：** 与目的主机在同一个IP子网内

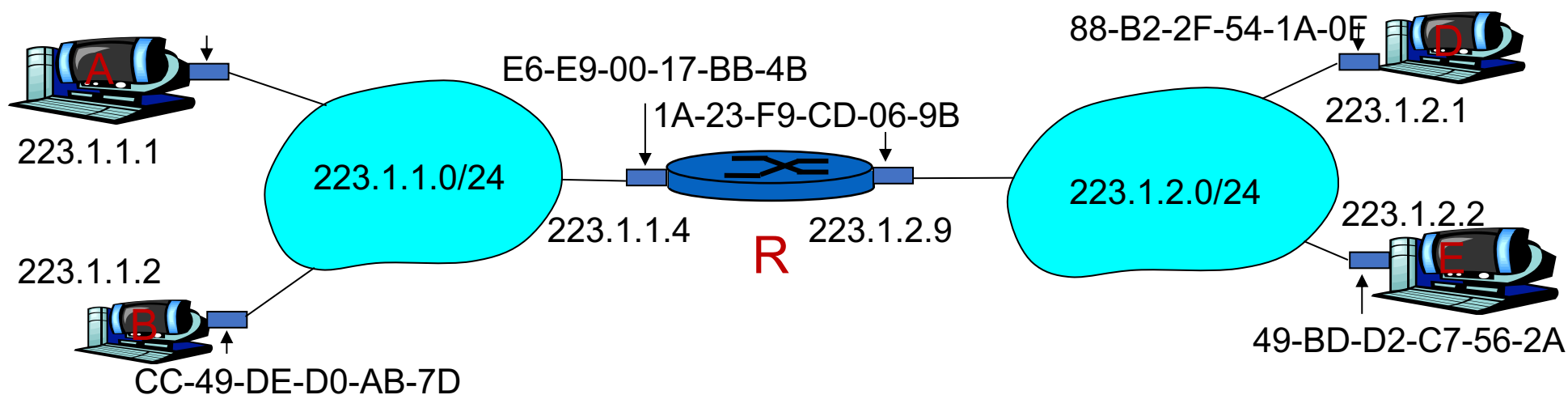




路由到另一个局域网

- A创建IP数据包（源为A、目的为E）
- 在源主机A的路由表中找到路由器R的IP地址223.1.1.4
- A根据R的IP地址223.1.1.4，使用ARP协议获得R的MAC地址
- A创建数据帧（目的地址为R的MAC地址）
- 数据帧中封装A到E的IP数据包
- A发送数据帧，R接收数据帧

例：从A经过R到E

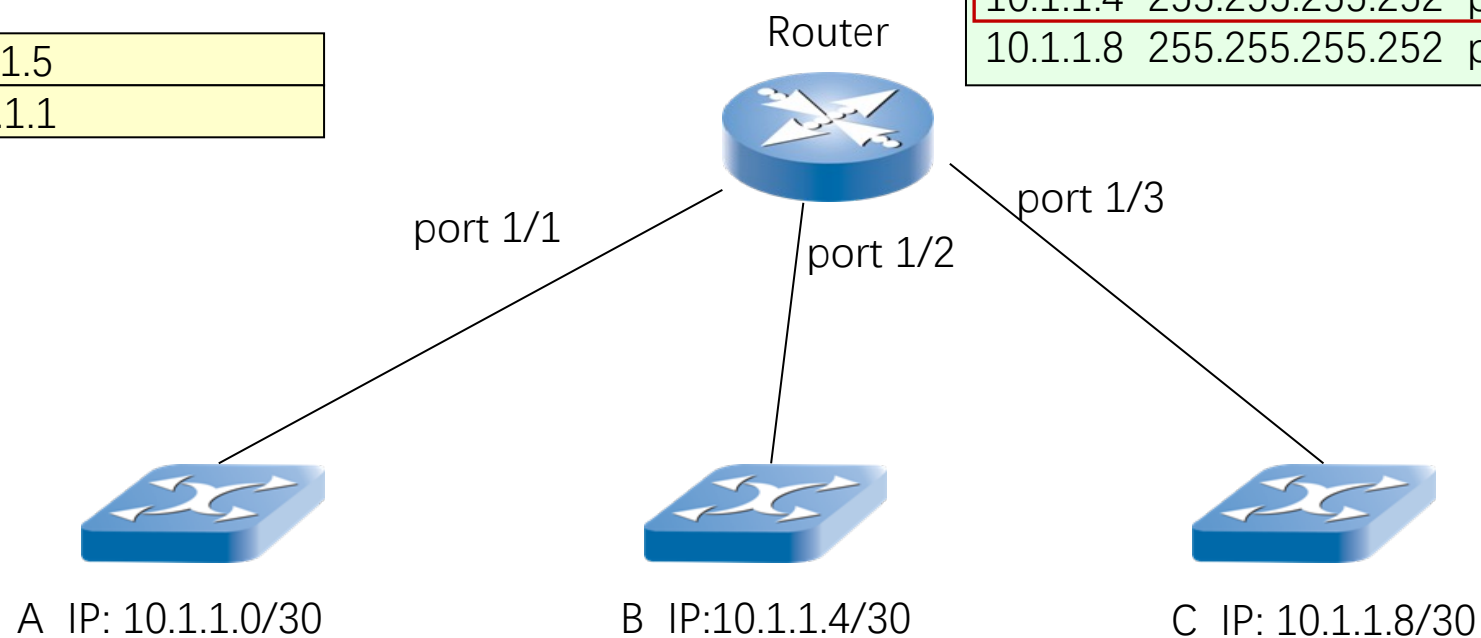


IP包转发

➤ 间接交付：与目的主机不在同一个IP子网内

目的IP: 10.1.1.5
源 IP: 10.1.1.1

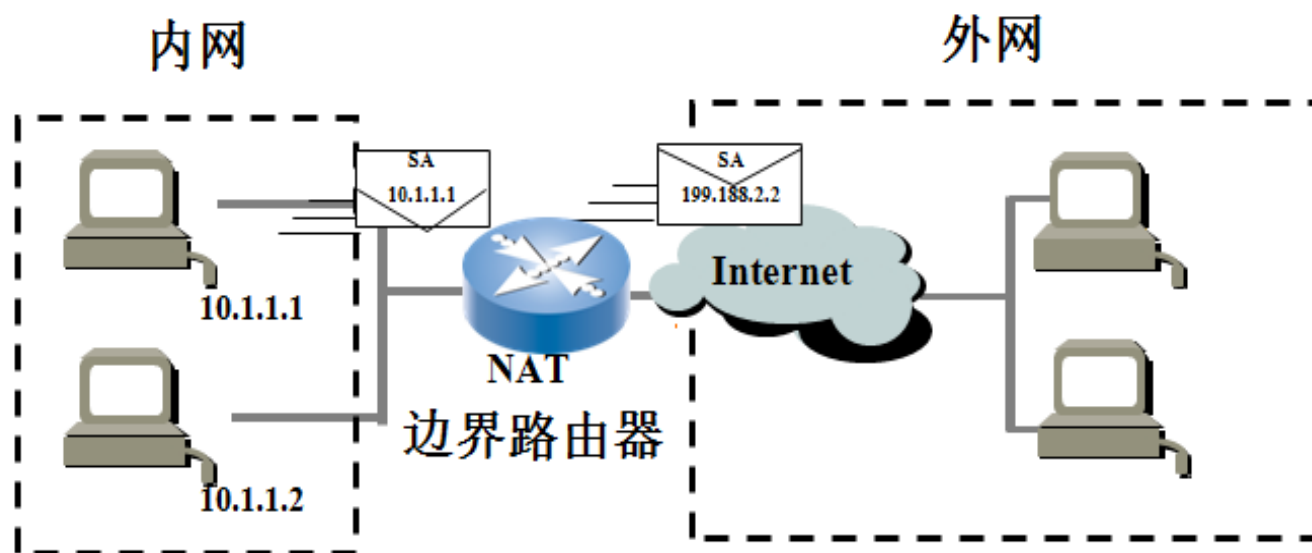
Dest	network	interface
10.1.1.0	255.255.255.252	port 1/1
10.1.1.4	255.255.255.252	port 1/2
10.1.1.8	255.255.255.252	port 1/3





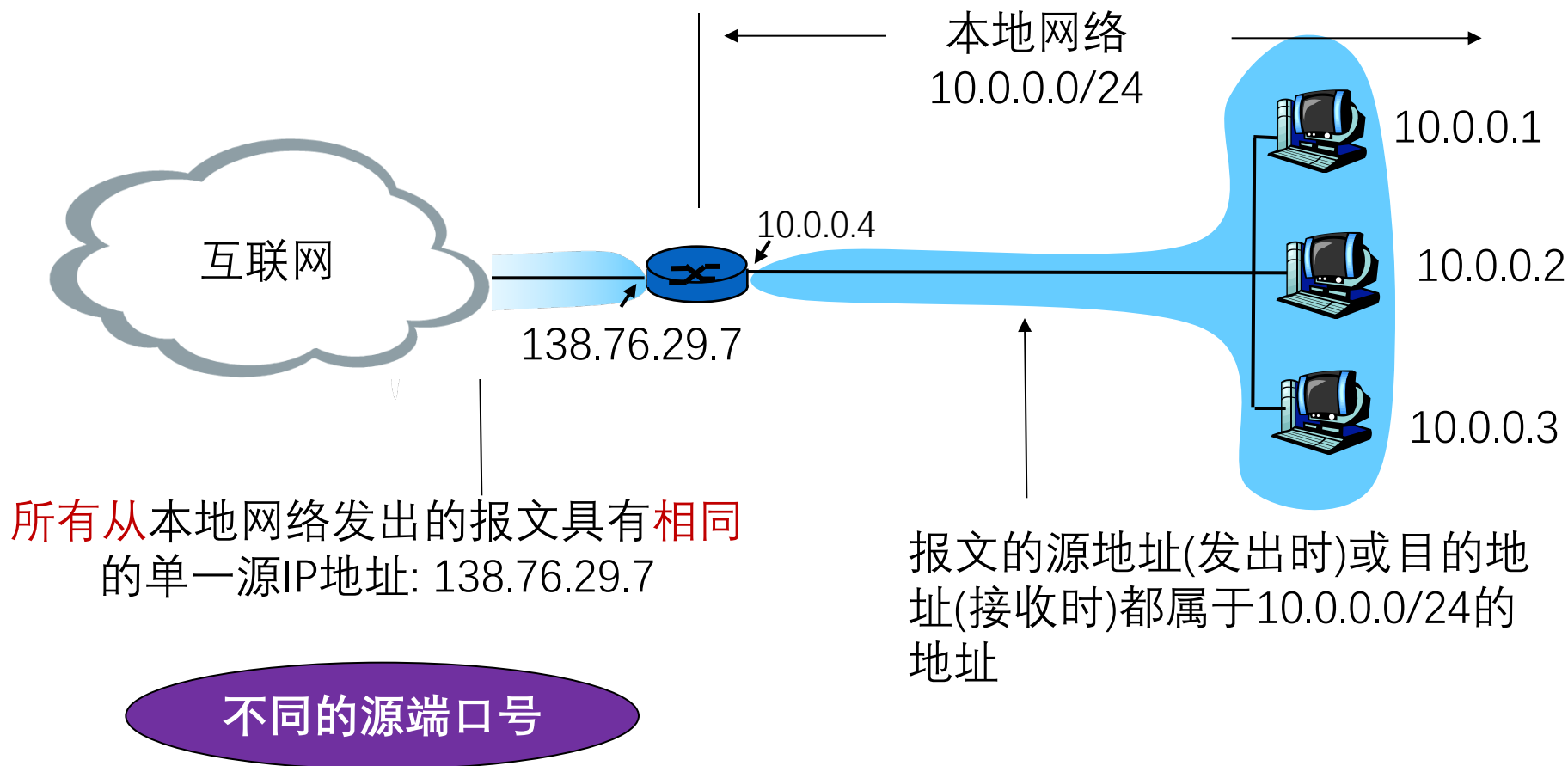
网络地址转换

- 网络地址转换(NAT, Network address translation)用于解决IPv4地址不足的问题, 是一种将私有 (保留) 地址转化为公有IP地址的转换技术
- 私有IP地址:
 - A类地址: 10.0.0.0--10.255.255.255
 - B类地址: 172.16.0.0--172.31.255.555
 - C类地址: 192.168.0.0--192.168.255.255





NAT工作机制





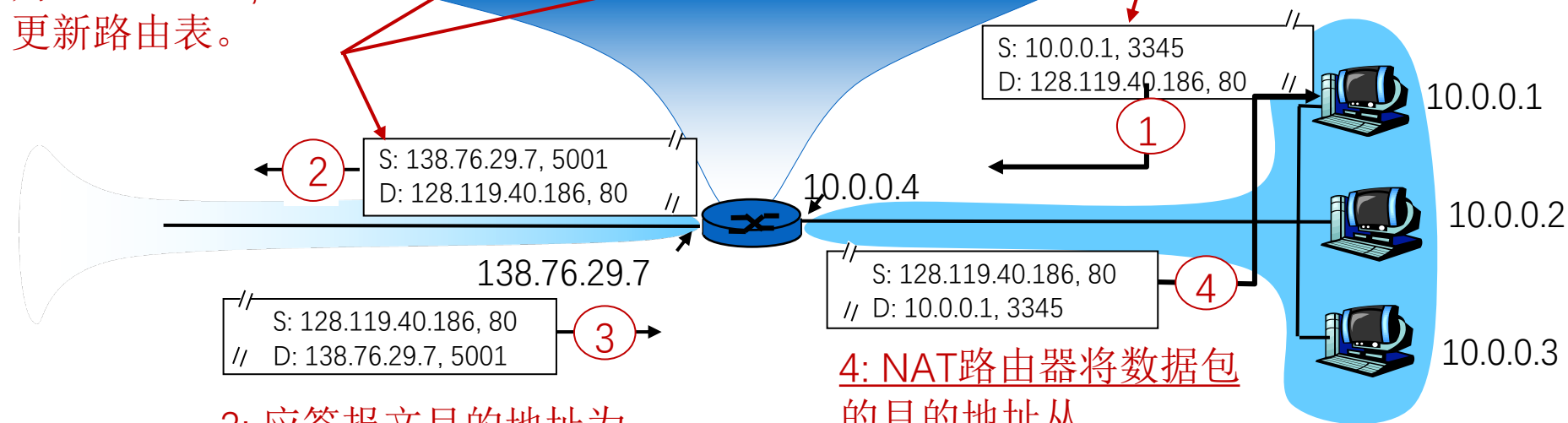
NAT工作机制



2: NAT路由器将数据包的源地址和源端口号从10.0.0.1, 3345改为138.76.29.7, 5001, 更新路由表。

NAT 转换表	
广域网地址及端口	局域网地址及端口
138.76.29.7, 5001	10.0.0.1, 3345
.....	

1: 主机10.0.0.1发送数据包到128.119.40.186, 80



3: 应答报文目的地址为:
138.76.29.7, 5001

4: NAT路由器将数据包的
目的地址从
138.76.29.7, 5001更新
为10.0.0.1, 3345



NAT工作机制



- **出数据包**: 外出数据包用 NAT IP地址(全局), 新port # **替代** 源IP地址(私有), port #
- **NAT转换表**: 每个 (源IP地址, port #)到(NAT IP地址, 新port #) 映射项
- **入数据包**: 对每个入数据包的地址字段用存储在NAT表中的(源IP地址, port #)**替代**对应的 (NAT IP地址, 新port #)



网络地址转换



- NAT根据不同的IP上层协议进行NAT表项管理
 - TCP, UDP, ICMP
- 传输层TCP/UDP拥有16-bit 端口号字段
 - 所以一个WAN侧地址可支持60,000个并行连接
- NAT的优势
 - 节省合法地址, 减少地址冲突
 - 灵活连接Internet
 - 保护局域网的私密性
- 问题或缺点
 - 违反了IP的结构模型, 路由器处理传输层协议
 - 违反了端到端的原则
 - 违反了最基本的协议分层规则

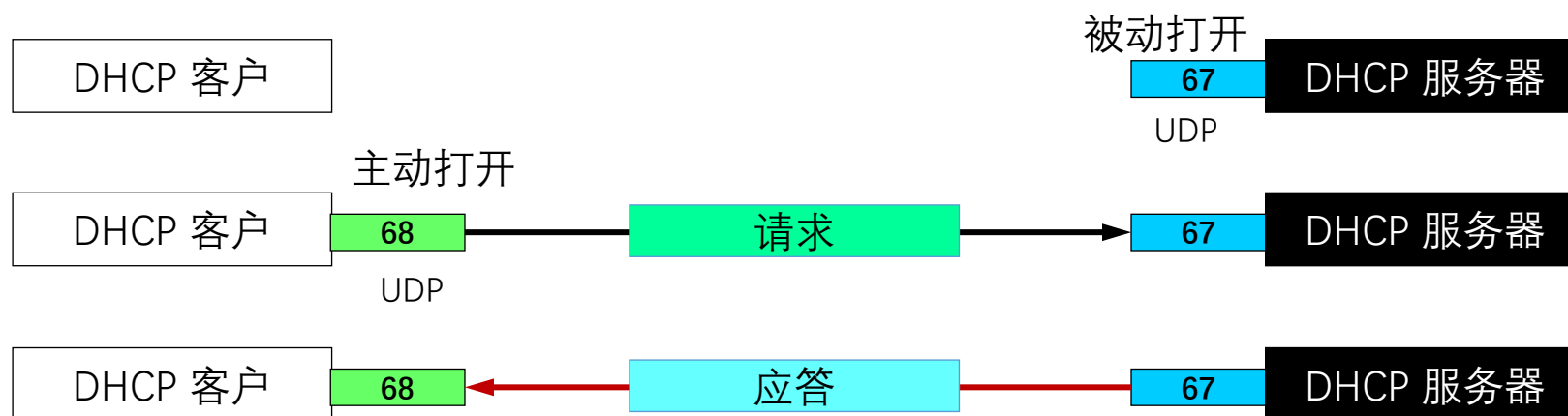


IPv4地址如何获取

- 公有IP地址要求全球唯一
 - ICANN (Internet Corporation for Assigned Names and Numbers) 即互联网名字与编号分配机构向ISP分配, ISP再向所属机构或组织逐级分配
- 静态设定
 - 申请固定IP地址, 手工设定, 如路由器、服务器
- 动态获取
 - 使用DHCP协议或其他动态配置协议
 - 当主机加入IP网络, 允许主机从DHCP服务器动态获取IP地址
 - 可以有效利用IP地址, 方便移动主机的地址获取

DHCP动态主机配置协议

- DHCP (Dynamic Host Configuration Protocol): **动态主机配置协议**
 - 当主机加入IP网络, 允许主机从DHCP服务器动态获取IP地址
 - 可以有效利用IP地址, 方便移动主机的地址获取
- 工作模式: **客服/服务器模式 (C/S)**
 - 基于 UDP 工作, 服务器运行在 67 号端口, 客户端运行在 68 号端口



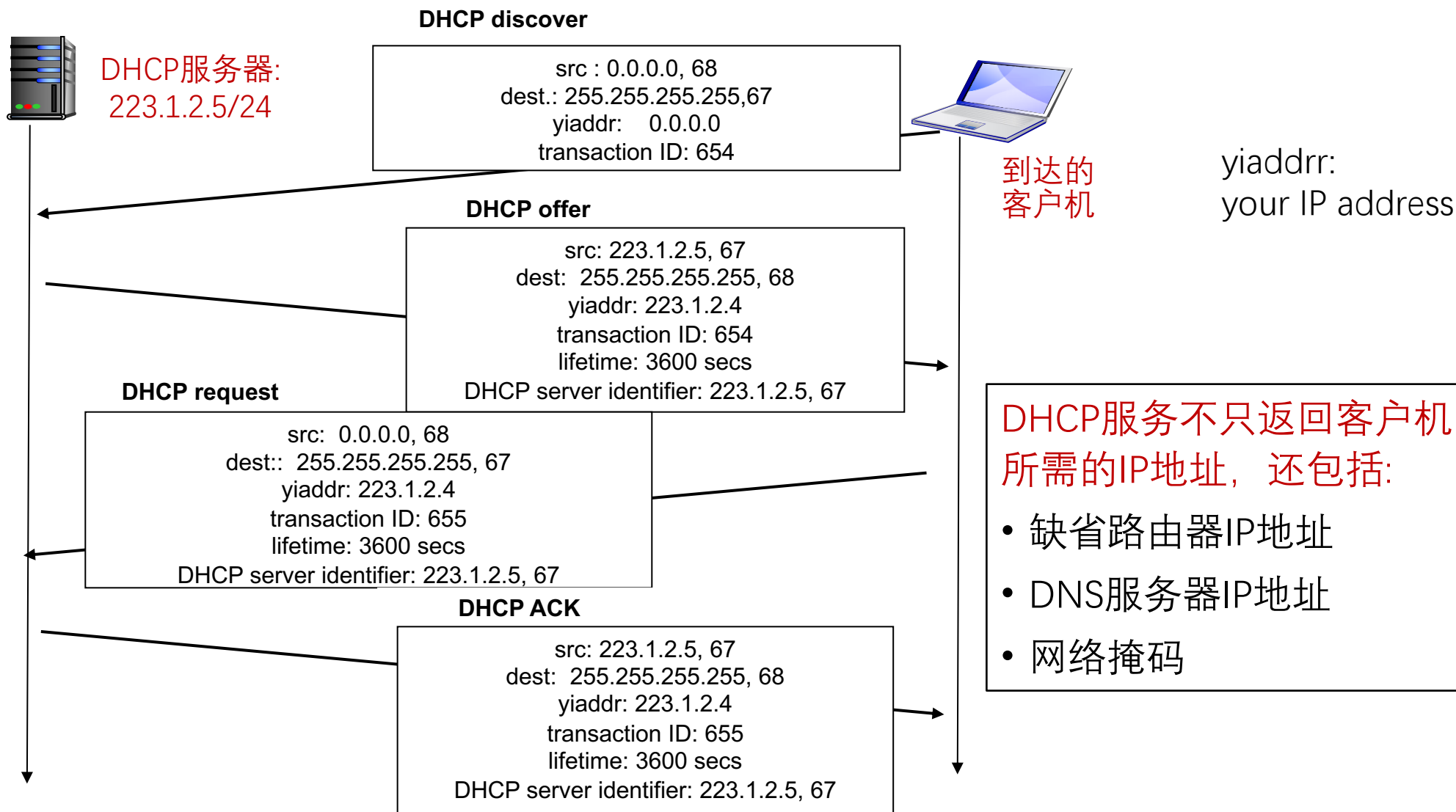


DHCP 工作过程

- DHCP 客户从UDP端口68以广播形式向服务器发送发现报文 (DHCPDISCOVER)
- DHCP 服务器以广播形式发出提供报文 (DHCPOFFER)
- DHCP 客户从多个DHCP服务器中选择一个, 并向其以广播形式发送DHCP请求报文 (DHCPREQUEST)
- 被选择的DHCP服务器以广播形式发送确认报文 (DHCPACK)



DHCP 工作过程





ICMP协议



➤ ICMP: 互联网控制报文协议

- ICMP 允许主机或路由器报告差错情况和提供有关异常情况的报告
- 由主机和路由器用于网络层信息的通信
- ICMP 报文携带在IP 数据包中： IP上层协议号为1

➤ ICMP报文类型

- ICMP 差错报告报文
 - 终点不可达：不可达主机、不可达网络，无效端口、协议
- ICMP 询问报文
 - 回送请求/回答 (ping使用)



Ping和ICMP



➤PING (Packet InterNet Groper)

- PING 用来测试两个主机之间的连通性
- PING 使用了 ICMP 回送请求与回送回答报文

```
C:\>ping www.baidu.com
```

```
正在 Ping www.a.shifen.com [110.242.68.4] 具有 32 字节的数据:
```

```
来自 110.242.68.4 的回复: 字节=32 时间=32ms TTL=53
```

```
来自 110.242.68.4 的回复: 字节=32 时间=29ms TTL=53
```

```
来自 110.242.68.4 的回复: 字节=32 时间=29ms TTL=53
```

```
来自 110.242.68.4 的回复: 字节=32 时间=31ms TTL=53
```

```
110.242.68.4 的 Ping 统计信息:
```

```
数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
```

```
往返行程的估计时间(以毫秒为单位):
```

```
最短 = 29ms, 最长 = 32ms, 平均 = 30ms
```

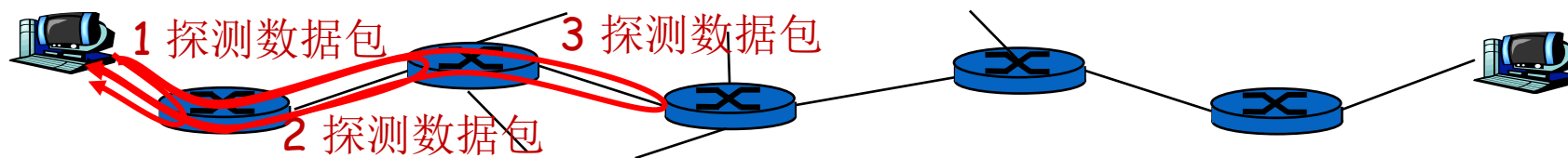
连通性

往返时延



Traceroute和ICMP

➤ 如何知道整个路径上路由器的地址？ 使用TraceRT命令



➤ 源向目的地发送一系列UDP段(不可能的端口号)

- 第一个 TTL = 1
- 第二个 TTL = 2, 等

➤ 当第n个数据包到达第n个路由器:

- 路由器丢弃数据包
- 并向源发送一个ICMP报文
- 报文的源IP地址就是该路由器的IP地址



Traceroute和ICMP



C:\>tracert www.taobao.com

通过最多 30 个跃点跟踪

到 www.taobao.com.danuoyi.tbcache.com [27.211.197.171] 的
路由:

1	3 ms	2 ms	4 ms	192.168.3.1
2	3 ms	6 ms	3 ms	SMBSHARE [192.168.1.1]
3	6 ms	9 ms	5 ms	27.215.136.1
4	6 ms	11 ms	7 ms	61.162.199.89
5	7 ms	18 ms	21 ms	61.162.199.9
6	23 ms	29 ms	22 ms	61.156.223.69
7	28 ms	31 ms	20 ms	112.230.160.54
8	19 ms	27 ms	36 ms	60.208.64.230
9	15 ms	15 ms	16 ms	119.164.254.86
10	19 ms	13 ms	13 ms	27.211.197.171

跟踪完成。

- 当源收到ICMP报文，计算 RTT
- Tracert针对同一RTT值执行上述过程3次

停止条件

- UDP段最终到达目的主机，目的主机返回ICMP“端口不可达”数据包(类型3, 编码3)。当源得到该ICMP，停止



本章内容



5.1 网络层服务

5.2 Internet 网际协议

5.3 路由算法

5.4 Internet路由协议

5.5 路由器工作原理

5.6 拥塞控制算法

5.7 服务质量

5.8 三层交换与VPN

5.9 IPv6技术

1. 优化原则
2. 最短路径算法
3. 距离向量路由
4. 链路状态路由
5. 层次路由



路由算法



- 路由算法须满足的特性：
 - 正确性
 - 简单性
 - 鲁棒性
 - 稳定性
 - 公平性
- 根据路由算法是否随网络的通信量或拓扑自适应划分
 - 静态路由选择策略（非自适应路由选择）
 - 动态路由选择策略（自适应路由选择）

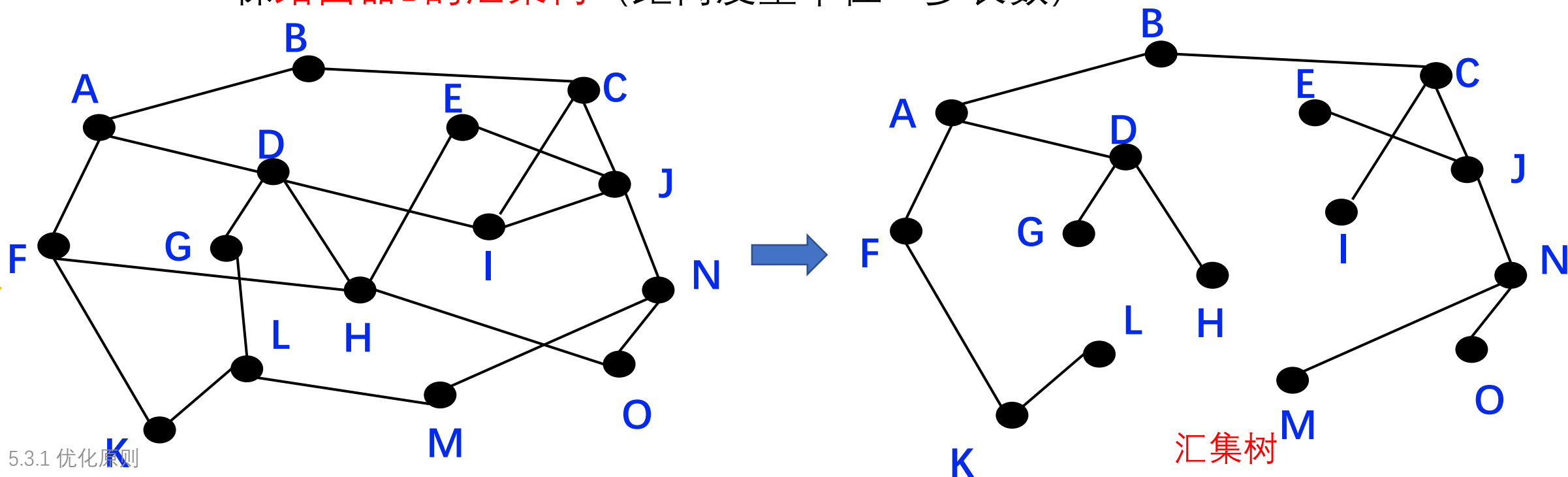


优化原则

➤ 汇集树(Sink Tree)

汇集树不是唯一的

- 所有的源节点到一个指定目标节点的最优路径的集合构成一棵以目标节点为根的树
- 一棵**路由器B的汇集树** (距离度量单位: 步长数)





最短路径算法



➤ 定义

- 用于计算一个节点到其他所有节点的最短路径，主要特点是以起始点为中心向外逐层扩展，直到扩展到终点为止

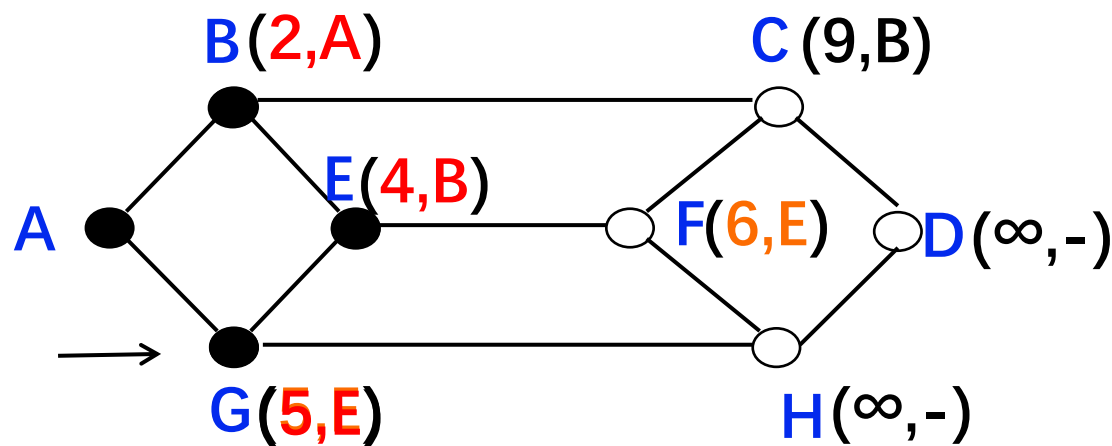
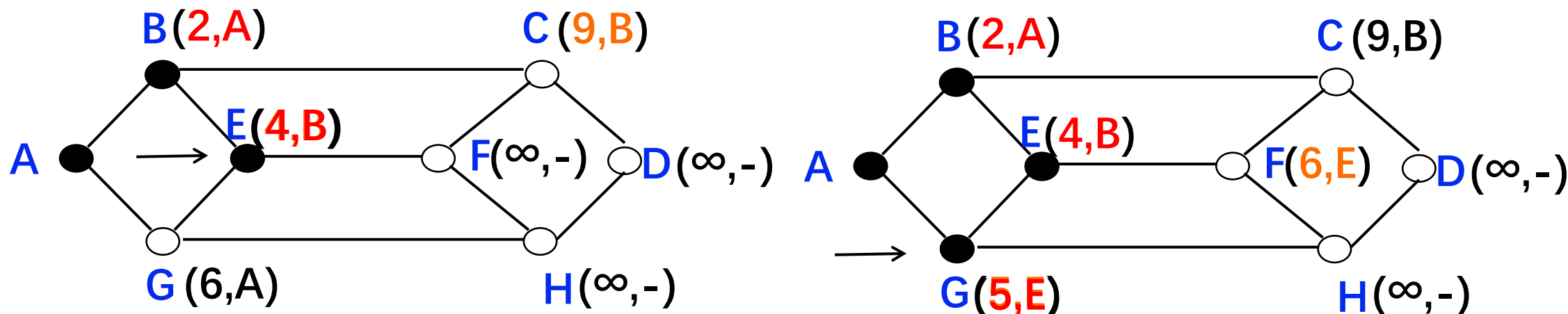
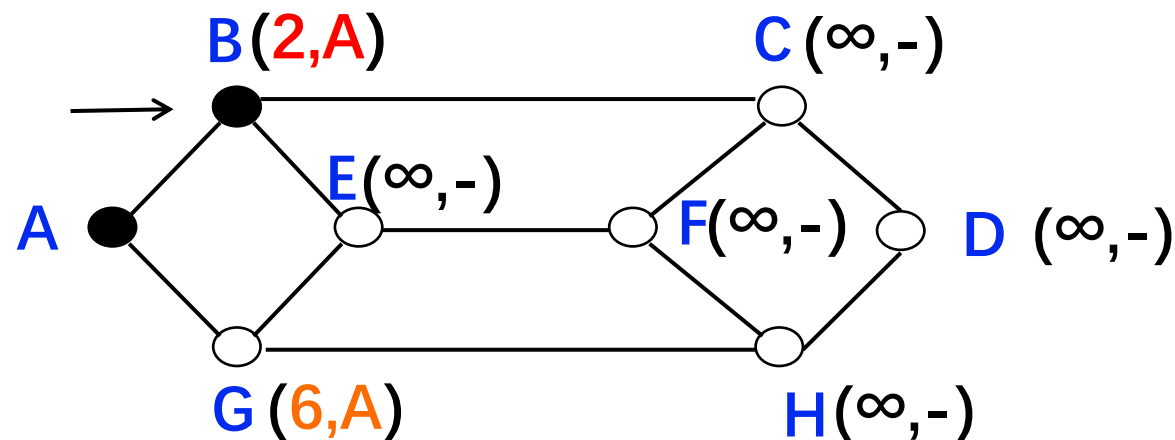
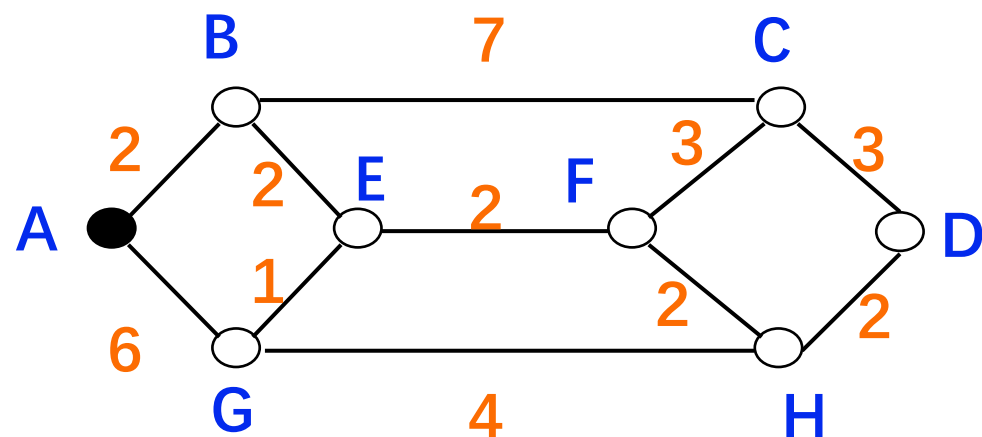
➤ Dijkstra算法思想

- 建立网络图
 - 节点表示路由器
 - 边表示通信线路/链路
 - 链路代价表示链路上的距离、信道宽度或通信开销等参数
- 根据算法在网络图上为某一对路由器找到最短路径



最短路径算法

➤ 具体例子 (A到D的最短路径过程)





距离向量路由

➤ Bellman-Ford 方程

- 假设 $D_x(y)$ 是从 x 到 y 最小代价路径的代价值
- 则: $D_x(y) = \min\{c(x, m) + D_m(y)\}$
其中 m 为 x 的邻居, $c(x, m)$ 为 m 到 x 的距离

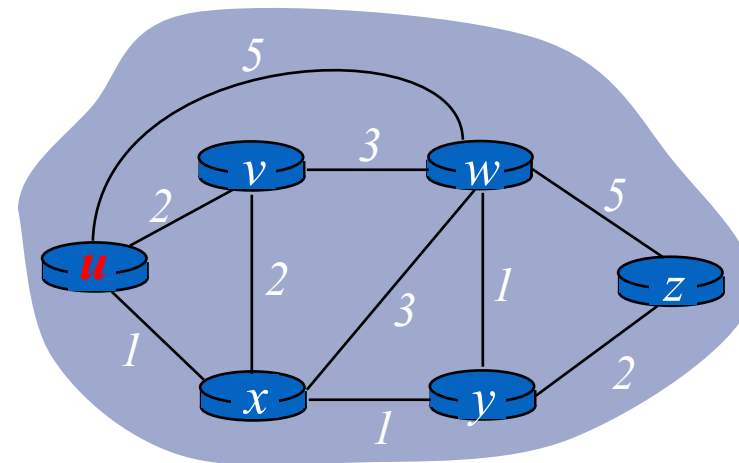
• 示例:

- 已知 $D_v(z) = 5, D_x(z) = 3, D_w(z) = 3$
- $D_u(z) = \min\{c(u, v) + D_v(z), \mathbf{c(u, x) + D_x(z)}, c(u, w) + D_w(z)\}$

$$= \min\{2 + 5, \mathbf{1 + 3}, 5 + 3\}$$

$$= \mathbf{4}$$

- 计算出代价最小的节点, 也就得到了对应转发项从 u 去往 z , 应从 x 转发



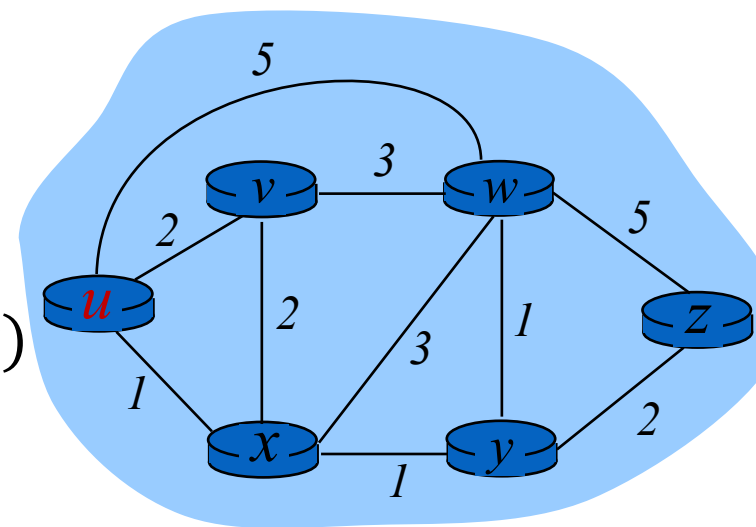


距离向量路由

- 距离向量 (Distance Vector) 算法基本思想:
 - 每个节点周期性地向邻居发送它自己到某些节点的距离向量;
 - 当节点 x 接收到来自邻居的新距离向量估计, 它使用Bellman-Ford方程更新其自己的距离向量:

$$D_x(y) \leftarrow \min_v \{c(x, v) + D_v(y)\} \quad \text{for each node } y \in N$$

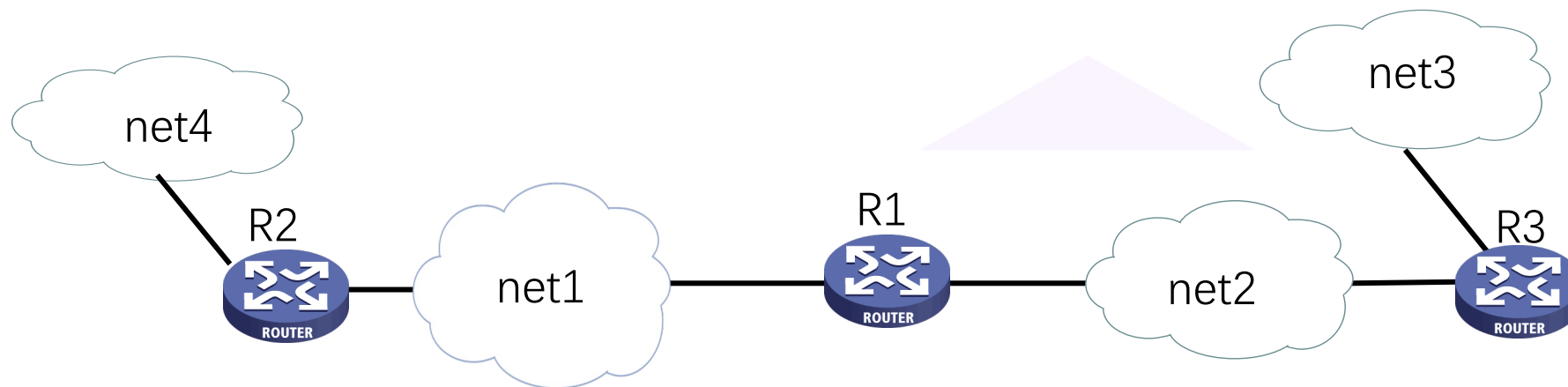
- 上述过程迭代执行, $D_x(y)$ 收敛为实际最小费用 $d_x(y)$



- 距离向量算法特点: 迭代的、分布式的
 - 每次本地迭代由下列引起: 本地链路费用改变、邻居更新报文
 - 分布式: 各节点依次计算, 相互依赖

距离向量路由

- 路由器启动时初始化自己的路由表
 - 初始路由表包含所有直接相连的网络路径，距离均为0



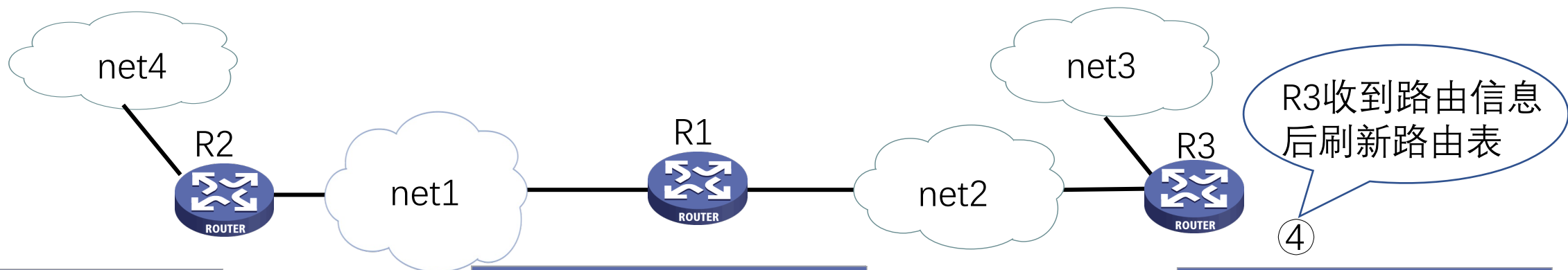
目的网络	距离	下一跳
net1	0	--
net4	0	--

目的网络	距离	下一跳
net1	0	--
net2	0	--

目的网络	距离	下一跳
net2	0	--
net3	0	--

距离向量路由

- 路由器周期性地向其相邻路由器广播自己知道的路由信息
- 相邻路由器可以根据收到的路由信息修改和刷新自己的路由表



目的网络	距离	下一跳
net1	0	--
net4	0	--

①
R2将路由信息发送给R1

目的网络	距离	下一跳
net1	0	--
net2	0	--
net4	1	R2

②
R1收到路由信息后刷新路由表

③
R1将路由信息发送给R3

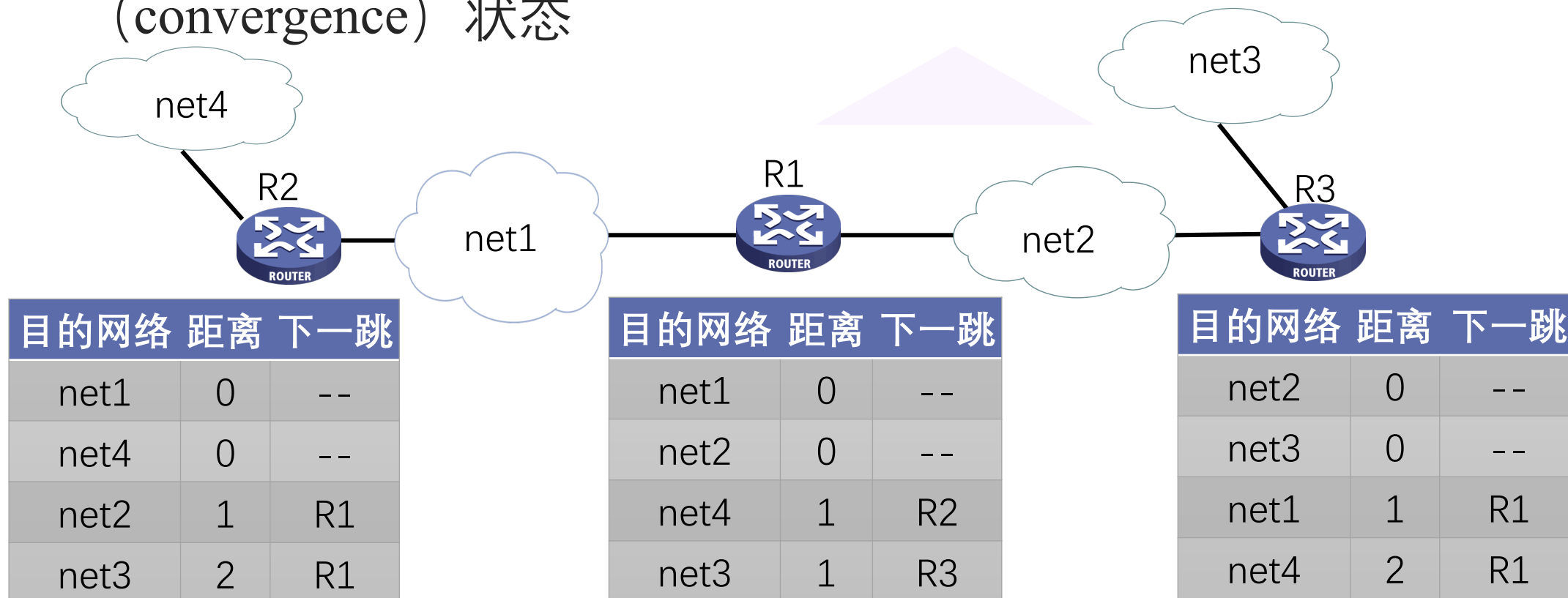
目的网络	距离	下一跳
net2	0	--
net3	0	--
net1	1	R1
net4	2	R1

R3收到路由信息后刷新路由表

④

距离向量路由

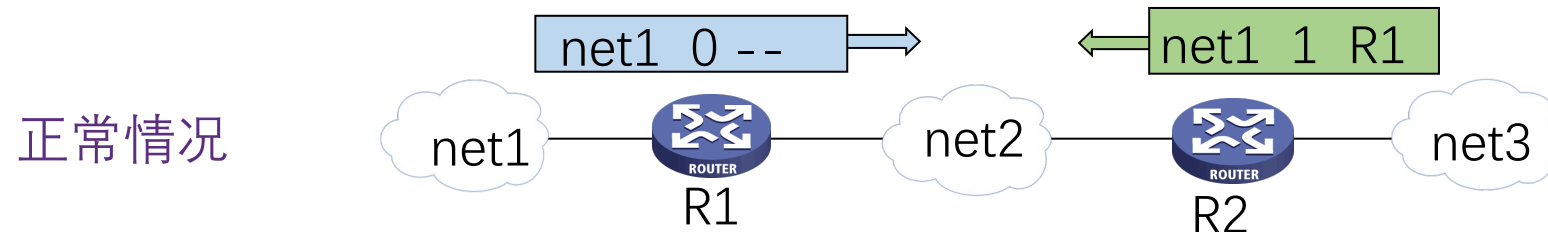
- 路由器经过若干次更新后，最终都会知道到达所有网络的最短距离
- 所有的路由器都得到正确的路由选择信息时网络进入“收敛” (convergence) 状态





距离向量路由

➤ 计数到无穷问题 (The Count-to-Infinity Problem)

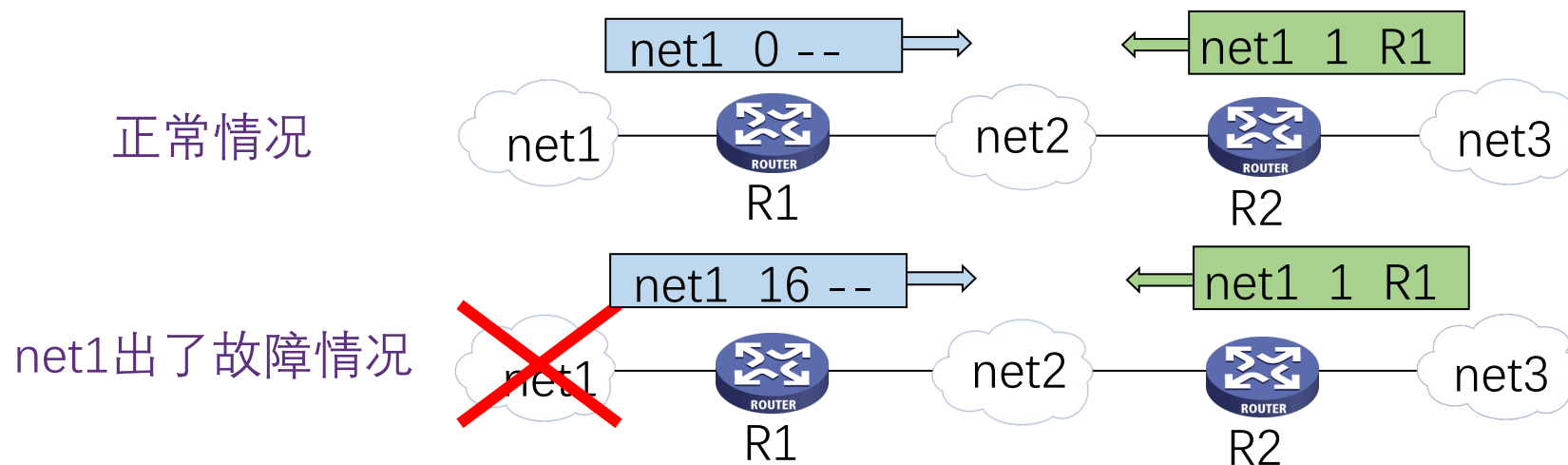


R1 说：“我到net1 的距离是 0，是直接交付。”

R2 说：“我到net1 的距离是 1，经过 R1。”

距离向量路由

➤ 计数到无穷问题 (The Count-to-Infinity Problem)



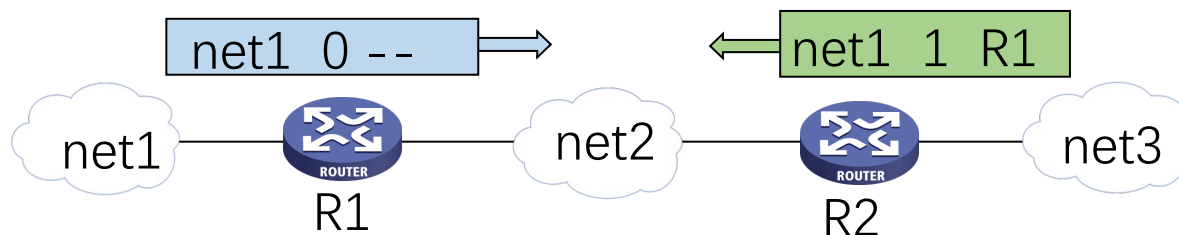
R1 说：“我到net1 的距离是 16
(表示无法到达)，是直接交付。”

但 R2 在收到 R1 的更新报文之前，还发送原来的报文，因为这时 R2 并不知道net1出了故障。

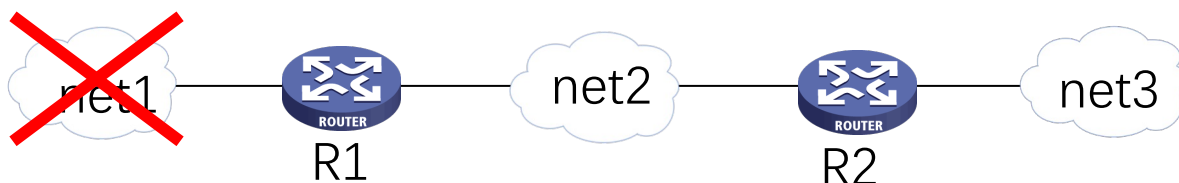
距离向量路由

➤ 计数到无穷问题 (The Count-to-Infinity Problem)

正常情况



net1出了故障情况

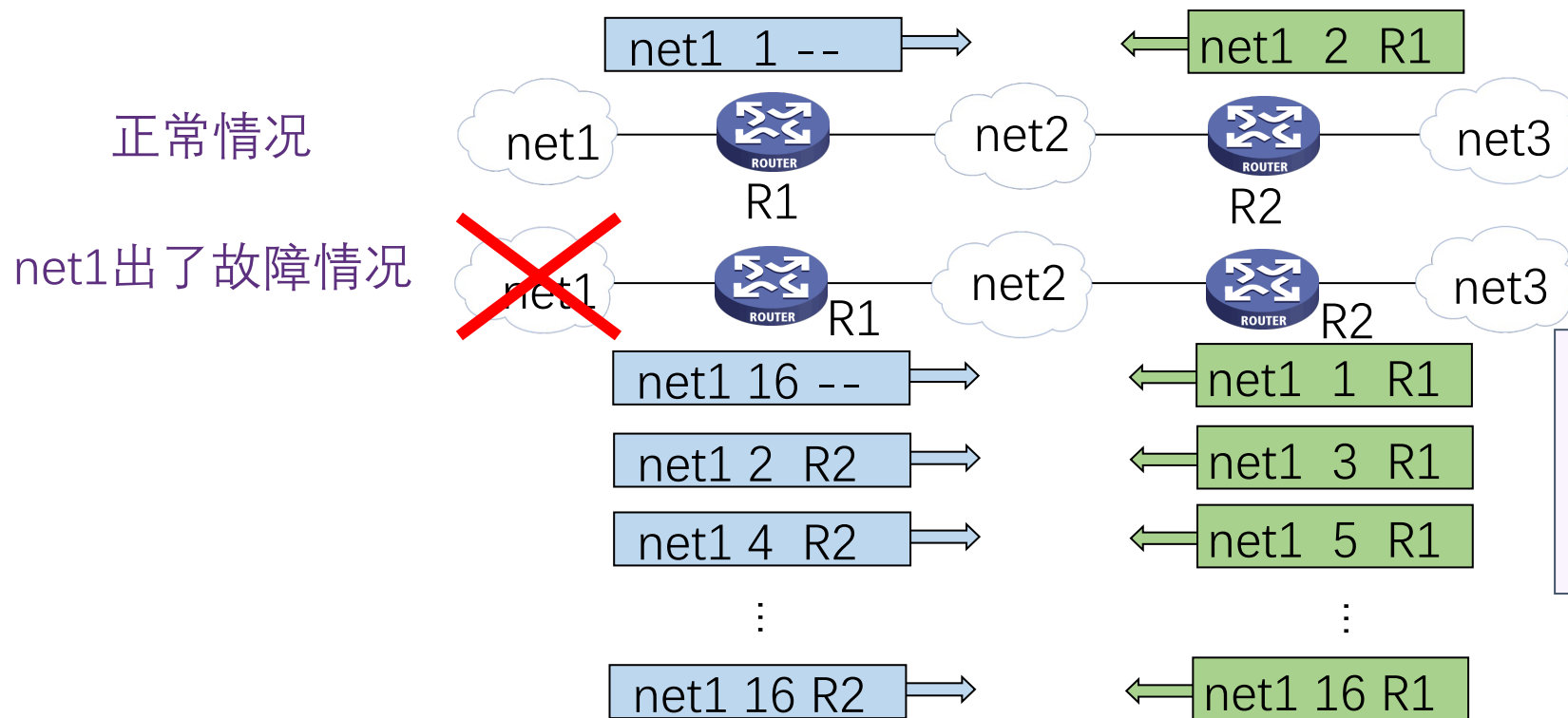


R2 以后又更新自己的路由表为“net1, 3, R1”，表明“我到net 1 距离是 3，下一跳经过 R1”。



距离向量路由

➤ 计数到无穷问题 (The Count-to-Infinity Problem)



这样不断更新下去，直到 R1 和 R2 到net 1 的距离都增大到16 时， R1 和 R2才知道net 1 是不可达的。

➤ 好消息传播快， 坏消息传播慢， 是距离向量路由的一个主要缺点



链路状态路由

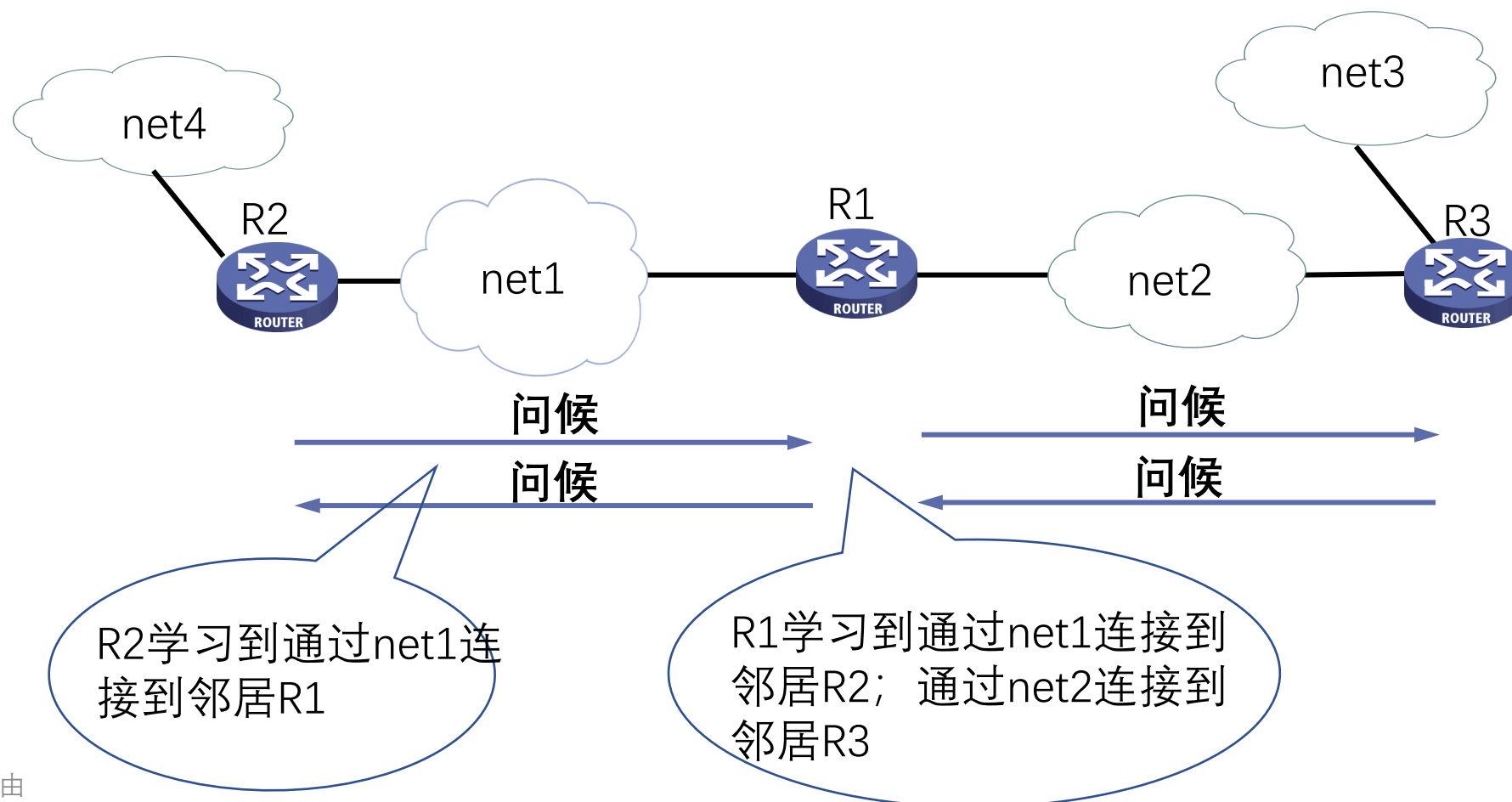


- 链路状态（Link State）路由可分为五个部分：
- 1. 发现邻居，了解他们的网络地址；
 - 2. 设置到每个邻居的成本度量；
 - 3. 构造一个数据包，数据包中包含刚收到的所有信息；
 - 4. 将此数据包发送给其他的路由器；
 - 5. 计算到其他路由器的最短路径。



链路状态路由

- 1.发现邻居，了解他们的网络地址





链路状态路由



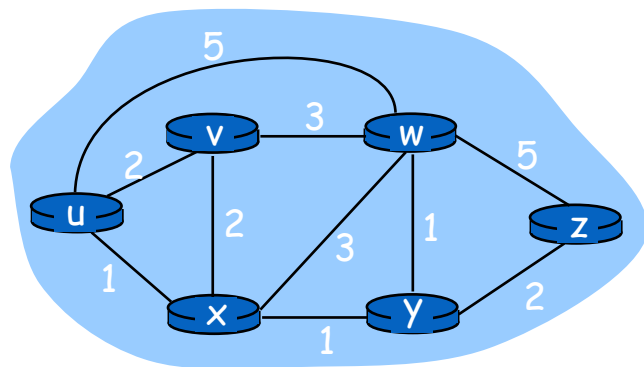
➤ 2. 设置到每个邻居的成本度量

- 开销/度量/代价：
 - 自动发现设置或人工配置
 - 度量：带宽、跳数、延迟、负载、可靠性等
- 常用度量：链路带宽（反比）
 - 例如：1-Gbps以太网的代价为1，100-Mbps以太网的代价为10
- 可选度量：延迟
 - 发送一个echo包，另一端立即回送一个应答
 - 通过测量往返时间RTT，可以获得一个合理的延迟估计值



链路状态路由

- 3.构造一个数据包，数据包中包含刚收到的所有信息
 - 构造链路状态数据包 (link state packet, LSP)
 - 发送方标识
 - 序列号
 - 生存期
 - 邻居列表



u		
Seq.		
Age		
v	2	
x	1	
w	5	

v		
Seq.		
Age		
u	2	
x	2	
w	3	

x		
Seq.		
Age		
u	1	
v	2	
w	3	
y	1	

y		
Seq.		
Age		
x	1	
w	1	
z	2	

w		
Seq.		
Age		
u	5	
v	3	
x	3	
y	1	
z	5	

z		
Seq.		
Age		
w	5	
y	2	



链路状态路由

- 4.将LSP数据包发送给其他的路由器
 - 每个LSP数据包包含一个序列号，且递增
 - 路由器记录所收到的所有（源路由器、序列号）对
 - 当一个新数据包到达时，路由器根据记录判断：
 - 如果是新数据包，洪泛广播
 - 如果是重复数据包，丢弃
 - 如果是过时数据包，拒绝



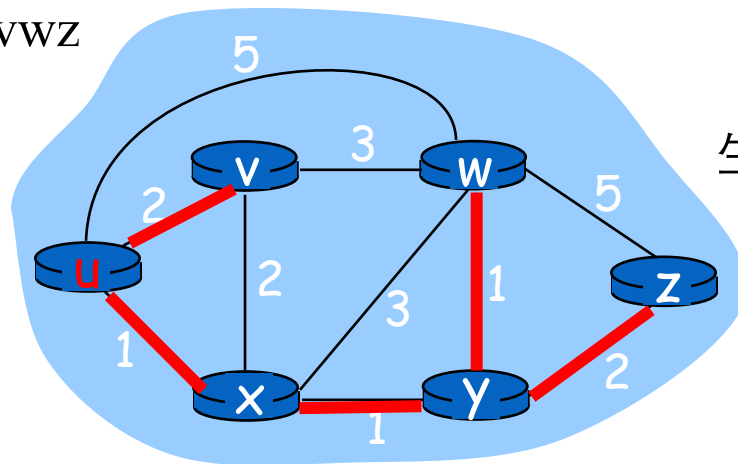
链路状态路由

➤ 5.计算到其他路由器的最短路径： Dijkstra算法示例

- $D(k)$: 从计算节点到目的节点k当前路径代价
- $p(k)$: 从计算节点到目的节点k的路径中k节点的前继节点

步骤	集合N	$D(v),p(v)$	$D(w),p(w)$	$D(x),p(x)$	$D(y),p(y)$	$D(z),p(z)$
0	u	2,u	5,u	1,u	∞	∞
➔ 1	ux	2,u	4,x		2,x	∞
➔ 2	uxy	2,u	3,y			4,y
➔ 3	uxyv		3,y			4,y
➔ 4	uxyvw					4,y
➔ 5	uxyvwz					

生成最短路径树



生成路由表

目的	下一跳	代价
v	v	2
w	x	3
x	x	1
y	x	2
z	x	4



链路状态路由

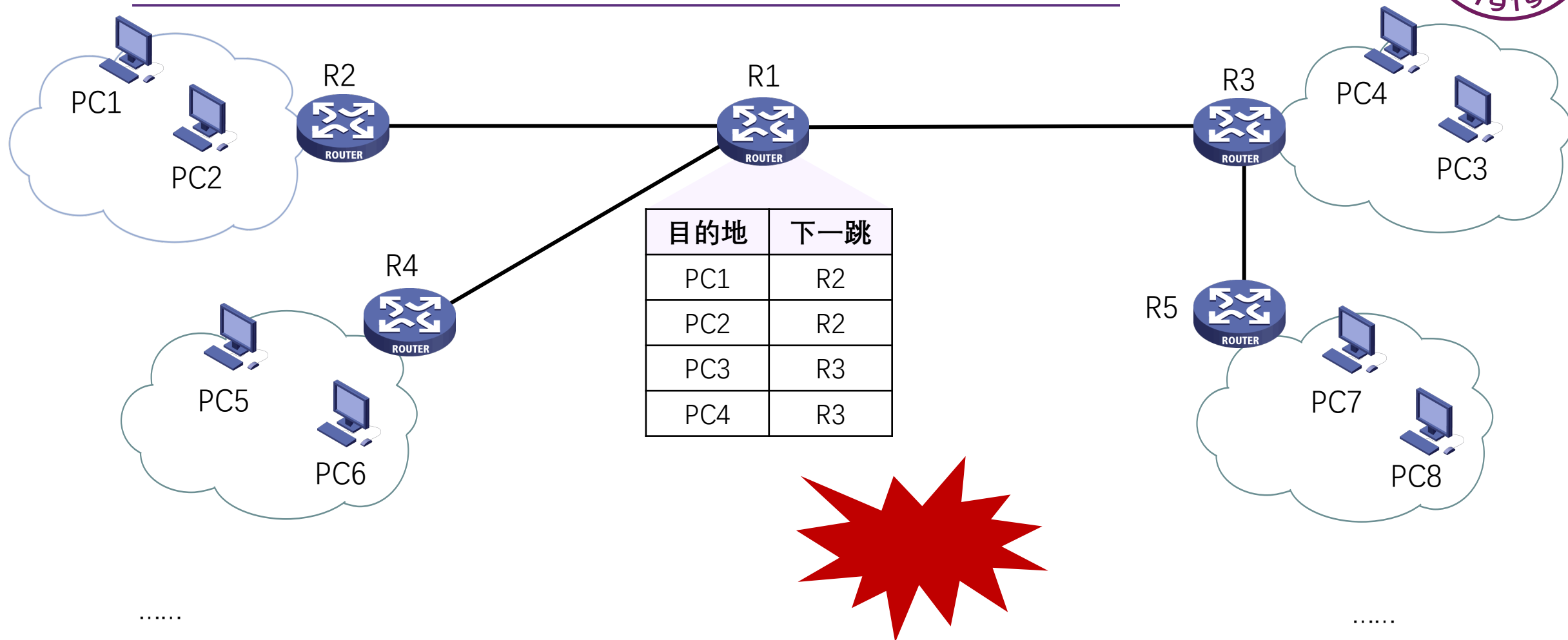


➤ 距离向量和链路状态算法比较：

- 网络状态信息交换的范围
 - 距离向量算法：邻居间交换
 - 链路状态算法：全网扩散
- 网络状态信息的可靠性
 - 距离向量算法：部分道听途说
 - 链路状态算法：自己测量
- 健壮性：
 - 距离向量算法：计算结果传递，健壮性差
 - 链路状态算法：各自计算，健壮性好
- 收敛速度：
 - 距离向量算法：慢,可能有计数到无穷问题
 - 链路状态算法：快



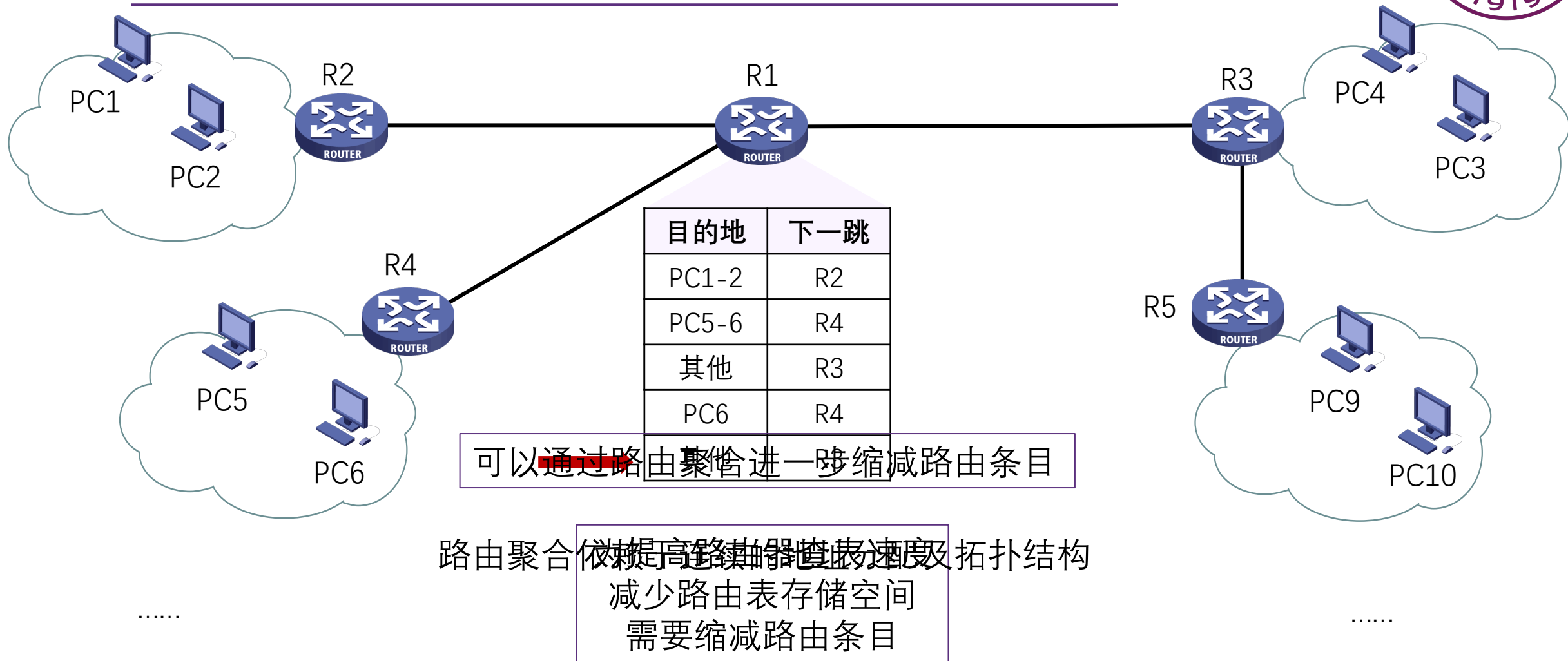
层次路由-产生原因



过于庞大的路由表存储、查找困难，路由信息交互开销高



层次路由-产生原因





层次路由-产生原因



➤ 现实情况：

- 地址分配往往是随机的，难以进行高效的路由聚合
- 每个网络的网络管理员有自己的管理方法和思路，并不希望每个路由器都干涉本网络内部的地址分配等问题

➤ 层次路由可以解决：

- 网络扩展性问题：当网络扩大时，控制路由表条目和路由表存储空间的增长
- 管理的自治问题：网络管理员可以控制和管理自己网络的路由



层次路由-基本思路

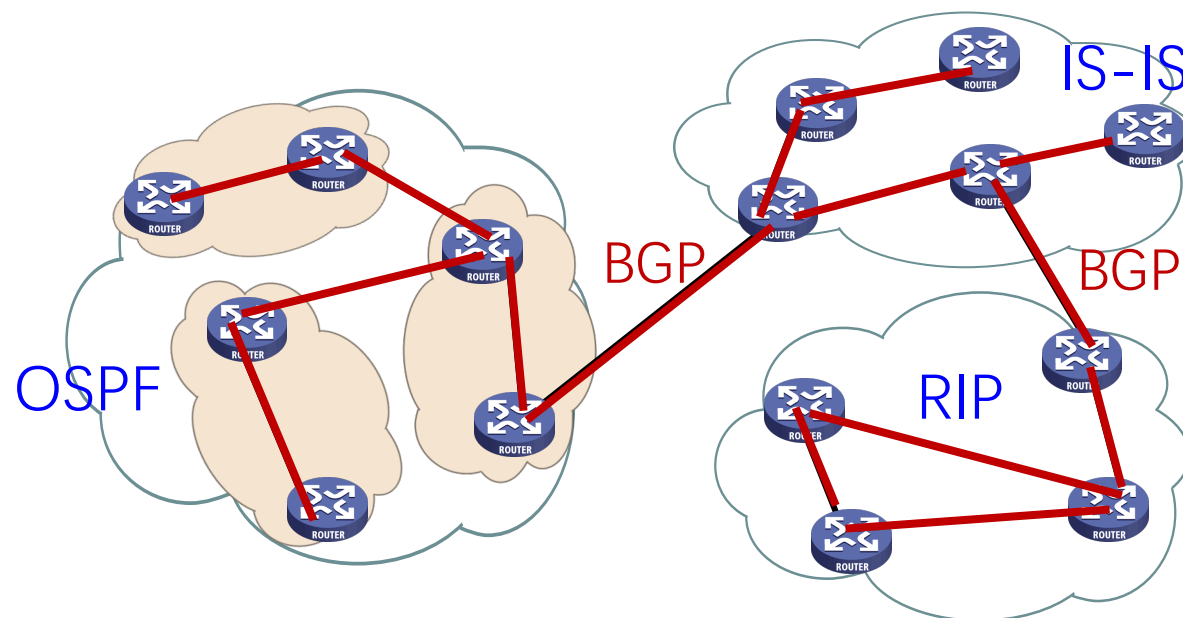


- 互联网由大量不同的网络互连，每个管理机构控制的网络是自治的
- 自治系统（AS, Autonomous System）
 - 一个管理机构控制之下的网络
 - 一个AS内部通常使用相同的路由算法/路由协议，使用统一的路由度量（跳数、带宽、时延 ...）
 - 不同的AS可以使用不同的路由算法/路由协议
 - 每个AS有一个全球唯一的ID号：AS ID
- 自治系统内还可以进一步划分层次：私有自治系统或区域



层次路由-基本思路

- 自治系统内部使用内部网关路由协议，Interior Gateway Protocols (IGP)
 - 每个自治系统域内路由算法可不同
 - 典型IGP协议：OSPF, RIP, IS-IS, IGRP, EIGRP……
- 自治系统之间使用外部网关路由协议，Exterior Gateway Protocols (EGP)
 - 各自治系统域之间的路由需统一
 - 典型EGP协议：BGP





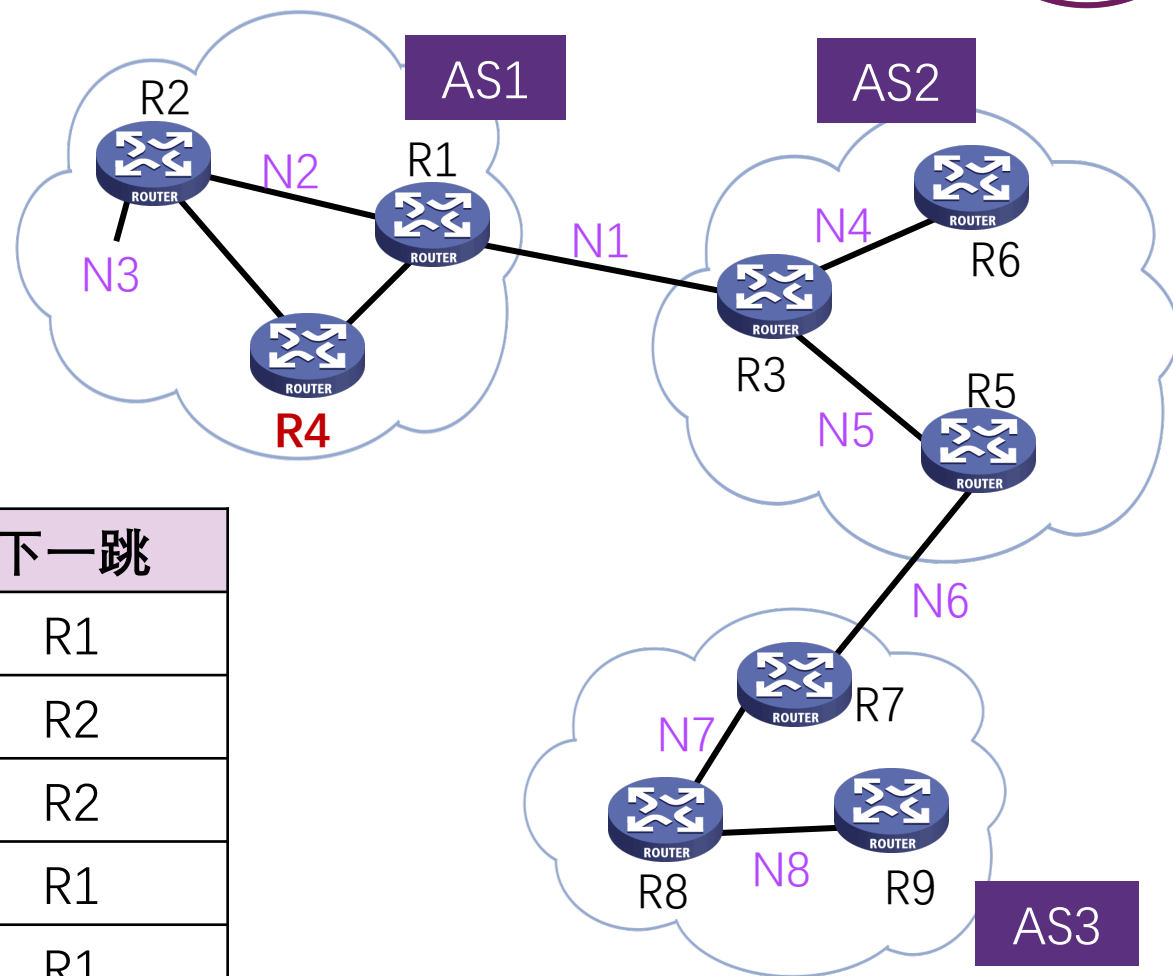
层次路由-效果

路由不分层的情况下
R4的路由表

目的地	下一跳
N1	R1
N2	R2
N3	R2
N4	R1
N5	R1
N6	R1
N7	R1
N8	R1

R4的层次表

目的地	下一跳
N1	R1
N2	R2
N3	R2
AS2内的所有网络	R1
AS3内的所有网络	R1





本章内容



5.1 网络层服务

5.2 Internet网际协议

5.3 路由算法

5.4 Internet路由协议

5.5 路由器工作原理

5.6 拥塞控制算法

5.7 服务质量

5.8 三层交换与VPN

5.9 IPv6技术

1. OSPF-内部网关路由协议
2. RIP-内部网关路由协议
3. BGP-外部网关路由协议



OSPF-概述

- OSPF (Open Shortest Path First) 开放最短路径优先协议于1989年开发
- 采用分布式的链路状态算法
- OSPF协议的基本思想
 - 向本自治系统中所有路由器洪泛信息
 - 发送的信息就是与本路由器相邻的所有路由器的链路状态
 - 只有当链路状态发生变化时路由器才用洪泛法发送此信息



OSPF-链路状态



- “链路状态”就是说明本路由器都和哪些路由器相邻，以及该链路的“度量”(metric)
 - OSPF度量值一般包括费用、距离、时延、带宽等
- 由于各路由器之间频繁地交换链路状态信息，因此所有的路由器最终都能建立一个链路状态数据库LSDB
- 这个数据库实际上就是区域内的拓扑结构图，它在区域内是一致的（这称为链路状态数据库的同步）



OSPF-区域的概念

- OSPF支持将一组网段组合在一起，称为一个区域
- 详细描述拓扑结构的链路状态信息仅在区域内传递
- 使用 **层次结构的区域划分**，上层的区域叫做 **主干区域** (backbone area)，其他区域都必须与主干区域相连
- 非主干区域之间不允许直接发布区域间路由信息
- 区域也不能太大，在一个区域内的路由器最好不超过 200 个
- 划分区域可以缩小LSDB (link state database)规模，减少网络流量

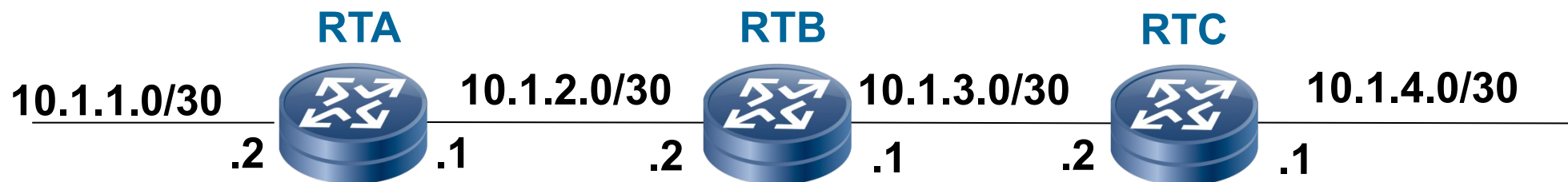


RIP-概述

- 路由选择协议RIP（Routing Information Protocol）是基于距离矢量算法的协议
- 使用跳数衡量到达目的网络的距离
 - RIP 认为一个好的路由就是它通过的路由器的数目少，即“距离短”
 - RIP 允许一条路径最多只能包含 15 个路由器
- RIP协议的基本思想
 - 仅和相邻路由器交换信息
 - 路由器交换的内容是自己的路由表
 - 周期性更新：30s

RIP-工作过程

➤ 初始化



目标网络	下一跳	距离
10.1.1.0	--	0
10.1.2.0	--	0

目标网络	下一跳	距离
10.1.2.0	--	0
10.1.3.0	--	0

目标网络	下一跳	距离
10.1.3.0	--	0
10.1.4.0	--	0

RIP-工作过程

➤ 周期性更新





RIP协议特点

- RIP协议的特点
 - 算法简单，易于实现
 - 收敛慢
 - 需要交换的信息量较大
- RIP协议的适用场合
 - 中小型网络



BGP-外部网关路由协议

➤ 路由协议

- 内部网关协议 IGP: 有 RIP、OSPF、ISIS 等多种具体的协议
- 外部网关协议 EGP: 目前使用的协议就是 BGP

➤ 边界网关协议 BGP (Border Gateway Protocol)

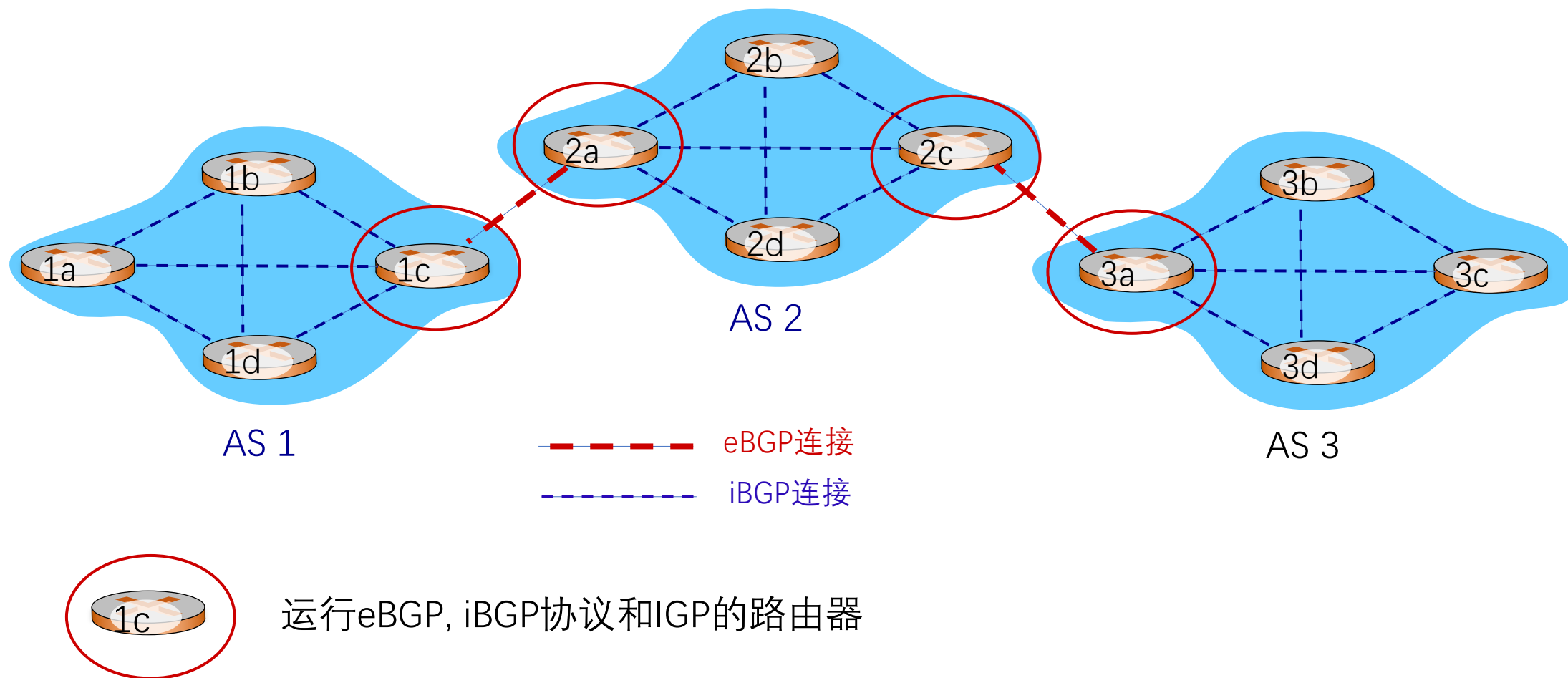
- 目前互联网中唯一实际运行的自治域间的路由协议

➤ BGP功能

- eBGP: 从相邻的AS获得网络可达信息
- iBGP: 将网络可达信息传播给AS内的路由器
- 基于网络可达信息和策略决定到其他网络的“最优”路由



eBGP&iBGP连接

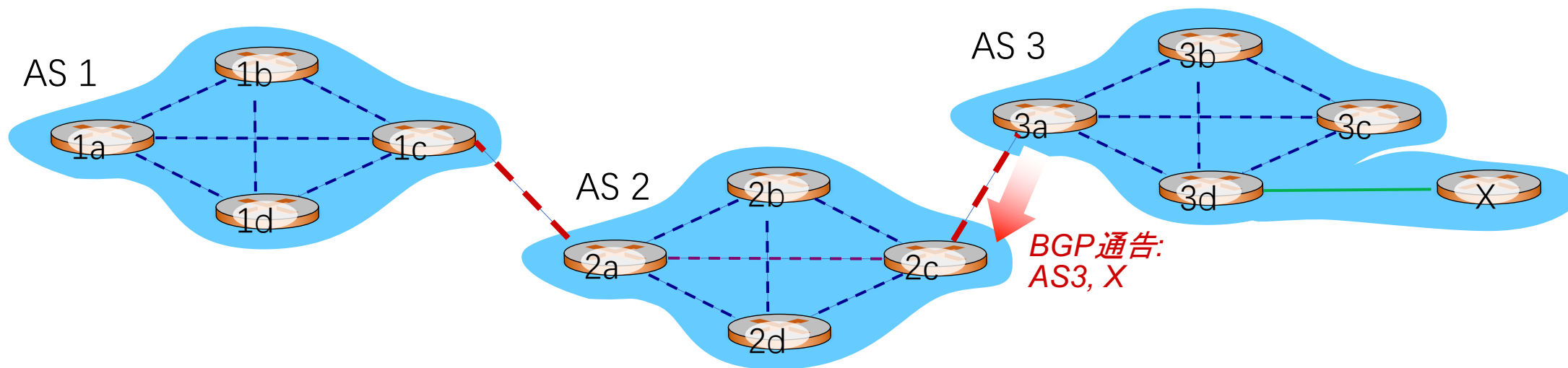




BGP基础

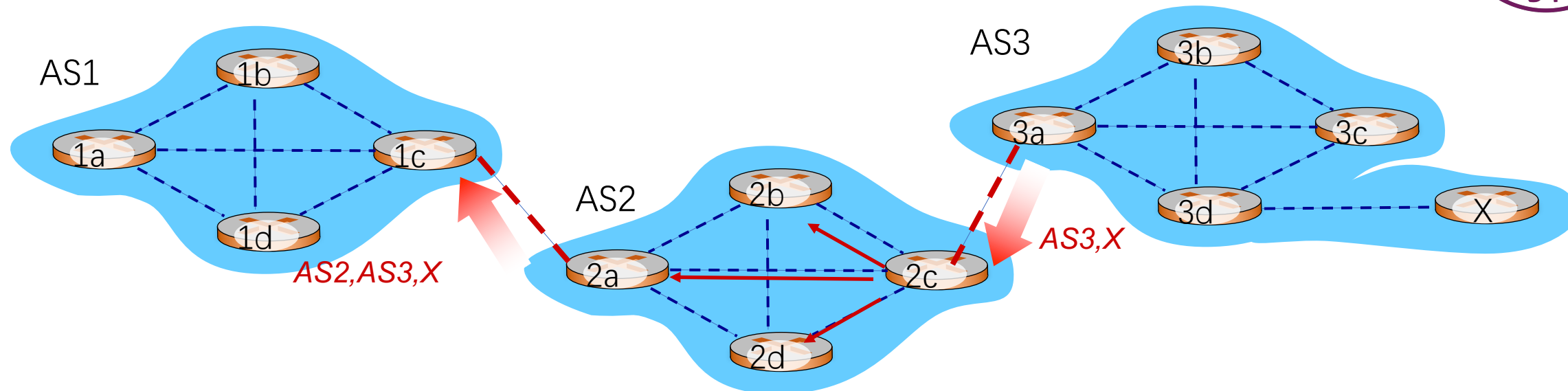


- BGP会话: 两个BGP路由器通过TCP连接交换BGP报文
 - 通告到不同网络前缀的路径，即路径向量协议
- 例：当AS3的路由器3a向AS2的路由器2c通告路径AS3, X时，AS3向AS2承诺它会向X转发数据包





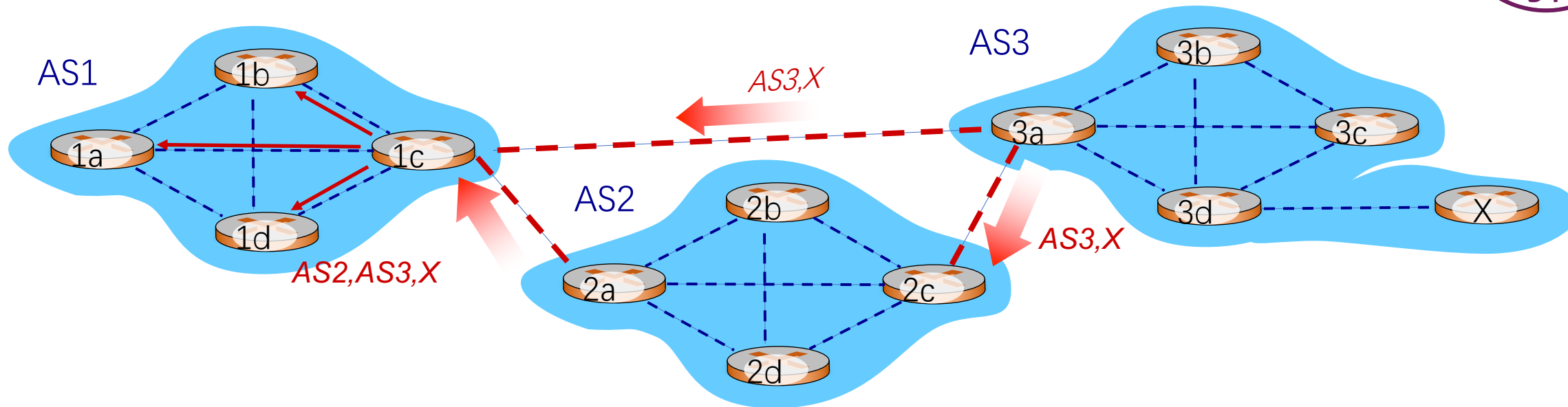
BGP路径通告



- AS2的路由器2c从AS3的路由器3a接收到路径AS3, X
- 根据AS2的策略, AS2的路由器2c接受路径AS3, X, 通过iBGP传播给AS2的所有路由器
- 根据AS2策略, AS2的路由器2a通过eBGP向AS1的路由器1c, 通告从AS3的路由器3a接收到路径AS2, AS3, X



BGP路径通告



路由器可能会学到多条到目的网络的路径:

- AS1的路由器1c从2a学到路径 *AS2, AS3, X*
- AS1的路由器1c从3a学到路径 *AS3, X*
- AS1路由器1c可能选择路径 *AS3, X*, 并在AS1中通过iBGP通告路径



BGP协议的特点

- BGP 协议交换路由信息的节点数量级是自治系统数的量级
- 每一个自治系统边界路由器的数目是很少的
- 在 BGP 刚刚运行时，BGP 的邻站交换整个的 BGP 路由表；以后只需要在发生变化时更新有变化的部分
- BGP为每个AS提供：
 - 从邻居AS获取网络可达信息（eBGP协议）
 - 传播可达信息给所有的域内路由器（iBGP协议）
 - 根据“可达信息”和“策略”决定路由



总结



1. 学习网络层服务：无连接和面向连接服务，数据包和虚电路网络
2. 学习Internet网络层协议：IPv4/IPv6, ICMP, DHCP, NAT, ARP
3. 掌握链路状态、距离矢量等路由算法，了解层次路由结构
4. 掌握Internet路由协议：OSPF、RIP、BGP
5. 了解路由器工作原理：控制层和数据层，报文转发机制，交换结构
6. 了解网络拥塞及拥塞控制思想
7. 了解网络服务质量及设计思想
8. 了解其它重要的网络层相关机制：三层交换、VPN
9. 了解IPv6相关机制



第六章 传输层

授课教师：张圣林

南开大学



本章目标



- 掌握传输层服务原理
 - 传输层复用和分用
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- 掌握因特网传输层协议：
 - UDP协议
 - TCP协议



本章内容



6.1 概述和传输层服务

6.2 套接字编程

6.3 传输层复用和分用

6.4 无连接传输：UDP

6.5 面向连接的传输：TCP

6.6 理解网络拥塞

6.7 TCP拥塞控制

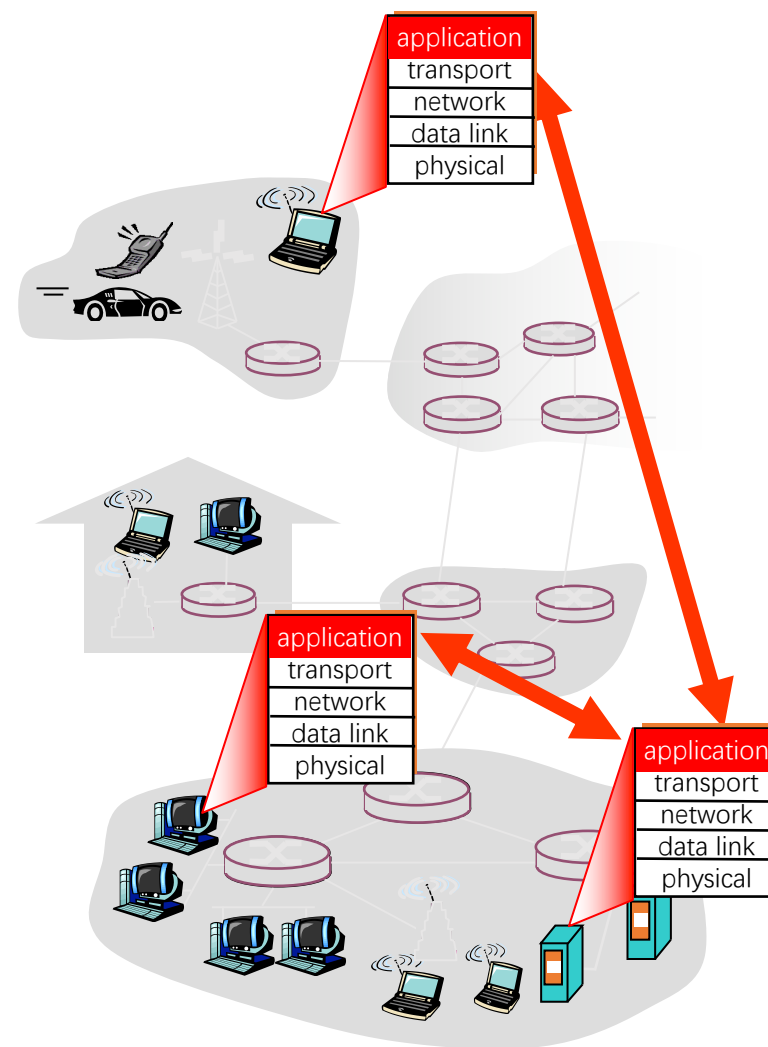
6.8 拥塞控制的发展

6.9 传输层协议的发展



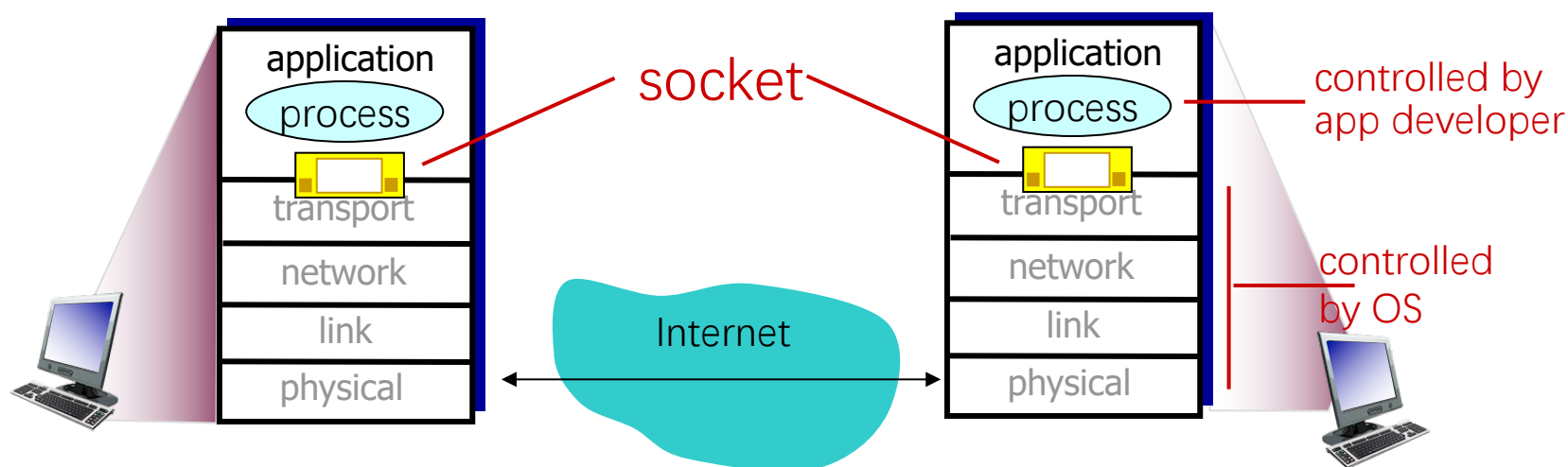
传输层的位置

- 传输层位于应用层和网络层之间：
 - 基于网络层提供的服务，向分布式应用程序提供通信服务
- 按照因特网的“端到端”设计原则：
 - 应用程序只运行在终端上，即不需要为网络设备编写程序
- 站在应用程序的角度：
 - 传输层应提供进程之间本地通信的抽象：
即运行在不同终端上的应用进程仿佛是直接连在一起的



不同终端上的进程如何通信？

- 设想在应用程序和网络之间存在一扇“门”：
 - 需要发送报文时：发送进程将报文推到门外
 - 门外的运输设施（因特网）将报文送到接收进程的门口
 - 需要接收报文时：接收进程打开门，即可收到报文
- 在TCP/IP网络中，这扇“门”称为**套接字（socket）**，是应用层和传输层的接口，也是应用程序和网络之间的API





传输层和网络层的关系：一个类比例子

- 设想A家庭有12个孩子，B家庭也有12个孩子，他们每周通过手写信件进行联系：
- A家庭推选Ann，B家庭推选Bill，分别负责各自家庭的信件收发
 - Ann和Bill定期收集家庭中其他孩子的信件，投递到门口的邮筒里
 - Ann和Bill定期打开家庭邮箱，取出里面的信件，分发给其他孩子
 - 邮政系统负责将邮筒里的信件投递到收信人的家庭邮箱里

➤ 类比：

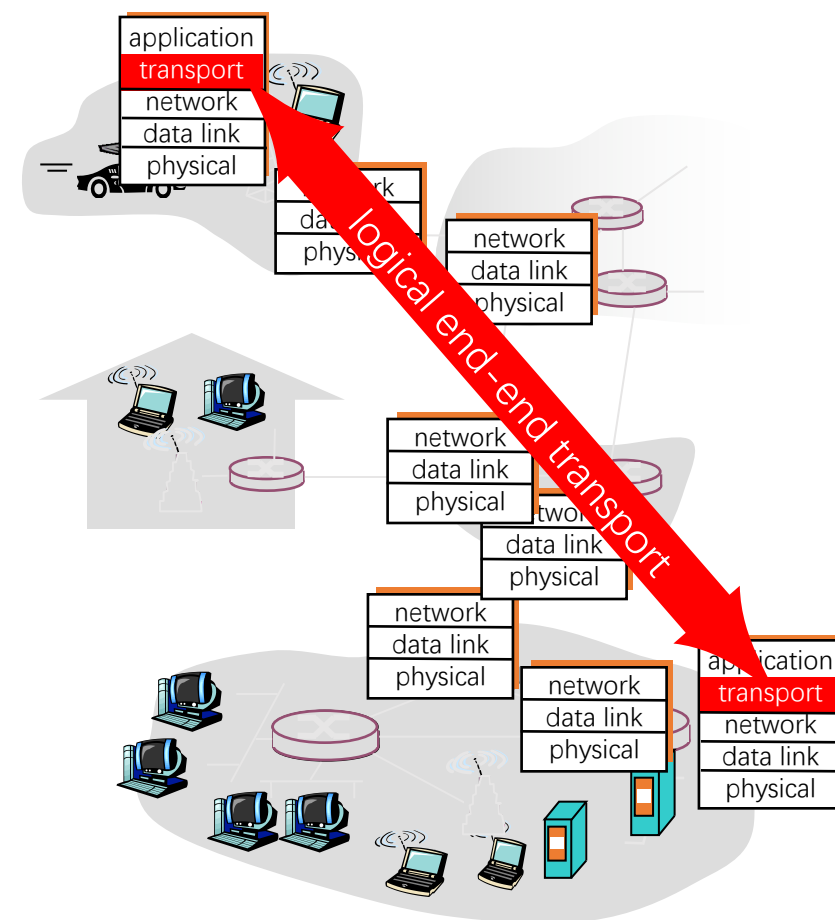
- 进程 = 孩子
- 进程间通信 = 孩子之间通信
- 应用报文 = 信件
- 传输层 = Ann 和 Bill（提供人到人的服务）
- 终端 = 家庭住宅
- 网络层 = 邮政系统（提供门到门的服务）

传输层依赖网络层服务，并扩展网络层服务



传输层提供什么服务？

- 因特网的网络层提供“**尽力而为**”的服务：
 - 网络层尽最大努力在终端间交付分组，但不提供任何承诺
 - 具体来说，不保证交付，不保证按序交付，不保证数据完整，不保证延迟，不保证带宽等
- 传输层的**有所为、有所不为**：
 - 传输层可以通过差错恢复、重排序等手段提供可靠、按序的交付服务
 - 但传输层无法提供延迟保证、带宽保证等服务





因特网传输层提供的服务



- 最低限度的传输服务：
 - 将终端-终端的数据交付扩展到进程-进程的数据交付（6.3节）
 - 报文检错
- 增强服务：
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- 因特网传输层通过UDP协议和TCP协议，向应用层提供两种不同的传输服务：
 - UDP协议（6.4节）：仅提供最低限度的传输服务
 - TCP协议（6.5节，6.7节）：提供基础服务和增强服务



本章内容



6.1 概述和传输层服务

6.2 套接字编程

6.3 传输层复用和分用

6.4 无连接传输：UDP

6.5 面向连接的传输：TCP

6.6 理解网络拥塞

6.7 TCP拥塞控制

6.8 拥塞控制的发展

6.9 传输层协议的发展



传输层基本服务

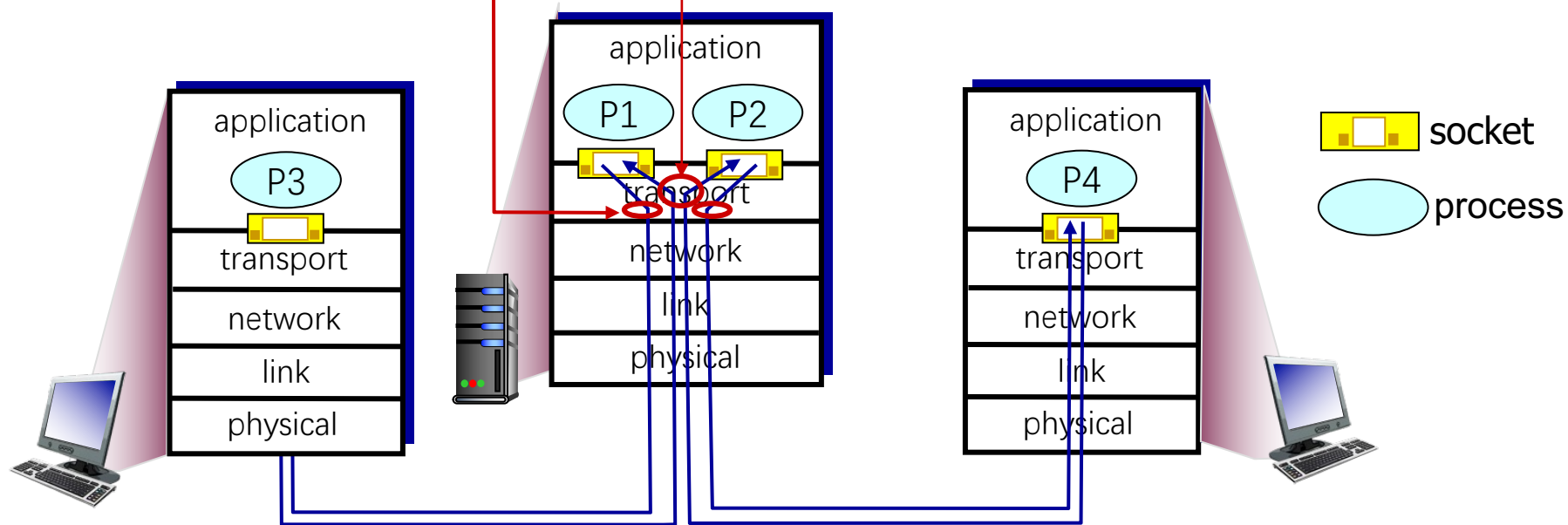
传输层基本服务：将主机间交付扩展到进程间交付，通过复用和分用实现

（发送端）复用：

传输层从多个套接字收集数据，交给网络层发送

（接收端）分用：

传输层将从网络层收到的数据，交付给正确的套接字





如何进行复用和分用：类比的例子

➤ 为将邮件交付给收信人：

- 每个收信人应有一个信箱，写有收信人地址和姓名（唯一标识）
- 信封上有收信人的地址和名字

➤ 为将报文段交付给套接字：

- 主机中每个套接字应分配一个唯一的标识
- 报文段中包含接收套接字的标识

➤ 复用：

- 发送方传输层将套接字标识置于报文段中，交给网络层

➤ 分用：

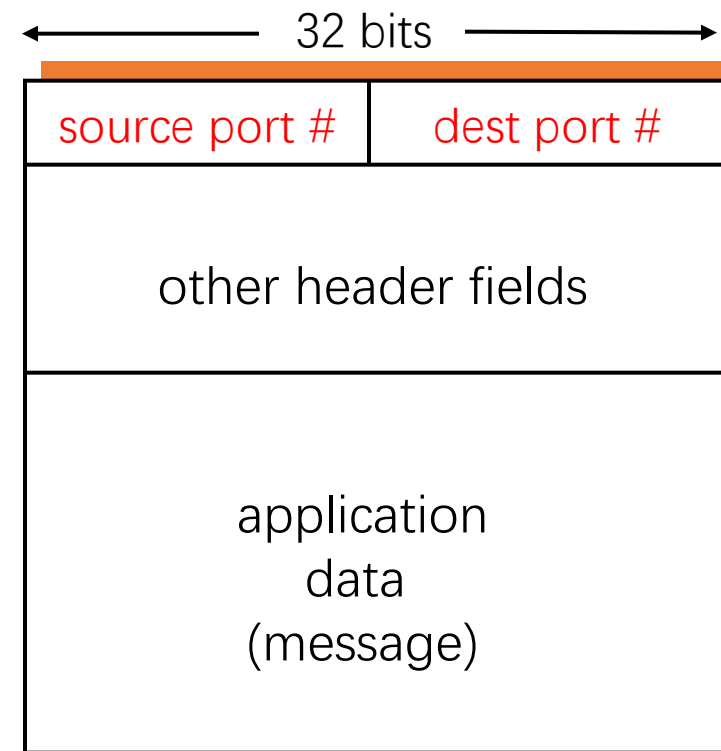
- 接收方传输层根据报文段中的套接字标识，将报文段交付到正确的套接字



套接字标识与端口号



- 端口号是套接字标识的一部分：
 - 每个套接字在本地关联一个端口号
 - 端口号是一个16比特的数
- 端口号的分类：
 - 熟知端口：0 ~ 1023，由公共域协议使用
 - 注册端口：1024 ~ 49151，需要向IANA (Internet Assigned Numbers Authority)注册才能使用
 - 动态和/或私有端口：49152 ~ 65535，一般程序使用
- 报文段中有两个字段携带端口号
 - 源端口号：与发送进程关联的本地端口号
 - 目的端口号：与接收进程关联的本地端口号



TCP/UDP报文段格式



套接字端口号的分配



➤ 自动分配:

- 创建套接字时不指定端口号
- 由操作系统从49152 ~ 65535中分配
- 客户端通常使用这种方法

➤ 使用指定端口号创建套接字:

- 创建套接字时指定端口号
- 实现公共域协议的服务器应分配众所周知的端口号 (0 ~ 1023)
- 服务器通常采用这种方法



UDP分用的方法

- UDP套接字使用<IP地址, 端口号>二元组进行标识
- 接收方传输层收到一个UDP报文段后:
 - 检查报文段中的目的端口号, 将UDP报文段交付到具有该端口号的套接字
 - <目的IP地址, 目的端口号> 相同的UDP报文段被交付给同一个套接字, 与 <源IP地址, 源端口号> 无关
 - 报文段中的 <源IP地址, 源端口号> 被接收进程用来发送响应报文

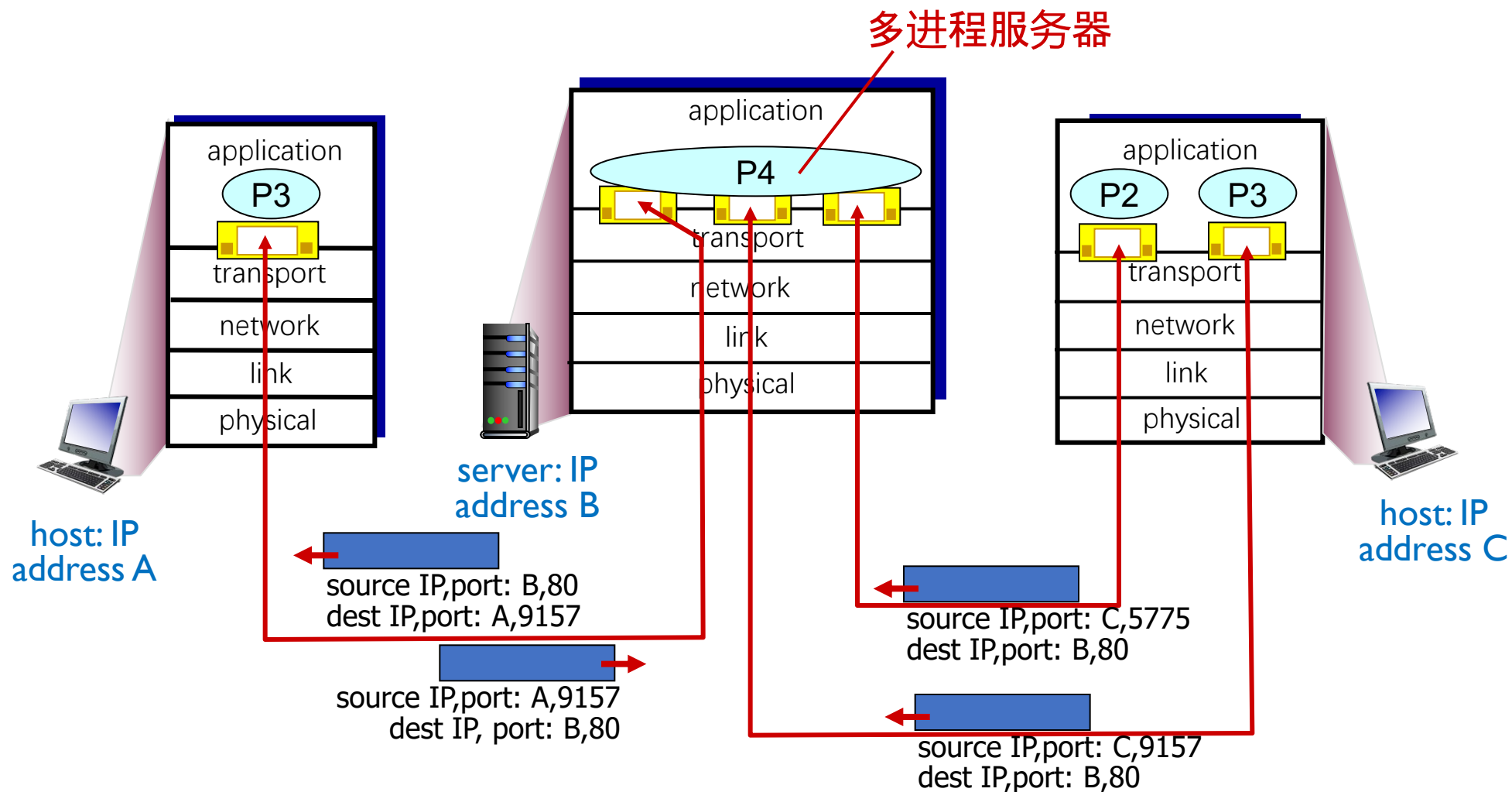


TCP服务器使用的套接字

- 一个TCP服务器为了同时服务很多个客户，使用两种套接字
- 监听套接字：
 - 服务器平时在监听套接字上等待客户的连接请求，该套接字具有众所周知的端口号
- 连接套接字：
 - 服务器在收到客户的连接请求后，创建一个连接套接字，使用临时分配的端口号
 - 服务器同时创建一个新的进程，在该连接套接字上服务该客户
 - **每个连接套接字只与一个客户通信**，即只接收具有以下四元组的报文段：
 - 源IP地址 = 客户IP地址，源端口号 = 客户套接字端口号
 - 目的IP地址 = 服务器IP地址，目的端口号 = **服务器监听套接字的端口号**
- **连接套接字需要使用<源IP地址，目的IP地址，源端口号，目的端口号>四元组进行标识**，服务器使用该四元组将TCP报文段交付到正确的连接套接字



TCP分用： 举例





小结



➤ UDP套接字

- 使用<IP地址, 端口号>二元组标识UDP套接字
- 服务器使用一个套接字服务所有客户

➤ TCP套接字

- 使用<源IP地址, 目的IP地址, 源端口号, 目的端口号> 四元组标识连接套接字
- 服务器使用一个监听套接字和多个连接套接字服务多个客户, 每个连接套接字服务一个客户



本章内容

6.1 概述和传输层服务

6.2 套接字编程

6.3 传输层复用和分用

6.4 无连接传输：UDP

6.5 面向连接的传输：TCP

6.6 理解网络拥塞

6.7 TCP拥塞控制

6.8 拥塞控制的发展

6.9 传输层协议的发展



UDP提供的服务



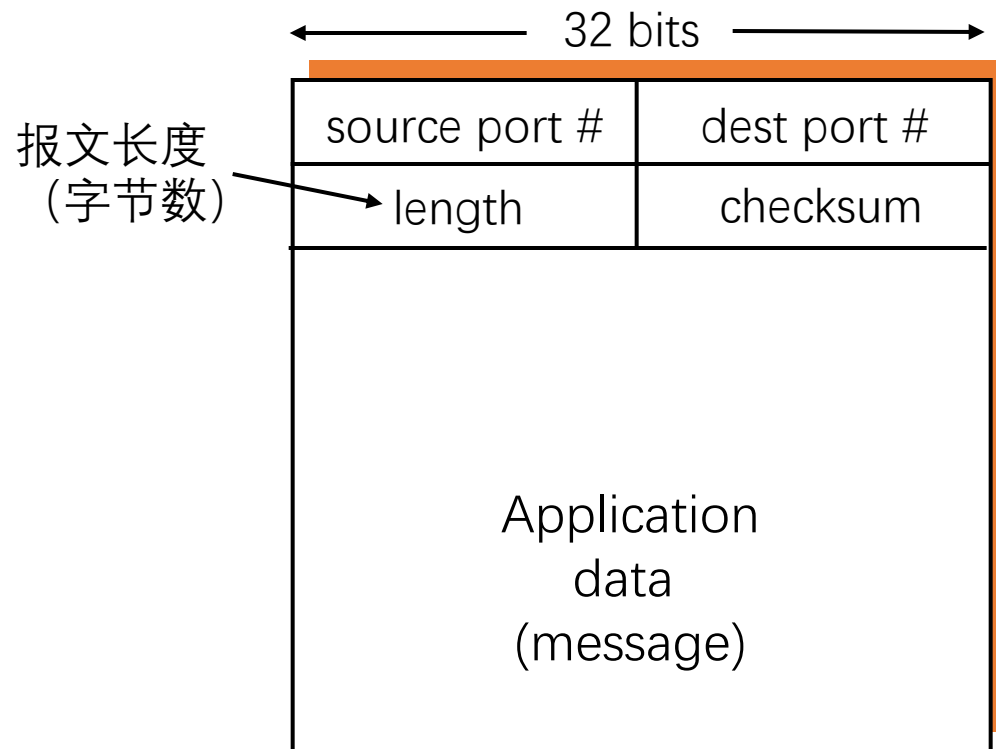
- 网络层提供的服务（best-effort service）：
 - 尽最大努力将数据包交付到目的主机
 - 不保证投递的可靠性和顺序
 - 不保证带宽及延迟要求
- UDP提供的服务：
 - 进程到进程之间的报文交付
 - 报文完整性检查（可选）：检测并丢弃出错的报文
- UDP需要实现的功能：
 - 复用和分用
 - 报文检错



UDP报文段结构



- UDP报文：
 - 报头：携带协议处理需要的信息
 - 载荷（payload）：携带上层数据
- 用于复用和分用的字段：
 - 源端口号
 - 目的端口号
- 用于检测报文错误的字段：
 - 报文总长度
 - 校验和（checksum）



UDP报文格式



UDP校验和 (checksum)



校验和字段的作用：对传输的报文段进行检错

发送方：

- 将报文段看成是由16比特整数组成的序列
- 对这些整数序列计算校验和
- 将校验和放到UDP报文段的checksum字段

接收方：

- 对收到的报文段进行相同的计算
- 与报文段中的checksum字段进行比较：
 - 不相等：说明报文段有错误
 - 相等：**认为**报文段没有错误



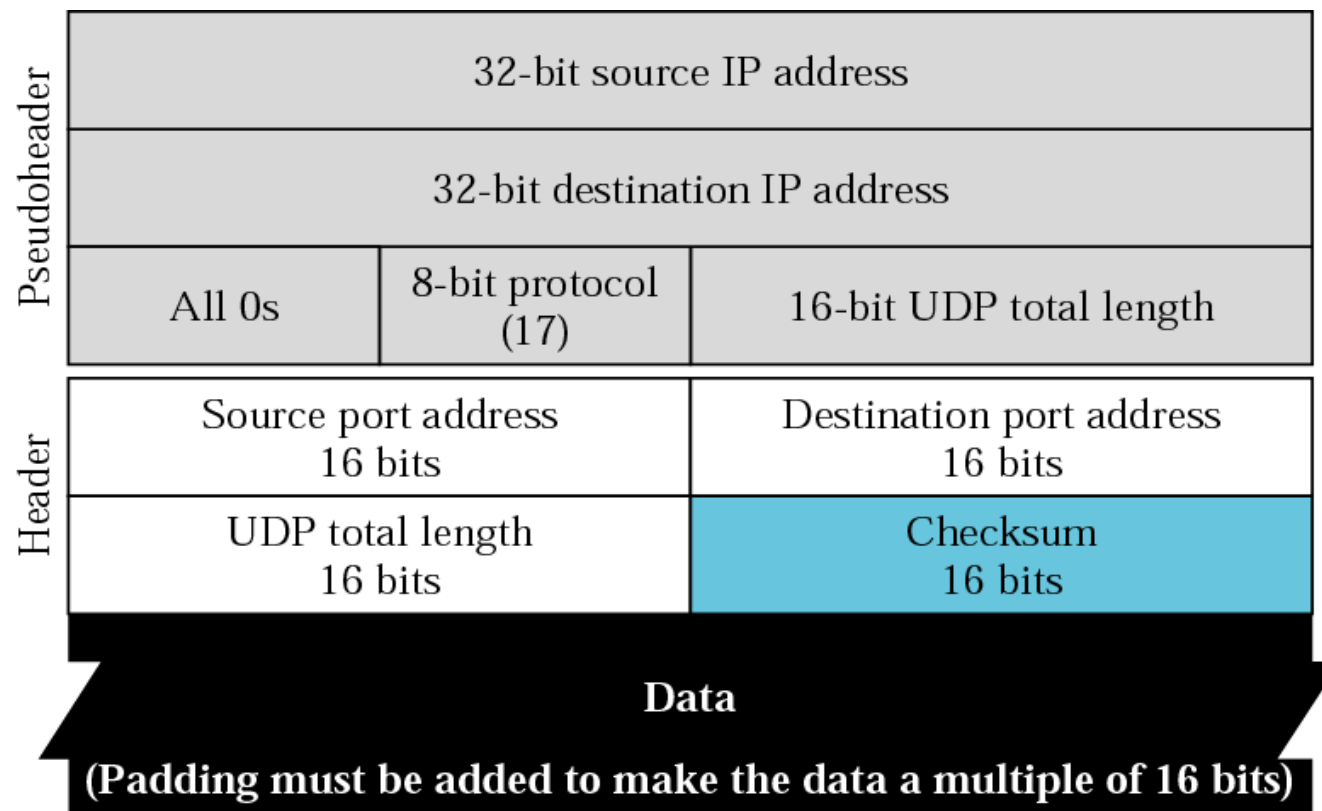
16 位二进制反码求和运算

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
环绕	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
求和	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
校验和	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1



UDP校验和计算

- 计算UDP校验和时，要包括伪头、UDP头和数据三个部分
- UDP伪头是一个虚拟数据结构，取自IP报头，并非UDP报文的实际有效成分，包括：
 - 源IP地址，目的IP地址
 - UDP的协议号
 - UDP报文段总长度
- 计算校验和时包含伪头信息，是为了避免由于IP地址错误等造成的误投递
- UDP校验和的使用是可选的，若不计算校验和，该字段填入0





UDP校验和计算举例

- 发送方计算校验和时，checksum字段填0
- 接收方对UDP报文（包括校验和）及伪头求和，若结果为0xFFFF，认为没有错误

153.18.8.105			
171.2.14.10			
All 0s	17	15	
1087		13	
15		All 0s	
T	E	S	T
I	N	G	All 0s

10011001	00010010	→	153.18
00001000	01101001	→	8.105
10101011	00000010	→	171.2
00001110	00001010	→	14.10
00000000	00010001	→	0 and 17
00000000	00001111	→	15
00000100	00111111	→	1087
00000000	00001101	→	13
00000000	00001111	→	15
00000000	00000000	→	0 (checksum)
01010100	01000101	→	T and E
01010011	01010100	→	S and T
01001001	01001110	→	I and N
01000111	00000000	→	G and 0 (padding)
10010110	11101011	→	Sum
01101001	00010100	→	Checksum



为什么需要UDP?



为什么需要UDP?

- 应用可以尽可能快地发送报文：
 - 无建立连接的延迟
 - 不限制发送速率（不进行拥塞控制和流量控制）
- 报头开销小
- 协议处理简单

UDP适合哪些应用?

- 容忍丢包但对延迟敏感的应用：
 - 如流媒体
- 以单次请求/响应为主的应用：
 - 如DNS
- 若应用要求基于UDP进行可靠传输：
 - 由应用层实现可靠性



本章内容

6.1 概述和传输层服务

6.2 套接字编程

6.3 传输层复用和分用

6.4 无连接传输：UDP

6.5 面向连接的传输：TCP

6.6 理解网络拥塞

6.7 TCP拥塞控制

6.8 拥塞控制的发展

6.9 传输层协议的发展

1. TCP概述

2. TCP报文段结构

3. TCP可靠数据传输

4. TCP流量控制

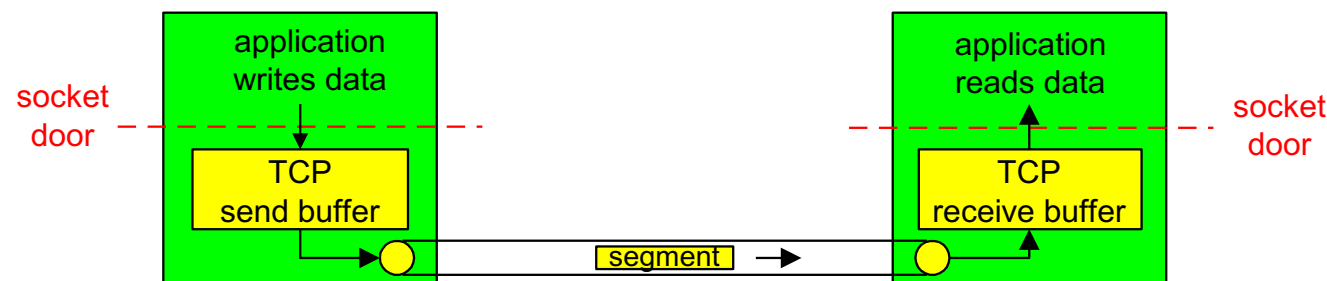
5. TCP连接管理

TCP 概述

- TCP服务模型：
 - 在一对通信的进程之间提供一条理想的字节流管道
- 点到点通信：
 - 仅涉及一对通信进程
- 全双工：
 - 可以同时双向传输数据
- 可靠、有序的字节流：
 - 不保留报文边界

需要的机制

- 建立连接：
 - 通信双方为本次通信建立数据传输所需的
状态（套接字、缓存、变量等）
- 可靠数据传输：
 - 流水线式发送，报文段检错，丢失重传
- 流量控制：
 - 发送方不会令接收方缓存溢出





本章内容

6.1 概述和传输层服务

6.2 套接字编程

6.3 传输层复用和分用

6.4 无连接传输：UDP

6.5 面向连接的传输：TCP

6.6 理解网络拥塞

6.7 TCP拥塞控制

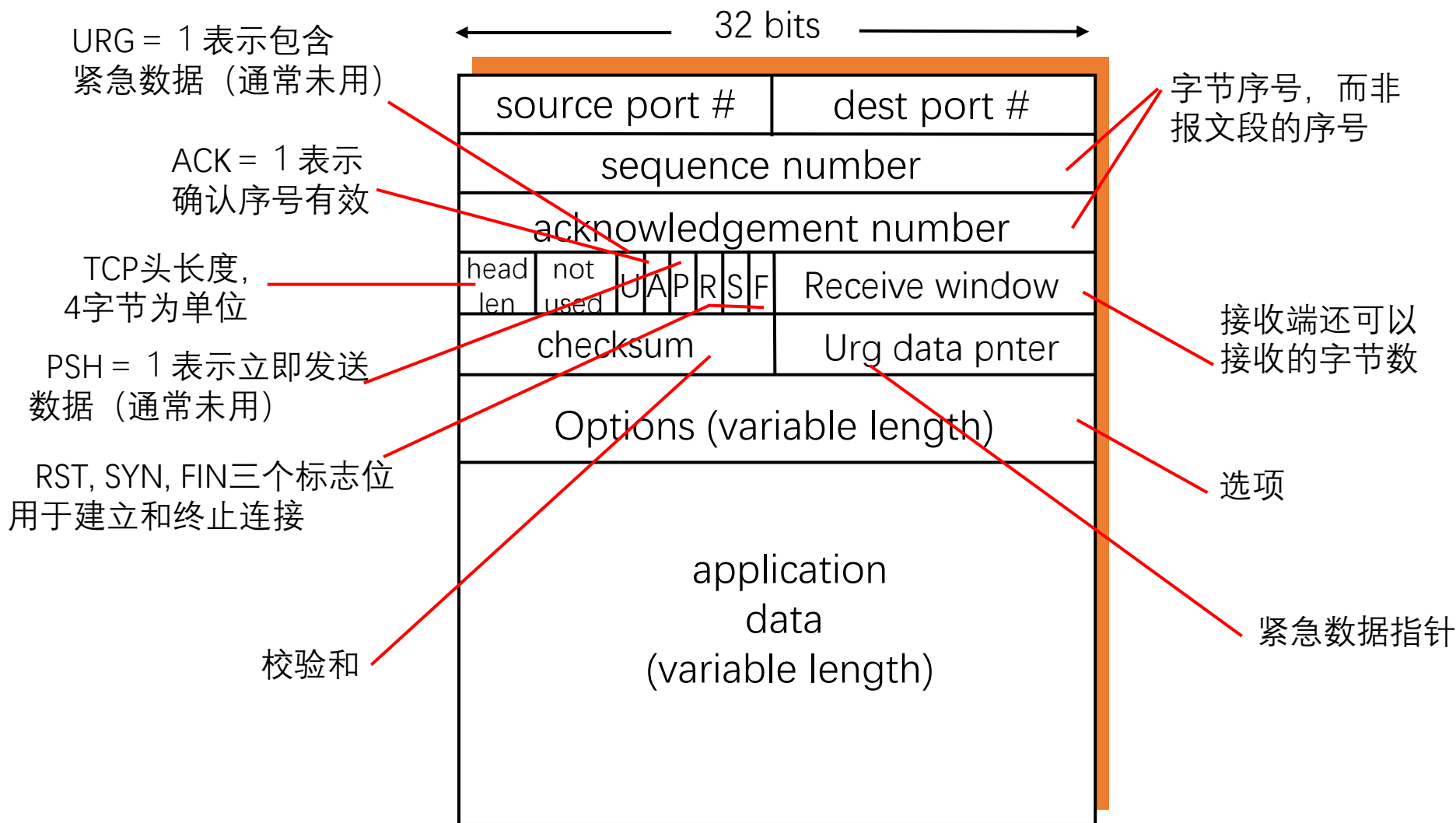
6.8 拥塞控制的发展

6.9 传输层协议的发展

1. TCP概述
2. TCP报文段结构
3. TCP可靠数据传输
4. TCP流量控制
5. TCP连接管理



TCP报文段结构



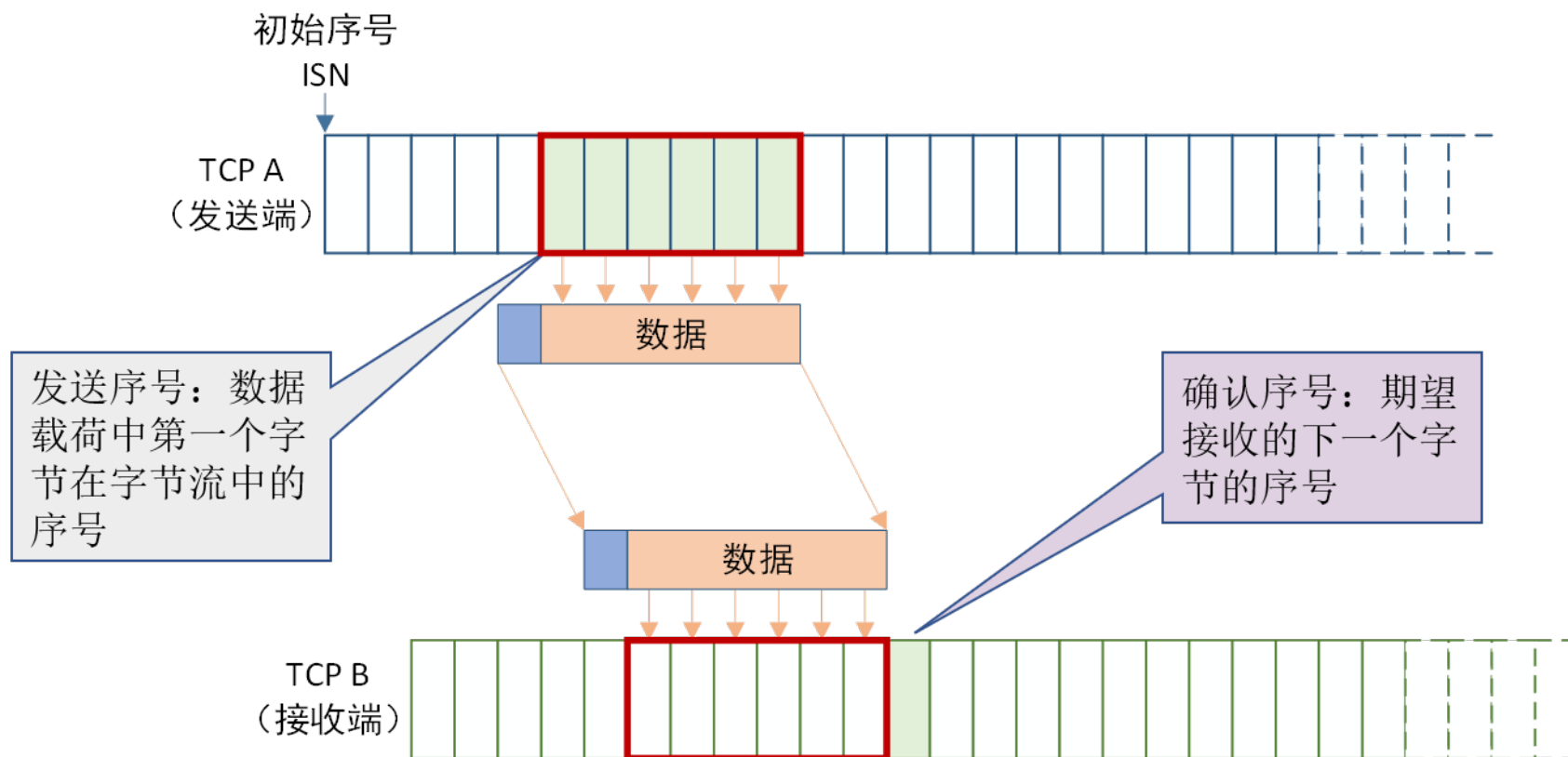


重要的TCP选项

- 最大段长度 (MSS) :
 - TCP段中可以携带的最大数据字节数
 - 建立连接时, 每个主机可声明自己能够接受的MSS, 缺省为536字节
- 选择确认 (SACK) :
 - 最初的TCP协议只使用累积确认
 - 改进的TCP协议引入选择确认, 允许接收端指出缺失的数据字节



发送序号和确认序号的含义

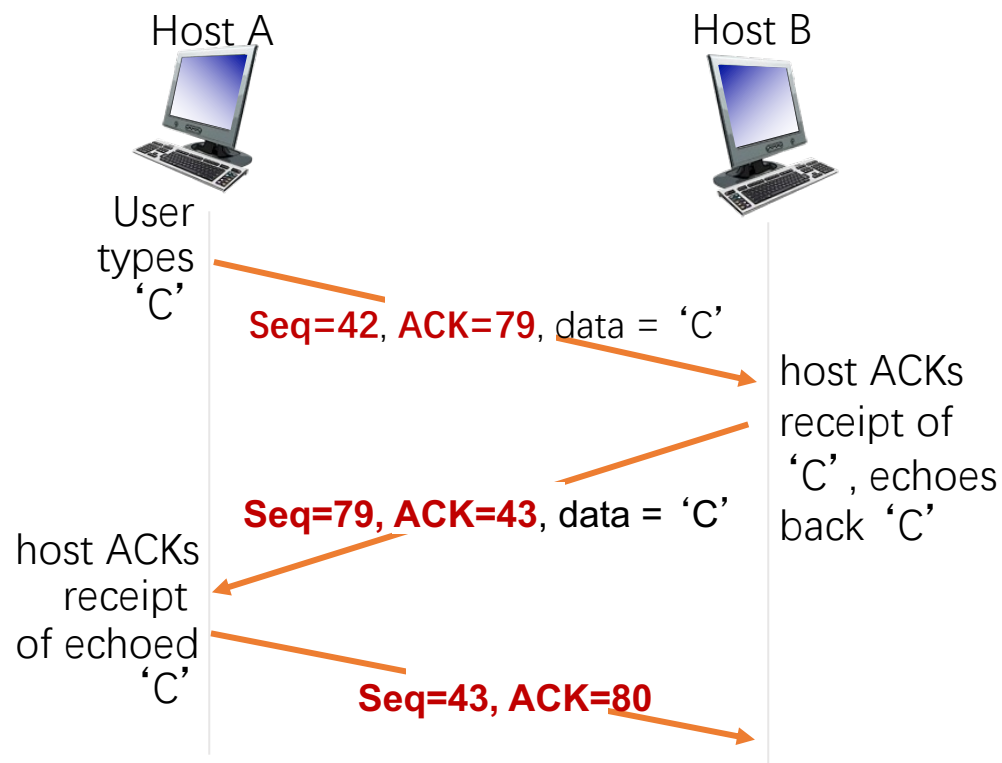


初始序号的选取: 每个TCP实体维护一个32位计数器, 该计数器每4微秒增1, 建立连接时从中读取计数器当前值 (依赖具体实现, 见连接管理)



发送序号和确认序号：使用举例

- 主机A向主机B发送仅包含一个字符‘C’的报文段：
 - 发送序号为42
 - 确认序号为79（对前一次数据的确认）
- 主机B将字符‘C’回送给主机A：
 - 发送序号为79
 - 确认序号为43（对收到‘C’的确认）
- 主机A向主机B发送确认报文段（不包含数据）：
 - 确认序号为80（对收到‘C’的确认）





本章内容

6.1 概述和传输层服务

6.2 套接字编程

6.3 传输层复用和分用

6.4 无连接传输：UDP

6.5 面向连接的传输：TCP

6.6 理解网络拥塞

6.7 TCP拥塞控制

6.8 拥塞控制的发展

6.9 传输层协议的发展

1. TCP概述
2. TCP报文段结构
3. TCP可靠数据传输
4. TCP流量控制
5. TCP连接管理



TCP可靠数据传输概述

- TCP 在不可靠的IP服务上建立可靠的数据传输
- 基本机制
 - 发送端：流水线式发送数据、等待确认、超时重传
 - 接收端：进行差错检测，采用累积确认机制
- 乱序段处理：协议没有明确规定
 - 接收端不缓存：可以正常工作，处理简单，但效率低
 - 接收端缓存：效率高，但处理复杂



一个高度简化的TCP协议



高度简化的TCP协议：仅考虑可靠传输机制，且数据仅在一个方向上传输

➤ 接收方：

- 确认方式：采用累积确认，仅在正确、按序收到报文段后，更新确认序号；其余情况，**重复前一次的确认序号**

➤ 发送方：

- 发送策略：流水线式发送报文段
- 定时器的使用：**仅对最早未确认的报文段使用一个重传定时器**
- 重发策略：仅在超时后重发**最早未确认**的报文段

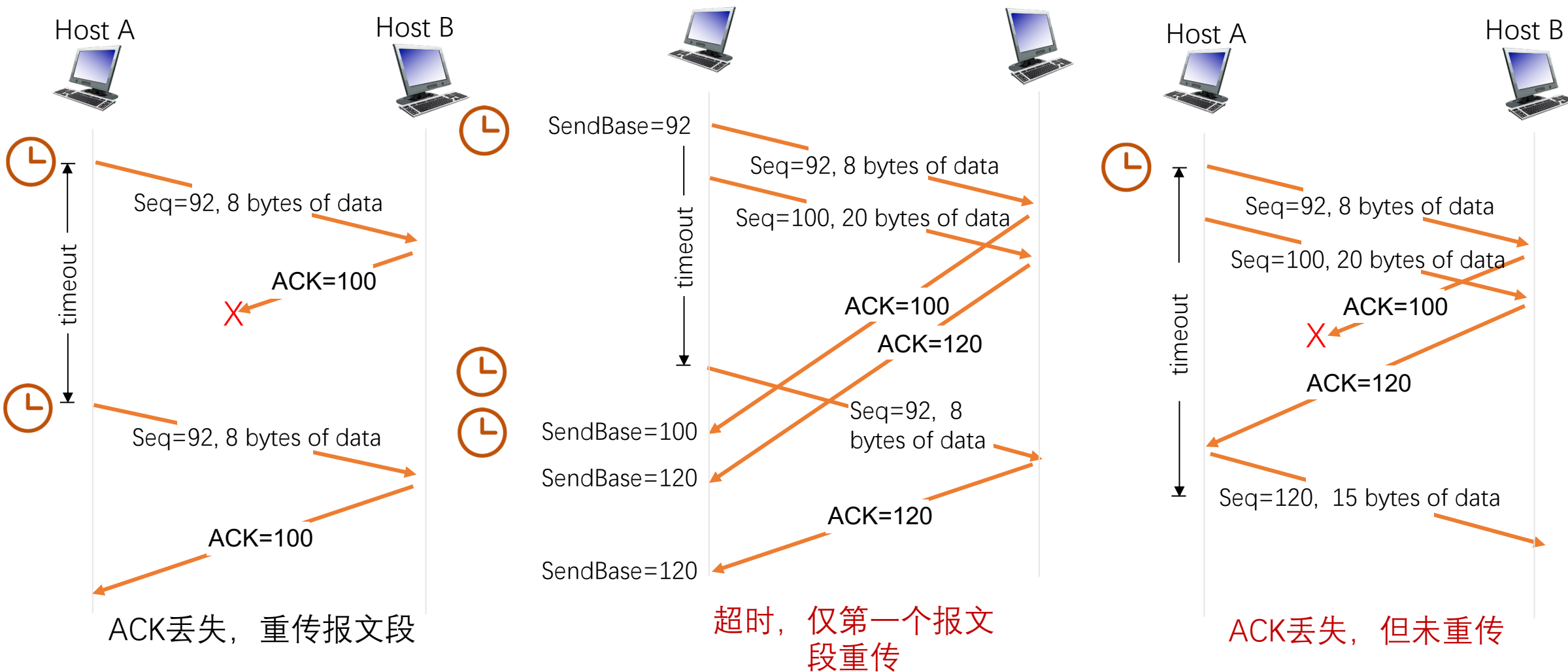


TCP发送方要处理的事件

- 收到应用数据：
 - 创建并发送TCP报文段
 - 若当前没有定时器在运行（没有已发送、未确认的报文段），启动定时器
- 超时：
 - 重传包含最小序号的、未确认的报文段
 - 重启定时器
- 收到ACK：
 - 如果确认序号大于基序号（已发送未确认的最小序号）：
 - 推进发送窗口（更新基序号）
 - 如果发送窗口中还有未确认的报文段，启动定时器，否则终止定时器



TCP可能的重传场景





TCP可靠数据传输概述

➤思考以下问题:

- 第二种情形, 如果TCP为每个报文段使用一个定时器, 会怎么样?
- 第三种情形, 采用流水式发送和累积确认, 可以避免重发哪些报文段?

➤TCP通过采用以下机制减少了不必要的重传:

- 只使用一个定时器, 避免了超时设置过小时重发大量报文段
- 利用流水式发送和累积确认, 可以避免重发某些丢失了ACK的报文段



TCP的接收端



- 理论上，接收端只需区分两种情况：
 - 收到期待的报文段：发送更新的确认序号
 - 其它情况：重复当前的确认序号
- 为减小通信量，TCP允许接收端推迟确认：
 - 接收端可以在收到若干个报文段后，发送一个累积确认的报文段
- 推迟确认带来的问题：
 - 若延迟太大，会导致不必要的重传
 - 推迟确认造成RTT估计不准确
- TCP协议规定：
 - 推迟确认的时间最多为500ms
 - 接收方至少每隔一个报文段使用正常方式进行确认



TCP接收端的事件和处理



接收端事件	接收端动作
收到一个期待的报文段，且之前的报文段均已发送过确认	推迟发送确认 ，在500ms时间内若无下一个报文段到来，发送确认
收到一个期待的报文段，且前一个报文段被推迟确认	立即发送确认（估计RTT的需要）
收到一个失序的报文段（序号大于期待的序号），检测到序号间隙	立即发送确认（快速重传的需要），重复当前的确认序号
收到部分或全部填充间隙的报文段	若报文段始于间隙的低端，立即发送确认（推进发送窗口），更新确认序号



快速重传



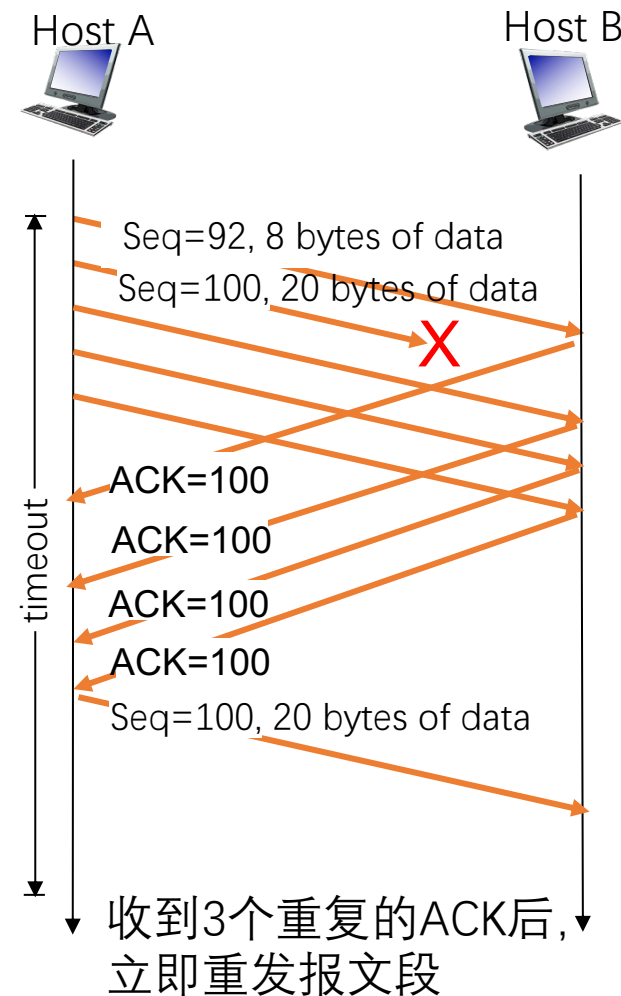
- 仅靠超时重发丢失的报文段，恢复太慢！
- 发送方可利用重复ACK检测报文段丢失：
 - 发送方通常连续发送许多报文段
 - 若仅有个别报文段丢失，发送方将收到多个重复序号的ACK
 - 多数情况下IP按序交付分组，重复ACK极有可能因丢包产生

➤ TCP协议规定：

- 当发送方收到对同一序号的3次重复确认时，立即重发包含该序号的报文段

➤ 快速重传：

- 所谓快速重传，就是在定时器到期前重发丢失的报文段





小结



➤ TCP可靠传输的设计要点:

- 流水式发送报文段
- 采用累积确认
- 只对最早未确认的报文段使用一个重传定时器
- 超时后只重传包含最小序号的、未确认的报文段

➤ 以上措施可大量减少因ACK丢失、定时器过早超时引起的重传

➤快速重传:

- 收到3次重复确认, 重发报文段



TCP结合了回退N协议和选择重传的优点

- TCP的差错恢复机制可以看成是回退N协议和选择重传的混合体：
 - 定时器的使用：与回退N协议类似，只对最早未确认的报文段使用一个定时器
 - 超时重传：与选择重传类似，只重传缺失的数据
- TCP在减小定时器开销和重传开销方面要优于回退N协议和选择重传！



本章内容

6.1 概述和传输层服务

6.2 套接字编程

6.3 传输层复用和分用

6.4 无连接传输：UDP

6.5 面向连接的传输：TCP

6.6 理解网络拥塞

6.7 TCP拥塞控制

6.8 拥塞控制的发展

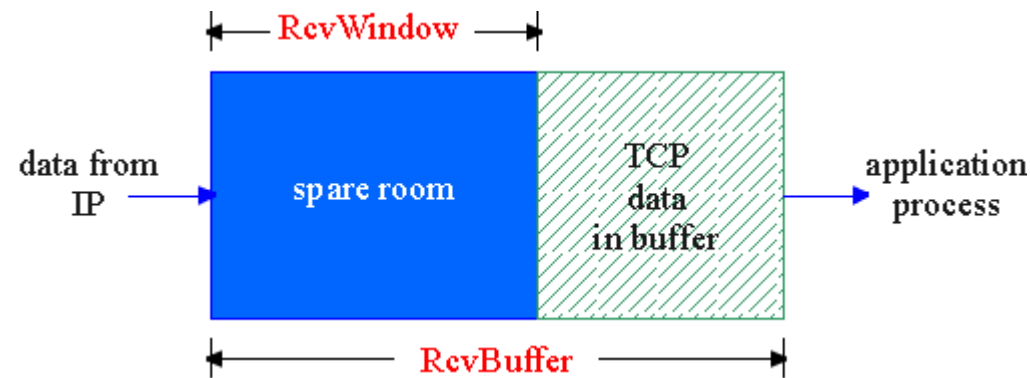
6.9 传输层协议的发展

1. TCP概述
2. TCP报文段结构
3. TCP可靠数据传输
4. TCP流量控制
5. TCP连接管理



TCP的接收端：接收缓存

- TCP接收端有一个接收缓存：
 - 接收端TCP将收到的数据放入接收缓存
 - 应用进程从接收缓存中读数据
 - 进入接收缓存的数据不一定被立即取走、取完
 - 如果接收缓存中的数据未及时取走，后续到达的数据可能会因缓存溢出而丢失



- 流量控制：
 - 发送端TCP通过调节发送速率，不使接收端缓存溢出



为什么UDP不需要流量控制



➤UDP不保证交付:

- 接收端UDP将收到的报文载荷放入接收缓存
- 应用进程每次从接收缓存中读取一个完整的报文载荷
- 当应用进程消费数据不够快时，接收缓存溢出，报文数据丢失，UDP不负责任

➤因此，UDP不需要流量控制

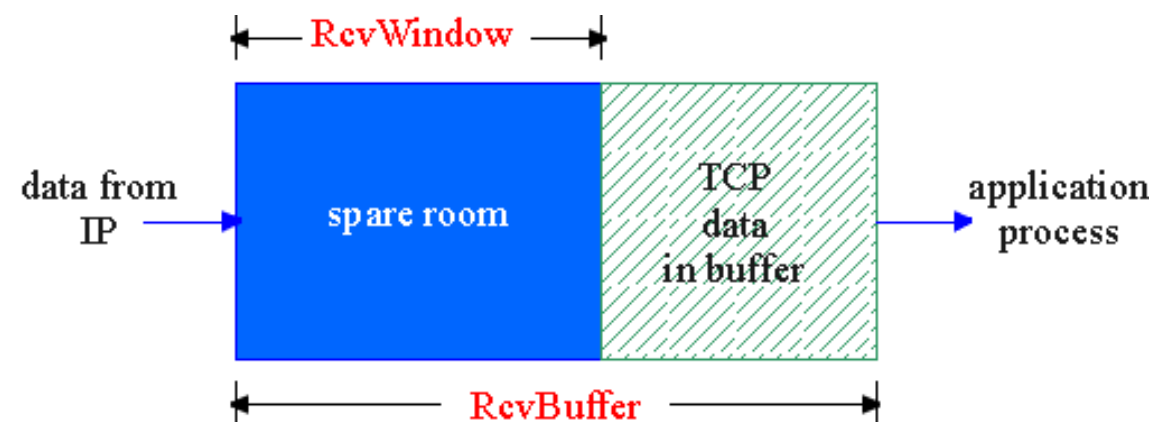
TCP如何进行流量控制

- 接收缓存中的可用空间称为**接收窗口**：

$$\text{RcvWindow} = \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$$

- 接收方将RcvWindow放在报头中，向发送方通告接收缓存的可用空间
- 发送方限制已发送、未确认的字节数不超过接收窗口的大小，即：

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{RcvWindow}$$



- 特别是，当接收方通告接收窗口为0时，发送方必须停止发送



非零窗口通告



- 发送方/接收方对零窗口的处理：
 - 发送方：当接收窗口为0时，发送方必须停止发送
 - 接收方：当接收窗口变为非0时，接收方应通告增大的接收窗口
- 在TCP协议中，触发一次TCP传输需要满足以下三个条件之一：
 - 应用程序调用
 - 超时
 - 收到数据或确认
- 对于单向传输中的接收方，只有第三个条件能触发传输
- 当发送方停止发送后，接收方不再收到数据，如何触发接收端发送“非零窗口通告”呢？
- TCP协议规定：
 - 发送方收到“零窗口通告”后，可以发送“零窗口探测”报文段
 - 从而接收方可以发送包含接收窗口的响应报文段



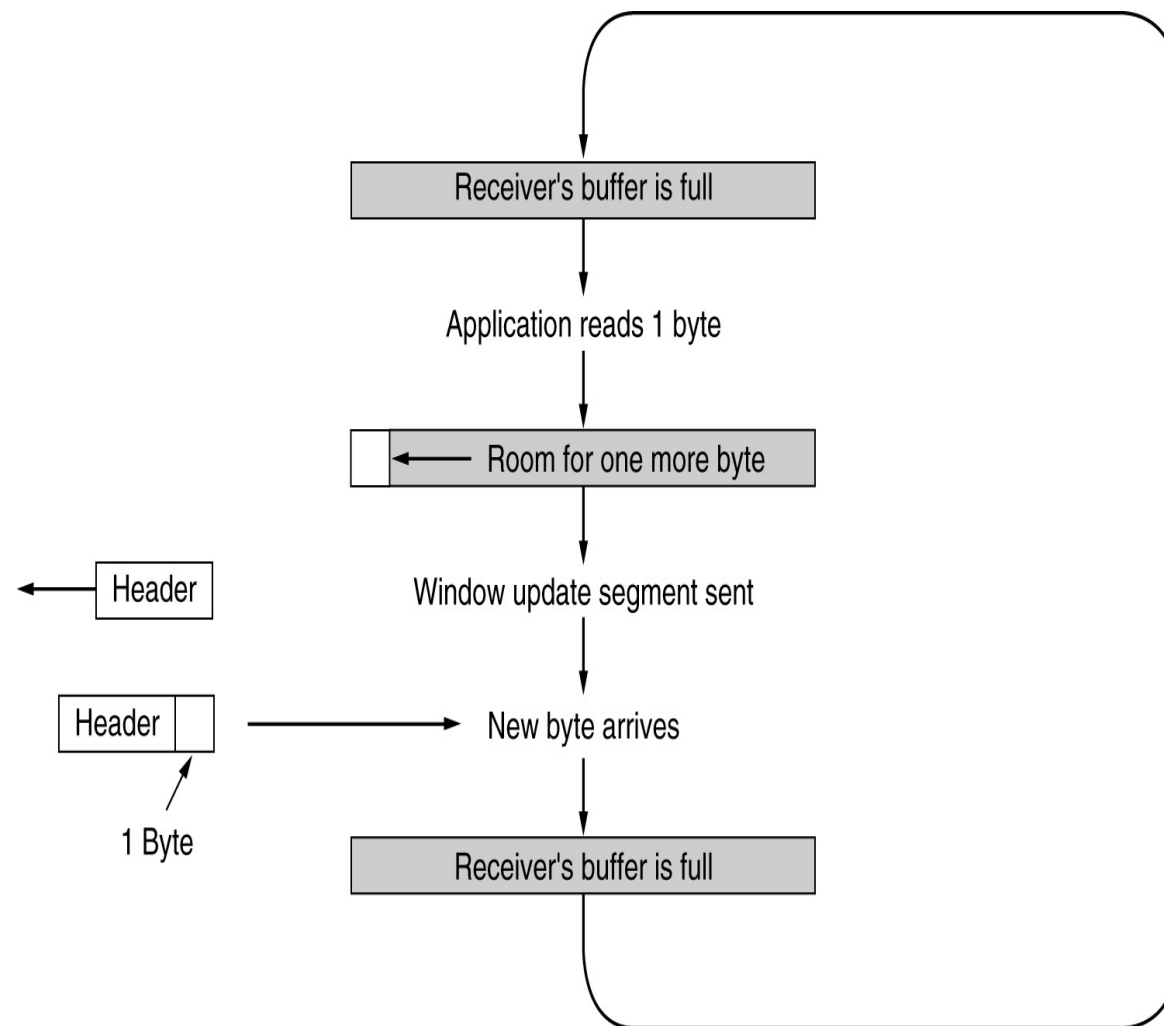
零窗口探测的实现

- 发送端收到零窗口通告时，启动一个坚持定时器
- 定时器超时后，发送端发送一个零窗口探测报文段
- 接收端在响应的报文段中通告当前接收窗口的大小
- 若发送端仍收到零窗口通告，重新启动坚持定时器



糊涂窗口综合症

- 当数据的发送速度很快、而消费速度很慢时，零窗口探测的简单实现带来以下问题：
 - 接收方不断发送微小窗口通告
 - 发送方不断发送很小的数据分组
 - 大量带宽被浪费
- 解决方案：
 - 接收方启发式策略
 - 发送方启发式策略





接收方启发式策略



- 接收端避免糊涂窗口综合症的策略：
 - 通告零窗口之后，仅当窗口大小**显著增加**之后才发送更新的窗口通告
 - 什么是显著增加：窗口大小达到缓存空间的一半或者一个MSS，取两者的较小值
- TCP执行该策略的做法：
 - 当窗口大小不满足以上策略时，推迟**发送确认**（但最多推迟500ms，且至少每隔一个报文段使用正常方式进行确认），寄希望于推迟间隔内有更多数据被消费
 - 仅当窗口大小满足以上策略时，才**通告新的窗口大小**



发送方启发式策略



➤ 发送方避免糊涂窗口综合症的策略：

- 发送方应积聚足够多的数据再发送，以防止发送太短的报文段

➤ 问题：发送方应等待多长时间？

- 若等待时间不够，报文段会太短
- 若等待时间过久，应用程序的时延会太长
- 更重要的是，TCP不知道应用程序会不会在最近的将来生成更多的数据

➤ Nagle算法的解决方法：

- 在新建连接上，当应用数据到来时，组成一个TCP段发送（那怕只有一个字节）
- 在收到确认之前，后续到来的数据放在发送缓存中
- 当数据量达到一个MSS或上一次传输的确认到来（取两者的较小时间），用一个TCP段将缓存的字节全部发走

➤ Nagle算法的优点：

- 适应网络延时、MSS长度及应用速度的各种组合
- 常规情况下不会降低网络的吞吐量



TCP流量控制小结

➤ TCP接收端

- 使用显式的窗口通告，告知发送方可用的缓存空间大小
- 在接收窗口较小时，推迟发送确认
- 仅当接收窗口显著增加时，通告新的窗口大小

➤ TCP发送端

- 使用Nagle算法确定发送时机
- 使用接收窗口限制发送的数据量，已发送未确认的字节数不超过接收窗口的大小



本章内容

6.1 概述和传输层服务

6.2 套接字编程

6.3 传输层复用和分用

6.4 无连接传输：UDP

6.5 面向连接的传输：TCP

6.6 理解网络拥塞

6.7 TCP拥塞控制

6.8 拥塞控制的发展

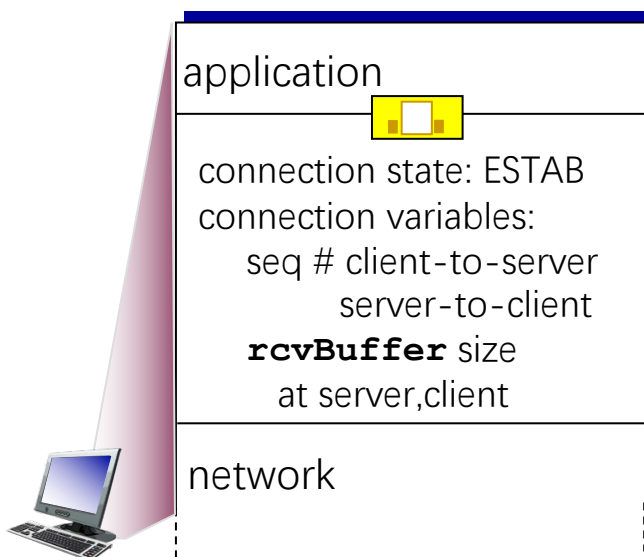
6.9 传输层协议的发展

1. TCP概述
2. TCP报文段结构
3. TCP可靠数据传输
4. TCP流量控制
5. TCP连接管理

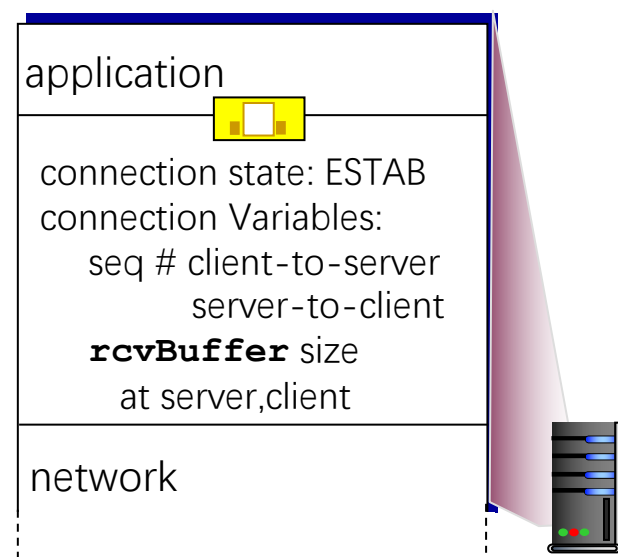


建立TCP连接

- 建立一条TCP连接需要确定两件事：
 - 双方都同意建立连接（知晓另一方想建立连接）
 - 初始化连接参数（序号，MSS等）



```
Socket clientSocket =
    newSocket("hostname", "port
number");
```



```
Socket connectionSocket =
    welcomeSocket.accept();
```



TCP三次握手建立连接



客户状态

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName,serverPort))
```

SYNSENT

ESTAB

选择初始化序列号X,
发送SYN消息

收到X的确认消息, 表明
服务器处于活跃状态,
并发送ACK消息, 以确认
Y已收到;
这一步可能包含客户端-
服务器数据



SYN bit=1, Seq=X

SYN bit=1, Seq=Y;
ACK bit=1, ACK num=X+1

ACK bit=1, ACK num=Y+1

服务器状态

```
serverSocket = socket(AF_INET,SOCK_STREAM)  
serverSocket.bind(('',serverPort))  
serverSocket.listen(1)  
connectionSocket, addr = serverSocket.accept()
```

LISTEN

SYN RCVD

ESTAB

发送SYNACK消息, 以
确认X已收到, 并选择
初始化序列号Y;

收到对Y的确认消息, 表明客
户端处于活跃状态



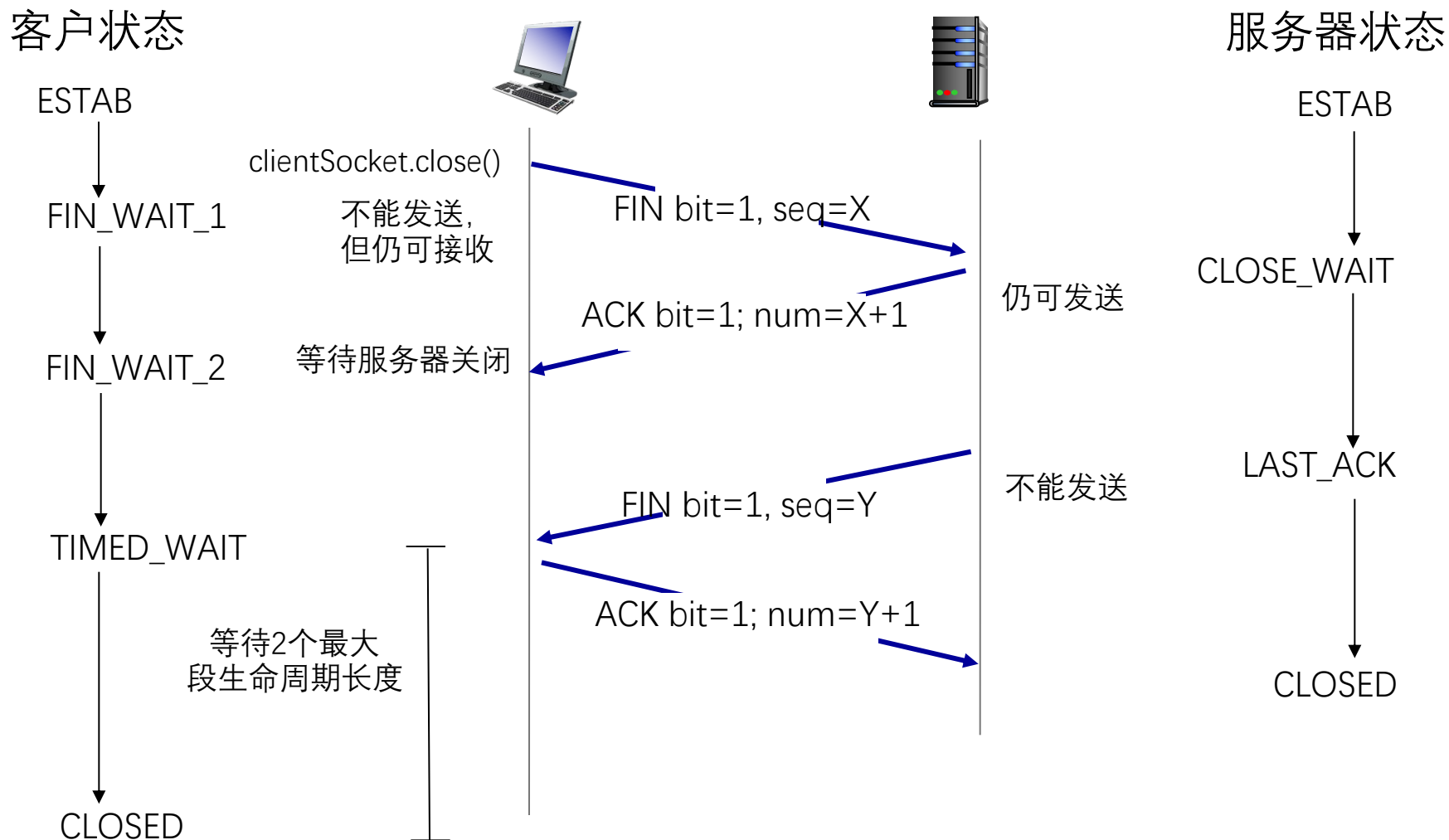
TCP三次握手建立连接



1. 客户端TCP发送SYN 报文段 (SYN=1, ACK=0)
 - 给出客户选择的起始序号
 - 不包含数据
2. 服务器TCP发送SYNACK报文段 (SYN=ACK=1) (服务器端分配缓存和变量)
 - 给出服务器选择的起始序号
 - 确认客户的起始序号
 - 不包含数据
3. 客户端发送ACK报文段 (SYN=0, ACK=1) (客户端分配缓存和变量)
 - 确认服务器的起始序号
 - 可能包含数据

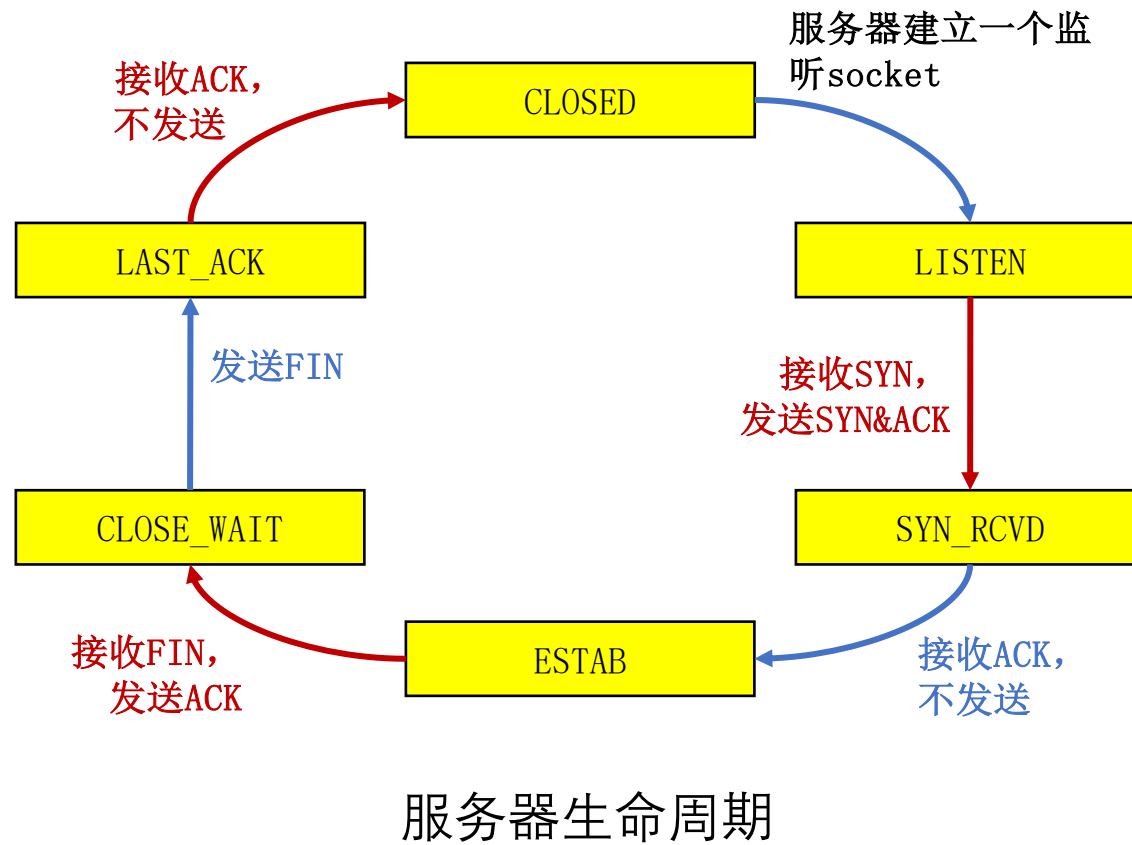
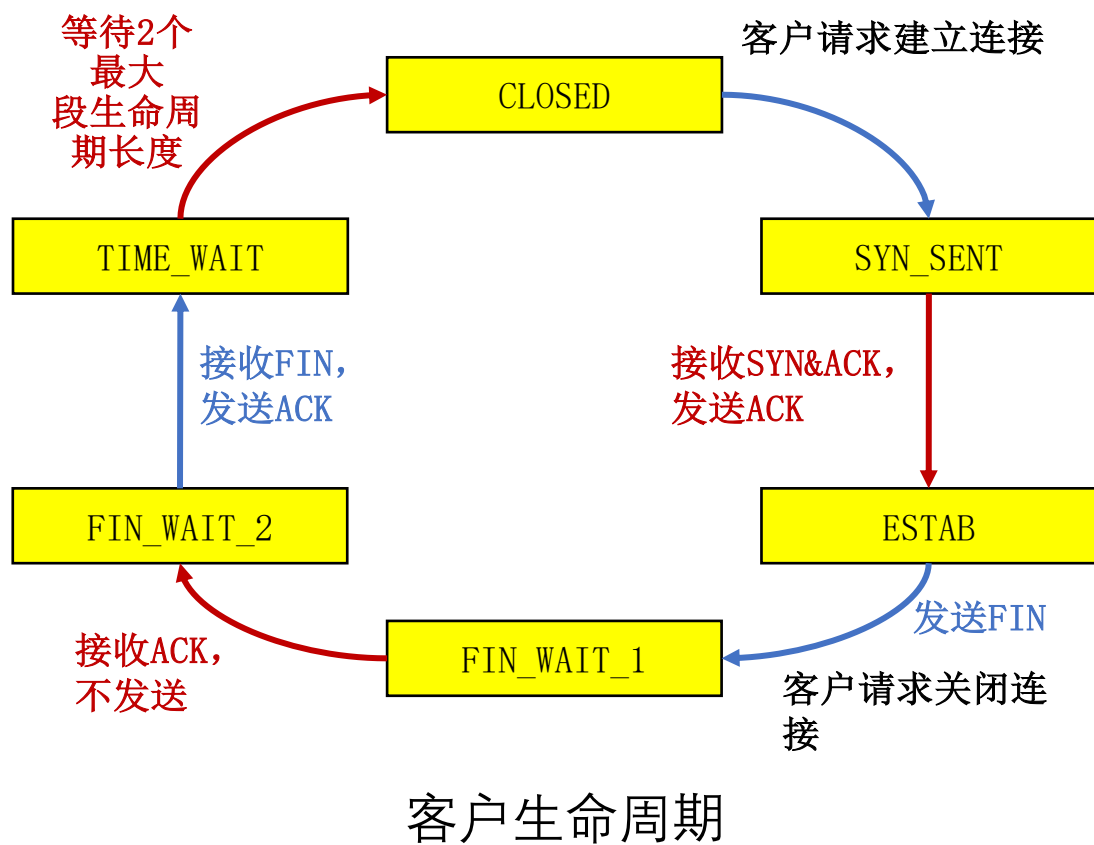


关闭TCP连接





客户/服务器经历的TCP状态序列





本章内容

6.1 概述和传输层服务

6.2 套接字编程

6.3 传输层复用和分用

6.4 无连接传输：UDP

6.5 面向连接的传输：TCP

6.6 理解网络拥塞

6.7 TCP拥塞控制

6.8 拥塞控制的发展

6.9 传输层协议的发展



TCP拥塞控制要解决的问题

- TCP使用端到端拥塞控制机制：
 - 发送方根据自己感知的网络拥塞程度，限制其发送速率
- 需要回答三个问题：
 - 发送方如何感知网络拥塞？
 - 发送方采用什么机制来限制发送速率？
 - 发送方感知到网络拥塞后，采取什么策略调节发送速率？



拥塞检测和速率限制

发送方如何感知拥塞？

- 发送方利用丢包事件感知拥塞：
 - 拥塞造成丢包和分组延迟增大
 - 无论是丢包还是分组延迟过大，对于发送端来说都是丢包了
- 丢包事件包括：
 - 重传定时器超时
 - 发送端收到3个重复的ACK

发送方采用什么机制限制发送速率？

- 发送方使用拥塞窗口 cwnd (congestion window) 限制已发送未确认的数据量：

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

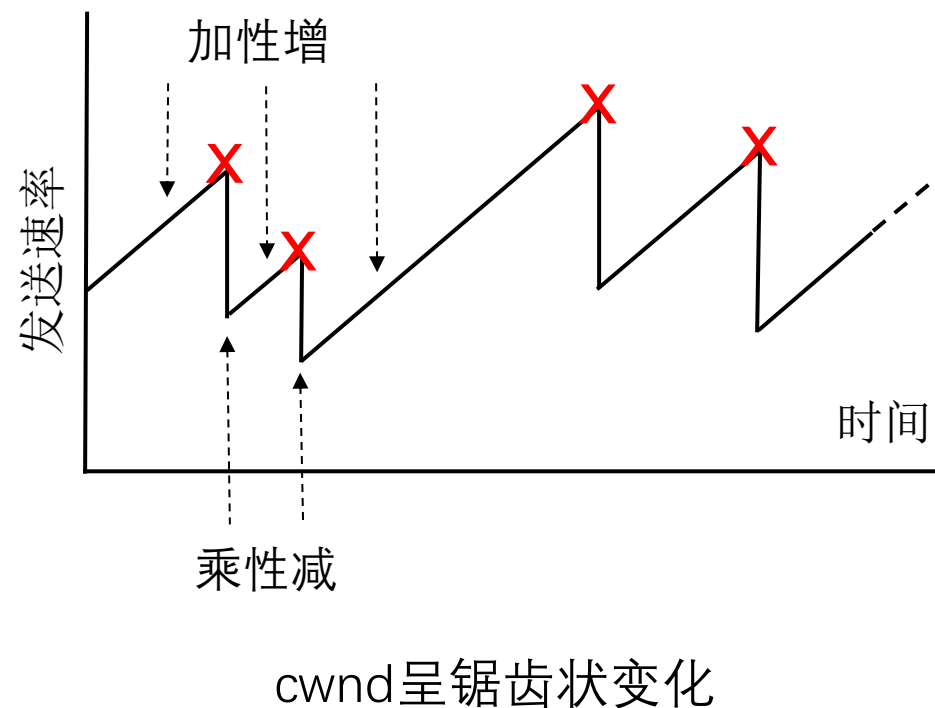
$$\text{rate} = \frac{\text{cwnd}}{\text{RTT}} \quad \text{Bytes/sec}$$

- cwnd随发送方感知的网络拥塞程度而变化



拥塞窗口的调节策略：AIMD

- 乘性减 (Multiplicative Decrease)
 - 发送方检测到丢包后，将cwnd的大小减半（但不能小于一个MSS）
 - 目的：迅速减小发送速率，缓解拥塞
- 加性增 (Additive Increase)
 - 若无丢包，每经过一个RTT，将cwnd增大一个MSS，直到检测到丢包
 - 目的：缓慢增大发送速率，避免振荡





TCP慢启动

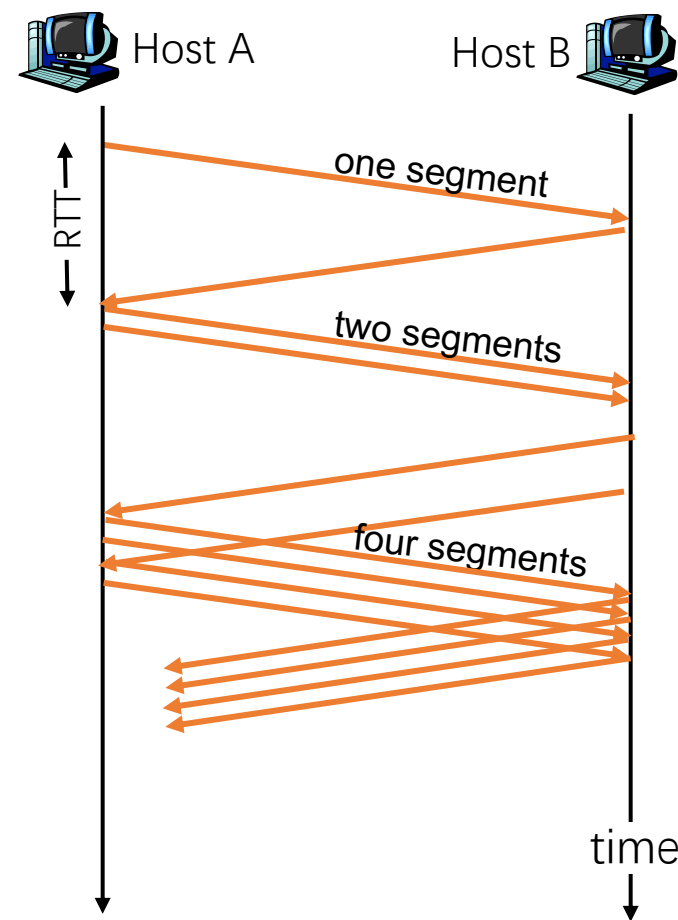


- 在新建的连接（或沉寂了一段时间的连接）上，以什么速率发送数据（此时接收窗口达最大值）？
- 早期的TCP协议：
 - 发送端仅以接收窗口大小限制发送速率，网络经常因为拥塞而崩溃！
- 采用“加性增”增大发送窗口，太慢！
 - 在新建连接上，令 $cwnd = 1 \text{ MSS}$ ，起始速度 = MSS/RTT
 - 然而，网络中的可用带宽可能远大于 MSS/RTT
- 慢启动的基本思想：
 - 在新建连接上指数增大 $cwnd$ ，直至检测到丢包（此时终止慢启动）
 - 希望迅速增大 $cwnd$ 至可用的发送速度



慢启动的实施

- 慢启动的策略：
 - 每经过一个RTT，将cwnd加倍
- 慢启动的具体实施：
 - 每收到一个ACK段，cwnd增加一个MSS
 - 相当于每经过一个RTT，将cwnd加倍
 - 只要发送窗口允许，发送端可以立即发送下一个报文段
- 特点：
 - 以一个很低的速率开始，按指数增大发送速率
- 慢启动比谁“慢”？
 - 与早期TCP按接收窗口发送数据的策略相比，采用慢启动后发送速率的增长较慢





区分不同的丢包事件

- 超时和收到3个重复的ACK，它们反映出来的网络拥塞程度是一样的吗？ **当然不一样！**
- **超时**：说明网络交付能力很差
- **收到3个重复的ACK**：说明网络仍有一定的交付能力
- 目前的TCP实现区分上述两种不同的丢包事件



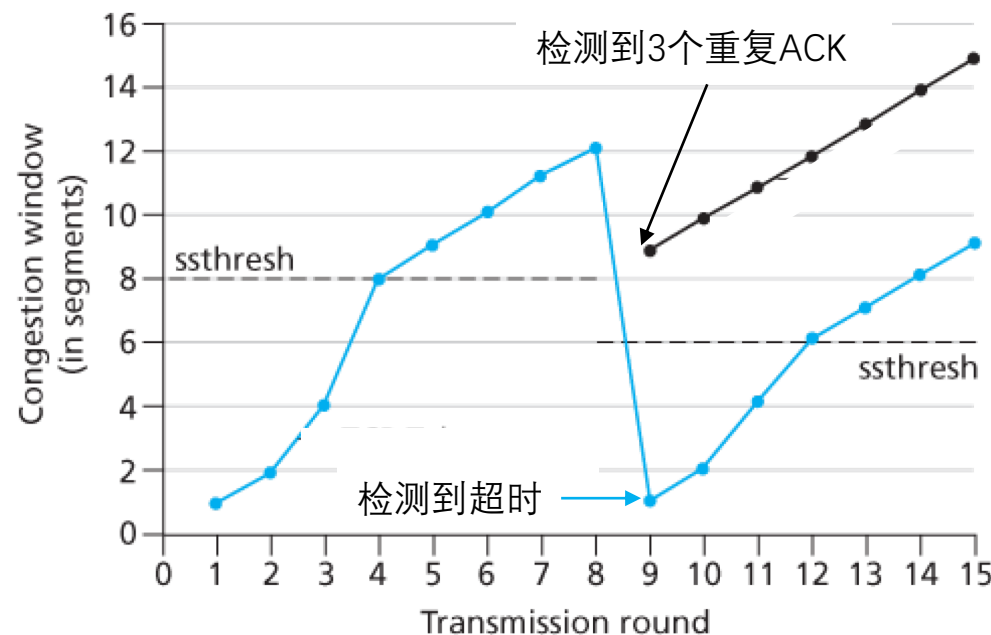
区分不同的丢包事件

- 超时和收到3个重复的ACK，它们反映出来的网络拥塞程度是一样的吗？ **当然不一样！**
- **超时**：说明网络交付能力很差
- **收到3个重复的ACK**：说明网络仍有一定的交付能力
- 目前的TCP实现区分上述两种不同的丢包事件
 - 收到3个重复的ACK：
 - 设置 $ssthresh$ (slow start threshold) $= cwnd/2$
 - 将 $cwnd$ 降至 $ssthresh + 3$
 - 使用AIMD调节 $cwnd$
 - 超时：
 - 设置 $ssthresh = cwnd/2$
 - $cwnd = 1MSS$
 - 使用慢启动增大 $cwnd$ 至 $ssthresh$
 - 使用AIMD调节 $cwnd$



TCP拥塞控制的实现

- 发送方维护变量ssthresh
- 发生丢包时, $ssthresh = cwnd / 2$
- ssthresh是从慢启动转为拥塞避免的分水岭:
 - cwnd低于ssthresh时, 执行慢启动
 - cwnd高于ssthresh时: 执行拥塞避免
- 拥塞避免阶段, 拥塞窗口线性增长:
 - 每当收到ACK, $cwnd = cwnd + MSS * (MSS / cwnd)$
 - 相当于每经过一个RTT, cwnd增加一个MSS



- 检测到3个重复的ACK后 (黑色线):
 - $cwnd = ssthresh + 3$, 线性增长
- 检测到超时后 (蓝色线):
 - $cwnd = 1$ MSS, 慢启动



TCP发送端的事件与动作

状态	事件	TCP发送方操作	说明
慢启动 (SS)	收到新的确认	$cwnd = cwnd + MSS$, If ($cwnd > ssthresh$) set state to "Congestion Avoidance"	每收到一个ACK段, cwnd增加一个MSS; 即每经过一个RTT, cwnd加倍
拥塞避免 (CA)	收到新的确认	$cwnd = cwnd + MSS * (MSS / cwnd)$	每经过一个RTT, cwnd 增加一个MSS
SS or CA	收到3个重复的 确认	$ssthresh = cwnd / 2$, $cwnd = ssthresh + 3$, Set state to "Congestion Avoidance"	cwnd减半后加3, 然后线 性增长
SS or CA	超时	$ssthresh = cwnd / 2$, $cwnd = 1 \text{ MSS}$, Set state to "Slow Start"	cwnd降为一个MSS, 进 入慢启动
SS or CA	收到一个重复的 确认	统计收到的重复确认数	cwnd 和 ssthresh 都不变



本章总结



- 掌握传输层服务原理
 - 传输层复用和分用
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- 掌握因特网传输层协议：
 - UDP协议
 - TCP协议



第七章 应用层

授课教师：张圣林

南开大学



本章目标



1. 掌握应用进程通信方式以及服务进程工作模式
2. 掌握域名系统基本原理和工作机制
3. 掌握电子邮件系统体系结构及基本工作原理
4. 了解WWW系统结构框架、静态和动态Web及其应用技术，掌握HTTP协议及其工作原理
5. 了解流媒体基本概念、数字音视频与编码、流式存储媒体、直播与实时音视频、流媒体动态自适应传输
6. 了解内容分发背景及内容分发网络、以及P2P网络的工作机制
7. 掌握Telnet、FTP、SNMP等应用层协议的工作原理



本章内容



➤ 7.1 应用层概述 (P7)

➤ 7.2 域名系统 (P25)

➤ 7.3 电子邮件 (P74)

➤ 7.4 WWW (P106)

➤ 7.5 流式音频和视频 (P147)

➤ 7.6 内容分发 (P165)

➤ 7.7 其它应用层协议 (P182)

1. 应用进程通信方式
2. 服务器进程工作方式



应用进程通信方式

- 每个应用层协议都是为了解决某一应用问题，通过位于不同主机中的多个应用进程之间的通信和协同工作来完成
 - 两台主机通信实际是其上对应的两个应用进程(process)在通信
 - 应用进程: 为解决具体应用问题而彼此通信的进程
- 应用层的具体内容就是规定应用进程在通信时所遵循的协议

客户/服务器 (C/S, Client/Server) 方式

浏览器/服务器 (B/S, Browser/Server) 方式

对等 (P2P, Peer to Peer) 方式



应用进程通信方式

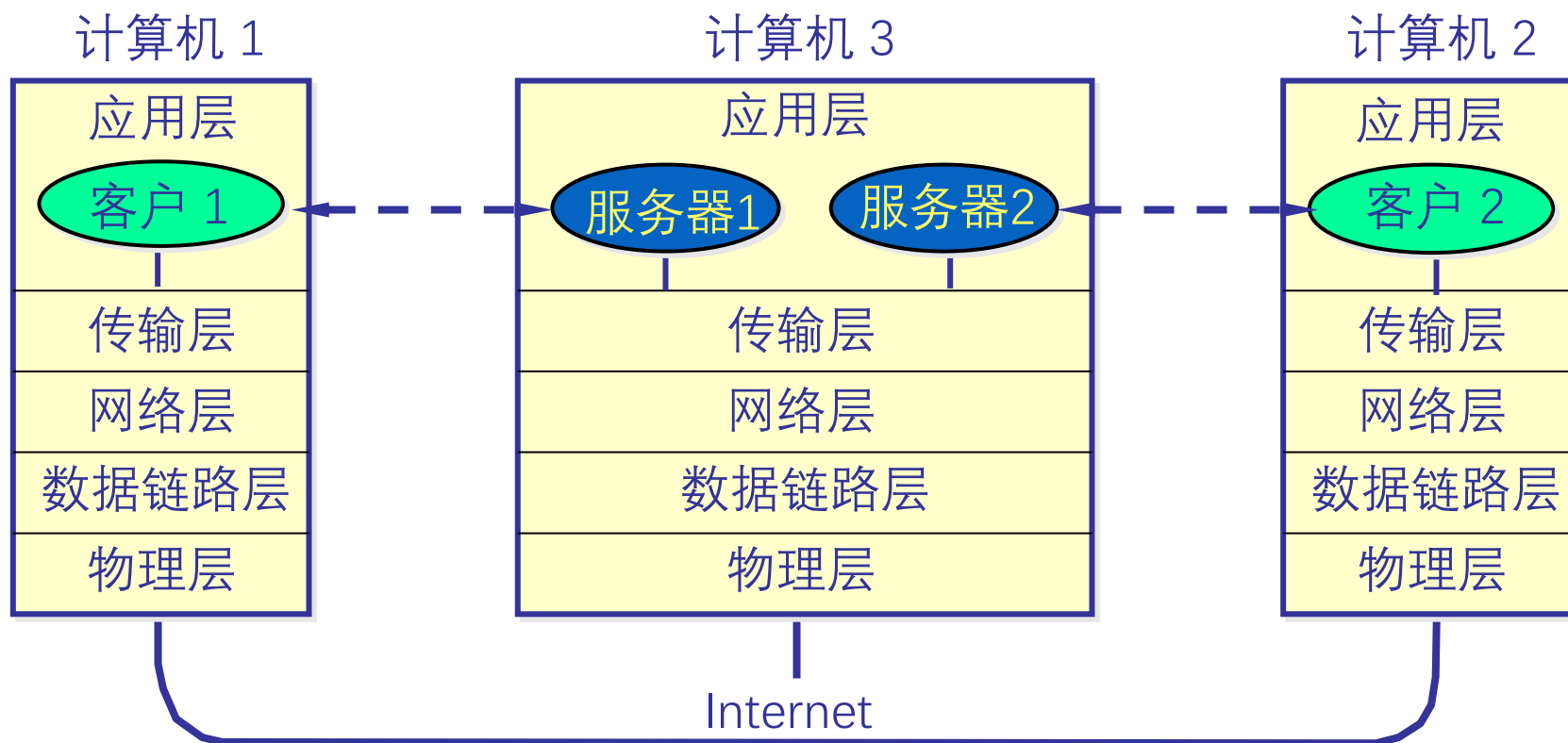
(1) 客户/服务器 (C/S, Client/Server) 方式

- 应用层的许多协议是基于C/S方式，例如，在移动互联网环境下，每个应用APP都是一个客户端
 - 客户(client)和服务端(server)是指通信中所涉及的2个应用进程
 - 客户/服务器方式描述的是应用进程之间服务和被服务的关系
 - 客户是服务请求方（主动请求服务，被服务）
 - 服务器是服务提供方（被动接受服务请求，提供服务）
- C/S方式可以是面向连接的，也可以是无连接的
- 面向连接时，C/S通信关系一旦建立，通信就是双向的，双方地位平等，都可发送和接收数据



应用进程通信方式

- 功能较强的计算机可同时运行多个服务器进程



应用进程通信方式

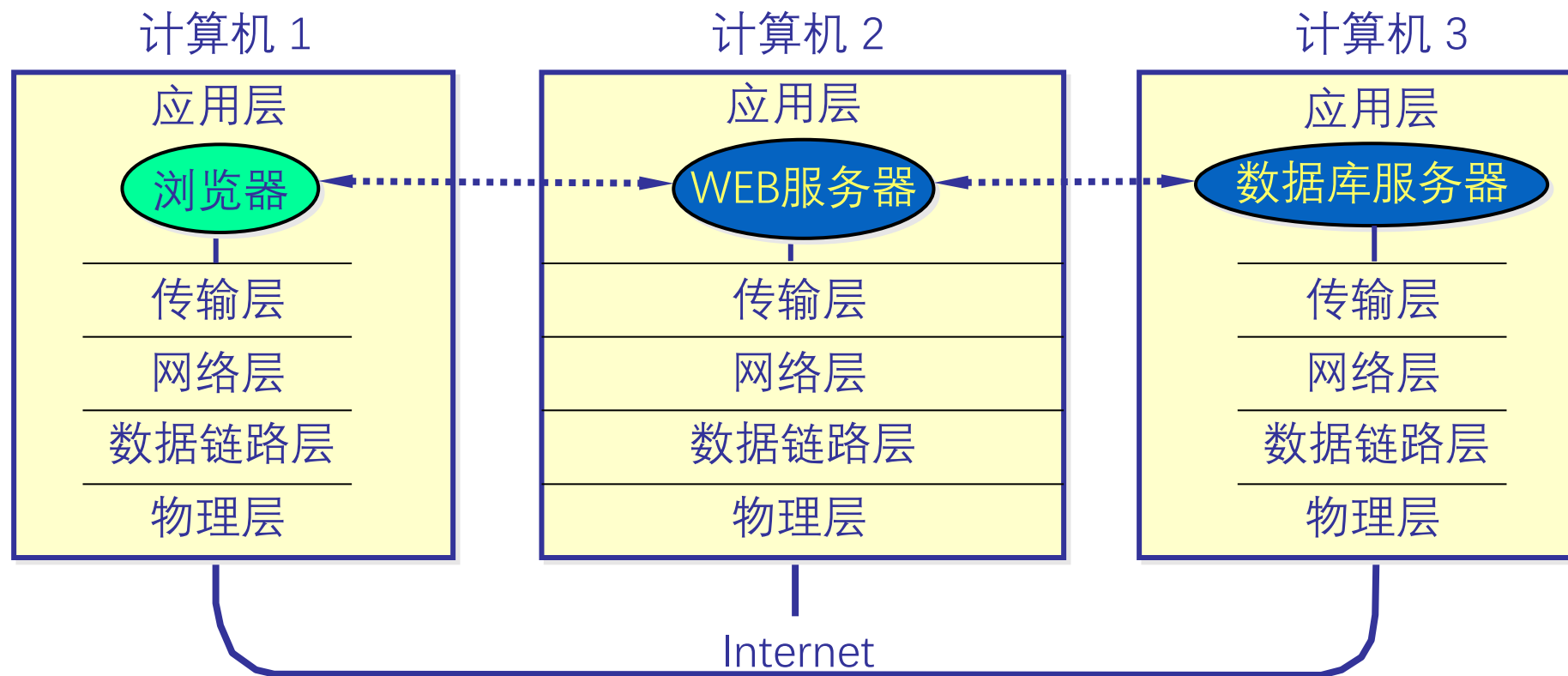
(2)浏览器/服务器(B/S, Browser/Server) 方式

- B/S方式可以看做C/S方式的特例，即客户软件改为浏览器了
- B/S方式采取浏览器请求、服务器响应的工作模式
- 在B/S方式下，用户界面完全通过Web浏览器实现，一部分事务逻辑在前端实现，但主要的事务逻辑在服务器端实现





应用进程通信方式



B/S方式的3层架构示意



应用进程通信方式



➤ B/S方式的特点

- 界面统一，使用简单。客户端只需要安装浏览器软件
- 易于维护。对应用系统升级时，只需更新服务器端的软件，减轻了系统维护和升级的成本
- 可扩展性好。采用标准的TCP/IP和HTTP协议，具有良好的扩展性
- 信息共享度高。HTML是数据格式的一个开放标准，目前大多数流行的软件均支持HTML
- 需要注意的是，在一种浏览器环境下开发的界面在另一种浏览器环境下可能有不完全适配的情况，这时需要安装对应的浏览器



应用进程通信方式



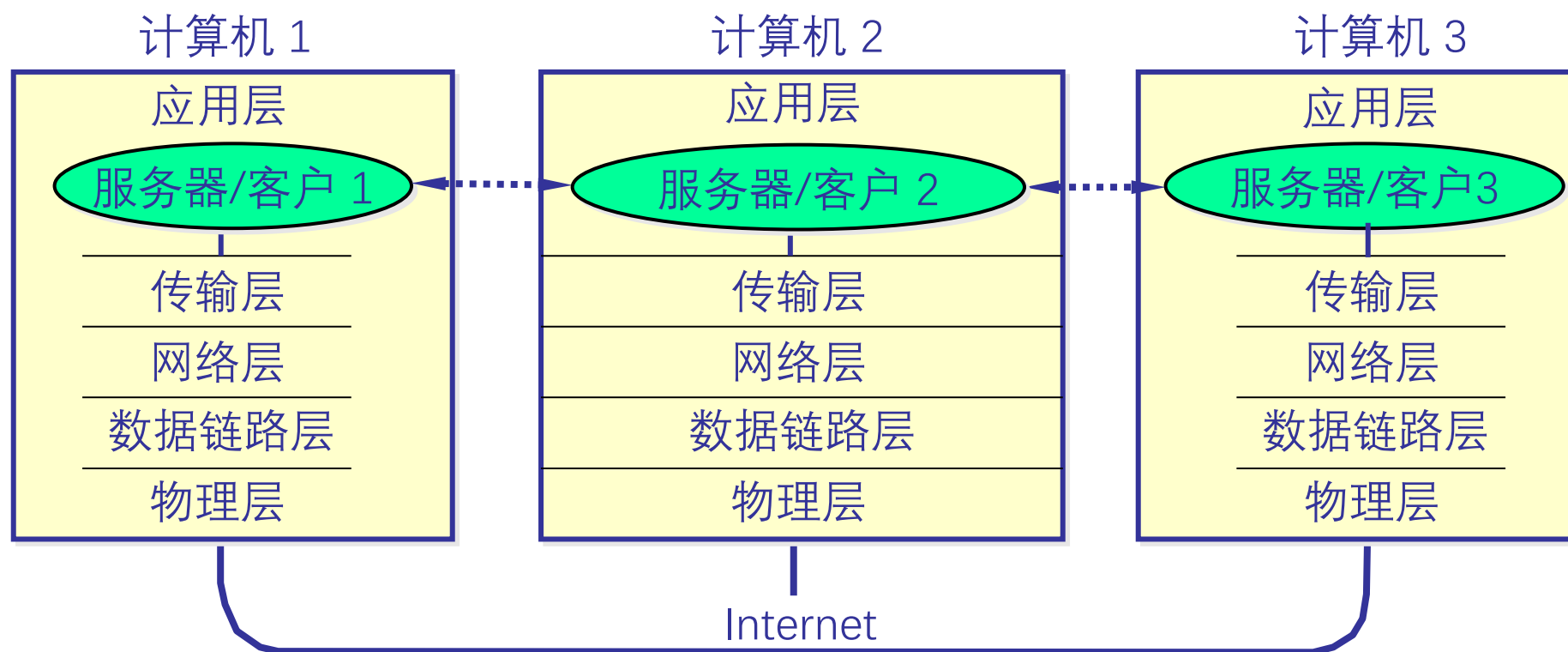
(3) 对等 (P2P, Peer to Peer) 方式

- **对等方式**是指两个**进程**在通信时并不区分服务的请求方和服务的提供方
 - 只要两个主机都运行P2P软件，它们就可以进行**平等、对等的通信**
 - 双方都可以下载对方存储在硬盘中的共享文档
- 音频/视频应用推动了P2P对等通信方式的发展
- 音频/视频流量已占主要比例



应用进程通信方式

- P2P方式从本质上看仍然是使用了C/S方式，但强调的是通信过程中的对等，这时每一个P2P进程既是客户同时也是服务器





服务器进程工作方式



- 循环方式(iterative mode)
 - 一次只运行一个服务进程
 - 当有多个客户进程请求服务时，服务进程就按请求的先后顺序依次做出响应 (阻塞方式)
- 并发方式(concurrent mode)
 - 可以同时运行多个服务进程
 - 每一个服务进程都对某个特定的客户进程做出响应 (非阻塞方式)



本章内容



7.1 应用层概述 (P7)

7.2 域名系统 (P25)

7.3 电子邮件 (P74)

7.4 WWW (P106)

7.5 流式音频和视频 (P147)

7.6 内容分发 (P165)

7.7 其它应用层协议 (P182)

1. 历史和概述
2. 域名系统名字空间和层次结构
3. 域名服务器
4. 域名解析过程
5. 域名系统查询和响应(选讲)
6. 域名系统高速缓存
7. 域名系统隐私(选讲)



历史与概述



➤ 域名系统概述

- **域名系统** (DNS, Domain Name System) 是互联网重要的基础设施之一, 向所有需要域名解析的应用提供服务, 主要负责将可读性好的**域名映射成IP地址**
- Internet采用**层次结构的命名树**作为主机的名字, 并使用**分布式的**域名系统 DNS
- Internet的DNS是一个**联机分布式数据库系统**
- 名字到域名的解析是由**若干个**域名服务器程序完成的。域名服务器程序在专设的节点上运行, 相应的节点也称为**名字服务器**(Name Server)或**域名服务器**(Domain Name Server)





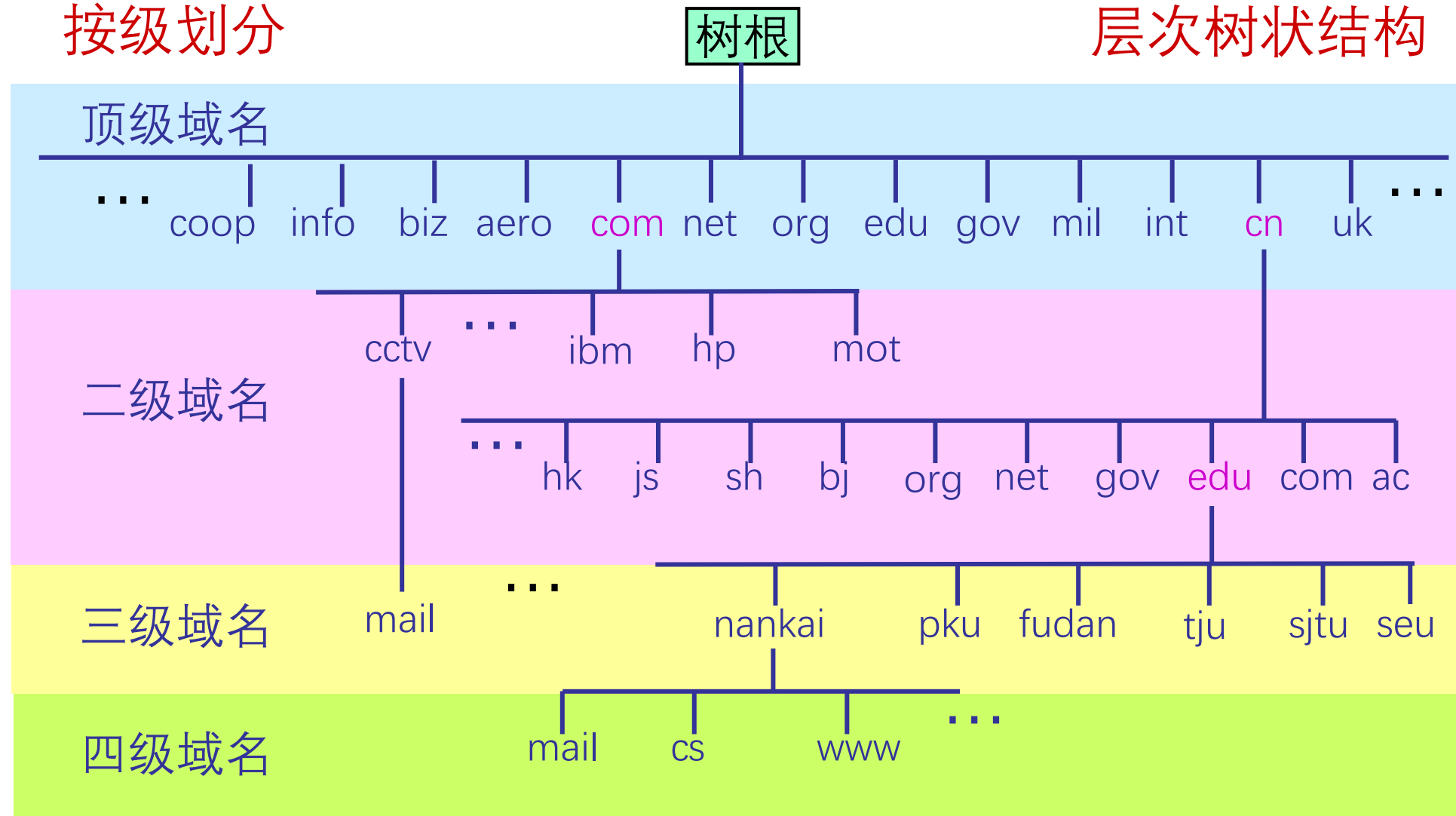
域名系统名字空间和层次结构

按级划分

树根

层次树状结构

域
树





域名系统名字空间和层次结构

Internet的域名结构

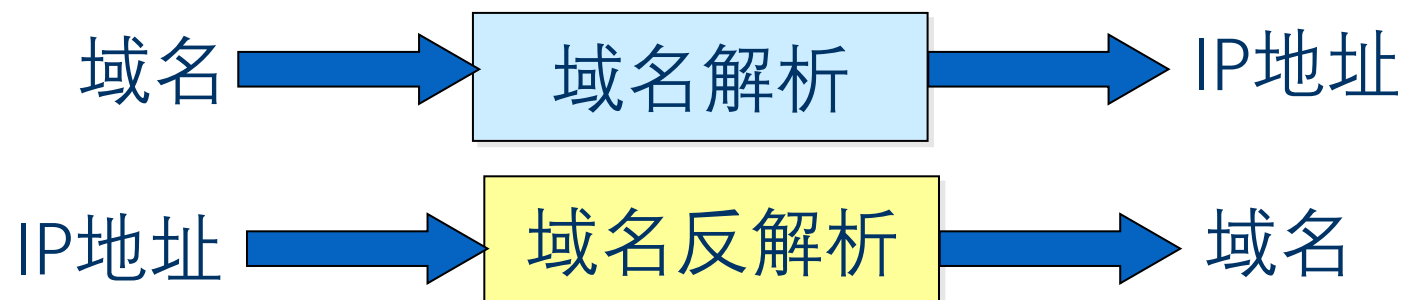
- Internet的域名结构采用了层次树状结构的命名方法
- 域名的结构由若干个分量组成，各分量之间用小数点(.)隔开，总长不超过255个字符
- 各分量分别代表不同级别的域名。(≤63字符)
- 合法域名中，点“.”的个数至少为一个
- 通常，点“.”对应的英文单词为dot，也可以读为point

... .三级域名.二级域名.顶级域名



域名服务器

- 保存关于域树(domain tree)的结构和设置信息的服务器程序称为名字服务器(name server)或域名服务器，负责域名解析工作
- 每个域名服务器具有连向其它有关域名服务器的信息，当需要查询其它域的信息时，能够知道或使其它域名服务器知道到哪里去找别的域名服务器，使域名解析过程能够完成
- 域名与IP地址可以是一对一、一对多或者多对一的关系
- 域名解析过程对用户透明





域名服务器



域名系统的区域

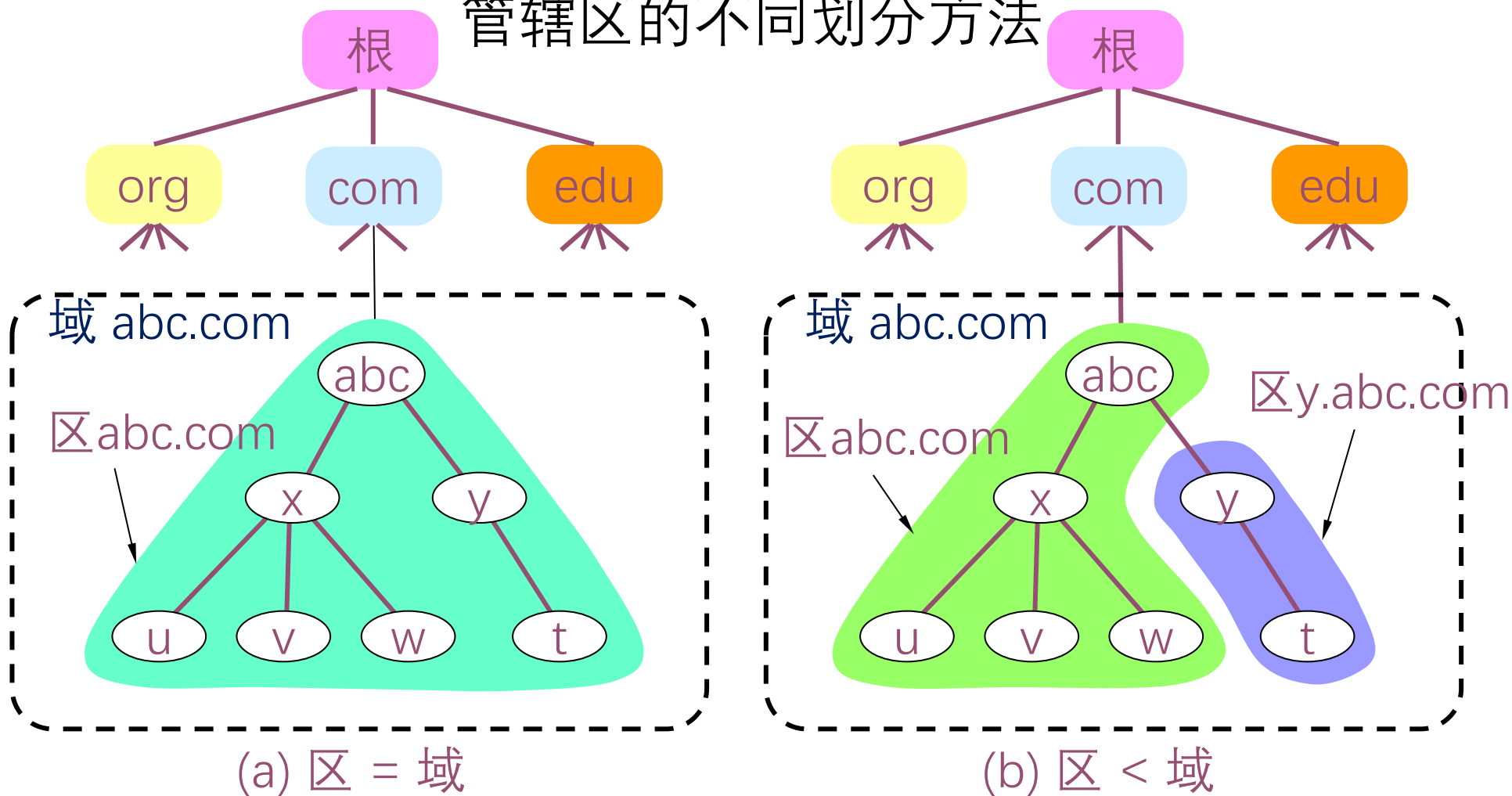
- Internet上域名服务器系统按域名层次树状安排，每个域名服务器管辖一部分域
- 一个域名服务器所负责或管辖（有权限的）范围称为管辖区(zone)，简称为区
- 各单位根据具体情况来划分自己管辖区，在一个区中的所有节点必须是能够连通的
- 域名服务器的管辖范围以“区”为单位，而不是以“域”为单位
- 管辖区是域名“域”的子集，管辖区可以小于或等于域，但不可能大于域



域名服务器



管辖区的不同划分方法





域名服务器

域名服务器类别

➤ 总体上，域名系统的域名服务器分为两大类

- **权威域名服务器**(authoritative name server)
 - 一种根据本地知识知道一个DNS区(zone)内容的服务器，它可以回答有关该DNS区的查询而无需查询其他服务器
 - 每个DNS区至少应有一个IPv4可访问的权威域名服务器提供服务
- **递归解析器**(recursive resolver)/递归服务器
 - 以递归方式运行的、使用户程序联系域(domain)名字服务器的程序。



域名服务器

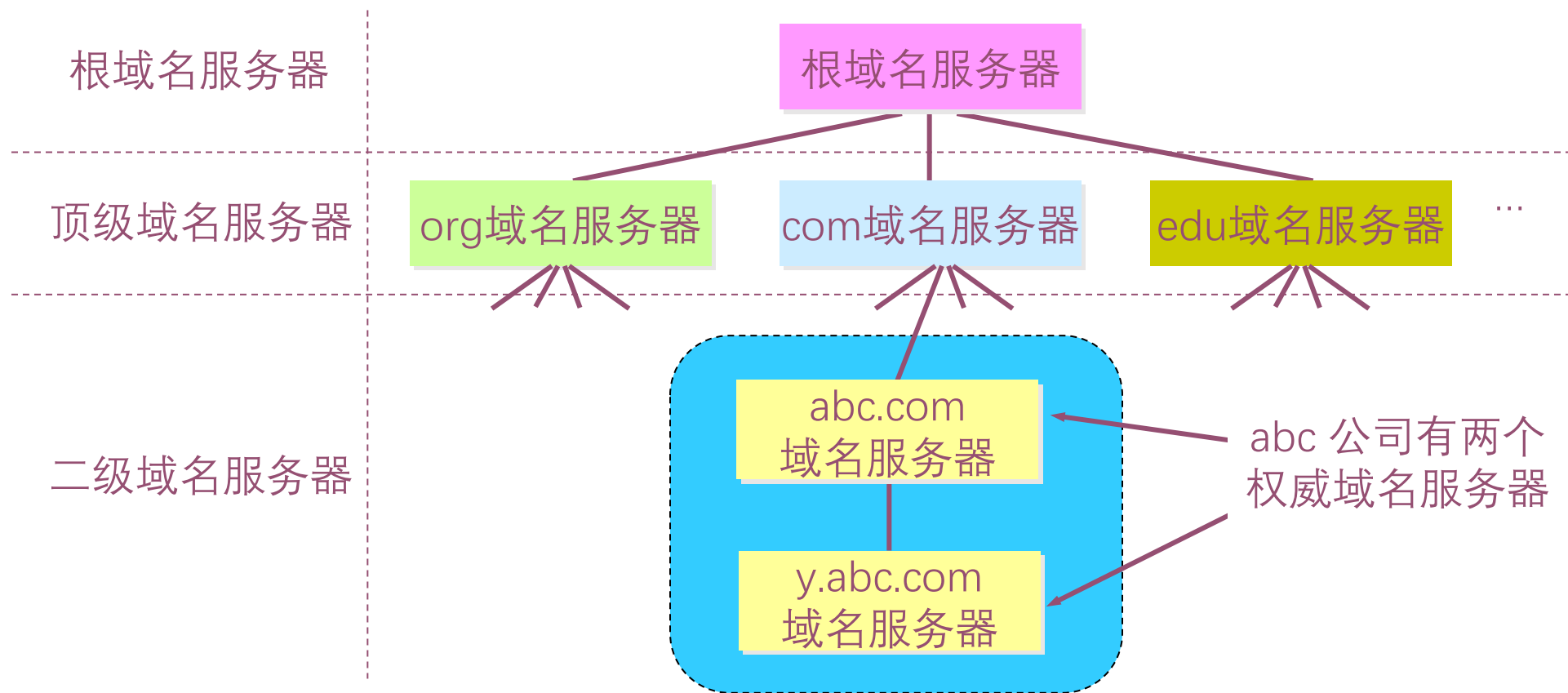


权威域名服务器类别

- 根据对应域的层次，权威域名服务器又进一步分为以下类别
 - 根域名服务器(root name server) /根服务器(root server)
 - 顶级域域名服务器(top-level domain name server)
 - 二级域域名服务器(second level domain name server)
 - 三级域域名服务器(third level domain name server)
 -
- 三级域及以下的域名服务器(例如nankai.edu.cn)通常在用户本地区域，因此三级域及以下的域名服务器也统称为**本地域名服务器**

域名服务器

层次树状结构的权威域名服务器



域名解析过程

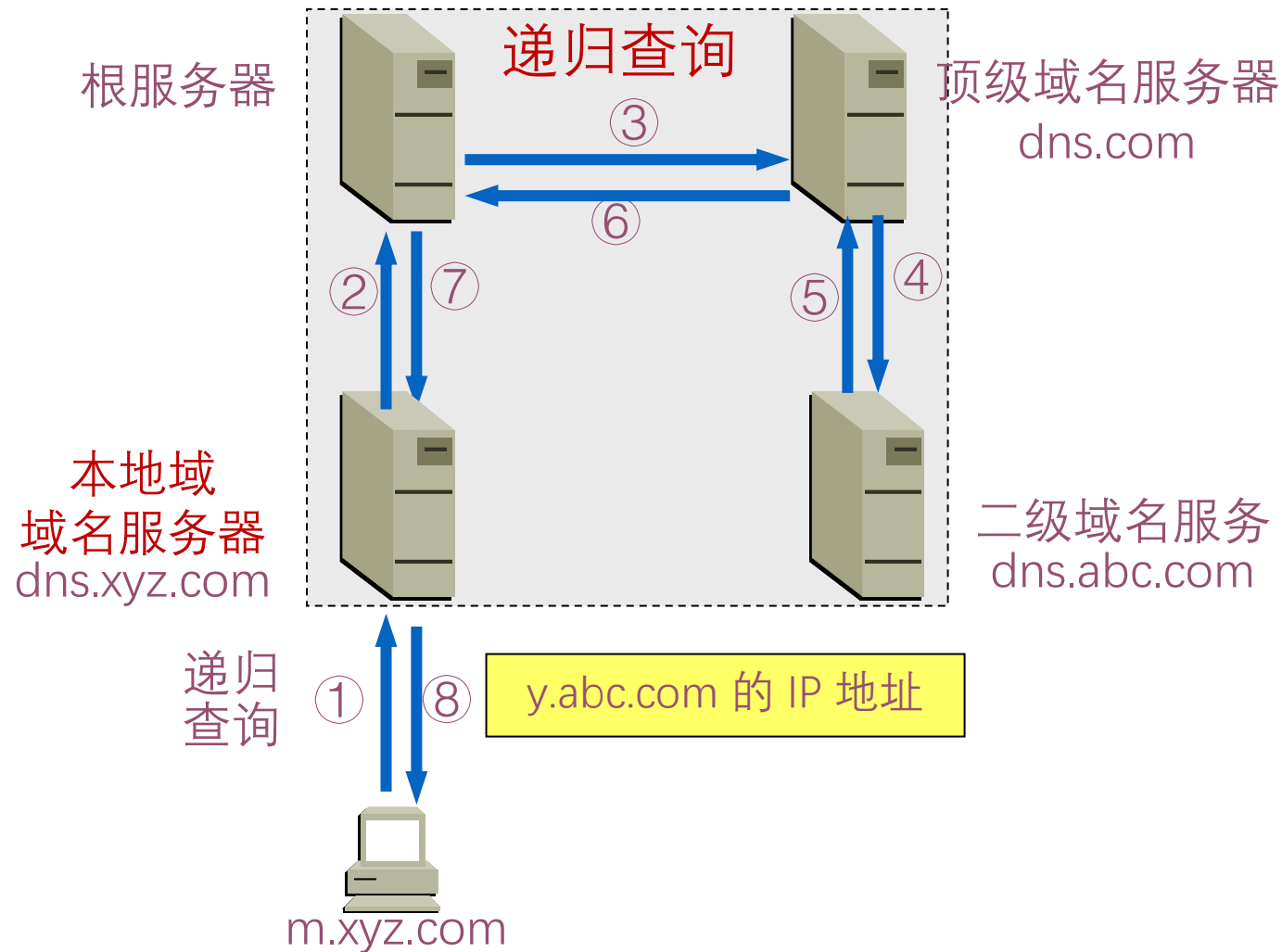
域名解析过程概述

- 当某一应用进程需要进行域名解析时，该应用进程将域名放在DNS请求报文（**UDP数据报，端口号为53**）发给递归服务器（使用UDP是为了减少开销）。递归服务器得到查询结果后，将对应IP地址放在应答报文中返回给应用进程
- 域名查询有**递归查询**(recursive query)和**迭代查询**(或循环查询，iterative query)两种方式
 - **主机向递归解析器/本地域域名服务器的查询**一般采用递归查询
 - **递归解析器/本地域域名服务器向根服务器**可以采用递归查询，但一般**优先采用迭代查询**

域名解析过程

➤ 递归查询

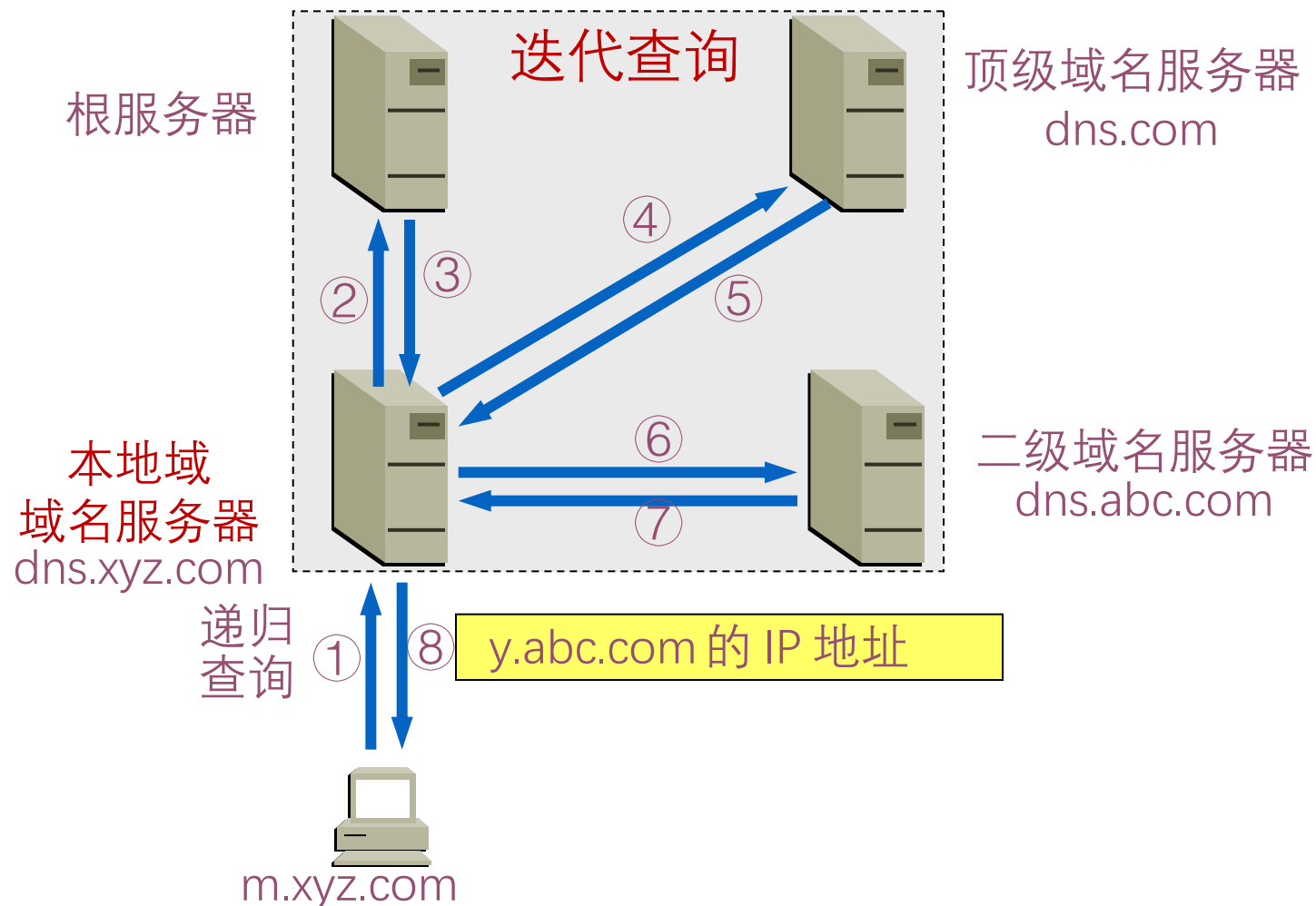
- 当收到查询请求报文的域名服务器不知被查询域名的IP地址时，该域名服务器就以DNS客户的身分向下一步应查询的域名服务器发出查询请求，即替递归服务器继续查询
- 较少使用



域名解析过程

➤ 迭代查询

- 当收到查询请求报文的域名服务器不知道被查询域名的IP地址时，就把自己知道的下一步应查询的域名服务器IP地址告诉**本地域名服务器**，由本地域名服务器继续向该域名服务器查询，直到得到所要解析的域名的IP地址，或者查询不到所要解析的域名的IP地址
- 通常使用





本章内容



- 7.1 应用层概述 (P7)
- 7.2 域名系统 (P25)
- 7.3 电子邮件 (P74)
- 7.4 WWW (P106)
- 7.5 流式音频和视频 (P147)
- 7.6 内容分发 (P165)
- 7.7 其它应用层协议 (P182)

1. 体系结构和服务
2. 用户代理
3. 邮件格式
4. 邮件传送
5. 最后传递



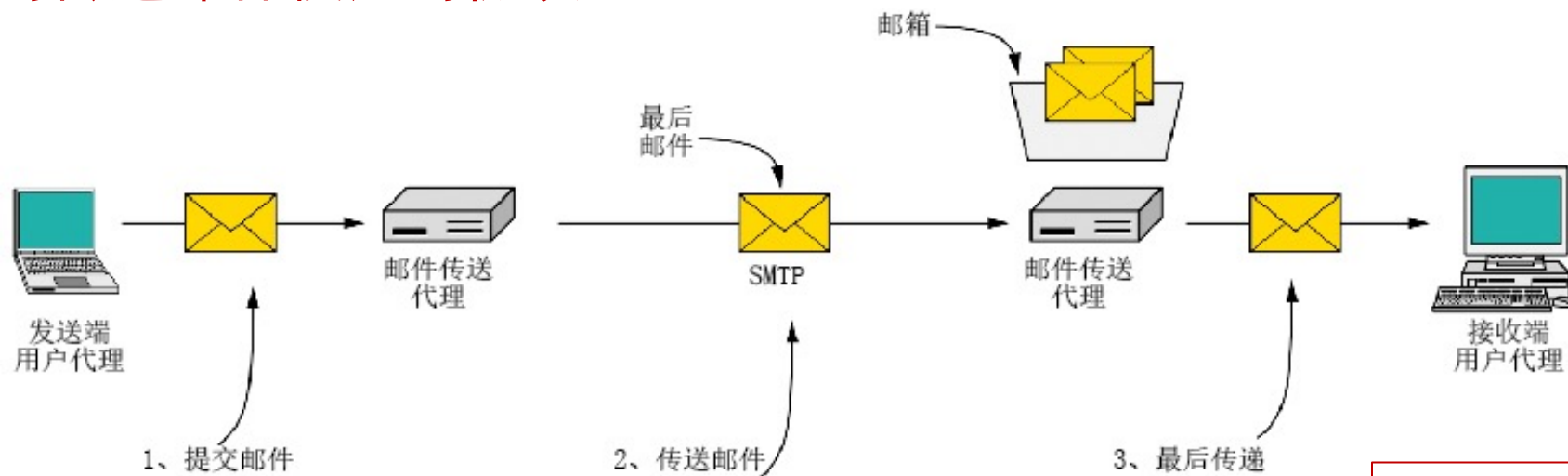
电子邮件概述

- 电子邮件（E-mail）是自早期Internet出现以来最广泛的应用，是一种异步通讯媒介
- 优越性
 - 比纸质信件更快更便宜
 - 使用门槛低
 - 收件人可随时上网到自己邮箱读取邮件
 - 电子邮件不仅可传送文字信息，而且还可附上声音和图像等



电子邮件系统体系结构

- 用户代理 (user agent) —— 邮件客户端
- 传输代理 (message transfer agent) —— 邮件服务器
- 简单邮件传输协议SMTP (Simple Mail Transfer Protocol) —— 邮件服务器之间传递邮件使用的协议



电子邮件系统的构成

电子邮件系统采用C/S工作模式



电子邮件系统体系结构



- 用户代理（邮件客户端）
 - 编辑和发送邮件
 - 接收、读取和管理邮件
 - 管理地址簿
 - 无统一标准
- 传输代理（邮件服务器）
 - 邮箱：保存用户收到的邮件
 - 邮件输出队列：存储等待发送的邮件
 - 运行电子邮件协议
- SMTP协议
 - smtp客户：发送邮件端
 - smtp服务器：接收邮件端

➤ 邮箱是邮件服务器中的一块内存区域，其标识即为电子邮件地址（邮箱名）

➤ 电子邮件地址是一个字符串，用于指定邮件接收者

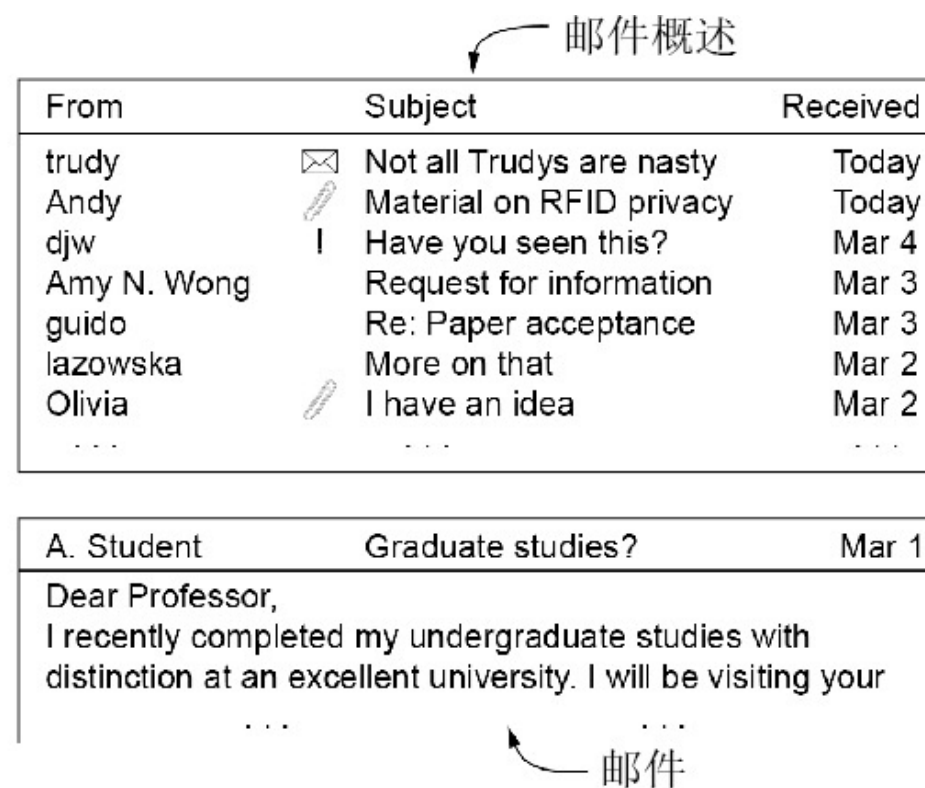
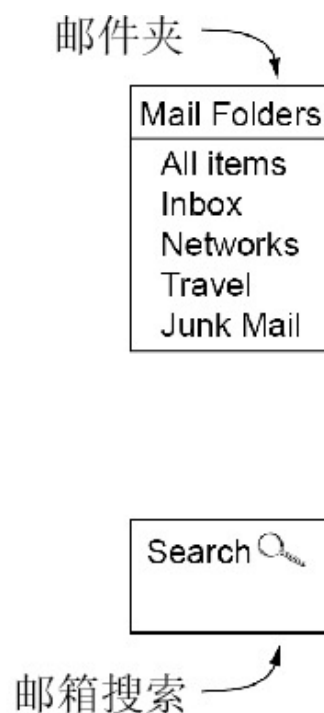
local-part@domain-name

- domain-name: 域名
- local-part: 域名中的邮箱名

例：aaa@hotmail.com、
david@nankai.edu.cn

用户代理

- 用户代理是一个程序，用户通过它和电子邮件系统交互
- 让用户能够阅读和发送电子邮件（email message）
- 与邮件服务器交互，接收、发送电子邮件
- 常用的用户代理：
Outlook, Thunderbird, 邮箱大师, Web客户端等



用户代理接口的典型元素



邮件传输

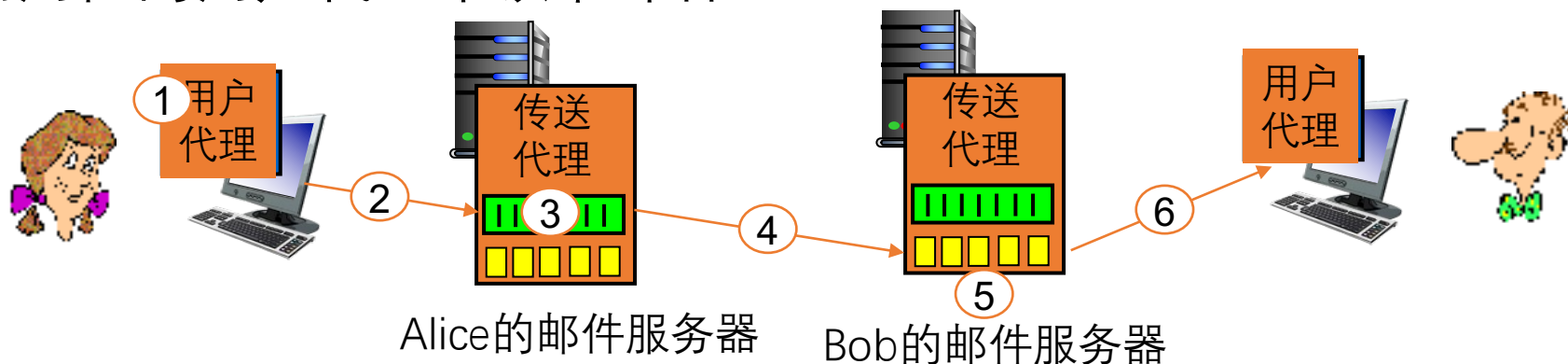


- 邮件传输代理将邮件从发件人中继给收件人
- 邮件传输采用的协议是SMTP
 - SMTP利用TCP可靠地从客户向服务器传递邮件，使用端口25
 - 直接投递: 发送端直接到接收端
 - SMTP的3个阶段：连接建立、邮件传送、连接关闭



场景：Alice发送邮件给Bob

- ① Alice使用用户代理撰写发送给 Bob@someschool.edu的邮件
- ② Alice的用户代理 发送邮件到她的邮件服务器；邮件存放在邮件队列
- ③ SMTP客户端打开与Bob的邮件服务器的TCP连接
- ④ SMTP客户端通过TCP连接发送Alice的邮件
- ⑤ Bob的邮件服务器把邮件存放在Bob的邮箱
- ⑥ Bob调用他的用户代理来读取邮件





POP3协议



- POP3由RFC1939定义，是一个非常简单的最终交付协议
- 当用户代理打开一个到端口110上的TCP连接后，客户/服务器开始工作
- POP3的三个阶段：
 - 认证(Authorization): 处理用户登录的过程
 - 事务处理(Transactions): 用户收取电子邮件，并将邮件标记为删除
 - 更新(Update): 将标为删除的电子邮件删除

POP3使用**客户/服务器**工作方式，在接收邮件的用户PC机中必须运行POP客户程序，而在用户所连接的邮件服务器中则运行POP服务器程序



IMAP



- IMAP—Internet邮件访问协议[RFC 2060]
 - 用于最终交付的主要协议
 - IMAP是POP3的改进版
 - 邮件服务器运行侦听端口为143的IMAP服务
 - 用户代理运行一个IMAP客户端
 - 客户端连接到服务器并开始发出命令



IMAP特性



- IMAP允许用户在不同的地方使用不同的计算机随时上网阅读和处理自己的邮件
- IMAP服务器把每个邮件与一个文件夹联系起来
 - 当邮件第一次到达服务器时，它与收件人的INBOX文件夹相关联
 - 收件人能够把邮件移到一个新的文件夹中，阅读邮件，删除邮件等
- IMAP还为用户提供了在远程文件夹中查询邮件的命令，按指定条件去查询匹配的邮件
- 与POP3不同，IMAP服务器维护了IMAP会话的用户状态信息
 - 例如，文件夹的名字、哪些邮件与哪些文件夹相关联



目 录



- 7.1 应用层概述 (P7)
- 7.2 域名系统 (P25)
- 7.3 电子邮件 (P74)
- 7.4 WWW (P106)
- 7.5 流式音频和视频 (P147)
- 7.6 内容分发 (P165)
- 7.7 其它应用层协议 (P182)

1. Telnet
2. FTP
3. SNMP



Telnet



- 远程登录是网络最早提供的基本服务之一
 - 通过终端仿真协议实现对远程计算机系统的访问，就像访问本地资源一样
 - 这个过程对用户是透明的
- 远程登录需要解决异构计算机系统的差异性问题上
 - 主要体现在对终端键盘输入命令的解释上
- Telnet协议最早出现在20世纪60年代后期
- 1983年由RFC 854确定为Internet标准
- Telnet协议使用C/S方式实现
 - 在本地系统运行Telnet客户进程，在远程主机运行Telnet服务器进程
- Telnet协议使用TCP连接通信
 - 服务器进程默认监听TCP 23端口，使用主进程等待新的请求，并产生从属进程来处理每一个连接



FTP



- 网络环境下复制文件的复杂性
 - 计算机存储数据的格式不同
 - 文件目录结构和文件命名规则不同
 - 对于相同的文件存取功能，操作系统使用的命令不同
 - 访问控制方法不同
- 文件传输协议FTP(File Transfer Protocol)是Internet上使用最广泛的应用层协议之一
 - FTP提供交互式的访问，允许用户指明文件的类型与格式，并允许文件具有存取权限
 - FTP屏蔽了各计算机系统的细节，适用于在异构网络中任意计算机之间传送文件
 - RFC 959早在1985年就已经成为Internet的正式标准



FTP



- FTP使用C/S方式实现
- FTP工作过程
 - 服务器主进程打开TCP 21端口，等待客户进程发出的连接请求
 - 客户可以用分配的任意一个本地端口号与服务器进程的TCP 21端口进行连接
 - 客户请求到来时，服务器主进程启动从属进程来处理客户进程发来的请求
 - 服务器从属进程对客户进程的请求处理完毕后即终止，但从属进程在运行期间根据需要还可能创建其他子进程
 - 服务器主进程返回，继续等待接收其他客户进程发来的连接请求，服务器主进程与从属进程并行工作



SNMP



- SNMP协议实现
 - SNMP使用无连接的UDP协议实现
 - SNMP使用C/S方式实现
- SNMP是有效的网络管理协议
 - 使用周期性轮询方式实现对网络资源的实时监控与管理



本章目标



1. 掌握应用进程通信方式以及服务进程工作模式
2. 掌握域名系统基本原理和工作机制
3. 掌握电子邮件系统体系结构及基本工作原理
4. 了解WWW系统结构框架、静态和动态Web及其应用技术，掌握HTTP协议及其工作原理
5. 了解流媒体基本概念、数字音视频与编码、流式存储媒体、直播与实时音视频、流媒体动态自适应传输
6. 了解内容分发背景及内容分发网络、以及P2P网络的工作机制
7. 掌握Telnet、FTP、SNMP等应用层协议的工作原理



谢谢！